你必须显式地共享这些数据，并明确 GPU 之间的数据访问顺序。SDK4.0 之前，CUDA 环境并不支持这些。如果有可能将程序分解成独立的块，那么尽量采取这种做法。

这种方法有几个替代方法。可以使用 SDK4.0 版本中提供的 GPU 对等通信模型，或者也可以在 CPU 层使用 CPU 级原语以进行合作。前者不适用于所有的操作系统，最明显的是 Windows 7 的消费类硬件产品。CPU 的解决方案需要特定 OS 的通信原码，或者使用一个共同的第三方解决方案。

8.6 按需选用 GPU

当系统有一个以上的 GPU 时，它们是相同还是不同的？程序员是如何知道的？这重要吗？

以上问题通常是很重要的，但这很大程度上取决于应用程序。在 CUDA 二进制文件中，通常嵌入多个二进制映像，分别对应每一代 GPU。二进制文件中至少包含适合于最低计算能力 GPU 的映像。然而，也可能存在更多的二进制映像，用于优化更高级别的计算设备。当执行内核时，CUDA 运行时会自动选择基于最高级别计算设备的二进制代码执行。

某些功能，如原子操作，只适用于特定计算水平的设备。在较低级别的计算设备运行这样的代码，会导致内核启动失败。因此，至少对于一些特定的程序，我们需要去关心哪个 GPU 是被使用的。其他程序在新的硬件上运行得更好或更坏，这取决于应用程序选择的高速缓存块大小。另外一些程序可能被写成使用大量 G80/G200 系列设备上的寄存器，在费米架构上这个数目减少了，但是在开普勒架构中又恢复了。

因此，掌握一些用户和管理员层次的知识是必须的，比如哪个是运行给定内核的最好平台，或者是程序员不得不改写程序使其能够在所有的平台上良好地运行。这可以通过忽略计算设备的特性来实现，不过这经常会让开发人员更难编程。或者可以提供一些具有可替代性的内核来避免计算层面的问题。然而，后者往往由商业问题驱动。程序员的工作时间消耗金钱，你必须评估你的细分市场是否包含了足够多的使用旧硬件的用户，以验证额外的开发和测试工作是否值得。在消费市场方面，截至 2012 年 8 月，大约 1/4 的市场仍然在使用费米架构之前的硬件，参见图 8-1。

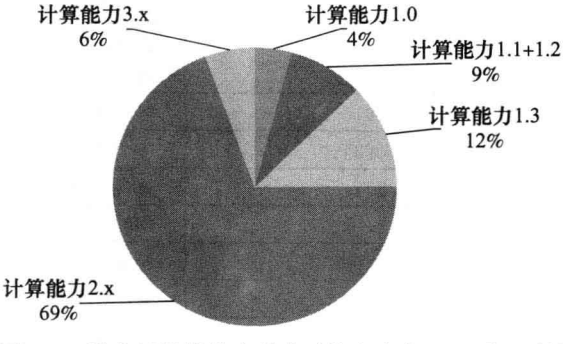

图 8-1 消费级计算能力分布（数字来自 2012 年 8 月）

程序员如何选择一个 GPU 设备呢？到目前为止，我们已经看到过很多例子了，其中我们使用了 4 种设备并且比较不同设备产生的结果。你应该从不同的代码示例中看出，我们需要通过调用以下代码来设置一个设备。

```
cudaError_t cudaSetDevice(int device_num);
```

或者使用如下的简化版本，

```
CUDA_CALL(cudaSetDevice(0));
```

参数 device_num 是一个从零（默认设备）到系统设备数量的一个数字。若要查询设备数量，只需调用下面的语句：

```
cudaError_t cudaGetDeviceCount(int * device_count);

CUDA_CALL(cudaGetDeviceCount(&num_devices));
```

请注意，在两种调用中我们都使用了第 4 章开发的 CUDA_CALL 宏。它仅仅是在遇到错误时获取返回值，检查错误类型，打印相应的错误信息并退出。若要获取 CUDA 究竟是如何工作的更多信息，请参阅第 4 章。

现在我们知道系统有多少设备，以及如何选择其中一个，剩下的问题是该选择哪一个。为此，我们需要知道一个特定设备的详细信息。我们可以使用下面的调用进行查询：

```
cudaError_t cudaGetDeviceProperties(struct cudaDeviceProp * properties,int device);

struct cudaDeviceProp device_0_prop;
CUDA_CALL(cudaGetDeviceProperties(&device_0_prop, 0));
```

properties 的结构特性由表 8-1 的结构成员所示。

表 8-1　设备属性说明

结构体成员	意　义	Tesla 专用	单　位
char name[256];	设备名称，如 GTX460		字符串
size_t totalGlobalMem;	设备全局内存最大值		字节
size_t sharedMemPerBlock;	每个块支持的共享内存最大值		字节
int regsPerBlock;	每个块支持的寄存器数量最大值		寄存器
int warpSize;	设备的线程束大小		线程
size_t memPitch;	在分配定宽内存时，memapy 操作支持的最大定宽		字节
int maxThreadsPerBlock;	每个块支持的线程数量最大值		线程
int maxThreadsDim[3];	每个维度支持的线程数量最大值		线程
int maxGridSize[3];	每个网络维度支持块的最大数量		块
int clockRate;	GPU 的时钟频率		KHz
size_t totalConstMem;	设备上恒定可用内存的最大值		字节
int major;	主计算版次		Int
int minor;	次计算版次		Int
size_t textureAlignment;	对纹理对齐的最低要求		字节
int deviceOverlap;	如果设备支持重叠的内存传输内核（不建议使用），则设置为 1		标志位
int multiProcessorCount;	设备的 SM 数		Int

（续）

结构体成员	意　义	Tesla 专用	单　位
int kernelExecTimeoutEnabled;	如果内核超时功能启用（默认启用），设置为 1		标志位
int integrated;	如果该装置是一个集成的装置，即一个设备共享 CPU，设置为 1		标志位
int canMapHostMemory;	如果该设备可以将 CPU 主机映射到 GPU 虚拟内存的内存空间，则设置为 1		标志位
int computeMode;	目前的计算模式（cudaComputeModeDefault、cudaComputeModeExclusive、cudaComputeModeProhibited） 允许共享的设备，不允许设备访问，或指定设备访问被禁止	x	枚举
int maxTexture1D;	支持最大的一维纹理大小		字节
int maxTexture2D[2];	支持最大的二维纹理大小		字节
int maxTexture3D[3];	支持最大的三维纹理大小		字节
int maxTexture1DLayered[2];	最大的一维分层纹理尺寸		字节
int maxTexture2DLayered[3];	最大的二维分层纹理尺寸		字节
size_t surfaceAlignment;	表面对齐要求		字节
int concurrentKernels;	如果在相同的上下文中支持并发内核设置为 1		标志位
int ECCEnabled;	如果启用了 ECC 内存，则设置为 1	x	标志位
int pciBusID;	PCI 总线上的设备的 ID		Int
int pciDeviceID;	PCI 设备的设备 ID		Int
int pciDomainID;	PCI 域的设备的 ID		Int
int tccDriver;	如果 TCC 驱动模式启用则设置为 1	x	标志位
int asyncEngineCount;	设备上异步复制引擎数量	x①	Int
int unifiedAddressing;	如果设备和主机共享一个统一的地址空间，则设置为 1	x②	标志位
int memoryClockRate;	最大支持内存时钟频率，单位为 KHz		KHz
int memoryBusWidth;	内存总线宽度		位
int l2CacheSize;	二级（L2）高速缓存的大小（0 为不存在）		字节
int maxThreadsPerMultiProcessor;	一个 SM 支持线程的最大数目		线程

①双复制引擎只支持 Telsa 设备。消费级设备限制于只支持单复制引擎。

②统一的地址仅支持 64 位平台。在 Windows 上，它需要 TCC 驱动程序，而这又需要一个 Telsa 卡。UNIX 平台上没有这种情况。

并非所有这些都是你感兴趣的，但一定会有一些值得你关注。这其中最重要的是主要和次要的计算能力的修订版次。还要注意的是这里的 warpSize，尽管实际中对于目前发布的所

有设备它的值保持为 32，然而它会随设备的不同而发生改变。

在选择设备时，没有必要检查每一个条目，以确认它是否有什么特殊的用户程序需要。你可以简单地填充结构体中你想要的属性，并且让（0 等于不关心）CUDA 运行时自动为你选择合适的设备。例如：

```
struct cudaDeviceProp device_prop;
int chosen_device;

memset(device_prop, 0, sizeof(cudaDeviceProp));
device_prop.major = 2;
device_prop.minor = 0;
if (cudaChooseDevice(&chosen_device, device_prop) != cudaErrorInvalidValue)
{
CUDA_CALL(cudaSetDevice(chosen_device));
}
```

在这段代码中，我们创建一个设备属性的结构，调用 memset 清空它，并请求计算能力 2.0 设备（任何费米架构设备）。然后，我们让 CUDA 将内容设置到指定的设备。

8.7　单节点系统

在 SDK4.0 之前版本的 CUDA 中，单节点系统是唯一支持多 GPU 模型的系统，如图 8-2 所示。一个基于单 CPU 的任务将与单 GPU 上下文相关联。在该上下文中的一个任务是一个进程或线程。后台的 CUDA 运行时将 CPU 进程 / 线程 ID 绑定到 GPU 上下文。因此，其随后所有 CUDA 调用（例如 cudaMalloc）将在绑定到该上下文的设备中分配内存。

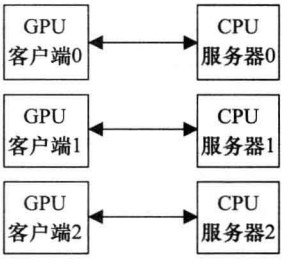

图 8-2　多个客户端、多服务器结构

这种方法有许多缺点，但也有些优点。从编程的角度看，在主机端的进程 / 线程模型被操作系统类型切分了。进程是一个作为一个独立 CPU 调度单位运行并且有其自己的数据空间的程序。为了节省内存，通常在同一进程的多个实例共享代码空间并且操作系统内每个进程保留了一组寄存器（或者上下文）。

相比而言，线程是一种更轻便的 CPU 调度元素。它同时共享父进程的代码及数据空间。然而，与进程不同的是，每个线程需要操作系统保持一个状态（指令指针、堆栈指针和寄存器等）。

线程可以与同一进程内的其他线程沟通和合作。进程可以通过进程间通信与其他进程沟通和合作。这样进程之间的通信可以是在一个 CPU 核内、一个 CPU 插槽内、一个 CPU 节点内、一个机架内、一个计算机系统内甚至是在不同计算机系统之间。

实际上 API 变化取决于通信水平和操作系统。Windows 上使用的 API 和 Linux 上使用的是完全不同的。POSIX 线程或者 pthreads，是一种 Linux 上经常使用的线程模型。尽管作为一个接口它是可用的，但是 Windows 中并不支持。C++Boost 库支持一个命名为 thread 的常

用线程包，这个包为 Linux 和 Windows 提供了支持。

执行内核时，CPU 线程和 GPU 线程是相似的，唯一的不同是它不像 GPU 一样划分为组或线程束执行。GPU 线程通过共享内存和显式同步以确保每一个线程对该内存进行了读 / 写。共享内存对于 SM 来说是本地的，这意味着线程只能（理论上）与同一 SM 上的其他线程进行通信。因为线程块是 SM 的调度单元，线程间通信实际上是限制在一个线程块内的。

我们可以以相同的方式看待 CPU 上的进程和 GPU 上的线程块，即一个进程被调度到 N 个 CPU 核中的一个上运行。一个线程块被调度到 GPU 上 N 个 SM 中的一个执行。从这个角度来看，SM 的行为很像 CPU 核。

CPU 进程可以通过同一个插槽上的主机内存和其他进程进行通信。然而，由于进程使用单独的内存空间，因此这只能在第三方的进程间通信库的协助下才能发生，因为进程无法看到其他进程实际地址空间。然而，对于 GPU 线程块却是不同的，因为它们访问 GPU 全局内存上共用的地址空间。

使用共享主机内存的多 CPU 系统也可以与另一个 CPU 通过该共享主机内存通信，但同样需要第三方的进程间通信库的帮助。同一个主机端的多 GPU 之间可以使用主机内存或 (截至 CUDA SDK 4.0) 直接通过 PCI-E 总线的对等通信模式进行通信。但是请注意，对等只支持使用费米架构或更新代次显卡的 64 位操作系统。对于 Windows，这仅被 TCC（Tesla 计算集群）驱动程序支持，这实际上意味着它仅被 Tesla 显卡支持。

然而，无法使用 CPU 核心 / 插槽间的共享主机内存，你不得不使用一些其他的网络传输机制（TCP/IP、InfiniBand 等）。这种类型的通信标准已经成为 MPI（消息传递接口）。也有一些替代选择，如 ZeroMQ（0MQ），它不太有名但却同样的有效。

请注意，在一个主机节点内通信时它们都利用了共享主机内存传输。然而，支持线程的模型（例如，pthreads、ZeroMQ）执行基于线程间的通信比那些基于进程的模型（如 MPI 等）更快。

我们关注一下如下情况：用一个 CPU 插槽，其上的 CPU 运行只含一个线程程序的多 GPU 环境。这是最常见的消费级硬件的使用案例，因此也是最有用的案例。如果想了解更高级的议题（如在多 GPU 间进行对等传输），请参阅第 10 章。

8.8 流

流是 GPU 上的虚拟工作队列。它们用于异步操作，也就是说你希望 GPU 独立于 CPU 进行操作。一些操作会隐式地引发一个同步点，例如，在主机和设备间默认的内存传入、传出操作。绝大多数情况下这都是程序员所希望的，因为从 GPU 中将结果复制回 CPU 后它们可以立刻在 CPU 上对这个结果进行处理。如果结果是部分地出现，那么应用程序只有在调试或单步情况下才能运行，而无法在全速下运行——这就是调试噩梦。

通过创建一个流，你可以将任务和事件压入流，然后按照它们会被压入流的顺序执行它们。流和事件与它们被创建时所在的 GPU 上下文关联。因此，为了展示在多 GPU 上如何建立一系列的数据流和事件，我们将建立一个小程序来说明这一点。

```
void fill_array(u32 * data, const u32 num_elements)
{
 for (u32 i=0; i< num_elements; i++)
 {
  data[i] = i;
 }
}

void check_array(char * device_prefix,
                 u32 * data,
                 const u32 num_elements)
{
 bool error_found = false;

 for (u32 i=0; i< num_elements; i++)
 {
  if (data[i] != (i*2))
  {
   printf("%sError: %u %u",
          device_prefix,
          i,
          data[i]);

   error_found = true;
  }
 }

 if (error_found == false)
  printf("%sArray check passed", device_prefix);
}
```

在第一个函数中，我们简单地使用0 ~ num_elements对该数组进行填充。第二个函数单纯检查GPU的结果是否是我们所期待的。显然，若要这两个函数能做一些更有用的工作，我们可以替换部分代码，实际上：

```
__global__ void gpu_test_kernel(u32 * data)
{
 const int tid = (blockIdx.x * blockDim.x)
     + threadIdx.x;
 data[tid] *= 2;
}
```

接下来我们声明内核函数本身。它仅仅是让每一个数据元素乘以2。这并没有其他功能，只是让我们可以很容易地检查是否每一个数组的元素都被正确处理了。

```
// Define maximum number of supported devices
#define MAX_NUM_DEVICES (4)

// Define the number of elements to use in the array
#define NUM_ELEM (1024*1024*8)

// Define one stream per GPU
```

```
cudaStream_t stream[MAX_NUM_DEVICES];

// Define a string to prefix output messages with so
// we know which GPU generated it
char device_prefix[MAX_NUM_DEVICES][300];

// Define one working array per device, on the device
u32 * gpu_data[MAX_NUM_DEVICES];

// Define CPU source and destination arrays, one per GPU
u32 * cpu_src_data[MAX_NUM_DEVICES];
u32 * cpu_dest_data[MAX_NUM_DEVICES];
```

最后，我们看看程序的主要部分。该函数声明了一些值，每个值通过 device_num 索引。这使我们能够对每个设备使用相同的代码并且仅需要增加索引的值。

```
// Host program to be called from main
__host__ void gpu_kernel(void)
{
 // No dynamic allocation of shared memory required
 const int shared_memory_usage = 0;

 // Define the size in bytes of a single GPU's worth
 // of data
 const size_t single_gpu_chunk_size = (sizeof(u32) *
 .          NUM_ELEM);

 // Define the number of threads and blocks to launch
 const int num_threads = 256;
 const int num_blocks = ((NUM_ELEM + (num_threads-1))
       / num_threads);

 // Identify how many devices and clip to the maximum
 // defined
 int num_devices;
 CUDA_CALL(cudaGetDeviceCount(&num_devices));
 if (num_devices > MAX_NUM_DEVICES)
  num_devices = MAX_NUM_DEVICES;
```

第一个任务是使用 cudaGetDeviceCount 调用确定有多少可用的 GPU。为了确保没有超出我们的计划，这个数字被减少为最大支持数目，它使用一个简单的 #define 宏。考虑到可能存在 4 个双芯 GPU 显卡，这里最大值使用 8 可能比选择 4 要更好。

```
// Run one memcpy and kernel on each device
for (int device_num=0;
     device_num < num_devices;
     device_num++)
{
 // Select the correct device
 CUDA_CALL(cudaSetDevice(device_num));
```

然后在每次循环的开始阶段，将当前设备上下文参数设置为device_num参数，以确保在该设备上所有随后的调用能够正常工作。

```
// Generate a prefix for all screen messages
struct cudaDeviceProp device_prop;
CUDA_CALL(cudaGetDeviceProperties(&device_prop,
                                  device_num));
sprintf(&device_prefix[device_num][0],"\nID:%d %s:",device_num,device_prop.name);

// Create a new stream on that device
CUDA_CALL(cudaStreamCreate(&stream[device_num]));

// Allocate memory on the GPU
CUDA_CALL(cudaMalloc((void**)&gpu_data[device_num],
                     single_gpu_chunk_size));

// Allocate page locked memory on the CPU
CUDA_CALL(cudaMallocHost((void **)
                         &cpu_src_data[device_num],
                         single_gpu_chunk_size));

CUDA_CALL(cudaMallocHost((void **)
                         &cpu_dest_data[device_num],
                         single_gpu_chunk_size));

// Fill it with a known pattern
fill_array(cpu_src_data[device_num], NUM_ELEM);


// Copy a chunk of data from the CPU to the GPU
// asynchronous
CUDA_CALL(cudaMemcpyAsync(gpu_data[device_num],
          cpu_src_data[device_num],
          single_gpu_chunk_size,
          cudaMemcpyHostToDevice,
          stream[device_num]));

// Invoke the GPU kernel using the newly created
// stream - asynchronous invokation
gpu_test_kernel<<<num_blocks,
                  num_threads,
                  shared_memory_usage,
  stream[device_num]>>>(gpu_data[device_num]);

cuda_error_check(device_prefix[device_num],
                 "Failed to invoke gpu_test_kernel");

// Now push memory copies to the host into
// the streams
// Copy a chunk of data from the GPU to the CPU
```

```
    // asynchronous
    CUDA_CALL(cudaMemcpyAsync(cpu_dest_data[device_num],
                              gpu_data[device_num],
                              single_gpu_chunk_size,
                              cudaMemcpyDeviceToHost,
                              stream[device_num]));
}
```

我们为系统中的每个 GPU 创建了一个流，或者工作队列。在这个流中，我们将从主机（CPU）内存复制到 GPU 全局内存的操作和紧接着的内核调用还有数据传回 CPU 的操作放入到流中。它们将按这个顺序执行，这样内核只有在前面的内存复制操作完成后才会执行。

请注意主机上锁页内存的使用，使用 cudaMallocHost 分配内存，而不要使用常规的 C 语言 malloc 函数。锁页内存是不能被交换到磁盘的。由于内存复制操作通过在 PCI-E 总线上的直接内存访问（DMA）实现，因此 CPU 端的内存一定总是存在于物理内存。用 malloc 分配的内存可以被交换到磁盘上，如果 DMA 试图对其进行操作，那么就会导致失败。由于我们用 cudaMallocHost 函数分配内存，因此我们还必须使用 cudaFreeHost 释放内存。

```
// Process the data as it comes back from the GPUs
// Overlaps CPU execution with GPU execution
for (int device_num=0;
  device_num < num_devices;
  device_num++)
{
 // Select the correct device
 CUDA_CALL(cudaSetDevice(device_num));
// Wait for all commands in the stream to complete
CUDA_CALL(cudaStreamSynchronize(stream[device_num]));
```

最终，一旦内核流被填满了，它就会等待 GPU 内核完成。在这个时刻 GPU 可能还没有启动，因为我们所做的仅仅是将命令加入到流或是命令队列。

```
  // GPU data and stream are now used, so
  // clear them up
  CUDA_CALL(cudaStreamDestroy(stream[device_num]));
  CUDA_CALL(cudaFree(gpu_data[device_num]));

  // Data has now arrived in
  // cpu_dest_data[device_num]
  check_array( device_prefix[device_num],
               cpu_dest_data[device_num],
               NUM_ELEM);

  // Clean up CPU allocations
  CUDA_CALL(cudaFreeHost(cpu_src_data[device_num]));
  CUDA_CALL(cudaFreeHost(cpu_dest_data[device_num]));

  // Release the device context
  CUDA_CALL(cudaDeviceReset());
 }
}
```

之后，CPU 会等待每个设备轮流完成任务。当全部完成时，它会检查内容，然后释放与每个流相关的 GPU 和 CPU 资源。然而，如果系统中的 GPU 各不相同并且执行内核花费的时间也不同会发生什么呢？首先，我们需要添加一些计时代码，以查看每个内核实际花费了多少时间。为了实现这一点，我们需要将事件添加到工作队列中。现在这些事件是很特殊的，因为我们可以查询事件而无须考虑当前选择的 GPU。要做到这一点，我们需要声明一个启动和停止事件：

```
// Define a start and stop event per stream
cudaEvent_t kernel_start_event[MAX_NUM_DEVICES];
cudaEvent_t memcpy_to_start_event[MAX_NUM_DEVICES];
cudaEvent_t memcpy_from_start_event[MAX_NUM_DEVICES];
cudaEvent_t memcpy_from_stop_event[MAX_NUM_DEVICES];
```

然后，需要将它们加入到流或工作队列：

```
// Push the start event into the stream
CUDA_CALL(cudaEventRecord(memcpy_to_start_event[device_num], stream[device_num]));
```

我们在执行从内存复制到设备、内核调用前、将内存复制回主机前以及内存复制操作结束后分别压入一个启动事件。这允许我们看到 GPU 操作的每个阶段。

最后，我们需要获得花费的时间，并打印到屏幕上：

```
// Wait for all commands in the stream to complete
CUDA_CALL(cudaStreamSynchronize(stream[device_num]));

// Get the elapsed time between the copy
// and kernel start
CUDA_CALL(cudaEventElapsedTime(&time_copy_to_ms,
 memcpy_to_start_event[device_num],
 kernel_start_event[device_num]));

// Get the elapsed time between the kernel start
// and copy back start
CUDA_CALL(cudaEventElapsedTime(&time_kernel_ms,
 kernel_start_event[device_num],
 memcpy_from_start_event[device_num]));

// Get the elapsed time between the copy back start
// and copy back start
CUDA_CALL(cudaEventElapsedTime(&time_copy_from_ms,
 memcpy_from_start_event[device_num],
 memcpy_from_stop_event[device_num]));

// Get the elapsed time between the overall start
// and stop events
CUDA_CALL(cudaEventElapsedTime(&time_exec_ms,
 memcpy_to_start_event[device_num],
 memcpy_from_stop_event[device_num]));
```

```
// Print the elapsed time
const float gpu_time = (time_copy_to_ms + time_kernel_ms + time_copy_from_ms);

printf("%sCopy To  : %.2f ms",
       device_prefix[device_num], time_copy_to_ms);

printf("%sKernel   : %.2f ms",
       device_prefix[device_num], time_kernel_ms);

printf("%sCopy Back  : %.2f ms",
       device_prefix[device_num], time_copy_from_ms);

printf("%sComponent Time : %.2f ms",
       device_prefix[device_num], gpu_time);

printf("%sExecution Time : %.2f ms",
       device_prefix[device_num], time_exec_ms);

printf("\n");
```

我们还需要重新定义内核以使它能做更多的工作，这样我们实际上就可以看到一些合理的内核的执行时间：

```
__global__ void gpu_test_kernel(u32 * data, const u32 iter)

{
 const int tid = (blockIdx.x * blockDim.x)
     + threadIdx.x;

 for (u32 i=0; i<iter; i++)
 {
  data[tid] *= 2;
  data[tid] /= 2;
 }
}
```

当运行程序时，我们看到以下结果：

```
ID:0 GeForce GTX 470:Copy To        :   20.22 ms
ID:0 GeForce GTX 470:Kernel         : 4883.55 ms
ID:0 GeForce GTX 470:Copy Back      :   10.01 ms
ID:0 GeForce GTX 470:Component Time : 4913.78 ms
ID:0 GeForce GTX 470:Execution Time : 4913.78 ms
ID:0 GeForce GTX 470:Array check passed

ID:1 GeForce 9800 GT:Copy To        :    20.77 ms
ID:1 GeForce 9800 GT:Kernel         : 25279.57 ms
ID:1 GeForce 9800 GT:Copy Back      :    10.02 ms
ID:1 GeForce 9800 GT:Component Time : 25310.37 ms
ID:1 GeForce 9800 GT:Execution Time : 25310.37 ms
ID:1 GeForce 9800 GT:Array check passed
```

```
ID:2 GeForce GTX 260:Copy To        :    20.88 ms
ID:2 GeForce GTX 260:Kernel         : 14268.92 ms
ID:2 GeForce GTX 260:Copy Back      :    10.00 ms
ID:2 GeForce GTX 260:Component Time : 14299.80 ms
ID:2 GeForce GTX 260:Execution Time : 14299.80 ms
ID:2 GeForce GTX 260:Array check passed

ID:3 GeForce GTX 460:Copy To        :   20.11 ms
ID:3 GeForce GTX 460:Kernel         : 6652.78 ms
ID:3 GeForce GTX 460:Copy Back      :    9.94 ms
ID:3 GeForce GTX 460:Component Time : 6682.83 ms
ID:3 GeForce GTX 460:Execution Time : 6682.83 ms
ID:3 GeForce GTX 460:Array check passed
```

根据结果可以看到，每个内存复制操作的时间差距很小。这并不奇怪，因为每个设备运行在 x8 PCI-E 2.0 链路上。PCI 链路速度甚至比最慢的设备内存的速度还要慢得多，所以我们的传输速度实际上被 PCI-E 总线的速度限制了。

然而，有趣的是内核执行速度却有着相当显著的不同，大约从 5 秒 ~ 25 秒。因此，如果我们轮流为每个设备提供数据，那么这个周期大概会花费大约 51 秒（5 秒 +25 秒 +14 秒 +7 秒）的时间。然而，在程序等待最慢设备 1（9800GT）的时候，设备 2（GTX260）和 3（GTX460）已经完成。在这段时间它们应该处理更多的任务。

我们可以通过查询结束事件来解决这个问题，而不是简单地等待结束事件。也就是说，我们检查内核是否完成，如果没有就转移到下一个设备，随后再回到较慢设备。这可以通过下面的函数实现：

```
cudaError_t cudaEventQuery (cudaEvent_t event);
```

这个函数接收并判断某一特定的事件。如果该事件已经发生就返回 cudaSuccess，如果事件尚未发生，则返回 cudaErrorNotReady。请注意，这意味着我们不能使用 CUDA_CALL 宏，因为 cudaErrorNotReady 的状态不是一个真正的错误状态，它只是状态信息。

我们还需要通过下面的调用说明 CUDA 如何跟踪处于等待状态的 GPU 任务：

```
// Give back control to CPU thread
CUDA_CALL(cudaSetDeviceFlags(cudaDeviceScheduleYield));
```

该调用在所有其他 CUDA 调用之前完成，并且告诉驱动程序，在任何情况下等待操作的时候，该 CPU 线程应该给其他 CPU 线程让步。这可能意味着额外的延迟，因为驱动程序要求此 CPU 线程在 CPU 工作队列中等待它的调度机会，但允许其他 CPU 任务继续进行。另一种方法是，驱动程序只围绕该 CPU 线程进行检测（轮询设备），当有其他设备就绪时，这肯定不是我们所期待的。

为了避免对事件队列进行轮询从而导致程序在涉及其他 CPU 任务时表现不佳，程序需要把自己休眠，过一段时间唤醒并且再次检查事件队列。在 Linux 和 Windows 中这个过程的执行略有不同，因此我们使用自定义的函数——snooze，它在这两个平台上都可以运行。

```
// Give up control of CPU threads for some milliseconds
void snooze(const unsigned int ms)
{
#ifdef _WIN32
 Sleep(ms);
#else
 if ((ms/1000) <= 0)
  sleep(1);
 else
  sleep(ms/1000);
#endif
}
```

最后，我们将重新排列数据的处理顺序，以移除 cudaStreamSynchronize 调用，并且把此代码放置到一个函数中。我们也将删除清理代码，并把代码放到主循环外。这种特定的操作是很重要的，因为在循环内这样做（这取决于函数本身），会导致串行的驱动程序调用。因此，修改后的关于查询事件队列的代码如下所示。

```
printf("\nWaiting");

u32 results_to_process = num_devices;
u32 sleep_count = 0;

// While there are results still to process
while(results_to_process != 0)
{
 // Process the data as it comes back from the GPUs
 // Overlaps CPU execution with GPU execution
 for (int device_num=0;
      device_num < num_devices;
      device_num++)
 {
  // If results are pending from this device
  if (processed_result[device_num] == false)
  {
   // Try to process the data from the device
   processed_result[device_num] =
        process_result(device_num);

   // If we were able to process the data
   if (processed_result[device_num] == true)
   {
    // Then decrease the number of pending
    // results
    results_to_process--;

    // print the time host waited
    printf("%sHost wait time : %u ms\n",
           device_prefix[device_num],
           sleep_count * 100);
```

```
 // If there are still more to process
 // print start of progress indicator
 if (results_to_process != 0)
  printf("\nWaiting");

 fflush(stdout);
}
else
{
 printf(".");
 fflush(stdout);
}
  }

  // Try again in 100ms
  sleep_count++;
  snooze(100);
 }
}

for (int device_num=0;
     device_num < num_devices;
     device_num++)
{
 cleanup(device_num);
}
```

while循环仅仅运行到每个设备都提供结果。我们建立了一个数组processed_results[num_devices]，它被初始化为false。随着每个GPU提供结果，未定的结果数量越来越少并且对已给出结果设备的对应数组元素进行标记，以说明这个GPU已经提供了结果。当GPU结果还不可用的时候，CPU线程睡眠100ms，然后再次尝试。这样就出现了下面的输出：

```
Waiting.....................................................
ID:0 GeForce GTX 470:Copy To   : 20.84 ms
ID:0 GeForce GTX 470:Kernel          : 4883.16 ms
ID:0 GeForce GTX 470:Copy Back       :   10.24 ms
ID:0 GeForce GTX 470:Component Time : 4914.24 ms
ID:0 GeForce GTX 470:Execution Time : 4914.24 ms
ID:0 GeForce GTX 470:Array check passed
ID:0 GeForce GTX 470:Host wait time : 5200 ms

Waiting.....
ID:3 GeForce GTX 460:Copy To         :   20.58 ms
ID:3 GeForce GTX 460:Kernel          : 6937.48 ms
ID:3 GeForce GTX 460:Copy Back       :   10.21 ms
ID:3 GeForce GTX 460:Component Time : 6968.27 ms
ID:3 GeForce GTX 460:Execution Time : 6968.27 ms
ID:3 GeForce GTX 460:Array check passed
ID:3 GeForce GTX 460:Host wait time : 7100 ms
```

```
Waiting.....................................
ID:2 GeForce GTX 260:Copy To         :    21.43 ms
ID:2 GeForce GTX 260:Kernel          : 14269.09 ms
ID:2 GeForce GTX 260:Copy Back       :    10.03 ms
ID:2 GeForce GTX 260:Component Time : 14300.55 ms
ID:2 GeForce GTX 260:Execution Time : 14300.55 ms
ID:2 GeForce GTX 260:Array check passed
ID:2 GeForce GTX 260:Host wait time : 14600 ms

Waiting.....................
ID:1 GeForce 9800 GT:Copy To         :    21.19 ms
ID:1 GeForce 9800 GT:Kernel          : 25275.88 ms
ID:1 GeForce 9800 GT:Copy Back       :    11.01 ms
ID:1 GeForce 9800 GT:Component Time : 25308.08 ms
ID:1 GeForce 9800 GT:Execution Time : 25308.08 ms
ID:1 GeForce 9800 GT:Array check passed
ID:1 GeForce 9800 GT:Host wait time : 25300 ms
```

注意结果出现的顺序是怎样与预期相吻合的。最快的设备 GTX470，只需 5 秒，而最慢的 9800 GT 需要 25 秒。在大多数情况下，CPU 线程在此期间处于闲置状态，它本来可以做一些有用的事情，比如当 GPU 完成任务时向它们分发更多的任务。让我们来看看在实际工作中是如何做到这一点的。

首先，我们需要将任务加入流或工作队列中的过程抽象出来。我们可以使用这一抽象进行初始的流填充，当工作完成时再次填充流。

```
__host__ void get_and_push_work(const int num_devices,
                                const size_t single_gpu_chunk_size,
                                const u32 new_work_blocks)
{
 // Work out the total number to process
 // Number already scheduled plus new work
 u32 results_to_process = num_devices +
                          new_work_blocks;

 // Keep track of the number of calculations in flow
 u32 results_being_calculated = num_devices;

 // Keep track of how long the CPU needs to sleep
 u32 sleep_count = 0;

 // While there are results still to process
 while(results_to_process != 0)
 {
  // Process the data as it comes back from the GPUs
  // Overlaps CPU execution with GPU execution
  for (int device_num=0;
       device_num < num_devices;
       device_num++)
  {
```

```
// Assume will process nothing
bool processed_a_result = false;

// If results are pending from this device
if (processed_result[device_num] == false)
{
 // Try to process the data from the device
 processed_result[device_num] =
     process_result(device_num);
// If we were able to process the data
if (processed_result[device_num] == true)
{
 // Then decrease the number of pending
 // results
 results_to_process--;

 // Increment the number this device
 // processed
 num_processed[device_num]++;

 // Decreate the number in flow
 results_being_calculated--;

 // Note we processed at least
 // one result
 processed_a_result = true;

 // print the time host waited
 printf("%sHost wait time : %u ms\n",
        device_prefix[device_num],
        sleep_count * 100);

 // If there are still more blocks
 // to process
 if (results_to_process >
     results_being_calculated)
 {
  // Give more work to the
  // finished GPU
  push_work_into_queue(device_num,
                       single_gpu_chunk_size);

  // Set flag to say GPU has work
  processed_result[device_num] =
          false;

  // Increment the number of
  // active tasks
  results_being_calculated++;

  // Format output
  printf("\n");
```

```
    fflush(stdout);
  }
 }

    // If we processed no results then sleep
    if (processed_a_result == false)
    {
     sleep_count++;
     printf(".");
     fflush(stdout);

     // Try again in 100ms
     snooze(100);
    }
   }
  }
 }
```

在这里，程序简单地跟踪活动 GPU 任务的数量，并且计算仍需要处理结果的数量。当我们分配 64 个工作单元时，这将导致如图 8-3 所示的 GPU 工作模块的分配。

正如你从该柱状图 8-3 所看到的，GTX470 可以在同一时间处理多于 25 个工作单元，相比之下，9800 GT 只能够处理多于 5 个的单元，其比例为 5 : 1。因为同步操作仅循环并等待流，因此当存在混合的 GPU 时，它会导致完全相等的工作分发，就和你在现实世界的系统中看到的一样。很多玩家都会为游戏准备一个显卡（GTX670），然后通常有一个比较旧的显卡专门用于 PhysX（GTX260），我们只是给出这样一个场景。事实上，如果多个次显卡总共贡献了 37 个工作单元，那么仅主显卡贡献 27 以外的 10 个工作单元。这反过来又给机器带来了两倍以上的可用工作吞吐量。

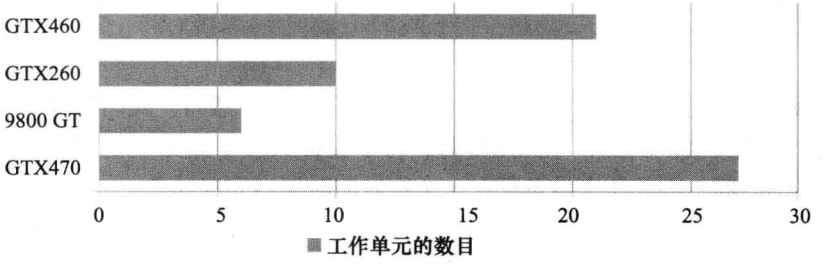

图 8-3　工作单元在多 GPU 上的分配

这一切都很不错，但实际上我们可以做得更好。我们实际上没有完全利用系统中的每个 GPU。多流既提供了流 0（默认流）之外的其他选择，又为 GPU 提供多个可以使用的工作队列。当内核太小而无法完全利用 GPU 时，这是非常有用的，但这并不常见。更常见的情况是，CPU 可能需要一段时间为 GPU 提供额外工作。在我们这个例子中，CPU 仅仅是根据一组预期值检查数组，但它可以做一个慢得多的操作，例如从磁盘中装载下一个工作单元。在这种情况下，我们希望在此期间 GPU 也保持忙碌。为此，我们使用一个名为双缓冲

的策略。

双缓冲通过让 CPU 与 GPU 工作在不同缓冲区而实现。因此，即使 CPU 正在处理一个数据集，而 GPU 仍然可以执行有用的工作，而不是等待 CPU。CPU 执行的操作可能是一些简单的装载或保存数据到磁盘上的操作。它可能还包括一些额外的对来自多 GPU 的数据的处理或组合。

要做到这一点，我们需要为每一个基于 MAX_NUM_DEVICES 的数组引入另一个维度。例如：

```
// Define N streams per GPU
cudaStream_t stream[MAX_NUM_DEVICES][MAX_NUM_STREAMS];
```

然后，我们需要选择每个设备支持两个或更多的数据流。每个设备使用两个流则会有一个小问题，如果我们以相同优先级将工作单元分配给 GPU，那么它们都得到总个数相同的工作单元。在实际中，这意味着我们到达工作队列最后时仍然在等待最慢的设备。这个问题的解决方案是为 GPU 分配的工作单元个数与它们的速度成比例。

如果回顾一下图 8-3，可以看到，GT9800 是最慢的设备。GTX260 的速度大约是它的 2 倍左右，GTX460 速度又是 GTX260 的两倍，GTX470 又比 GTX460 快 20%左右。考虑到我们希望有至少两个流以实现双缓冲，如果按照与设备的速度成比例的方式分配流的数目，这样我们得到使所有设备在同样时间里保持忙碌的工作分配方案。我们可以通过一个简单的数组做到这一点：

```
// Define the number of active streams per device
const u32 streams_per_device[MAX_NUM_DEVICES] =
{
 10, /* GTX470 */
 2, /* 9800 GT */
 4, /* GTX260 */
 8, /* 460 GTX */
};
```

因此，我们最初分配设备 0（GTX470）10 个工作单元。然而，我们只分配给 GT9800 两个工作单元，依此类推。当工作单元用尽时，每个设备队列的长度约为数组所示的值。这等价于大约相同的时间，因此所有设备都在一个很短的时期内完成任务。

这里，各种 GPU 相对速度的列表是静态构造的。如果你总是在目标机器中使用相同的硬件，那么这种方法是很好的。但是，如果你不知道目标硬件是什么，那么你可以做一些初始计时测试，然后在运行时完成这样的表。要记住的重要一点是为了实现双缓冲，列表中的最小值应至少为 2。其他的值应该是 2 的倍数，它反映了对于最慢设备的相对时间。

通过前面的例子我们可以看出（每个 GPU 只使用一个单独的数据流）GPU 的负载是变化的。有时它甚至会下降为 25%或更少。实际上，我们看到的是 GPU 工作量的失速。如果在没有进一步的 CPU 干预的情况下，为 GPU 提供多个任务进行处理，会将 GPU 的负载持续增加到 100%且充分利用所有的设备。这也有利于减少 GPU 内核对其他任务的 CPU 负载的敏感度，因为在需要 CPU 再次服务之前，内核将大量的工作交给每个 GPU 进行处理。

事实上，如果你运行单流内核与多流内核进行比较，我们看到结果从151秒降到139秒，在执行时间上减少了8%。CPU端的任务很少，因此它能够相对迅速地填充单个入口队列。然而，若是一个更复杂的CPU任务，CPU和GPU时间的重叠变得更加重要，你会看到这个8%的数值会非常显著地增长。

随着我们在程序中加入额外的复杂性，它的开发时间会增加并且会引入更多的错误。对于大多数程序，每个设备至少使用两个流能够有助于提高整体吞吐量，这样我们所做的额外的努力就是值得的。

8.9 多节点系统

单台计算机形成网络上的一个节点，将许多机器连接起来就得到了一个机器集群。通常，这种集群将由一套机架式节点组成，这个机架本身可以与一个或多个其他的机架相互连接。

截止到2012年，世界上最大的GPU系统——天河1A，包含超过14 000个CPU和超过7000个Tesla费米架构GPU。它们分成112个机柜（机架），其中每个包含64个计算节点，它们之间使用一个自制的、支持多达160GB/s通信带宽的互连结构。

目前，实际上大多数研究者和商业组织无法接触到这种规模的系统。然而，他们通常可以购买联网于16 ~ 48端口千兆以太网交换机上的节点，这通常是放置在一个空调机房里的19英寸机架单元。

理想的CPU核与GPU的比率取决于应用程序和代码串行部分的百分比。如果串行部分很少，那么简单的单CPU核和多GPU就足够了，并不用麻烦地进行任何额外的编程。然而，如果CPU负载很大，那么它很可能限制吞吐量。为了克服这个问题，我们需要为每个CPU核分配更少的GPU，变成应用程序需要的1∶2或1∶1的比例，最简单、最可扩展的方法是为节点上的每一组CPU/GPU分配一个进程。

一旦我们使用了这个模型，它将允许更大规模的扩展。因为我们可以有两个节点，其中每个节点包含4个GPU和一个或多个CPU核。如果问题可以被进一步分解成8个块而不是4个，那么其性能理论上会翻倍。事实上，正如我们之前所见到的，由于通信开销的存在，这通常是不会发生的。随着节点数的增长，网络通信对问题的影响也会增加。因此，你通常会发现每个节点具有更多数量GPU的网络，其性能会超过虽有同样数量的GPU但是分布在更多节点上的网络。本地节点上的资源（磁盘、内存、CPU）会对给定问题的最佳拓扑产生重大的影响。

为了转移到这样的系统上，我们需要一种通信机制以允许将工作调度到给定的CPU/GPU集合上，而不用关心它们处于网络的什么地方。为了达到这一目的，我们将使用ZeroMQ，它是一个非常轻量级的和相当容易使用的通信函数库。我们当然也可以使用套接字函数库，但这可能更加低级一些，而且大多数情况下不容易正确地编程。我们也可以使用MPI，它是一个Linux平台上相当标准的协议定义，但一般需要进行一些设置并且更加适用

于受控环境。ZeroMQ 可以很好地处理错误，允许节点消失和再现且不会使整个程序陷入混乱之中。

ZeroMQ(或 0MQ) 是一个小型、轻量级的函数库，你只需简单地与其连接即可。它没有编译器、封装器或其他类似的东西，仅仅是一个简单的函数库。一旦被初始化了，ZeroMQ 便会在后台运行，并且允许应用程序使用同步或异步通信而不必担心缓冲区的管理。如果你想发送一个 10MB 的文件到另一个节点，那么直接发送它，ZeroMQ 将在内部处理任何缓冲。它是编写分布式应用程序的一个很好的接口。从 http://www.zeromq.org/ 我们可以免费获取它。

ZeroMQ 支持大量的线程间 (INPROC) 传输、进程间 (IPC) 传输、广播到多个节点 (MULTICAST) 和基于网络的系统 (TCP)。我们会使用后者，因为它有最好的灵活性，允许连接网络（或 Internet）上任何地方的多个节点。

使用 ZeroMQ 的第一个任务是创建一个连接点。我们将使用主 / 从范型（master/worker paradigm），如图 8-4 所示，其中有一个主机器（服务器) 将工作包分发给从机器 (客户端)，每个客户端机器连接到网络上的某一特定访问点（由服务器提供），然后等待指定给它的工作。注意，这里的客户端是一套 CPU/GPU，而不是一个物理节点，因此，有 4 个 GPU 且以 1 : 1 比例将 CPU 核映射到 GPU 设备的 4 核 CPU 代表 4 个客户端。同样的，以 1 : 4 比例将 CPU 核映射到 GPU 设备的 4 核 CPU 将显示为一个客户端。

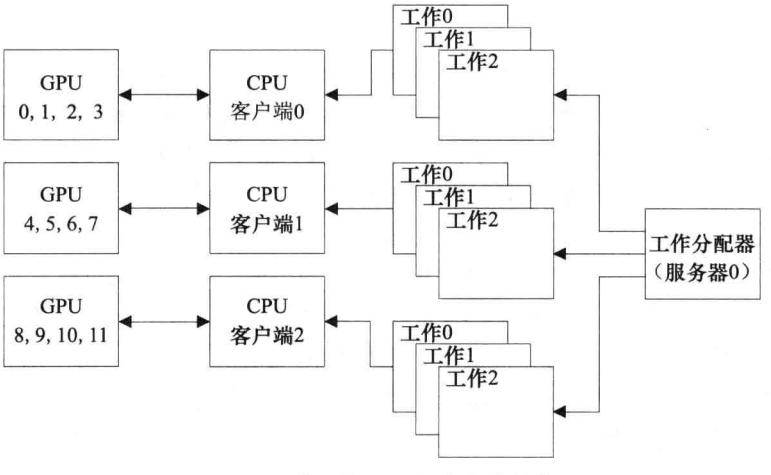

图 8-4 单服务器、多客户端结构

ZeroMQ 术语中，服务器将绑定一个端口，也就是说它将创建一个访问点。所有客户将连接到已知的访问点。至此还没有传输应用数据。然而在后台，ZeroMQ 将为每一个连接到端口的客户端建立一个内部队列。

下一步是确定一个消息传递模式，最简单的是请求 / 响应模式（request/reply pattern）。这类似于 MPI 中的 send 和 recv 函数，且对于每个发送，必须有一个响应。这是通过下列方

式实现的：

客户端：

```
zmq::context_t context(1);
zmq::socket_t socket(context, ZMQ_REQ);
socket.connect("tcp://localhost:5555");
```

服务器：

```
zmq::context_t context(1);
zmq::socket_t socket(context, ZMQ_REP);
socket.bind("tcp://*:5555");
```

然后 CPU 客户端维护一个工作队列，通常至少有两项，以允许 GPU 双缓冲，此外至少一个入队和一个出队的网络消息。

应用程序中使用的协议是，CPU 客户端连接到服务器并向服务器请求一批工作，服务器返回包含需要客户端处理的范围的响应，然后客户端在 CPU 上执行任何必要的工作来为工作包生成数据。例如，这可能是对一个给定模型的值生成所有可能的组合，以对某些预测进行测试。

```
// Host program to be called from main
__host__ void gpu_kernel_client(const u32 pid)
{
 printf("\nRunning as Client");

 // Init Network
 zmq::context_t context(1);
 zmq::socket_t socket(context, ZMQ_REQ);
 socket.connect("tcp://localhost:5555");

 // GPU params
 size_t chunk_size;
 u32 active_streams;
 u32 num_devices;
// Setup all available devices
setup_devices(&num_devices,
              &active_streams,
              &chunk_size);

u32 results_to_process;
get_work_range_from_server(pid,
                           &results_to_process,
                           &socket);

// Generate CPU data for input data
generate_cpu_data_range(0, results_to_process);

// Keep track of pending results
u32 pending_results = results_to_process;
```

```
// While there is still work to be completed
while (pending_results != 0)
{
 // Try to distribute work to each GPU
 u32 work_distributed = distribute_work(num_devices,
                                        chunk_size,
                                        pending_results);

 // Collect work from GPU
 u32 work_collected = collect_work(num_devices,
                                   chunk_size);

 // Decrement remaining count
 pending_results -= work_collected;

 // Post completed work units to server
 if (work_collected > 0)
 {
  send_completed_units_to_server(pid,
                                 chunk_size,
                                 &socket);
 }

 // If no work was distributed, or collected
 // and we've not finished yet then sleep
 if ( (work_distributed == 0) &&
      (work_collected == 0) &&
      (pending_results != 0) )
 {
  printf(".");
  fflush(stdout);
   snooze(100);
  }
 }

 // Print summary of how many each device processed
 for (u32 device_num=0u;
      device_num < num_devices;
      device_num++)
 {
  printf("%s processed: %u",
         device_prefix[device_num],
         num_processed[device_num]);
 }

 printf("\nTotal: src:%u dest:%u",
        unprocessed_idx, completed_idx);

 cleanup_devices(num_devices);
}
```

在收到来自服务器的初始化工作并生成GPU的工作队列之后，客户端代码循环运行，直到工作完成。该循环会为可用的GPU分配工作、接手已经完成的工作并且将完成的工作提交给服务器。最后，如果它没有上述任务处理时，会睡眠100 ms后继续尝试。在程序退出时，我们打印出诸如每个设备处理了多少个工作单元等总结信息。

请注意这里的调度不同于前面的示例。我们现在需要一些额外的缓冲空间以向服务器提交完成的工作单元，并需要一段时间将数据推送到传输队列。因此，我们不再立刻将工作重新调度到GPU上而是稍后再调度额外的工作。这允许我们使用更简单的途径分发任务：即收集任何完成的任务，如果需要的话在本地进行加工处理，然后将其提交给服务器。

```
__host__ u32 distribute_work(const int num_devices,
                             const size_t chunk_size,
                             u32 pending_results)
{
 u32 work_units_scheduled = 0;

 // Cycle through each device
 for (u32 device_num = 0;
      device_num < num_devices;
      device_num++)
 {
  u32 stream_num = 0;
  bool allocated_work = false;

  while ( (allocated_work == false) &&
          (stream_num < streams_per_device[device_num]) )
  {
   // If there is more work to schedule
   if (pending_results > 0)
   {
    // If the device is available
    if (processed_result[device_num][stream_num] == true)
    {
     // Allocate a job to the GPU
     push_work_into_queue(device_num,
                          chunk_size,
                          stream_num);

     // Set flag to say GPU has work pending
processed_result[device_num][stream_num] = false;

     // Keep track of how many new
     // units were issued
     work_units_scheduled++;

     // Move onto next device
     allocated_work = true;

     pending_results--;
    }
```

```
    }
    stream_num++;
   }
  }

  return work_units_scheduled;
}
```

这里我们对 processed_results 数组进行逐一遍历，观察任何流中是否存在前一个周期已处理过而且现在又再次使用的元素。然后我们分配未完成的工作来把每一个 GPU 设备的工作单元分配到可用的流槽中。

```
__host__ void push_work_into_queue(const u32 device_num,
                                   const size_t chunk_size,
                                   const u32 stream_num)
{
 // No dynamic allocation of shared memory required
 const int shared_memory_usage = 0;

 // Define the number of threads and blocks to launch
 const int num_threads = 256;
 const int num_blocks = ((NUM_ELEM + (num_threads-1))
                         / num_threads);

 // Copy in the source data form the host queue
 memcpy(cpu_src_data[device_num][stream_num],
        cpu_unprocessed_data[unprocessed_idx % MAX_IN_QUEUED_PACKETS],
        chunk_size);

 // Processed this packet
 unprocessed_idx++;

 // Select the correct device
 CUDA_CALL(cudaSetDevice(device_num));

 // Push the start event into the stream
 CUDA_CALL(cudaEventRecord(memcpy_to_start_event[device_num][stream_num], stream
[device_num][stream_num]));

 // Copy a chunk of data from the CPU to the GPU
 // asynchronous
 CUDA_CALL(cudaMemcpyAsync(gpu_data[device_num][stream_num],
cpu_src_data[device_num][stream_num], chunk_size, cudaMemcpyHostToDevice, stream
[device_num][stream_num]));

 // Push the start event into the stream
CUDA_CALL(cudaEventRecord(kernel_start_event[device_num][stream_num], stream
[device_num][stream_num]));
```

```
 // Invoke the GPU kernel using the newly created
 // stream - asynchronous invokation
 gpu_test_kernel<<<num_blocks,
                   num_threads,
                   shared_memory_usage,
                   stream[device_num][stream_num]>>>
                   (gpu_data[device_num][stream_num],
                   kernel_iter);

 cuda_error_check(device_prefix[device_num],
                  "Failed to invoke gpu_test_kernel");

 // Push the start event into the stream
 CUDA_CALL(cudaEventRecord(memcpy_from_start_event[device_num][stream_num], stream
[device_num][stream_num]));

 // Copy a chunk of data from the GPU to the CPU
 // asynchronous
 CUDA_CALL(cudaMemcpyAsync(cpu_dest_data[device_num][stream_num],gpu_data
   [device_num]
[stream_num], single_gpu_chunk_size, cudaMemcpyDeviceToHost, stream[device_num]
[stream_num]));

 // Push the stop event into the stream
CUDA_CALL(cudaEventRecord(memcpy_from_stop_event[device_num][stream_num], stream
[device_num][stream_num]));

}
```

push_work_into_stream 函数和以前的函数基本相似。然而，它现在接收 stream_num 参数，这允许我们在流中填入任何可用的槽。它也可以通过 cpu_unprocessed_data 将数据复制到 CPU 内存中，其中 cpu_unprocessed_data 是 CPU 主机端一组常规的内存。注意，这不是 GPU 异步内存操作使用的页映射主机内存。CPU 主机需要处于自由状态以计算 / 更新此内存而无须担心运行中的内核对它进行同步。

```
__host__ u32 collect_work(const int num_devices,
       const size_t chunk_size)
{
 // Keep track of the number of results processed
 u32 results_processed = 0;

 // Cycle through each device
 for (u32 device_num=0;
   device_num < num_devices;
   device_num++)
 {
  // Then cycle through streams
  for(u32 stream_num=0;
      stream_num < streams_per_device[device_num];
```

```
        stream_num++)
    {
     // If results are pending from this device
     if (processed_result[device_num][stream_num] == false)
     {
      // Try to process the data from the device
      processed_result[device_num][stream_num]=process_result(device_num,
  stream_num, chunk_size);

      // If we were able to process the data
      if (processed_result[device_num][stream_num] == true)
      {
       // Increment the number this device
       // processed
       num_processed[device_num]++;

       // Increment this run's count
       results_processed++;
      }
     }
    }
   }
   return results_processed;
  }
```

collect结果函数简单地对每个设备及每个设备的每个流进行迭代，并且调用process_result函数来处理任何可用的结果。

```
  __host__ bool process_result(const u32 device_num,
                               const u32 stream_num,
                               const size_t chunk_size)
  {
   bool result;

   bool stop_event_hit = (cudaEventQuery(memcpy_from_stop_event[device_num]
  [stream_num])== cudaSuccess);

   // Space is avaiable if network_out_idx is not
   // more than the total queue length behind
   bool output_space_avail = ((completed_idx - network_out_idx) <
  MAX_OUT_QUEUED_PACKETS);

   // If the stop event has been hit AND
   // we have room in the output queue
   if (stop_event_hit && output_space_avail)
   {
    float time_copy_to_ms = 0.0F;
    float time_copy_from_ms = 0.0F;
    float time_kernel_ms = 0.0F;
    float time_exec_ms = 0.0F;
```

```
  // Select the correct device
  CUDA_CALL(cudaSetDevice(device_num));

  // Get the elapsed time between the copy
  // and kernel start
  CUDA_CALL(cudaEventElapsedTime(&time_copy_to_ms, memcpy_to_start_event
    [device_num]
[stream_num], kernel_start_event[device_num][stream_num]));

  // Get the elapsed time between the kernel start
  // and copy back start
  CUDA_CALL(cudaEventElapsedTime(&time_kernel_ms,
kernel_start_event[device_num][stream_num],
memcpy_from_start_event[device_num][stream_num]));

  // Get the elapsed time between the copy back start
  // and copy back start
  CUDA_CALL(cudaEventElapsedTime(&time_copy_from_ms,
memcpy_from_start_event[device_num][stream_num],
memcpy_from_stop_event[device_num][stream_num]));

  // Get the elapsed time between the overall start
  // and stop events
  CUDA_CALL(cudaEventElapsedTime(&time_exec_ms,
memcpy_to_start_event[device_num][stream_num],
memcpy_from_stop_event[device_num][stream_num]));

  // Print the elapsed time
  const float gpu_time = (time_copy_to_ms +
                          time_kernel_ms +
                          time_copy_from_ms);

  printf("%sCopy To   : %.2f ms",
         device_prefix[device_num], time_copy_to_ms);

  printf("%sKernel    : %.2f ms",
         device_prefix[device_num], time_kernel_ms);

  printf("%sCopy Back : %.2f ms",
         device_prefix[device_num],
         time_copy_from_ms);

  printf("%sComponent Time : %.2f ms",
         device_prefix[device_num], gpu_time);

  printf("%sExecution Time : %.2f ms",
         device_prefix[device_num], time_exec_ms);
  fflush(stdout);

  // Data has now arrived in
  // cpu_dest_data[device_num]
```

```
check_array( device_prefix[device_num],
             cpu_dest_data[device_num][stream_num],
             NUM_ELEM);

// Copy results into completed work queue
memcpy(cpu_completed_data[completed_idx % MAX_OUT_QUEUED_PACKETS],
  cpu_dest_data[device_num][stream_num],
  chunk_size);

printf("\nProcessed work unit: %u", completed_idx);
fflush(stdout);

// Incremenet the destination idx
// Single array per CPU
completed_idx++;
 result = true;
}
else
{
 result = false;
}

return result;
}
```

在 process_results 函数中，处理流有两个条件。第一个条件是该流已经被完成，换言之，我们在流上遇到了停止事件。另一个条件是用于传输的输出队列目前拥有空闲的槽。如果这两种情况都不出现，那么函数会简单地返回并且不执行任何操作。

否则，该函数收集一些计时信息并打印它。然后它将接收的数据复制到输出队列从而释放主机上的锁页内存并释放一个 GPU 流槽以供后续使用。

最后，我们看看将数据发送给服务器必须要做哪些事情。

```
__host__ void send_completed_units_to_server(
 const u32 pid,
 const size_t chunk_size,
 zmq::socket_t * socket)
{
 for (u32 packet=network_out_idx;
      packet < completed_idx;
      packet++)
 {
  // Define a client message
  CLIENT_MSG_T client_msg;
  client_msg.id.pid = pid;
  client_msg.id.ip = 0;
  client_msg.id.msg_type = 0;
  client_msg.id.msg_num = packet;
  memset(client_msg.data, 0, CLIENT_MSG_DATA_SIZE);
```

```
SERVER_MSG_T server_msg;
memset(&server_msg, 0, sizeof(SERVER_MSG_T) );

// Create object to send to server
zmq::message_t request(sizeof(CLIENT_MSG_T));
zmq::message_t reply;

// Copy in the output data
memcpy(client_msg.data,
       cpu_completed_data[packet % MAX_OUT_QUEUED_PACKETS],
       chunk_size);

// Copy the total message to ZEROMQ data area
 memcpy( (void*) request.data(), &client_msg, sizeof(CLIENT_MSG_T) );

 // Send to server
 printf("\nSending data %u to server", packet);
 socket->send(request);

 // Free output buffer
 network_out_idx++;

 // Wait for a reply
 socket->recv(&reply);

 // Decode the reply
 memcpy( &server_msg, (void*) reply.data(), sizeof(SERVER_MSG_T) );
 printf("\nReceived acknowledge from server");
 }
}
```

为了使用 ZeroMQ 发送一条消息，我们只需使用 zmq::message_t 构造函数来创建一个请求和应答消息。然后将相关元素以及一些头信息从 cpu_completed_data 数组复制到消息的有效载荷区域。头信息允许服务器知道谁是发送者。然后我们把消息提交给服务器并等待服务器的确认。

现在从调度和工作负载角度而言，我们对这种方法有一些警告。其中主要的问题在于网络加载和通信开销。我们在网络上发送数据的数量会对系统的性能产生巨大的影响。收到任意入站数据，在 CPU 上改变它并在 CPU 上再次将它发送出去的总体时间必须比 GPU 内核执行所用时间要少。否则，应用程序可能是 CPU 密集的或网络密集的。

在这个示例中，服务器发送给客户端一系列数据。我们假定客户端知道如何处理这些数据，可能是以生成一个工作所用的数据集，或从本地磁盘加载一些数据的方式。你需要尽可能避免的做法是简单地将数据本身发送给客户端。要利用节点上的本地资源，该资源可能是 CPU、主机内存、本地存储空间或任何可能的资源。

第二，输出数据被全部运送回服务器。这个问题可能是这样的：输出数据不是一个巨大的数据块，而仅仅是单个值，比如来自归约操作的。通常情况下输入空间是很大的。然而，

如果输入空间可以划分为 N 个本地磁盘，这时网络流量是非常小的，通过使用多个 GPU 节点你可以看到很大的扩展性。

8.10　本章小结

我们已经看到在计算机系统上使用多 GPU 的两个例子。在第 1 个例子中，一切都被包含在一个单独的机箱或节点里。第 2 个例子是在多节点下的使用，每个节点上包含多个 GPU。我们介绍了 ZeroMQ 的使用，因为它是一个更简单和更灵活的方法，可以替代传统的 MPI 方法。

我们使用流来实现双缓冲系统，这意味着 CPU 在准备下一个数据块和处理前一个数据块时，GPU 总是忙碌的。我们将流的使用从双流延伸到多流，以允许我们对同一节点中不同速度的 GPU 设备的工作进行平衡。

每个节点使用两个或 4 个 GPU，可能会使绑定 GPU 的应用程序的当前吞吐量变成两倍或 4 倍。为了进一步提升，你需要使用多个节点并估计将要在网络上通信的数据量。然而，就像天河系统 -1A 展示的那样，如果你的问题和预算允许，你可以扩展到成千上万的 GPU。

问题

1. 给定例子使用了同步网络通信，特别是基于发送 / 确认的协议。这种方法的优点和缺点是什么？我们还能通过其他方法实现吗，它带来的效益 / 成本是多少？

2. 当使用多个 GPU 系统时，使用线程相对使用进程的优点和缺点是什么？

3. 为了将第 2 个例子从 ZeroMQ 转换到 MPI，你必须考虑什么问题？

答案

1. 同步模型是最简单的、可用于工作和调试的模型。在主机内存和 GPU 之间存在同步和异步内存传输，同样的，我们可以在一个同步或异步的模型下进行操作以实现通信。如果内存是锁页的，那么网络控制器可以使用 DMA 模式访问它，这不会给 CPU 添加任何负载。它的优点是释放 CPU 去做其他的任务，但它增加了管理另一个异步设备的程序复杂性。

至于发送 / 确认方法，代价可能是很高的。你不会在一个小的局域网上看到它，因为服务器可能会超负荷并且需要很长时间才能回复，而客户端工作队列将会阻塞。增加每个设备上流的个数可能会有些帮助，但是服务器可以处理的客户端个数是有上限的。同时，等待确认消息也是有延迟的，这实际上不是我们想要的。服务器可能会简单地重新启动那些没有接收成功的工作单元。然后我们可以在客户端使用 post 方法，和异步通信结合在一起，可以让客户端继续工作，并将通信工作转移到通信栈。

2. 当线程之间有公用的数据时，使用多线程是最好的选择，这类似于在 SM 处理中使用共享内存。进程最好使用在通信更加标准时，例如，使用多个节点时 MPI 能够使扩展更加容易。

3. MPI 是为封闭系统设计的，因此对于客户端会退出、重启和重现的情况存在问题。通常 MPI 的实现有固定大小的及有限缓冲区。在一个消息中，塞进太多数据往往会使 MPI 栈崩溃。ZeroMQ 是隐式异步的，因为你的消息被复制到本地存储，然后由一个后台线程将其推送到网卡。只有当其内部缓冲达到最大值时它才会阻塞。MPI 的同步通信会立刻阻塞而其异步通信要求应用程序的数据保持到 MPI 将它完成。这意味着更少的数据复制，但却使 MPI 编程变得更为复杂。

从转换的角度来说，MPI_Init 调用取代了创建 ZeroMQ 上下文。创建并绑定到 ZeroMQ 中套接字的操作等价于 MPI_Comm_size(MPI_COMM_WORLD) 调用。你可以使用简单的 MPI_Comm_rank 调用获得整个系统的唯一 ID 而不是使用 PID 来识别一个消息 (在多个节点上你需要一个 IP 以及一个 PID)。ZeroMQ 的 send 和 recv 调用与 MPI 的 MPI_Send 和 MPI_Recv 调用非常相似。在 MPI 的实现中唯一需要做的额外工作是在函数结尾调用 MPI_Finalize，而对于 ZeroMQ 这是没有必要的。

对于 ZeroMQ 内置的存在一定风险的基于缓冲的异步通信，我们可以使用 MPI_Bsend 和适当的应用程序级缓冲管理来实现。

注意，截止到 SDK 4.0，CUDA 分配的锁页内存默认情况下是可以被其他设备（例如网卡）访问的。因此，我们可以让网卡和 GPU 使用相同的锁页内存，这样就消除了主机内存之间不必要的复制操作。

此外，在 Linux 系统或基于费米 Tesla 的 Windows 系统上，也可以从 GPU 直接将数据发送到网卡或在 GPU 之间发送而无须通过主机内存。这样大大避免了使用有限带宽的 PCI 总线或通过主机端。这不是我们这里要讨论的，因为目前并不是所有的平台都支持它。然而，在 SDK 样例中有一个对等通信的例子，我们会在第 10 章详细介绍，请希望使用这些功能的读者留意。

第 9 章

应用程序性能优化

本章详尽地剖析 CUDA 中限制性能的主要因素。每小节都特设一些例子，用于展示其中的问题。应该按序阅读它们。前面的章节已经介绍了 CUDA 和 GPU 程序设计的基本知识，本章假设你已经学习了前面章节并掌握了其中的概念，或者你已经有 CUDA 的大量经验，急于学习提高程序执行性能的技术。

本章针对程序的性能优化，分解成如下 7 个策略：

策略 1，理解问题，并正确分解为串行 / 并行的工作负载。

策略 2，理解并优化内存带宽、延迟和缓存使用问题。

策略 3，理解与主机端传输数据的玄机。考查锁页内存、零复制内存的表现和某些硬件的带宽限制。

策略 4，理解线程结构和计算能力，并了解它们对性能的影响方式。

策略 5，结合一些通用算法的优化实例，讨论如何实现算法。

策略 6，关注性能分析，定位应用程序的瓶颈所在及其来源。

策略 7，考察如何让应用程序根据各种硬件实现自我调优。

9.1 策略 1：并行 / 串行在 GPU/CPU 上的问题分解

9.1.1 分析问题

首要考虑的是，对问题尝试并行化是否是正确的解决方案。让我们看看这里涉及的一些问题。

9.1.2 时间

界定算法执行时间“可接受的”时间段是很重要的。目前可接受的并不一定意味着是现实可能的最好时间。当考虑到优化，作为专业软件开发人员的你必须认识到，你的时间就是金钱，而且如果你在西方国家工作，你的时间成本就更不便宜。需要程序执行得更快，那么

实现它就需要付出更多的努力（如图 9-1 所示）。

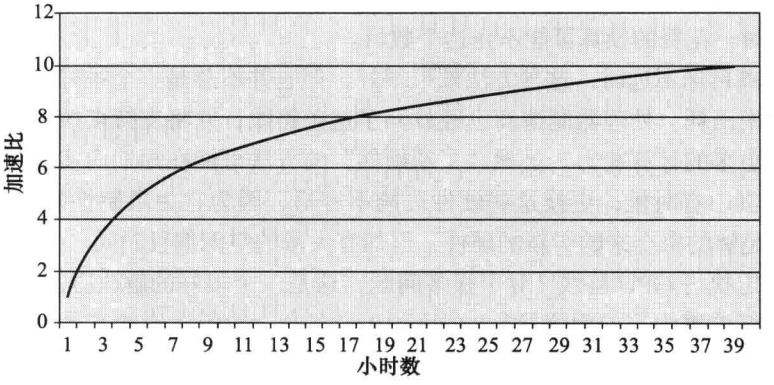

图 9-1 编码时间与加速比关系

你通常会发现在任何优化活动中都会有一定量所谓的“唾手可得的东西”。在优化中上述这些改变是很容易的且会带来一个合理的加速。当这些被处理掉，就逐渐变得更难找到优化之处，只有采用更复杂的重组才能进一步优化，这不仅花费更多的时间而且引入更多潜在错误。

在大多数西方国家，编程工作是相当昂贵的。即使你的编程时间是免费的（例如，如果你是一个工作于项目中的学生），优化代码的时间也可以用来做其他事情。作为工程师，我们有时会执着于把事情做到精益求精，甚至超出它应该达到的程度。所以我们要了解什么是需要的，并设置一个合适的目标。

在设置适当的加速目标时，在给定一组硬件的前提下，你必须知道什么是合理的。如果你需要在几秒钟内处理 20TB 的数据，那么单 GPU 机器是不能够达到要求的，当你考虑互联网搜索引擎，就会碰到这样的问题：它们必须在几秒钟内，搜索一组结果返回给用户。然而，与此同时，搜索引擎也必须用几天的时间去更新它们的索引，也就是利用这些时间去收集新的内容，在当时这认为是可接受的。而在现代世界，则认为这是缓慢的。因此，今天可以接受的事实，也有可能不被明天、下个月或明年接受。

在考虑可接受的时间段是多少时，问问你自己要达到该时间段还需付出多少努力。如果没有超过 2 倍的量级，往往就值得花时间来优化 CPU 的实现方式，而不是创建一个全新的、并行的方法来解决该问题。多线程引出了各种与依赖关系、死锁、同步、调试等有关的问题。如果你可以忍受串行 CPU 版本，那么这在短期内可能是一个更好的解决方案。

再来看下过去 30 年里解决问题的简单办法：只需买更快的硬件。使用分析技术来查看运行的应用程序在什么地方花费时间，从而决定哪里是瓶颈。是否有输入 / 输出（I/O）瓶颈、内存瓶颈或处理器瓶颈？若是 I/O 瓶颈，则可购买一个高速的 PCI-E RAID 卡并使用 SATA 3/SAS 的固态硬盘驱动器去解决该问题。如果遇到内存带宽问题，则可移植到一个具有高时钟速率内存的 LGA 2011 插槽系统。如果它只是受制于计算吞吐量，则可安装你可以买到的最

高时钟速率的至尊版和黑盒版处理器。例如，购买一个现成的、液冷版沙桥K或X系列超频处理器。这些解决方案的成本一般远低于3000 ~ 6000美元。在你花时间编写由串行到并行转变的程序时，花费的预算可能不止这个数目。

尽管当你离目标不远时，这种方法效果很好，但它并不总是一个很好的方法。高时钟速率带来了高功率消耗。处理器制造商已经放弃了这条路线，并将支持多核作为唯一的长期解决方案来提供更多的计算能力。虽然“买新硬件”的方法可能在短期内实用，但这不是一个长期的解决方案。有时候，更换这些硬件可能不容易，因为它由限制性的IT部门提供，或者因为你没有足够的资金来购买新的硬件，但却有大量的空闲编程时间。

如果你决定执行GPU路线（对于很多问题，这是一个很好的解决方案），那么通常应该将你的设计目标设置为当前程序执行时间的10倍左右。你所达到的实际量级取决于编程者的知识、可用的编程时间以及应用程序中的并行程度。最后一个因素具有巨大的决定性，我们下面就要谈到。即使对那些CUDA新手，至少加速2倍或3倍是一个相对容易的目标。

9.1.3 问题分解

这里的基本议题是：这个问题可以被分解成并行运行的组块吗？也就是，是否有机会发掘出问题中的并发性？如果答案是否定的，那么GPU就不是要考虑的方案。相反，你需要看看CPU优化技术，如缓存优化、内存优化、SIMD优化等。这些技术的某些方面已经结合GPU在之前的章节已涉及，本章将介绍其他的方面。其中的很多优化技术能很好地应用于串行CPU代码。

假设你能够把问题分为多个并发块，那接下来的问题是有多少个并发块？CPU并行化的一个主要限制因素经常是没有足够大粒度（或粗粒度）的并行工作要做。GPU运行成千上万的线程，所以问题需要被分解成上千个块，而不只是像CPU那样只执行少数并发任务。

问题分解应该总是先从数据开始，然后考虑执行的任务。你应该试图用输出数据集来表示问题。你能否构建一个公式，描述如何由对应的输入数据集中的数据转换为数据集的某个输出点的吗？你可能需要不止一个公式，例如，一个用于大多数数据点，一个用于处理问题空间边缘的数据点。如果你能做到这一点，那么把问题转换到GPU空间就相对容易了。

这类方法的一个问题在于，为了取得最好的效益，你需要完全理解问题。你不能简单地瞥一眼最高占用CPU的东西，然后把它们并行化。这一方法的真实益处在于把从输入数据点到输出数据点的链完全并行化。这里可能有部分链在你有硬件的情况下本应该使用100 000个处理器的，而这些数据点实际只使用了几百个处理器。很少有问题是真正单线程的。只是作为程序员、科学家和工程师，这是许多年前我们在大学中学过的一种解决方案。因此，看到问题潜在的并行性通常是第一个障碍。

现在有一些问题，不适合采用单输出数据点视图，例如，H264视频编码。在这个特殊的问题中，设置有许多阶段，而每一阶段定义了一个变长的输出数据流。也有些图像编码/处理的过程，特别是过滤，非常适合这一方法。这里的目标像素是N个源像素的函数。这一

类比在许多科学问题中都很适用。一个给定目标原子受力的量值可以由来自全部施力原子的力量之和表示。如果输入集合非常大，则可以简单地使用一个阈值或截止点，使得贡献小的那些输入数据点排除出数据集。这将会带来少量误差，但是在一些问题上，允许数据集中的大部分从计算中忽略。

优化通常用于执行数据的操作或函数。然而，随着计算能力相较内存带宽的飞速增长，数据是现在要首要考虑的因素。尽管事实上 GPU 有着 5 ~ 10 倍于 CPU 的内存带宽，你也必须分解问题才能利用这个带宽。这就是我们接下来将讨论的部分。

如果你打算使用多个 GPU 或多个 GPU 节点，这里最后一个需要考虑的是如何在处理器元素上分解问题和数据集。以计算周期的角度来看，节点之间的通信将是非常昂贵的，所以需要尽可能地将它最小化并跟计算重叠起来执行。这是我们稍后要涉及的内容。

9.1.4 依赖性

依赖就是一些计算需要用到以前计算的结果，可以是针对问题域的计算也可能是数组下标的计算。在这两种情况下，依赖都会引起并行执行的问题。

依赖关系主要有两种主要形式，要么一个元素是依赖于它附近的若干元素，要么在多轮遍历数据集时下一轮依赖于当前轮。

```
extern int a,c,d;
extern const int b;
extern const int e;

void some_func_with_dependencies(void)
{
 a = b * 100;
 c = b * 1000;
 d = (a + c) * e;
}
```

如果你考虑这个例子，可以看到，a 和 c 都依赖 b。你也可以看到 d 依赖于 a 和 c。a 和 c 的计算可以并行获得，但计算 d 时需要 a 和 c 都已计算完成。

在一个典型的超标量 CPU 中，有多个独立的流水线。a 和 c 的计算是独立的，可能会被分派到不同的执行单元去完成它们所需的乘法。然而，这些计算的结果需要在计算 a 和 c 的和之前得到。这个加法的结果也需要在最后的乘法之前得到。

鉴于一个指令的返回结果必须注入到下一个指令，这种类型的代码排放方式只允许较小的并行性并会导致流水线的失速。处于失速时，CPU 和 GPU 将处于闲置状态，很显然这是一种浪费。CPU 和 GPU 都使用多线程来隐藏这个问题。

在 CPU 方面，来自其他虚拟 CPU 核的指令流填补指令流水线的空隙 (如超线程技术)。然而，这要求 CPU 知道流水线中的指令属于哪个线程，但这会使硬件变得复杂。在 GPU 上，也使用多个线程，但采用时间切换方式，这样算术运算的延迟时间被以极小地甚至可以没有代价地隐藏掉。事实上，GPU 上你需要大约 20 个时钟来隐藏这样的延迟。然而，这种

延迟不一定必须来自另一个线程。考虑以下示例：

```
extern int a,c,d,f,g,h,i,j;
extern const int b;
extern const int e;

void some_func_with_dependencies(void)
{
 a = b * 100;
 c = b * 1000;

 f = b * 101;
 g = b * 1001;

 d = (a + c) * e;
 h = (f + g) * e;

 i = d * 10;
 j = h * 10;
}
```

这里的代码已经重新排放并引入了一些新的变量。注意，如果你在计算变量 a、c 与使用它们计算 d 之间的位置插入一些独立的指令，将需要更久的时间才能获得 d 的计算结果。计算 f、g 和 h 的值与计算 d 是重叠的。实际上，你是通过重叠执行非依赖指令达到隐藏算术运算的延迟。

循环融合（loop fusion）是一种处理依赖关系并引入额外非依赖指令的技术，如下所示。

```
void loop_fusion_example_unfused(void)
{
 unsigned int i,j;

 a = 0;
 for (i=0; i<100; i++)  /* 100 iterations */
 {
  a + = b * c * i;
 }

 d = 0;
 for (j=0; j<200; j++)  /* 200 iterations */
 {
  d += e * f * j;
 }
}

void loop_fusion_example_fused_01(void)
{
 unsigned int i;   /* Notice j is eliminated */

 a = 0;
 d = 0;
```

```
 for (i=0; i<100; i++)  /* 100 iterations */
 {
  a += b * c * i;
  d += e * f * i;
 }

 for (i=100; i<200; i++) /* 100 iterations */
 {
  d += e * f * i;
 }
}

void loop_fusion_example_fused_02(void)
{
 unsigned int i;   /* Notice j is eliminated */

 a = 0;
 d = 0;
 for (i=0; i<100; i++)  /* 100 iterations */
 {
  a += b * c * i;
  d += e * f * i;
  d += e * f * (i*2);
 }
}
```

在这个例子中，我们有两个独立的计算结果 a 和 d。在计算第二个结果时迭代次数多于第一个。不过，两个计算的迭代空间是互相重叠的。因此你可以将第二个计算的一部分移动到第一个计算的循环体内部，正如函数 loop_fusion_example_fused_01 所显示的那样。这样就可以引入额外的、无依赖性的指令，另外能够降低总体的循环次数，在本例中降低了 1/3 的迭代次数。循环迭代不是免费的，因为它们需要一个循环迭代值和一个分支。因此，降低 1/3 的迭代次数会为我们在减少执行的指令数方面带来显著益处。

在 loop_fusion_example_fused_02 函数中，我们可以通过进一步移除第二个循环并把其中的操作合并到第一个循环以达到融合两个循环的目的，相应地只需调整循环的索引即可。

现在的 GPU，很可能将这些循环展开，放到线程内，并由单个内核程序计算 a 和 d 的值。存在多种解决方案，但最可能的情况是使用一个包含 100 个线程的线程块计算 a，然后另外一个包含 200 个线程的线程块计算 d。通过融合两个计算，你不再需要另一个计算 d 的线程块。

然而，谨慎使用这种方法。通过执行这些操作，你同时减少了可用于线程 / 线程块调度的整体并行度。如果这个数目很小的话，会浪费执行时间。另外要注意，使用融合的内核时，通常会消耗更多的临时寄存器。由于寄存器的使用增加了，会限制一个 SM 上可调度的线程块的数目，从而可能会限制实际可融合的数量。

最后，你应该好好考虑需要多轮遍历的算法。它们通常被实现为一些内核调用的序列，每一次调用在数据上循环一遍。由于每轮要读 / 写全局数据，效率通常较低下。许多这样的

算法可以写成只涉及单个或少量目标数据点的内核程序。这为把数据放入共享内存或寄存器提供了可能，并且相较给定内核需要较多次全局内存访问的方式，可以大大提高完成的工作量。这将明显改善多数内核的执行时间。

9.1.5 数据集大小

数据集的大小使选取问题的解决方案差别巨大，一个典型的CPU实现可以分成如下几类：

- 数据集小于一级缓存（~ 16KB ~ 32KB）
- 数据集小于二级缓存（256KB ~ 1MB）
- 数据集小于三级缓存（~ 512K ~ 16MB）
- 数据集小于单台主机内存大小（~ 1GB ~ 128GB）
- 数据集小于主机端持久性存储大小（500GB ~ 20TB）
- 数据集分布在多台机器上（>20TB）

对GPU而言，列表稍有不同：

- 数据集小于一级缓存（16KB ~ 48KB）[⊖]
- 数据集小于二级缓存（512KB ~ 1536MB）[⊜]
- 数据集小于GPU内存大小（~ 512K ~ 6GB）
- 数据集小于单台主机内存大小（~ 1GB ~ 128GB）
- 数据集小于主机端持久性存储大小（500GB ~ 20TB）
- 数据集分布在多台机器上（>20TB）

对于非常小的问题集，可以增加更多的CPU核，可能会带来超线性的加速比。这时增加更多的CPU核，你将得到比线性更大的加速比。实际上，每个处理器核的数据集会比现在更小。对于一个具有16核的CPU，通常问题空间会减少16倍。如果现在把问题从内存转移到三级缓存或从三级缓存转移到二级缓存，你将看到一个非常明显的加速，这不是由于并行性，而是使用的缓存有高得多的内存带宽。这样的问题同样适用于把问题完全从一级缓存移到二级缓存。

GPU的主要问题不是缓存，而是你能在一张GPU卡上保存多少数据。将数据从主机系统传入或传出会耗费大量的计算时间。为了隐藏这一时间，你应该把计算与数据传输重叠起来执行。在更先进的显卡上，你可以做到同时传入与传出数据。然而，为了实现这一点，你需要使用主机上的锁页内存。由于锁页内存不会被虚拟内存管理系统换出，所以它必须是存在于主机上的真正的DRAM内存。

对于拥有6GB内存的Tesla系统，你可能会分配1GB输入缓冲区、1GB输出缓冲区和4GB计算或工作内存。尽管一些商用显卡支持多达4GB的全局内存，但在商用硬件上，你

⊖ 一级缓存仅在费米架构及之后可用，可以配置为16KB ~ 48KB。GT200/G80的一级缓存由纹理内存充当，共24KB。

⊜ 在计算能力1.x的设备上，没有二级缓存；在计算能力2.x（费米架构）设备上，达到768KB；而到了计算能力3.x（开普勒架构）设备，多达1536KB。

最多能获得 2GB 的可用空间，所以可用空间要比总量少得多。

在主机端，你的内存至少需要与为输入和输出缓冲区分配的锁页内存等量。你通常拥有的可用内存，在 Nehalem 平台上最多是 24GB（6 个 4GB 的 DIMM）、在沙桥—EPI7 上是 32GB（8 个 4GB 的 DIMM 的），在 AMD 平台上是 16GB（4 个 4GB 的 DIMM）。由于你通常使用最多 2GB 的锁页内存，因此剩余的内存量可以轻松地支持多个 GPU，多数系统支持至少两个 GPU 卡。1 个机箱内采用 4 个物理显卡通常是高端系统的实际上限。

当问题的规模远远大于主机的内存大小时，你必须要考虑实际单台主机上的存储容量的限制。TB 级的磁盘可以让节点保存数十 TB 的数据。大多数主板都配备了 6 个或更多的 SATA 接口并支持超过 4 TB 的磁盘。如果数据集需要传送到较远的地方，那么磁盘是很容易运送的。隔日达快递通常是在不同地点间传输这类数据的最快方式。

最后，当你的数据集可能因为计算、内存、存储或者能源方面的因素，无法放置于单台机器时，你必须考虑使用多个节点。这就需要进行节点间通信了。节点间通信是非常耗时的，相比任何内部的数据通信至少慢一个数量级。你还必须掌握另一套 API，所以如果问题可以放在单个节点上，最好避免节点间通信这一步骤。

9.1.6 分辨率

考虑一下使用 10 倍或 50 倍的处理能力可以做什么。说得实际点，以前需要花费 1 小时来解决的问题能只用 1 分钟解决。对于一个给定的数据集，可以探索的议题是如何改变？有哪些过去实现不了，现在则可以实时或者接近实时做到的呢？之前需要批量提交的问题，现在变成了可以互动的问题。

这种变化可以让问题暂缓一步处理，而是考虑还有什么是可能实现的。是否有算法是因为计算代价太大而被丢弃？你现在可以处理更多的数据，或者将数据提高到更高的分辨率、产生更精确的结果吗？如果你对之前运行几个小时或一天的情况很欣然，因为你正好可以利用这段时间做其他的事情，那么提高问题的分辨率是否比提高速度更有吸引力？一个更精准的结果在你的问题里能得到什么？

在金融领域的应用程序中，如果你的算法领先于市场中的竞争者，那么你可以对变化做出比别人更快的反应，它可以直接转化为更多商业利益。

在医疗应用程序中，在病人结束会诊离开前就能够预测结果，能够更有效地节约医生和病人的时间，因为它避免了重复的预约时间。

在仿真应用程序中，不必等待很长时间就可以在相同的时间单位内探索一个更大的问题空间。它也允许猜测执行。就像你在一个给定的数据集里求所有在 n 和 m 之间的 x 值。同样，你可以探索二维或三维空间中的变量。在复杂问题或非线性系统中，它并不总是很清楚什么是最佳的解决方案，尤其是当改变一个参数会影响许多其他参数的情况下。它可能会更快地搜索问题空间和观察结果，不像以前那种需要专家坐下来，制定出最佳的解决方案。这种直截了当的暴力方法是非常有效的，往往能得到“专家”无法找到的解决方案。

作为一名学生，你现在可以开始通过个人桌面超级计算机中不同的讲座解决问题，而不

是提交给大学机房一份任务，等待一天来运行它，却发现工作进行一半就失败了。你可以大大快于 CUDA 盲的同行，开发出原型方案并得到问题的解决方案。想一想，原本在批量方式下需要一天的任务，现在你在本地一小时就能做完了。

9.1.7 识别瓶颈

1. Amdahl 定律

Amdahl 定律常常被并行架构的论文引用。它是很重要的，因为它告诉我们，当数据流中仍然存在串行执行的元素时，将限制速度的提升。

考虑一个简单的例子。我们的程序中 50% 的执行时间花费在可以并行化的代码上，另外 50%必须串行执行。如果你有一组无限快的并行处理单元，你可以把程序中并行部分的执行时间减少到 0，但仍然会留有 50%的串行代码要执行。在这种情况下，可能的最大加速比是 2×，即程序的执行时间是之前的一半。尽管使用了大量的并行处理，但说实话，加速的效果并不明显。

即使我们将 90%的程序并行化，但仍然存在 10%的串行代码。因此，最大的加速比是 9×，也就是比原来完全串行的程序快 9 倍。

无限扩展程序的唯一办法是消除程序执行中的所有串行瓶颈。我们来考虑一下图 9-2，其中所有的方块表示待处理的数据条目。

在这个例子中，有 10 个线程，每个线程处理一列数据。图 9-10 的中心位置含有相互依赖的关系，因此，在继续向下运行之前，所有线程把它们的已有结果送到某个单值。

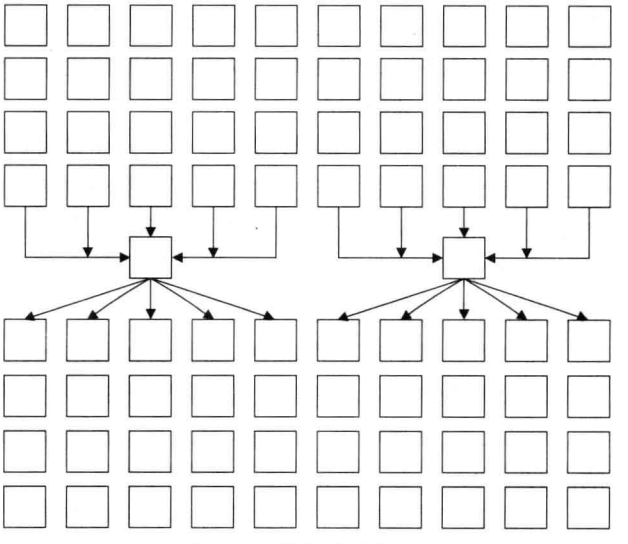

图 9-2 数据流的瓶颈

设想一下，在某个时刻，这是一片麦地，而每一列是一行作物。每个线程就像一个联合收割机，向下移动，并且收集每个方块内的作物。但是到了麦地场中心有一个带有两个大门的围墙。

在 1 个甚至 2 个联合收割机作用下，大门只构成一个小问题，每个联合收割机可以从一行作物转移到另一行作物。使用 10 个联合收割机，每个对应一列，让每个收割机都经过大门，将比较耗时并会减慢整个过程的速度。这就是为什么在又大又空旷的麦场比在小而有限的麦场更高效的原因之一。

上述比喻与软件设计有何关联呢？每个大门就像是代码中的一个串行点。该程序通过运行大量的工作，做得很好，当突然遇到一个串行点或同步点，一切都堵塞了。这就像在只有

有限数量出口的停车场，每个人都试图在同一时间离开。

针对这类问题，把瓶颈部分并行化就能解决。如果我们在麦场有 10 个大门或者在停车场有 10 个出口，将不会再有瓶颈，而是变成了一个有序的队列，在 N 个周期内即可完成。

当考虑如计算直方图这样的算法时，你会看到如果把所有的线程都加入同一个桶里，就形成了同样的瓶颈。通常会采用原子操作，这样一组并行线程就要串行执行。相反，如果分配给每个线程属于它自己的一组桶，后面再将这些桶合并起来，就能消除串行瓶颈问题。

仔细分析你代码中存在这样瓶颈的地方，并考虑如何将它们尽量消灭掉。这种瓶颈往往会限制应用程序的最大扩展程度。虽然在两个甚至 4 个 CPU 核上，可能算不上什么问题，但对 GPU 的代码来说，你需要考虑到这里有成千上万的并行线程。

2. 分析

分析是确定你当前在哪儿以及应该在什么地方多花点时费的最有用的任务之一。通常人们认为他们知道瓶颈在哪里，然后就去优化那个函数，最后却发现，应用程序的总体执行时间只有 1%或 2%的差别。

在现代软件开发中，通常有多个组，分别负责软件包的不同方面。它可能无法与接触到的每个开发软件的程序员都保持联系，特别是大型的团队。通常你认为的瓶颈，实际上可能并不重要。

优化应根据确切的数字和事实，而不是猜测哪“可能”是最应该优化的地方。英伟达提供了两个很好的工具，分别是 CUDA Profiler 和 Parallel Nsight，以提供分析信息。

分析器通过读取硬件计数器，来发现代码花费的时间和在 GPU 上的占用水平。它会提供非常有用的数据，如总共合并读或写的次数、缓存命中 / 失败率、分支频率、线程束串行化程度等。CUDA Memcheck 工具在识别内存带宽的使用效率是否低下时也是非常有用的。

使用分析器做完一个初步的检查之后，你应该先查看花费总时间最多的代码段。典型的未优化的程序，80%的时间是花在 20%的代码上的。优化这 20%代码是有效减少使用时间的关键，分析器是确定这 20%代码所在的一把钥匙。

当然，一旦上述问题已被优化为最佳，如果不进行重新设计，后面的为提供加速化的进一步优化将会变得越来越耗时。你需要度量加速比并且知道什么时候你花的时间不会再得到良好的回报。

Parallel Nsight 在这方面是一个非常有用的工具，因为它提供了一些默认的“实验”。能够把内核实际所做的工作显示出来。如图 9-3 所示，你可以从实验中获得很多有用的信息。

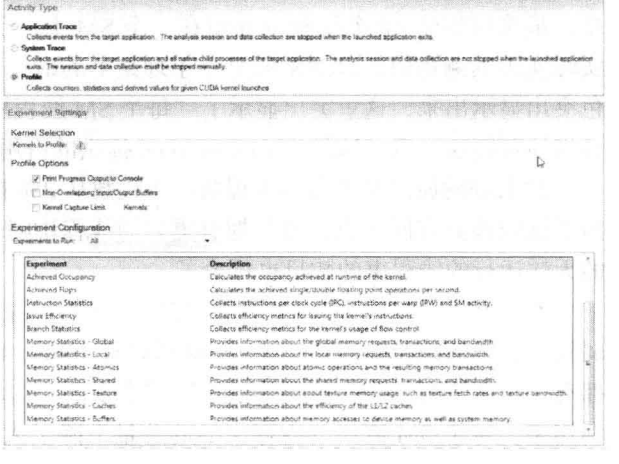

图 9-3 Parallel Nsight 实验

第一个实验是关于 CUDA Memory Statistics（CUDA 内存统计）的，以友好的图形化展示了缓存是怎样分布的以及设备的不同部分达到了怎样的带宽。

这个特别的例子（参见图 9-4）来自奇偶排序，我们在后面还会进一步考察。值得一提的是缓存的比率。当我们获得 54%的一级缓存命中率时，我们与全局内存的平均吞吐量达到了 310GB/s，这是全局内存实际可用带宽的两倍。它还列出了交易的数量，这是非常重要的。如果我们可以通过更好地合并内存操作或者发起更大的读 / 写操作来降低所需事务的数量，则可以显著地提升内存的吞吐量。

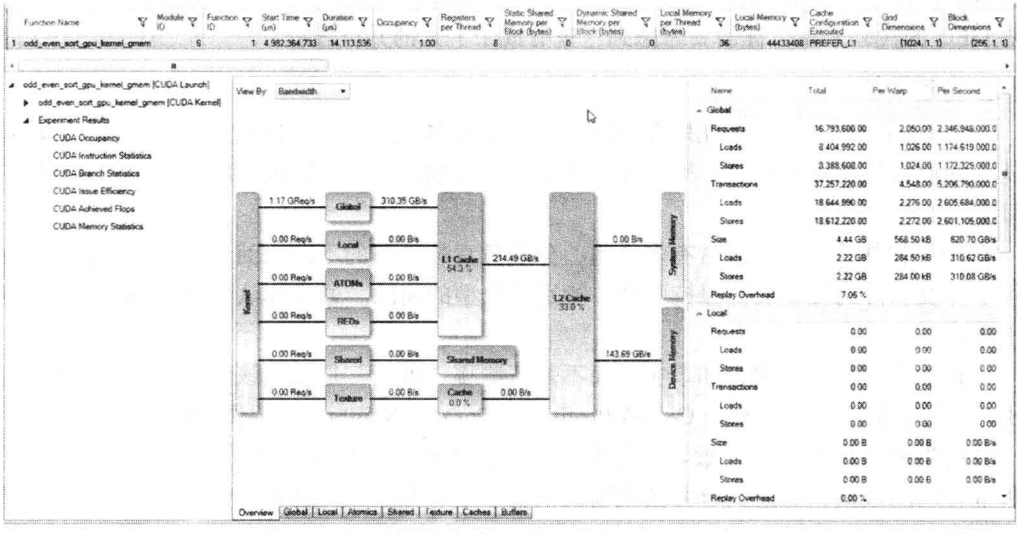

图 9-4 Parallel Nsight 内存概要

另外一个重要的实验是占用率（参见图 9-5）。在这个实验中，注意到 Achieved occupancy（达到占用率）一栏，特别是活动线程束的数量。由于这是一个计算能力 2.0 的设备，最多可以有 48 个线程束驻留在一个 SM 上。所取得的占用量，跟理论占用量相对，即实际实现的测量值，这通常明显小于理论上的最大值。另请注意，任何的限制因素都会以红色突出显示出来，这里突出显示了“每个 SM 线程块的数量是 6”。占有图形选项卡允许你更详细地理解这一点。这是从 CUDA SDK 提供的占用率计算电子表格中提取出来的。

此限制的原因是实际的线程数。线程数从 256 降低到 192，会允许硬件调度 8 个线程块。由于该内核具有同步点，因此拥有更多的线程块则会带来更好的指令混合。由于同步点造成的无法运行的线程束的数目也更少了。

在实践中，这种变化的作用是相当显著的。占用率从 98.17%提高到 98.22%，虽只有微弱提高，然而执行时间从 14ms 下降到只有 10ms。时间降低的原因在于内存使用。通过每块拥有 192 个线程，我们访问的地址区域变小了，这增加了局部性访问，并最终可以提高缓存利用率。每个 SM 需要的内存交易的总量下降了大约 1/4。因此，我们看到了一个执行时间相应比例的下降。

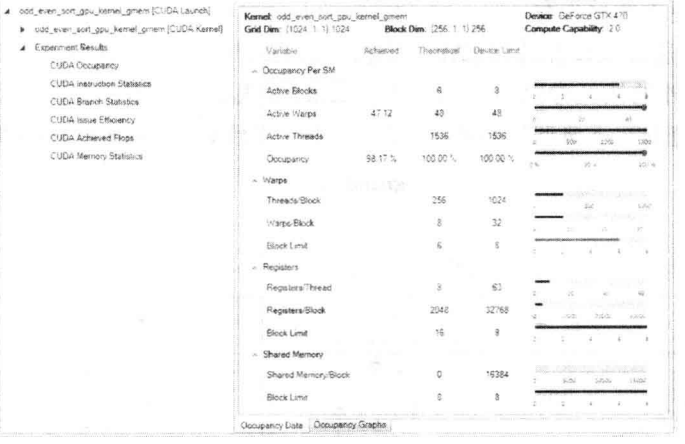

图 9-5 Parallel Nsight 占用数据

9.1.8 CPU 和 GPU 的任务分组

来自英伟达的 M.Fatica 博士在 GTC2010 会议中做了一个关于在 GPU 上优化 Linpack 的精彩演讲。Linpack 是一个基于线性代数的测试基准。它用在 Top500 超级计算机基准测试中（www.top500.org），以评价世界各地的超级计算机。这次演讲中一个有趣的事是，那个时候使用的 GPU 是费米 TeslaC2050 卡，可以每秒运算约 3500 亿次双精度矩阵乘法（Double-precision Matrix Multiply，DGEMM）。所使用的 CPU 可以每秒运算约 800 亿次 DGEMM。每秒 800 亿次 DGEMM 比 GPU 贡献的 1/4 还略低。所以这个差别不可忽略。在执行时间上减少 1/4 的额外运算是需要大费周折的。

事实上，最好的应用程序往往可以充分利用 CPU 和 GPU 两者的优势，并相应地划分数据。任何基于 GPU 的优化也必须考虑 CPU，因为这对于总的应用程序时间很重要。如果你有 4 核、6 核或 8 核的 CPU，却只有一个核忙于处理 GPU 应用程序，为何不使用其他闲置的核来处理该问题呢？可以使用的核数量越多，通过分流一些工作给 CPU 的潜在收益越大。

如果说一个 CPU 核可以处理 GPU 工作的 1/10，那么当仅只有 3 个 CPU 核时，你的 GPU 就获得额外 30%的吞吐量。如果你有一个 8 核的设备，有可能这是一个在性能上 70%的增益，这几乎是和两个 GPU 以串联方式工作是相同的。然而，实际上其他的方面可能限制整体速度，如内存、网络或者 I/O 带宽。不过，即便如此，当应用程序已不再主机端的这些限制约束时，你很可能会看到这个应用程序有着显著的加速。

这些限制中，I/O 的限制是一个有趣的例子，因为引入更多的 CPU 的线程或进程，经常可以显著提高整体的 I/O 吞吐量。这似乎是一个奇怪的结论，因为 I/O 设备的输入 / 输出上限决定了它的吞吐量。在现代具有大内存的计算机中，大多数 I/O 操作都是进行缓存的。因此，I/O 操作大都在内存上移动数据而不是在设备上移动。一个常规的 RAID 控制器有自己的处理器来执行 I/O 操作。多个 CPU 核允许多个独立的存储器之间传输数据，相对单核而

言，这提供一个更大的总带宽。

独立的 CPU 进程或线程可以创建一个独立的 GPU 上下文，且启动它们自己的内核到该 GPU 中。这些额外的核经常以队列的方式在 GPU 中去执行。当所需资源变为可用时，内核开始执行。如果查看一下典型的 GPU 使用率，你会看到如图 9-6 所示的情形。

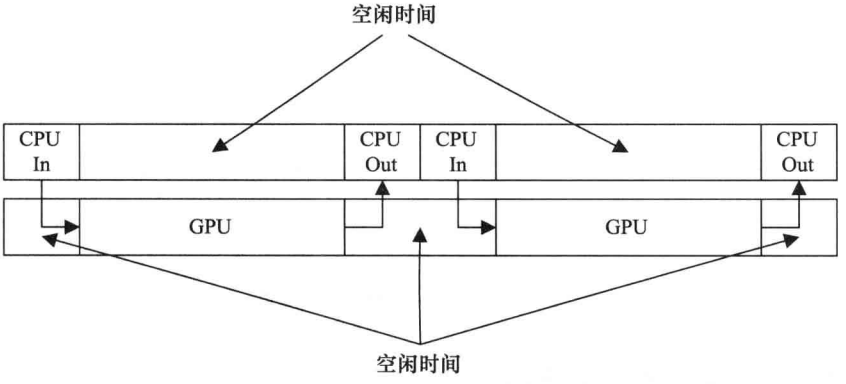

图 9-6 CPU 和 GPU 的空闲时间

注意图 9-6 所示，GPU 和 CPU 都有显著的空闲时间。GPU 上的空闲时间更昂贵，因为它的吞吐量通常在 CPU 时间的 10 倍以上。诸如 Parallel Nsight 的工具允许你查看这样的时间线，你会惊讶地发现特定内核存在如此多某一内核创建的空闲时间。

通过在一个 GPU 中放入多个内核，这些内核就可以伺机占用空闲硬件槽位。这将在一定程度上增加第一组内核的延迟，但会大大提高应用程序的整体吞吐量。在很多应用程序中，可能有多达 30%的空闲时间。我们来考虑一下一个典型的应用程序需要做些什么。首先，从某处获取数据，通常是来自一个如硬盘这样的慢速 I/O 设备。然后将数据传送到 GPU 并一直等到 GPU 内核完成当前工作。一旦 GPU 内核完成当前工作，主机把数据从 GPU 端传回。接着主机把数据保存到某个地方，通常是慢速的 I/O 存储设备。获取下一个数据块，依此类推。

GPU 正在执行内核的同时，为什么不从慢速 I/O 设备读取下一个数据块，以便当 GPU 内核完成时，数据已经就绪了呢？实际上，在执行多个进程时就采用了这一策略。当你为第 1 个进程读取数据时，I/O 设备会阻止第 2 个进程的执行。当第 1 个进程在传输数据并调用内核时，第 2 个进程会访问 I/O 硬件。接着就开始数据传输，与此同时，进程 1 在进行计算且进程 2 的内核调用的处于排队状态。当进程 1 开始把数据传回主机时，进程 2 的内核就开始执行。因此，仅通过引进一对进程，你巧妙地重叠了 I/O、CPU、GPU 和传输时间，整体吞吐量获得了显著的改善。这方面的详细解释，请查看第 8 章关于流的示例。

请注意，不管你使用线程或是进程，都可以达到同样的效果。线程允许应用程序数据共享为一个公用数据区，并提供更快的同步原语。进程允许设置为处理器关联（processor affinity），可以把进程绑定到一个给定的 CPU 核。这样做往往会提高性能，因为它可以更好地重用该核的缓存。选择线程还是进程，在很大程度上取决于 CPU 之间需要同步任务的个数。

在权衡 CPU/GPU 的使用过程中，也需要知道如何最优地划分任务。当数据是稀疏分布的或者是小数据集的时候，CPU 很擅长处理这类串行任务。但是考虑到一个典型的 GPU 与 CPU 在性能上 10：1 的表现，就不得不小心你有可能支持不了 GPU。出于这个原因，许多应用程序只使用 CPU 来加载和存储数据。根据 GPU 上所需计算时间，有时会让一个 CPU 上的一个核处于满载状态。

你有时会看到 CPU 用在归约操作的最后阶段。通常在每轮迭代，归约操作涉及的元素数会下降为原来的一半。如果你开始使用一百万个元素，6 轮迭代之内，可供调度的线程数量就小于一个 GPU 可调度的最大线程数了。如果再继续迭代几轮，一些 SM 就开始闲置。当使用 GT200 和之前的硬件时，内核不支持重叠，所以内核不得不继续迭代直到得到最后的元素，才可能释放空闲的 SM 去做更多的工作。

因此，一种优化策略是，当迭代到一定的阈值，剩余部分的计算就转交给 CPU 来完成。事实上，如果 CPU 空闲并且剩余的数据传输量也不是很大，那么这种策略相比等待 GPU 完成整个归约过程，会有显著的收益。从费米架构开始，英伟达公司解决了上述问题，能够让那些闲置的 SM 在下一个排队的内核中使用。但是，要让 SM 变得空闲，必须要保证其上的所有线程块已经完成它们的任务。一些非最优的内核可能残留一个或者数个活跃线程（即使在归约操作的最后一层），从而导致内核牵制住该 SM，直到整个归约操作完成。对于类似归约操作的一些算法，请确保每次迭代都在减少活跃线程束的数量，而不单单是活跃线程的数目。

9.1.9 本节小结

- 理解问题并基于你的编码时间和熟练程度定义你的加速目标。
- 识别问题中的并行性，并思考如何以最佳方式在 CPU 和一个或多个 GPU 之间分配。
- 考虑一下，是较少的执行时间还是处理数据以获得更高分辨率更重要。
- 理解任何串行代码段的实现，并思考如何处理它们最合适。
- 分析你的应用程序，以确保你的理解确实反映实际情况。如果可以帮助你加强理解，请重复你之前的分析。

9.2 策略 2：内存因素

9.2.1 内存带宽

内存带宽和延迟是所有应用程序都要考虑的关键因素，尤其是 GPU 应用程序。带宽是指与某个给定目标之间传输的数据量。在 GPU 的情况下，我们主要关心的是全局内存带宽。延迟则是指操作完成所用的时间。

GPU 上的内存延迟设计为由运行在其他线程束中的线程所隐藏。当线程束访问的内存位置不可用时，硬件向内存提交一次读或者写请求。如果同一线程束上其他线程访问的是相邻内存位置并且内存区域的开始位置是对齐的，那么该请求会自动与这些线程的请求组合或者合并。

费米架构和其他旧版本硬件的内存事务规模有着显著的不同。在计算能力1.x设备上（G80和GT200），每次内存访问时的合并内存事务大小最小为128字节。如果被合并线程访问的总区域足够小并且在同一个32字节对齐的块中，那么这将减少到64或32字节。这个内存是没有被缓存的，因此如果线程没有访问连续的内存地址，会导致内存带宽快速地下降。例如，如果线程0读取地址0、1、2、3、4、…、31，线程1读取32、33、34、…、63地址，它们将不会被合并。实际上，对于每个线程，硬件会提交一个至少32字节的读请求。未被使用的字节将从内存中取出并直接丢弃。因此，如果不仔细考虑内存是如何被使用的，你就只能获得设备上实际可用带宽的很小一部分。

从这个角度看，费米架构和开普勒架构的情况已经有了很大的改善。与计算能力1.x的设备不同，费米架构从事务中获取32字节或128字节的内存，而不支持64字节的内存读取。在默认情况下，每个内存事务就是一个128字节的缓存行读取。因此，一个关键的区别是，当采用一组128字节而不是单独1个访存请求时，现在将通过访问缓存，而不是另一个内存存取。这使得GPU模型从费米架构开始与前几代相比更加容易进行编程。

我们需要考虑的一个关键领域是运行过程中的内存事务的数量。每一个内存事务被送入一个队列中然后由内存子系统单独执行。这当然会有一定的开销。一个线程一次提交对4个浮点数或整型数的一个读操作比提交4个单独的读操作花费的代价更小。实际上，如果你看一看NVIDIA提供的一些图表会发现，为了在费米架构和开普勒架构上接近峰值带宽，你可以采取两种方法。方法1，使用线程束完全加载处理器，实现接近100%的占用率。方法2，通过float2/int2或float4/int4向量类型使用64/128位读操作，此时占用率小了很多，但仍然能达到接近100%的峰值内存带宽。实际上，通过使用向量类型，你提交了少量可以由硬件有效处理的较大事务。通过每线程处理多个元素，还引入了一定的指令级并行。

然而，需要注意的是，向量类型（int2、int4等）分别引入了8字节和16字节的隐式对齐。数据必须支持此功能，例如，你不能将一个int类型的指针从int数组元素int[5]转换为int 2*然后期望它工作正常。在这种情况下，你最好执行并排32位读操作或者对数据结构的边缘进行填充从而允许对齐访问。正如我们在优化排序的例子中看到的那样，如果内存吞吐量增加了，那么每线程处理4个元素经常能在额外的寄存器使用和更好地利用处理器的指令级并行的可能性上提供最佳平衡。

9.2.2 限制的来源

内核通常被两个关键因素限制：内存延迟/带宽和指令延迟/带宽。当某一个是关键的限制因素，却对另一个进行优化时，会花费大量的精力并且只会得到很少的回报。因此，理解这两类关键限制因素中哪一个正在限制系统的性能，对于指导你合理的分配精力是很关键的。

最简单的能够看到代码平衡位置的方法，是简单地注释掉所有算术指令，然后用直接赋值成结果替代。算术指令包括所有的计算、分支、循环等操作。如果输入值和计算的输出值存在一一映射，那么这种方法很容易实现并且一一赋值效果也很好。如果存在某种形式的归约操作，你只需将它替换成普通的求和操作。一定要确保包括所有从内存中读取到最终输出

的参数，否则编译器将删除明显冗余的内存读 / 写操作。对内核执行重新定时，你会看到花费在算术和算法部分的近似百分比时间。如果这个百分比很高，那么你就受到了算术限制。相反，如果这个百分比在整体计时中变化很小，那么你就受到了内存限制。

在算术代码仍被注释掉时，使用 Parallel Nsight，启用 Analysis 函数和 Profile 设置执行内核。考查得到的指令统计数据（参见图 9-7）。如果条形图中有大量的蓝色，那么表明内核内存模式没能很好地合并，GPU 不得不串行地执行指令流以支持分散的内存读 / 写。

如果是这种情况，那么还有可能重新安排内存模式，以允许 GPU 将线程的内存访问模式合并吗？记住，要做到这一点，线程 0 必须访问地址 0，线程 1 访问地址 1，线程 2 访问地址 2，以此类推。理想情况下，你的数据模式应该生成一个基于列的线程访问模式，而不是基于行的访问。如果你不能轻易地重新安排数据模式，你能够重新安排线程模式吗（这样你就能在访问数据前将线程载入到共享内存中）？如果可以，那么在从共享内存中访问数据时你就不用再关心合并读操作了。

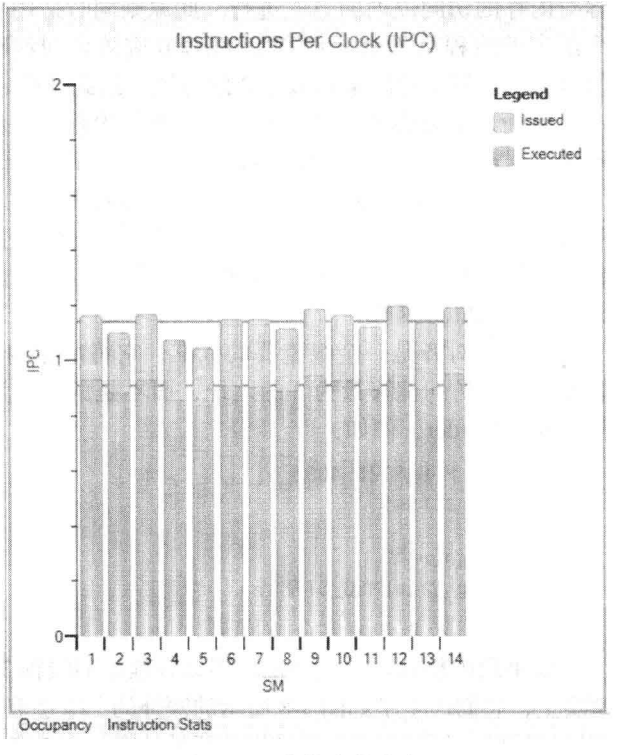

图 9-7 高指令补发率

是否可以扩大一个单一线程处理的输出数据集的元素数目呢？这通常同时有助于内存受限型和算术受限制的内核。如果你这样做，请不要在线程中引入循环，而是要通过复制代码实现。如果代码是很重要的，这也可以作为设备函数或宏来实现。确保将读取操作提前到内核开始处，这样在需要数据时就已经完成了对它们的读取。这将增加寄存器的使用，所以一定要监控正在被调度的线程束个数以确保它们不会突然地退出。

至于算术限制的内核，查看源代码并思考如何将其翻译成 PTX 汇编代码。不要害怕实际产生的 PTX 代码。数组索引通常被替换为基于指针的代码，将速度较慢的乘法替换成更快的加法。使用 2 的幂次的除法和乘法指令分别可以被替换为速度更快的右移和左移位运算。循环体中的所有常量（不变量）应该被移到循环体外部。如果线程包含一个循环，那么展开循环通常会实现加速。什么样的展开因素效果最好？在这一章后面我们会详细地考查这些优化策略。

你实际使用的是单精度浮点数还是双精度浮点数，哪个是你想要使用的呢？注意编译器会把没有 F 后缀的浮点常量当作双精度处理。你真地在所有的计算中都需要使用 32 位的精度吗？试

着使用 -use_fast_math 编译器开关，看看结果的准确度是否满足你的需求。此开关可以启用 24 位浮点运算，它明显快于标准的 IEEE 32 位浮点数学逻辑。

最后，你是否使用“发布”版本的代码测试速度呢？和我们之前看到的那些例子一样，单凭这一点就可以提高 15% 以上的性能。

9.2.3 内存组织

在许多 GPU 应用程序中，使用正确的内存模式往往是关键的考虑因素。CPU 程序通常在内存中以行的方式安排数据。尽管费米架构和开普勒架构能容忍非合并的读 / 写操作，然而像我们之前提到的一样，计算能力 1.x 设备则不会。你必须尝试安排内存模式以使连续线程对内存的访问以列的方式进行。此原则同时适用于全局内存和共享内存。这意味着对于一个给定的线程束（32 个线程），线程 0 应该访问偏移量为 0 的地址，线程 1 访问偏移量为 1 的地址，线程 2 访问偏移量为 2 的地址，以此类推。想一想从全局内存获取数据的方式。

然而，假设你有一个对齐访问，全局内存一次会有 128 字节的数据。每个线程对应一个单精度浮点数或整型数，那么线程束中的所有 32 个线程都会被准确地给予一个数据元素。

注意，cudaMalloc 函数将以 128 字节对齐的块为单位分配内存，所以对于大部分情况，对齐并不是一个难题。但是，如果使用结构会越过这个边界，那么有两种方法解决这一问题。首先，你可以在结构中添加填充的字节 / 字。或者，你可以使用我们在第 6 章提到的 cudaMallocPitch 函数。

注意，对齐是一个很重要的标准，它将决定内存事务或缓存行需要获取一次还是两次。假设线程 0 访问偏移量为 2 的地址，而不是偏移量为 0 的。也许你正在访问的一些数据结构起始处有 header，例如：

```
#define MSG_SIZE 4096
typedef struct
{
 u16 header;
 u32 msg_data[MSG_SIZE];
} MY_STRUCT_T;
```

如果用内核处理 msg_data，那么线程束的线程 30 和 31 无法通过一次访存得到所需数据。事实上，它们产生了如图 9-8 所示的额外的 128 字节的内存事务。所有后续的线程束都会遇到同样的问题。仅仅由于数据结构的开始处有一个 2 字节的头，内存带宽就变成了原来的一半。

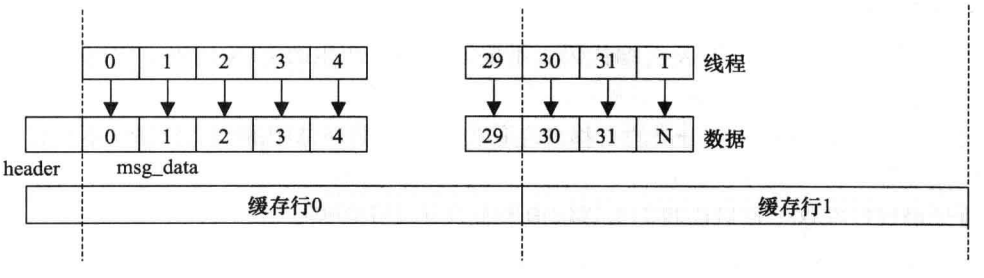

图 9-8 结构内的缓存行 / 内存事务的使用

在计算能力 1.x 的设备中，情况最为严重。为线程 30、31 产生的额外读取，甚至没有被预先填入缓存，而只是被丢弃。将 header 加载到一个位于其他位置的单独内存块，可以允许对数据块进行对齐访问。如果无法做到这一点，那么请手动将填充字节插入到结构定义中，以确保 msg_data 对齐到 128 字节的边界。需要注意的是，如果该结构随后没有被用来创建结构数组，那么简单地重新排序结构元素，将“header”移动到 msg_data 后也是有效的。突然之间线程就会和内存组织相匹配并且使用结构中 msg_data 时，内存吞吐量将加倍。

我们仍考虑使用前缀求和的情况。前缀求和允许多个独立的进程或线程对独立的内存区域进行读写而不会相互干扰。从同一地址的多次读取实际上是有着很大的好处，因为 GPU 将很容易地转发该值给线程束中的其他线程，而无须额外的内存读取。多次写操作当然是一个问题，它们需要按顺序进行。

现在我们假设数据类型是整型或单精度浮点型，数组中每一个元素大小均为 4 字节。如果前缀数组的分布是完全相等的，那么我们并不需要前缀数组来访问数据，因为你可以简单地为每个线程使用一个固定的偏移。如果你使用的是前缀求和来计算数据集的偏移量，那么极有可能出现每个桶里的元素数不相等。如果知道每个桶中元素数目的上界并且有充足的可用内存，则可以填充每个桶，让它达到边界对齐的要求。使用另外一个数组保存桶中元素数目或者从前缀和索引中计算该值。这样我们以很多桶尾部存储了无用的单元为代价，达到了对内存的对齐访问。

针对对齐问题的一个非常简单的解决方案是使用填充值。只要保证该值对于计算结果没有影响即可。例如，如果你在每个桶内执行加和操作，那么填充零意味着最终的结果不会变化，但是为所有线程束中的所有的元素带来了统一的内存模式和执行路径。对于求最小值的操作，你可以填充 0xFFFFFFFF；相反的，对于求最大值操作你可以填充 0。通常找出可以处理且对结果没有影响的填充值并不困难。

当你转而处理固定大小的桶时，很容易就能确保数据集按列生成和进行访问，而不是按行。由于不满足合并的需求，通常，使用共享内存作为临时缓冲是明智的。然后，可以将其用于对全局内存进行合并的读 / 写操作。

9.2.4 内存访问以计算比率

内存操作与计算操作的比率是值得思考的问题。你所期望的理想比例至少是 10 : 1。也就是说，对于每一个内核，从全局内存执行的读取操作需要执行 10 条或更多的指令。这些指令可能是数组索引计算、循环计算、分支或条件判断。每个指令都应该对有效地输出起到一定的贡献。特别是循环没有展开时，它经常会增加指令开销但并不会有助于任何有用的工作。

如果看一看 SM 的内部结构，我们看到线程束会被奇偶指令调度器调度到 CUDA 核集合上。计算能力 1.x 设备有一个线程束调度器，而计算能力 2.x 设备有两个。GF100/GF110 芯片组（费米架构 GTX480/GTX580）中有 32 个 CUDA 核并且每个 SM（参见图 9-9）有 4 个 SFU（Special-Function Unit，特殊功能单元）。基于 GF104/GF114 的设备（GTX460/GTX560）有 48 个 CUDA 核，每个 SM（参见图 9-10）有 8 个 SFU。计算能力 2.0 和计算能力 2.1 设备具都有一组共 16 个 LSU（Load Store Unit，负载存储单元），它用于将值到加载内存或从内存加载值（全局内存、常量内存、共享内存、本地内存和缓存）。

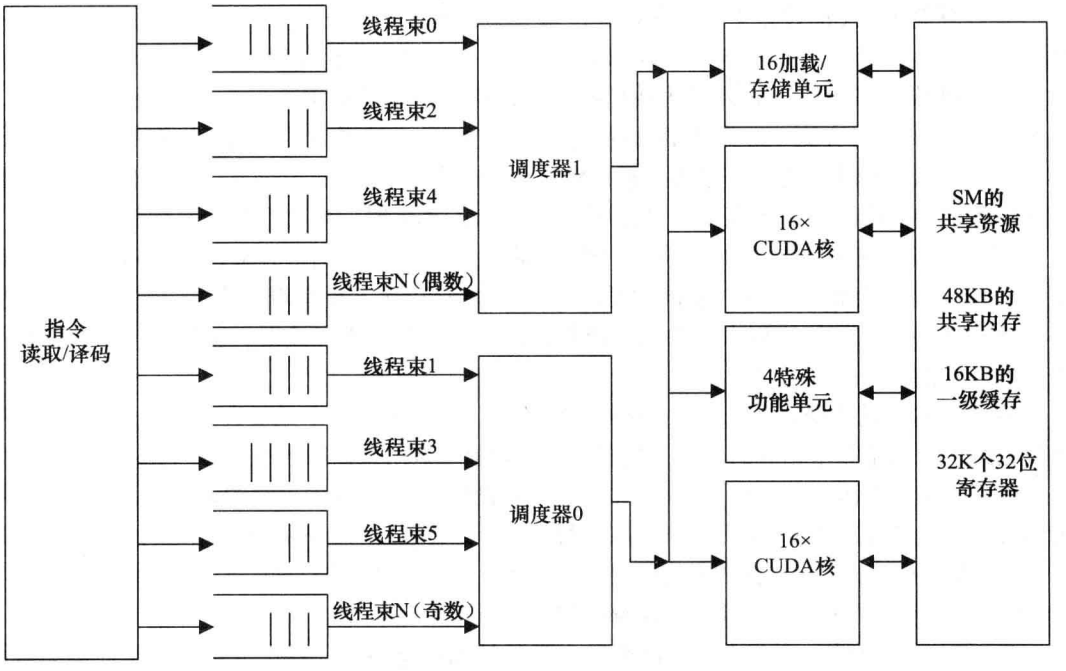

图 9-9 CUDA 线程束的调度（GF100/GF110，计算能力 2.0）

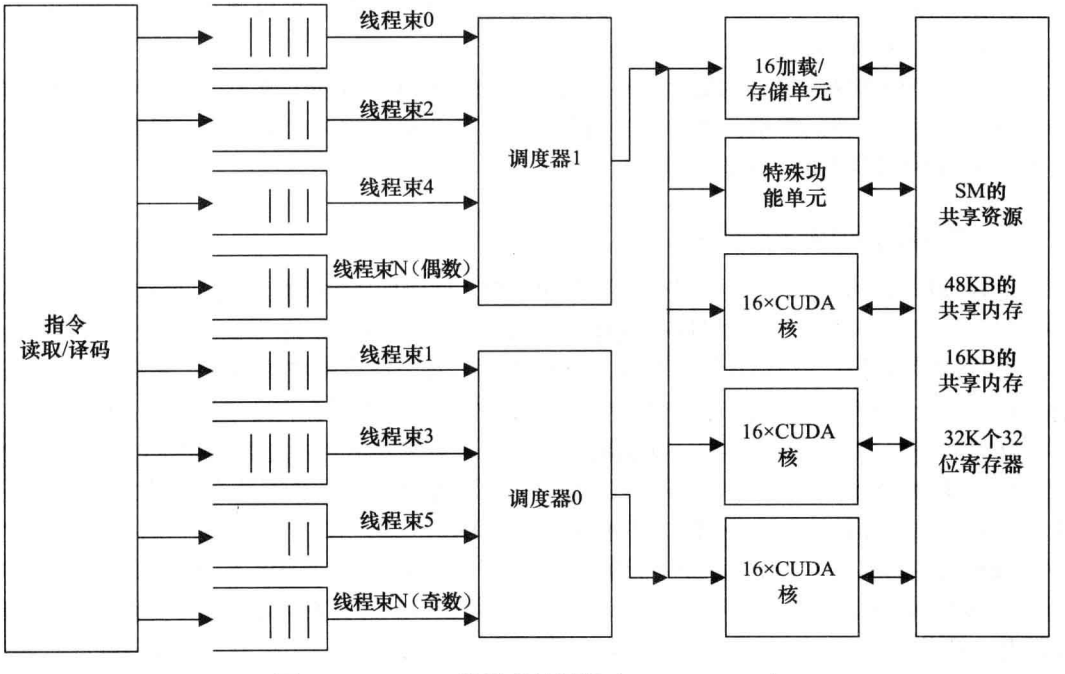

图 9-10 CUDA 线程束的调度（GF104/GF114）

因此，在每个周期内线程束调度器提交（或调度）总共 2 条（计算能力 2.0）或 4 条（计算能力 2.1）指令。由于这些指令来自于不同的线程束，它们之间是相互独立的，因此将它们放入执行单元（CUDA 核、SFU 和 LSU）流水线中。

这种设计有些玄机。首先，对于 GF100 系列（计算能力 2.0）的硬件，必须存在的最小线程束个数为 2，而对于 GF104 系列（计算能力 2.1）的硬件最少个数是 4。这意味着每个 SM 中最少线程数分别为 64 或 128 个。使用比上述少的数量意味着一个或多个指令调度单元将会保持闲置状态，这会将（GF100）指令调度速度减半。使用的线程个数不是 32 的倍数将意味着 CUDA 核中的一些元素会闲置，同样，这也是我们不愿看到的。

基于切换到其他线程束的能力，使用最少个数的常驻线程束无法隐藏内存或指令延迟。指令流的失速实际上将会使 CUDA 核失速，这是我们非常不想看到的。实际上，多个线程块会被分配到一个 SM 上，以试图确保这个问题永远不会发生并且更重要的是生成各种形式的混合指令。

第二个要点是共享的资源限制了持续执行相同操作的能力。由于 CUDA 核和 LSU 都被纳入了流水线，但是它们只有 16 个单元的宽度。因此，将整个线程束调度到这两个单元之一会花费两个周期。计算能力 2.0 的设备中，每个调度器只能调度一条指令。因此，为了将一个操作加入到 LSU 中，流水线中某一个 CUDA 核的槽必须是空的。这个调度有 4 个可能的接收者（CUDA、SFU 和 LSU），但是每个周期只有两个提供者。

在计算能力 2.1 硬件上，情况有了大幅好转，它的两个调度器分别可以调度两条指令，也就是每周期共计 4 条指令。采用 3 组 CUDA 核，则很可能在供应 3 个算术指令和 1 个加载 / 存储指令，而不会导致流水线出现空隙。

然而，如果所有线程束都想提交一条指令到相同的执行单元（例如 LSU 或 SFU），就会出现一个问题。每两个时钟周期内只有一个线程束能够使用 LSU。由于计算能力 2.0 硬件的 SFU 只有 4 个单元，那么线程束被 SFU 完全消化掉要花费 8 个周期。

因此，计算能力 2.1 设备上 LSU 的可用的带宽是拥有同样个数 CUDA 核的计算能力 2.0 设备的一半。因此，LSU 或 SFU 成为了瓶颈。流中需要其他的指令，这样在内存和超越指令在 LSU 或 SFU 流水线执行的同时 CUDA 核可以做一些有用的工作。

开普勒 GK104 设备（GTX680/Tesla K10）通过将 CUDA 核数量从 48 增加到 96 然后将其中每两个放到一个 SM 中，进一步扩展了 GF104/114（GTX460/560）的设计。因此，每个 SM 有 4 个线程束调度器、8 个调度单元、两个 LSU 和两个 SFU。

现在，让我们扩展一下之前看到的例子。考虑一个典型的内核。在内核开始处，所有线程束中的所有线程从内存取出一个 32 位的值。地址可以进行合并访问，例如：

```
int tid = (blockIdx.x * blockDim.x) + threadIdx.x;
data[tid] = a[tid] * b[tid];
```

这将分解成一个整型的乘以及加法指令（MADD）、为变量 tid 计算值并将其存入寄存器、变量 data、b 和 c 是全局内存中的数组。它们通过 tid 进行索引，因此写入的地址需要通过将 tid 乘以数组的元素大小计算得到。假设它们都是整型数组，因此每个元素的大小是 4 字节。

在计算 tid 时（参见图 9-11）我们很快地遇到了第一个依赖。线程束将 blockIdx.x 和

blockDim.x 的乘法调度到 CUDA 核中的整型 MADD 单元。在计算 tid 的乘法和加法指令完成之前，我们无法继续执行其他工作，因此线程束被标记为阻塞和挂起的状态。

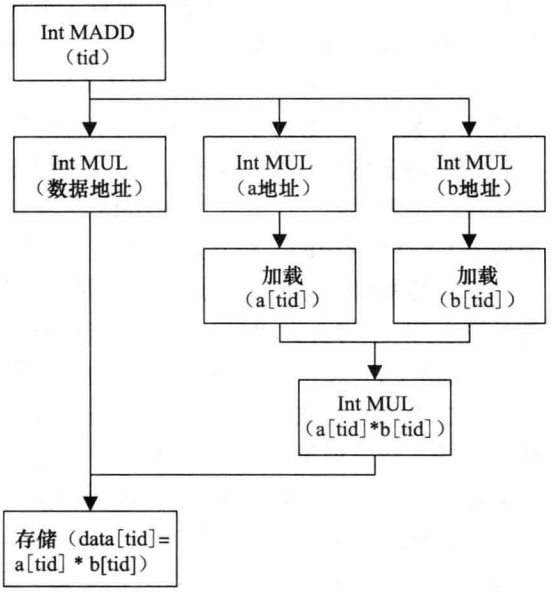

图 9-11 数据流的依赖关系

此时，选择下一个线程束，它会执行同样的操作并且同样会在计算 tid 时暂停。在所有线程束都执行到这一点后。经过了足够多的时钟，此时线程束 0 中 tid 的值是已知的并且可以放到乘法中计算目的地址。因此，三个额外的 MADD 指令被调度到 CUDA 核用来计算地址偏移量。下一条指令可能是一对加载指令，但是对此我们需要来自乘法指令的 a 和 b 的地址。此时我们再一次使该线程束挂起并让其他线程束执行。

一旦 a 的地址计算出来，就能够调度加载指令。由于 a 和 b 的地址计算是相继提交的，很可能加载 a 的指令被调度时，b 的地址计算已经完成了。因此，我们马上提交对“b”的加载。流中的下一条指令可能是“a”和“b”的乘法，一段时间内这两个元素可能都是不可用的，因为它们必须要从主存中取出放到 SM。因此，线程束会被挂起，然后后续的线程束执行同一点。

由于内存读取花费很长的时间，因此所有的线程束将需要加载的指令调度到 LSU，然后挂起。如果其他线程块没有其他的工作要做，那么 SM 将会闲置，等待内存事务完成。

一段时间后，a 会作为 128 字节的合并读操作（单个的缓存行或 1 次内存事务）的一部分从内存子系统到达。16 个 LSU 将 128 个字节中的 64 个分发到线程束 0 中第一个半线程束使用的寄存器中。在下一个周期中，16 个 LSU 将余下的 64 字节分发到另一个半线程束使用的寄存器中。然而，线程束 0 仍然不能继续进行，因为它只有乘法运算所需的两个操作数中的一个。接着合并读取其他线程束的 a 并分发到这些线程束的相应寄存器上。

当所有对 a 合并读取的数据都被分发到其他线程束相应的寄存器上时，b 的数据可能已经到达一级缓存。再一次，16 个 LSU 将 64 字节分发到线程束 0 的第一个半线程束，在随后的一个周期中，它将剩余的 64 字节分发到第二个半线程束上。

在第二个周期开始时，第一个半线程束能够执行乘法指令 a[tid] * b[tid]。在第三个周期中，空闲的 LSU 开始为线程束 0 的第一个半线程束提供数据。同时，线程束 0 的第二个半线程束开始执行乘法。由于线程束 0 的下一条指令是存储指令并且依赖于乘法，因此线程束 0 挂起。

假设大约有 18 ~ 22 个常驻线程束，在最后一个线程束调度最终乘法时，线程束 0 的乘法将已经完成。然后它能将存储指令调度到 16 个 LSU 中并且完成其执行。然后其他的线程束也会完成相同的工作，这样内核就完成了。现在考虑以下情况（参见图 9-12）：

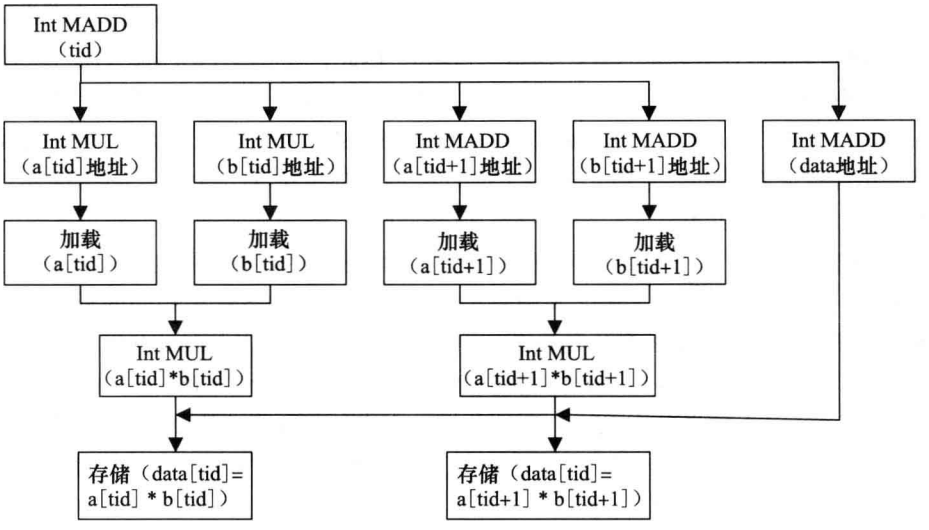

图 9-12　双数据流的依赖关系

```
int tid = (blockIdx.x * blockDim.x) + threadIdx.x;
data[tid] = a[tid] * b[tid];
data[tid+1] = a[tid+1] * b[tid+1];
```

通过将线程块的个数减半，我们可以让每个线程处理两个元素。注意，这将独立的执行流引入到了线程束的每个线程中。因此，算术操作开始与加载操作重叠。

然而，正如例子中的 C 代码所写的一样，这并不会有帮助。这是因为代码所包含的依赖并不是很明显。对 data 第一个元素的写操作可能会影响 a 数组或 b 数组中的元素。这是因为 data 的地址空间可能与 a 或 b 重叠。在数据流中有全局内存写操作时，你需要将读操作提前到内核开始处。试用以下的代码：

```
int tid = (blockIdx.x * blockDim.x) + threadIdx.x;
int a_0 = a[tid];
int b_0 = b[tid];
int a_1 = a[tid+1];
int b_1 = b[tid+1];
data[tid] = a_0 * b_0;
data[tid+1] = a_1 * b_1;
```

或者

```
int tid = (blockIdx.x * blockDim.x) + threadIdx.x;
int2 a_vect = a[tid];
int2 b_vect = b[tid];
data[tid] = a_vect * b_vect;
```

我们有两个选择，标量方法或向量方法。GPU 只在硬件上支持向量的加载和保存而不支持向量操作。因此，乘法操作实际上要像 C++ 中的重载操作符一样完成并且只是将两个相互独立的整数相乘。然而，向量方法分别执行两个 64 位加载和一个单独的 64 位存储而不是非向量版本

的4个独立的32位加载和一个的32位存储。因此，40%的内存事务被节省了。内存带宽使用是相同的，但是更少的内存事务意味着更小的内存延迟。因此，等待内存的总体失速时间减少了。

为了使用向量类型，只需声明数组为int2类型，它是一种两个整型的内置向量类型。被支持的类型为int2、int3、int4、float2、float3和float4。当然可以创建自己的类型（例如uchar4），并且定义自己的操作符。每一个向量类型实际上都是一个对齐的结构体，包含N个声明为基类型的成员元素。

因此，我希望你能真正看到不同类型的指令之间是需要平衡的。这在计算能力2.1的设备（GF104系列）上显得更为关键，其中有三组CUDA核共享SM中相同的资源。计算能力2.0转变为计算能力2.1设备的变化是显著地增加了SM的运算能力而没有提供额外的数据传输能力。计算能力2.0的设备在384位宽的总线上有多达512个CUDA核，核与内存带宽比为1∶3。计算能力2.1的设备在256位宽的总线上有多达384个CUDA核，其核与内存带宽比为1∶5。因此，计算能力2.0的设备更适合于内存密集应用程序，而计算能力2.1的设备更适合于计算密集型应用程序。

实际上，计算能力2.0的设备通过增加33%的CUDA核数达到了平衡。然而，计算能力2.1的设备通常在较高的时钟频率下执行，这包括内部时钟速度和外部内存总线速度。这有助于重新平衡较小的内存总线宽度，但是一般仍然不足以使计算能力2.1的设备超越计算能力2.0的设备。

重要的是要认识到（特别是对于计算能力2.1的设备），指令流需要有足够的计算密度以充分利用SM上的CUDA核。内核只是简单地执行加载/存储操作和少量的其他工作，因此无法达到这些设备可用的峰值性能。通过每个线程处理两个、4个或8个元素，从而扩展这些内核使其包括独立的指令流。因此尽可能地使用向量操作。

9.2.5 循环融合和内核融合

另一个可以显著地节省内存带宽的技术是基于9.2.4节看到的循环融合。循环融合是指两个明显独立的循环在一段范围内交错地执行。例如，循环1从0执行到100，循环2从0执行到200。至少在前100次迭代中，循环2的代码可以被融合到循环1的代码中。这样增加了指令级的并行，同样也减少了总体迭代次数的1/3。

内核融合是循环融合的演变。如果你有一系列按顺序执行的内核（一个接一个执行），这些内核的元素可以被融合吗？对于那些不是你编写的或没有完全理解的内核，这样做时千万要小心。调用两个连续的内核会在它们之间生成隐式地同步。这样设计可能是有目的，然而由于它是隐式的，可能只有原来的设计者才会意识到。

开发内核的时候，将操作分解成几个阶段或几轮是很常见的。例如，第一轮你可能针对整个数据集计算结果。第二轮，你可能使用特定的标准对数据进行过滤，然后在特定的点进行深入的处理。如果第二轮能够本地化到一个线程块，那么第一轮和第二轮通常能够组合成一个单独的内核。这就消除了将第一个内核写入主存、随后读取第二个内核的操作以及调用额外内核的开销。如果第一个内核能够将结果写入共享内存，那么只在第二轮需要这些结

果，这样就完全消除了对全局内存的读取、写入。归约操作经常被划分到这一类并且能从这样的优化中显著的受益，因为第二个阶段的输出通常比第一阶段的输出小很多，因此它显著地节约了内存带宽。

内核融合技术如此有效的部分原因是它所带来的数据重用。从全局内存取出数据的速度很慢，大约为 400 ~ 600 个时钟周期。考虑一下诸如从磁盘读取这样的内存访问。如果你曾经执行过任何磁盘 I/O 操作，可能会知道以每次读取一个字符的方式读取一个文件是非常慢的。使用 fread 读取大的文件块比重复调用 fgetch 这样每次读取一个字符的函数要高效很多。读取数据后，你就将它保存在了内存中。访问全局内存可以使用相同的方法。以每线程 16 字节（float4、int4）为一组读取数据而不是以单字节或字的方式。一旦你让一个线程成功地处理了一个单独的元素，则尝试切换到 int2 或 float2，然后处理两个。切换到 4 个可能有帮助也可能没有帮助，但是更经常的是由 1 个切换为两个。一旦有数据存储在共享内存或寄存器集中，那么就尽可能地重用它。

9.2.6 共享内存和高速缓存的使用

相比与全局内存，使用共享内存可以提供 10 : 1 的速度提升。但是共享内存的大小是受限的，在费米 / 开普勒设备上为 48KB，而之前的设备则是 16KB。这或许听起来不是一个很大的空间，特别是对于主机上的多个 GB 的内存系统，但实际上这是按每 SM 计算的。GTX580 或者 Tesla M2090 设备每 GPU 有 16 个活动的 SM，每 SM 可以提供 48KB 共享内存，总计 768KB。这是与一级缓存速度接近的内存。除此之外，你有 768KB 的二级缓存用于所有 SM 间共享（16 个 SM 设备上）。这样带来的结果就是全局存储和原子操作的速度比前一代的 GPU 上升了一个数量级。

当你考虑一个拥有 1.5GB 内存的 GTX580 设备时，768KB 意味着在任意时间点，内存空间中只有很小的一部分存放在缓存中。同级别的 Tesla 卡有 6GB 的内存。因此，在数据集上迭代的内核如果没有重用数据，那么需要意识到它们可能正在以低效地方式使用缓存或者共享内存。

与在一个大型数据集执行多轮不同，内核融合这样的技术可用于在数据间移动而非多次传入它。思考一下输出数据的问题而不是输入数据。构建该问题是将线程分配给输出数据项而不是输入数据项。在数据流方面，建立流入而非流出。优先选择聚集（收集数据）原语（gather primitive）而不是分散（分发数据）原语（scatter primitive）。GPU 会同时从全局内存和二级缓存直接将数据广播到每个 SM，这一点支持高速度聚集型的操作。

在费米架构和开普勒架构上，我们有一个非常有趣的选择：可以配置共享内存以优先使用一级缓存（48KB 一级缓存和 16KB 共享内存）或优先使用共享内存（48KB 共享内存和 16KB 一级缓存）。设备默认情况下是优先使用共享内存，因此你会有 48KB 的共享内存可用。这个设置并非是固定的，而是在运行时设定的，因此每次内核调用时可以设置它。那些不使用共享内存的内核或者保持 16KB 上限来确保与之前的 GPU 兼容的内核，通常通过启用额外的 32KB 缓存能显著地受益（10% ~ 20% 的性能提升），而在默认情况下是禁用的。配置方法如下：

```
cudaFuncSetCacheConfig(cache_prefer, kernel_name);
```

上句中，参数 cache_prefer 设置为 cudaFuncCachePreferShared，表示使用 48KB 共享内存和 16KB 一级缓存，而设置为 cudaFuncCachePreferL1 时，表示 48KB 一级缓存和 16KB 共享存储内存。注意，开普勒架构也允许 32KB/32KB 的划分。

但是，在一些领域中缓存致使费米架构和开普勒架构运行速度较上一代 GPU 慢。在计算能力 1.x 的设备上，如果数据项很小，则内存事务可以逐步将规模减小读取，直至每次访问 32 字节。因此，从十分分散的内存区域访问一个数据元素的内核，在任何基于缓存的架构，包括 CPU 或 GPU，表现会十分糟糕。原因在于单个元素的读取会载入 128 字节的数据。对于大多数程序来说，存入缓存的数据会在下一次循环迭代中命中，这是由于程序常常访问与之前访问过的数据临近的数据。因此对于大多数程序，这是一个显著的优点。但是，对那些只需要单个数据元素的程序来说，剩余的 124 字节是多余的。对于这种内核，你需要为内存子系统只读取其所需的内存事务而不是缓存行大小的，只能在编译的时候通过 -Xptxas–dlcm=cg 标志来完成此工作。这就将所有访问减少到每次事务 32 字节并且令一级缓存失效。对于只读数据，考虑使用纹理内存或者常量内存。

对于 G80/GT200 这些计算能力 1.x 的硬件，将共享内存作为内核设计不可或缺的一部分是很必要的。无法对数据进行缓存访问（通过共享内存显式的或通过硬件管理隐式的），内存延迟时间将会很长。开普勒架构中 GPU 上缓存的出现使得编写在 GPU 上运行较好的程序或内核变得十分容易。

让我们看一些使用共享内存时遇到的障碍。首先是可用空间的大小。在计算能力 1.x 硬件上是 16KB，而在计算能力 2.x 的硬件上多达 48KB。它可在编译时通过对变量添加 __shared__ 前缀来静态分配。它也是内核调用中的一个可选参数，即：

```
kernel<<<num_blocks, num_threads, smem_size>>>(a,b,c);
```

使用运行时分配，你需要额外的指向内存起始处的指针。例如：

```
extern volatile __shared__ int s_data[];

__global__ my_kernel(const int * a,
                     const int * b,
                     const int num_elem_a,
                     const int num_elem_b)
{
 const int tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 // Copy arrays 'a' and 'b' to shared memory
 s_data[tid] = a[tid];
 s_data[num_elem_a + tid] = b[tid];

 // Wait for all threads
 __syncthreads();

 // Process s_data[0] to s_data[(num_elem_a-1)] - a
 // Process s_data[num_elem_a] to s_data[num_elem_a + (num_elem_b-1)] - array 'b'
}
```

注意费米架构中二级缓存的大小并非总是像“CUDA C programmer guide”（CUDA C编程指南）中所说的768KB。事实上，二级缓存是基于所用设备的种类和现有SM的数量的。计算能力2.1设备的二级缓存少于计算能力2.0的设备。甚至没有启用所有SM的计算能力2.0设备（GTX470、GTX480、GTX570）的二级缓存也少于768KB。我们用来测试的GTX460设备的二级缓存为512KB，GTX470则为640KB。

二级缓存的大小作为l2CacheSize的成员在对cudaGetDeviceProperties API调用时返回。

9.2.7 本节小结

- 仔细考虑你的内核处理的数据并且如何将其以最佳的方式安排在内存中。
- 针对128字节的合并访问，优化访存模式，对齐到128字节的内存读取大小和一级缓存行大小。
- 注意权衡单、双精度及其对内存使用的影响。
- 在适当的时候将多内核融合成单内核。
- 以最适当的方式使用共享内存和缓存，以确保你能充分利用更高计算能力设备上的扩展容量。

9.3 策略3：传输

9.3.1 锁页内存

为对某一数据集进行操作，你需要将数据从主机传输到设备上、在数据集上进行操作，然后将结果传输回主机。由于是在完全串行的方式下执行的，这将导致主机和GPU在一段时间内都是闲置的，白白浪费了传输能力和计算能力。

在本章，我们详细介绍了多GPU的使用，包括如何使用流以确保GPU总是有工作可做。使用简单的双缓冲技术，尽管GPU正在将结果传输回主机并且请求一个新的工作包，但另一个缓冲仍然能被计算引擎用来处理下一个数据块。

主机处理器支持虚拟内存系统，其中物理内存页可以被标记为换出状态。然后将它更换到磁盘上。一旦主机处理器访问到该页，处理器将会从磁盘将该页加载回来。它允许程序员使用比硬件上实际空间更大的虚拟地址空间。考虑到程序通常有很好的局部性，这就可以使全部的内存空间比物理限制允许的大很多。然而，如果程序确实需要8GB而主机只有4GB，那么其性能通常是很差的。

可以认为使用虚拟内存是内存容量受限的时候留下的后遗症。今天可以花100多欧元（美元、英镑）就能购买16GB的内存，这意味着对大多数应用程序来说，主机基本上不再需要虚拟内存了。

除了大数据问题，大多数程序一般是能够放入主机内存空间的。如果不行，还有特殊的服务器解决方案，即每节点可以容纳高达128GB内存。这样的解决方案通常是比较值得推荐的，因为它允许你将数据保存在一个节点上而不像多节点解决方案一样增加复杂度。当然，

以块的方式加载数据集也是完全可行的，但是这样你最终会受到 I/O 硬件吞吐量的限制。

你应该总是使用有较大数量的主机内存的系统上的锁页内存。锁页内存允许 GPU 上的 DMA（直接内存访问）控制器请求主机内存传输而不需 CPU 主机处理器的参与。因此，在管理传输或从磁盘将换出的页面调回时，没有加载操作需要劳烦主机处理器处理。

PCI-E 传输实际上只能使用基于 DMA 的传输执行。在不直接使用锁页内存时，驱动程序会在后台执行。因此，驱动程序必须分配一块锁页内存，执行一个从常规内存到锁页内存的主机端复制操作、初始化传输、等待传输完成，然后释放锁页内存。所有这些操作都花费一定时间并且会消耗宝贵的 CPU 周期，而这些 CPU 周期本可以更加有效地使用。

在 GPU 上分配的内存默认情况下为锁页内存，这只是因为 GPU 不支持将内存交换到磁盘上。我们关心的是在主机处理器上如何分配锁页内存。为了分配锁页内存，我们需要使用特殊的 cudaHostMalloc 函数或者使用常规的 malloc 函数，然后将其注册为锁页内存。

注册内存只是设置一些内部标志以确保内存从不被换出，并且告诉 CUDA 驱动程序，该内存为锁页内存，所以能够直接使用它，而不需要使用一个临时的缓冲区。

像 malloc 一样，如果使用 cudaHostAlloc，则需要使用 cudaFreeHost 函数释放这一块内存。不要调用常规的 C 语言函数释放 cudaHostAlloc 分配的内存，否则程序随后可能崩溃或出现一些未定义的行为和不常见的错误。

cudaHostAlloc 函数的原型是：

```
cudaError_t cudaHostAlloc (void ** host_pointer, size_t size, unsigned int flags)
```

其中，在特定的硬件有如下标志：

cudaHostAllocDefault——大多数情况下使用，简单地指定默认行为。

cudaHostAllocWriteCombined——用于只被传输到设备的内存区域。当主机要从这块内存区域读取时不要使用这个标志。它在主机处理器上关闭了内存区域的缓存，这意味着在传输时它完全忽略了内存区域。这在特定的硬件配置上能够加速到设备的传输。

cudaHostAllocPortable——锁页内存在所有 CUDA 上下文中变成锁页的和可见的。默认情况下，内存分配属于创建它的上下文。如果你打算在 CUDA 上下文之间或主机处理器的线程之间传递指针，则必须使用这个标志。

cudaHostAllocMapped——我们不久就会看到它。它将主机内存分配到设备内存空间，这允许 GPU 内核直接读取和写入，所有的传输将隐式地处理。

为了演示分页内存与非分页内存效果的不同，我们编写了一个简短的程序。它只是执行一些到达设备或来自设备的大小不一的传输，并调用一个虚拟内核以确保传输确实发生了。其结果如图 9-13 所示。

Y 轴为传输速度，单位是 MB/s；X 轴为传输数据大小，单位为字节。从图 9-13 中我们可以看到，使用分页内存和非分页内存有显著的区别。锁页内存写入速度快 1.4 倍，读取速度快 1.8 倍。使用锁页内存将 512MB 数据发送到显卡花费 194ms，相反，使用换页内存为 278ms。从设备端传输数据的时间，换页内存为 295ms，相比而言，锁页内存为 159ms。

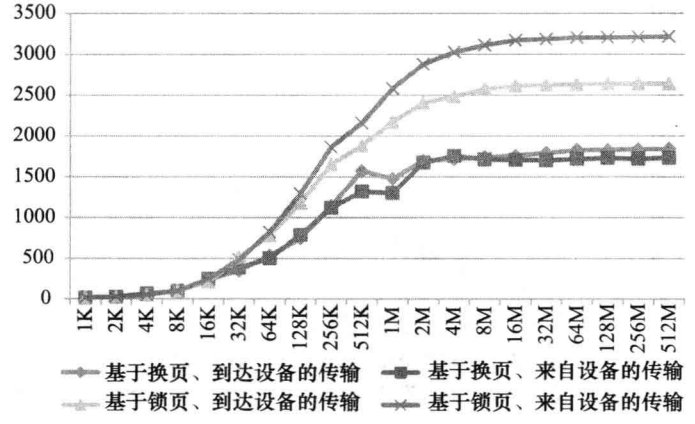

图 9-13 到达设备和来自设备的传输速度（AMD Phenom II X4 905e，PCI-E 2.0 X8 链路）

在输入端，我们看到了一个奇怪的问题。使用锁页内存，来自设备的带宽比到达设备的带宽高 20%。由于 PCI-E 提供两个方向（来自设备以及传输到设备的）速度相同的全双工通信，你可能期望读取和写入的传输速度相似。就像将在随后的例子中看到的一样，这种差异是依赖于硬件的。除了 Intel Nehalem I7，所有测试系统都显示出不同程度的差异。

4 种设备的传输速率（到达设备和来自设备的）几乎是相同的，这是意料之中的，因为所有显卡上的全局内存带宽至少比 PCI-E 带宽大一个数量级。

同样值得注意的是，要获得接近峰值的带宽（即使使用锁页内存），传输数据的大小需要是 2MB 左右。事实上，只有传输数据大小为 16MB 或以上时我们才能获得绝对峰值带宽。

为了进行比较，图 9-14、图 9-15 和图 9-16 也给出了我们对一些系统进行测试的结果。

图 9-14 显示了一个基于英特尔低功耗 ATOM 设备的小型上网本，它配备了专用的 NVIDIA ION 显卡 GT218。使用 2.0 X16 链路时，通常可以看到 PCI-E 峰值带宽高达 5GB/s。由于这款上网本采用 X1 链路，我们预期的最大值为 320MB/s 而实际我们所看到的大约为 200MB/s。

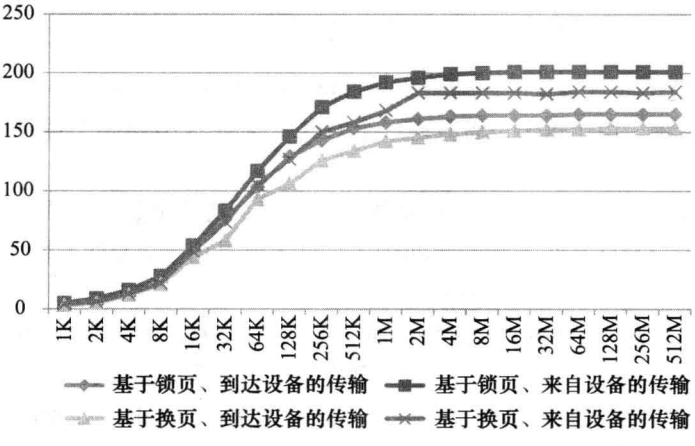

图 9-14 到达设备和来自设备的传输速度（英特尔 Atom D525，PCI-E 2.0 X1 链路）

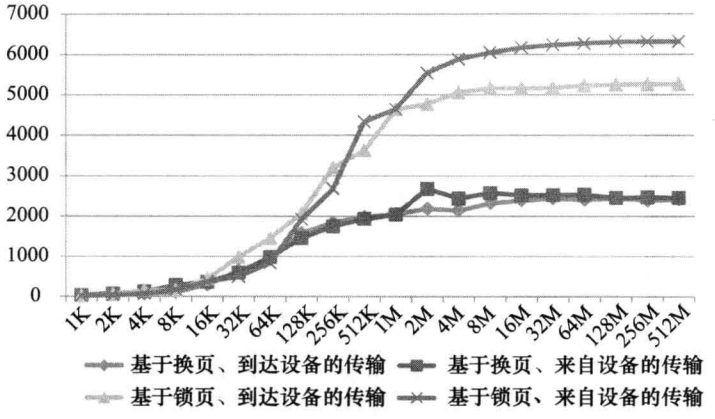

图 9-15 到达设备和来自设备的传输速度（英特尔 I3540，PCI-E X16 链路）

然而，我们看到了与 AMD 系统非常类似的模式。因为在开始达到峰值传输速率时，我们需要大约 2MB 的传输大小。我们观察到的唯一不同是到达设备和来自设备的传输差别。

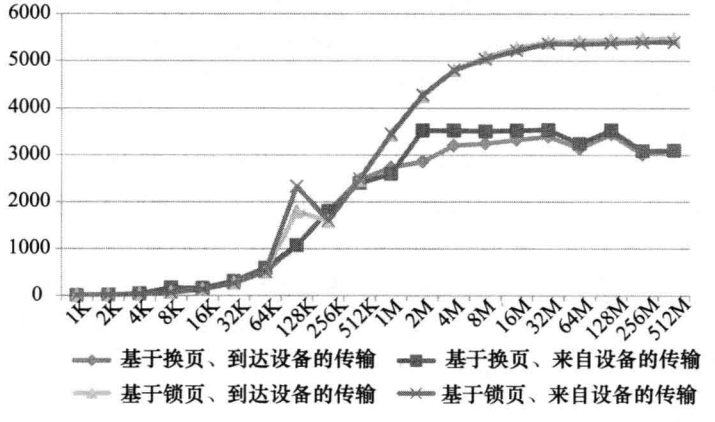

图 9-16 到达设备和来自设备的传输速度（英特尔 I7 920，PCI-E X16 链路）

在消费级环境中非常常见的中端系统是 Intel 的 i3/i5 系统。图 9-15 显示的是运行在 H55 芯片组的 i3540 系统。由于这个设备只有一个 GPU，它在 X16 链路上运行时达到了 PCI-E2.0 的峰值速度（参见图 9-15）。

我们再一次看到了锁页和非锁页传输的很大不同，其差距超过了两倍。然而，请注意绝对速度的差异，在 AMD 系统上大约是两倍多。这主要是由于 AMD 系统使用了 X8 PCI-E 链路，而这里的 Intel 系统使用了 X16 PCI-E 链路。

Intel I3 是一个典型的消费级处理器。所有编写基于消费级应用程序的人都应该清楚，他们需要使用锁页内存传输，因为我们已经看到它带来了很大的速度提升。

最后，我们看一看服务器场合。它使用了 Intel I7920 Nehalem 处理器和 ASUS 超级计算机 1366 插槽的主板。该主板是高端 GPU 经常使用的，它允许多达 4 个 PCI-E 插槽。该系统

配备了三个 GTX290 GPU，每一个 GPU 使用 PCI-E 2.0 X16 连接。

我们再一次从图 9-16 中看到了很有趣的一点。在传输小于 512KB 的数据时，锁页和换页内存传输的速度是相等的。当数据大于 512KB 时，锁页内存传输的速度约为基于换页内存传输速度的 1.8 倍。与 Nehalem I3 系统不同，请注意 Nehalem I7 系统在两个传输方向上更加一致，传入和传出的速度并没有明显的不同。然而，尽管设备都在 X16 PCI-E 2.0 链路上，我们仍然注意到，峰值传输速度仅为 5400MB/s，而与之相比，I3 达到了 6300MB/s 的峰值速度（参见图 9-16）。

因此，我们可以说，选择当前可用硬件时，锁页内存的传输速度大约为非锁页内存传输的两倍。同样，我们看到不同设备的读写速度也有显著的不同。我们需要使用更大的（而不是更小的）内存块，可以考虑将多个传输组合起来以增加总线的整体使用带宽。

9.3.2 零复制内存

零复制内存是一种特殊形式的内存映射，它允许你将主机内存直接映射到 GPU 内存空间上。因此，当你对 GPU 上的内存解引用时，如果它是基于 GPU 的，那么你就获得了到全局内存的高速带宽（180GB/s）。如果 GPU 代码读取一个主机映射变量，它会提交一个 PCI-E 读取事务，很长之间之后主机会通过 PCI-E 总线返回数据。

在上一节考查过 PCI-E 总线带宽之后，乍看之下，这并不合理。大的传输是高效的而小的传输效率很低。如果我们再次执行之前例子中使用的测试程序，可以看到 AMD Phenom X4 平台传输时间的中值为 0.06ms。然而，这些是显式的、独立的传输，因此很可能零复制内存的实现更高效。

计算能力 2.x 的硬件访问全局内存时，整个缓存行被从内存取出。即使是在计算能力 1.x 的硬件上，同样的 128 字节（很可能减少到 64 字节或 32 字节）也从全局内存取出。

NVIDIA 并没有公布其使用的 PCI-E 传输大小以及零复制技术的具体实现细节。然而，全局内存使用的合并方法也可以用于 PCI-E 传输。如果计算密度很大，足够隐藏 PCI-E 传输延迟，那么线程束内存延迟隐藏模型同样也可以用于 PCI-E 传输。实际上，这是使其工作的关键。如果对每次全局内存的读取量很小并且程序已经是内存密集型的，那么这种方法可能不会有帮助。

然而，如果你的程序是计算密集型的，那么零复制内存可能是一项非常有用的技术。它节省了设备显式传输的时间。事实上，是将计算与数据传输操作重叠了，而且无须执行显式的流管理。当然，问题是必须高效地使用数据。如果你多次读取或写入相同的数据，这会创建多个 PCI-E 传输事务。这些事务的延迟都是很高的，因此执行的次数越少越好。

这也可以高效地用在主机和 GPU 共享同一内存空间的系统中。例如，低端的基于 NVIDIA ION 的上网本。这里对 GPU 全局内存的分配操作实际上会导致在主机内存分配空间。很显然，从主机的一个内存区域复制到主机内存的另一个区域是没有意义的。零复制内存能够避免在系统中执行这些复制并且不会影响 PCI-E 总线传输。

零复制内存也有一个非常有用的使用场合。就是将 CPU 应用程序移植到 GPU 的初始阶段。在这个开发阶段，经常会有主机上的若干段代码没有被移植到 GPU。将这样的数据声明为零复制内存区域就能够允许代码整段地移植并且仍然能工作。在所有代码都真正移植到在 GPU 之前，程序的性能通常是很差的。它允许更小的移植步骤，所以这不是一个要么全做要么全不做的问题。

让我们从现有的 memcpy 程序开始并且扩展内核，这样它就会读取数据而无须依赖显式的复制操作。对此，我们一定要对内存进行合并访问，这样读取一个简单的一维数组是很容易的。因此，我们的内核变成这样：

```
__global__ void kernel_copy(u32 * const gpu_data,
                            const u32 * const host_data,
                            const u32 num_elements)
{
 const u32 idx = (blockIdx.x * blockDim.x) + threadIdx.x;
 const u32 idy = (blockIdx.y * blockDim.y) + threadIdx.y;
 const u32 tid = ((gridDim.x * blockDim.x) * idy) + idx;

 if (tid < num_elements)
   gpu_data[tid] = host_data[tid];
}
```

在内核中，我们只是将网格维度 x 和 y 变成单独的线性数组并且将源数据集的一个元素赋值到目的数据集。然后使用零复制或主机映射内存做三件重要的事情。第一是启用它，第二是使用它分配内存，最后将常规的主机指针转换成指向设备内存空间的指针。

我们需要在任何 CUDA 上下文创建之前进行下面的调用：

```
// Enable host mapping to device memory
CUDA_CALL(cudaSetDeviceFlags(cudaDeviceMapHost));
```

当 CUDA 上下文被创建时，驱动程序会知道它需要支持主机映射内存（host-mapped memory）。没有驱动程序的支持，主机映射（零复制）内存将无法工作。如果该支持在 CUDA 上下文创建之后完成，内存也无法工作。请注意对 cudaHostAlloc 这样的函数的调用，尽管在主机内存上执行，也仍然创建一个 GPU 上下文。

虽然大多数设备支持零复制内存，但是一些早期的设备却不支持。这并不是计算能力的一部分，因此需要显式地检查：

```
struct cudaDeviceProp device_prop;
CUDA_CALL(cudaGetDeviceProperties(&device_prop, device_num));
zero_copy_supported = device_prop.canMapHostMemory;
```

下一个阶段是分配主机上的内存，这样它就可以被映射到设备内存。我们对 cudaHostAlloc 函数使用额外的标志 cudaHostAllocMapped 就可以实现。

```
// Allocate zero copy pinned memory
CUDA_CALL(cudaHostAlloc((void **) &host_data_to_device, size_in_bytes,
cudaHostAllocWriteCombined | cudaHostAllocMapped));
```

最后，我们需要通过 cudaHostGetDevicePointer 函数将主机指针转换成指向设备的指针：

```
// Convert to a GPU host pointer
CUDA_CALL(cudaHostGetDevicePointer(&dev_host_data_to_device,host_data_to_device,0));
```

在这个调用中，我们将之前在主机内存空间分配的 host_data_to_device 转换成 GPU 内存空间的指针。不要将两种指针混淆。在 GPU 内核中，只使用转换后的指针；原始的指针只出现在主机上执行的代码中。因此，为了之后释放内存，需要在主机上执行一个操作，其

他的调用保持不变：

```
// Free pinned memory
CUDA_CALL(cudaFreeHost(host_data_to_device));
```

由于我们使用的内存块大小为 512MB，在不管每个线程块分配多少线程的情况下，为了使每个线程都可以访问一个元素，则意味着线程块个数要超过 64K。这个线程块数量超出了任何单一维度的硬性限制。我们不得不引入另一个维度，因此采用了第 5 章已经提到过的线程网格。我们可以很简单地将线程网格数固定为某个足够大的、允许我们灵活地选择每个线程块线程个数的值。

```
const int num_elements = (size_in_bytes / sizeof(u32));
const int num_threads = 256;
const int num_grid = 64;
const int num_blocks = (num_elements + (num_threads-1)) / num_threads;
int num_blocks_per_grid;

// Split blocks into grid
if (num_blocks > num_grid)
 num_blocks_per_grid = num_blocks / num_grid;
else
 num_blocks_per_grid = 1;

dim3 blocks(num_grid, num_blocks_per_grid);
```

dim3 操作只是将我们计算出的常规标量数值赋给一个包含三个元素的结构体类型，这个结构体类型可以在内核启动时当做一个参数。它会导致内核启动 64 个包含 N 个线程块的线程网格。这确保任意一个给定的线程块索引不会超过 64K 的限制。因此，在内核启动时我们使用 blocks（dim3 类型）替换 num_blocks（标量类型）：

```
// Run the kernel
kernel_copy<<<blocks,num_threads>>>(gpu_data,dev_host_data_to_device,num_elements);
```

我们看到，到达设备的传输的整体性能与使用显式内存复制传输的性能相同。这是个很重要的暗示。大多数不使用流 API 的应用程序只是开始时将内存复制到 GPU，然后在内核完成时再复制回主机。使用锁页内存复制，我们可以显著地缩减这个时间。但是这个时间仍然是增加的，因为它是串行操作。

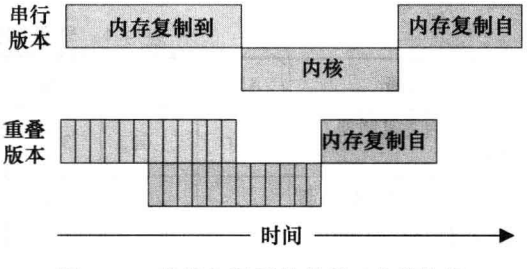

图 9-17 串行与重叠的传输 / 内核执行

实际上，使用零复制内存，我们将传输和内核操作分解成更小的块，然后以流水线的方式执行它们（参见图 9-17）。整体时间减少得非常显著。

请注意，我们并没有对来自设备的复制操作执行同样的优化。这样做是因为消费者级 GPU 只有一个启用了复制引擎。因此，它们只支持一个内存流。当你执行读取操作、内核操作、写入操作时，如果写操作在随后的读操作之前放入流中，该写操作会阻塞读操作直到挂

起的写操作完成。请注意，对于 Tesla 设备却不是这样的。由于两个复制引擎都启用了，因此 Tesla 显卡能够支持相互独立的流。在费米架构之前，所有显卡都只有一个复制引擎。

然而，采用零复制内存的传输实际上是相当小的。PCI-E 总线在两个方向上的带宽都是相同的。由于基于 PCI-E 的内存读取存在高延迟，因此实际上大多数读操作应该在所有写操作之前被放入读队列。我们可能获得比显式内存复制版本节省大量执行时间的效果。

请注意图 9-18 中显示的是简化过的，它列出了单独的“到达 / 来自设备的锁页内存访问”曲线，然而我们却显示了每个设备的零复制时间。锁页内存的时间实际上对于所有设备是相同的，所以它并没有显示每个设备。

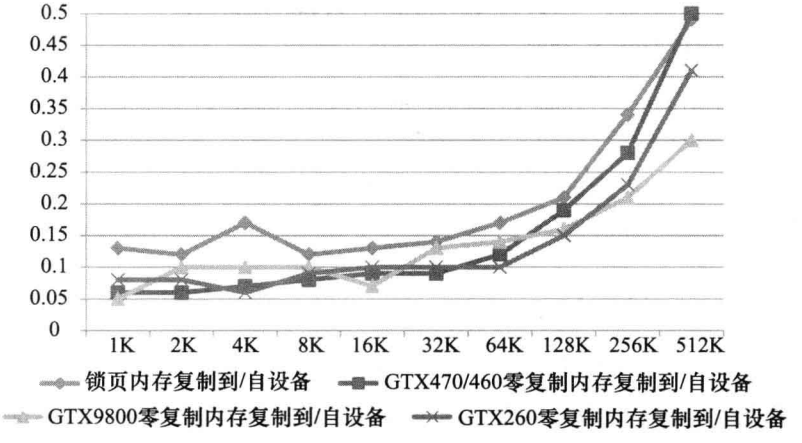

图 9-18 不同代 GPU 上的零复制时间与显式锁页复制时间

我们已经列出了内存复制到设备、内核执行、从设备复制回内存操作的整体执行时间。因此，单纯地测量设备传输时间会忽略其他的开销。由于我们使用了零复制内存，内存事务和内核时间则不能够分开。然而，由于内核的工作很少，因此零复制和显式内存复制版本的整体执行时间差别并不明显。

这里有相当大的可变性。然而，我们看到的是对于数据量较小的传输（小于 512KB），零复制比使用显式复制更快。现在我们看看表 9-1 和图 9-19 中比 512KB 大的数据传输情况。

表 9-1 零复制运行结果（时间单位为 ms）

设备	1M	2M	4M	8M	16M	32M	64M	128M	256M
锁页内存复制到 / 自设备	0.96	1.62	3	5.85	11.52	22.97	45.68	91.14	182.04
GTX470/460 零复制内存复制到 / 自设备	0.5	0.9	1.72	3.34	6.61	13.11	26.15	52.12	103.99
GTX9800 零复制内存复制到 / 自设备	0.56	1.09	2	3.94	7.92	15.63	31.6	61.89	123.81
GTX260 零复制内存复制到 / 自设备	0.68	1.38	2.52	4.48	8.74	18.84	38.96	74.35	160.33

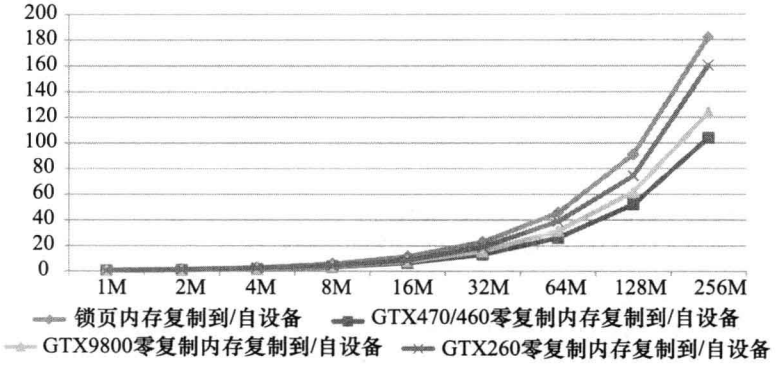

图 9-19 零复制曲线图（时间（ms）与传输大小）

很有趣的是，在这里我们看到执行时间大幅减少。在费米架构硬件上，内核操作和内存复制操作的重叠执行使执行时间从 182ms 减少到 104ms，达到了 1.75 倍的加速。早期设备的结果并没有如此显著，但仍体现出了明显的加速。

当然，你也可以像第 8 章一样使用流和异步内存复制实现。零复制只是提供了一种选择，一个更简单的使用接口。

不过，也有一些注意事项。要知道数据到底需要从内存读取多少次。重新从全局内存中读取数据通常不使用零复制内存。

如果我们在 9800GT 和 GTX 260（计算能力 1.x 的设备）平台上修改程序，从主机内存读取值两次而不是一次，那么其性能会减少到原来的一半。这是因为平台上每一个从全局内存的读取都没有被缓存。因此 GPU 访问零复制内存区域的时间加倍了，从而使提交的 PCI-E 事务的个数翻倍。

而在费米架构设备上的情况却有所不同。它有一个一级和二级缓存，很可能之前内核取出的数据，在后来访问相同的内存地址时仍然在缓存中。可以肯定，你需要显式地将打算重新使用的数据复制到共享内存。因此，在费米架构设备中，取决于数据模式，通常看不到设备提交多个 PCI-E 事务，因为这些事务中很多命中了内部缓存，因此不会创建全局内存事务。

因此，只要关心数据重用并且对每个数据项执行了足够的工作，那么零复制内存就可以很容易地对现有的串行代码进行加速而无须深入地学习流 API。

然而，要知道 PCI-E 总线的带宽远小于 CPU 上的可用带宽。最新的沙桥 i7 处理器（Socket 2011）能达到大约 37GB/s 的内存带宽，其理论峰值为 51GB/s。在 PCI-E2.0 总线上，我们从 8GB/s 的理论峰值获得了 5 ~ 6GB/s 的带宽。必须在应用程序中花费大量的工作验证 PCI-E 总线上转移数据的代价。在每个元素执行很少工作的情况下，CPU 可能是更好的选择。

用于这些测量的程序如下所示，以供参考。

```
void memcpy_test_zero_to_from(const int device_num,
                              const size_t size_in_bytes,
                              TIMER_T * const kernel_time,
                              const u32 num_runs,
                              const bool pinned)
```

```
{
 char device_prefix[256];
 int major, minor;
 int zero_copy_supported;

 // Init

 // Enable host mapping to device memory
 CUDA_CALL(cudaSetDeviceFlags(cudaDeviceMapHost));

 // Get the device properties
 get_device_props(device_prefix, device_num, &major,
                  &minor, &zero_copy_supported);

 // Exit if zero copy not supported and is requested
 if (zero_copy_supported == 0)
 {
   printf("%s Error Zero Copy not supported", device_prefix);
   wait_exit(1);
 }

 // Select the specified device
 CUDA_CALL(cudaSetDevice(device_num));

 printf("%s Running Memcpy Test to device using",
        device_prefix);

 if (pinned)
     printf(" locked memory");
 else
   printf(" unlocked memory");

 printf(" %lu K", size_in_bytes / 1024);

 (*kernel_time) = 0;

 init_device_timer();

 // Allocate data space on GPU
 u32 * gpu_data;
 CUDA_CALL(cudaMalloc((void**)&gpu_data,
                      size_in_bytes));
 u32 * dev_host_data_to_device;
 u32 * dev_host_data_from_device;

 // Allocate data space on host
 u32 * host_data_to_device;
 u32 * host_data_from_device;

 if (pinned)
 {
```

```
   // Allocate zero copy pinned memory
   CUDA_CALL(cudaHostAlloc((void **) &host_data_to_device, size_in_bytes,
cudaHostAllocWriteCombined | cudaHostAllocMapped));

   CUDA_CALL(cudaHostAlloc((void **) &host_data_from_device, size_in_bytes,
cudaHostAllocDefault | cudaHostAllocMapped));
 }
 else
 {
   host_data_to_device = (u32 *) malloc(size_in_bytes);
   host_data_from_device = (u32 *) malloc(size_in_bytes);
 }

 // Convert to a GPU host pointer
 CUDA_CALL(cudaHostGetDevicePointer(&dev_host_data_to_device,host_data_to_
 device,0));
 CUDA_CALL(cudaHostGetDevicePointer(&dev_host_data_from_device,host_data_from_
 device,0));

 // If the host allocation did not result in
 // an out of memory error
 if ( (host_data_to_device != NULL) &&
      (host_data_from_device != NULL) )
 {
   const int num_elements = (size_in_bytes / sizeof(u32));
   const int num_threads = 256;
   const int num_grid = 64;
   const int num_blocks = (num_elements + (num_threads-1)) / num_threads;
   int num_blocks_per_grid;

   // Split blocks into grid
   if (num_blocks > num_grid)
       num_blocks_per_grid = num_blocks / num_grid;
   else
       num_blocks_per_grid = 1;

   dim3 blocks(num_grid, num_blocks_per_grid);

   for (u32 test=0; test < num_runs+1; test++)
   {
    // Add in all but first test run
    if (test != 0)
     start_device_timer();

    // Run the kernel
    kernel_copy<<<blocks, num_threads>>>(dev_host_data_to_device,
dev_host_data_to_device, num_elements);

    // Wait for device to complete all work
    CUDA_CALL(cudaDeviceSynchronize());
```

```
        // Check for kernel errors
        cuda_error_check(device_prefix, " calling kernel kernel_copy");

        // Add in all but first test run
        if (test != 0)
           (*kernel_time) += stop_device_timer();
      }

      // Average over number of test runs
      (*kernel_time) /= num_runs;

      if (pinned)
      {
        // Free pinned memory
        CUDA_CALL(cudaFreeHost(host_data_to_device));
        CUDA_CALL(cudaFreeHost(host_data_from_device));
      }
      else
      {
        // Free regular paged memory
        free(host_data_to_device);
        free(host_data_from_device);
      }
    }

    CUDA_CALL(cudaFree(gpu_data));
    destroy_device_timer();

    // Free up the device
    CUDA_CALL(cudaDeviceReset());

    printf(" KERNEL:%.2f ms", (*kernel_time));

    const float one_mb = (1024 * 1024);
    const float kernel_time_for_one_mb = (*kernel_time) * (one_mb / size_in_bytes);

    // Adjust for doing a copy to and back
    const float MB_per_sec = ((1000.0F / kernel_time_for_one_mb) * 2.0F);

    printf(" KERNEL:%.0f MB/s", MB_per_sec );
  }
```

9.3.3 带宽限制

对于绝大多数的程序而言，最终的带宽限制来源于设备获取输入数据和写回输出数据的I/O速度。这也是应用程序无法继续加速的限制。如果你的程序在串行CPU实现下运行需要消耗20分钟，并且具有一定的并行性，那么很有可能该应用程序通过GPU实现后所消耗的时间比它从当前存储设备上加载和保存数据所消耗的时间还要少。

在带宽方面我们遇到的第一个问题就是简单地从机器上存入和取出数据。如果你使用的是网络连接存储，那么该限制还包含网络链路传输速度的限制。对于这个问题，最好的解决方案就是使用一个包含多个高速固态硬盘的高速 SATA3 RAID 控制器。然而，除非对硬盘的使用非常高效，否则这种方案也无法解决带宽问题。每个硬盘传输数据到主机端内存的速度都存在一个峰值，其实际上是硬盘、控制器、路由器到主机端内存的传输速率的一个函数。

对硬盘上执行一个标准检查程序，比如常见的 ATTO 检查程序。这样我们就能看到使用不同大小的块时产生的影响（参见图 9-20）。这个标准测试模拟了对硬盘基于不同大小块的读 / 写访问。程序分别以 1KB，2KB，4KB 等大小的块对一个 2GB 的文件进行读 / 写操作，以此来观察使用不同大小的块所带来的影响。

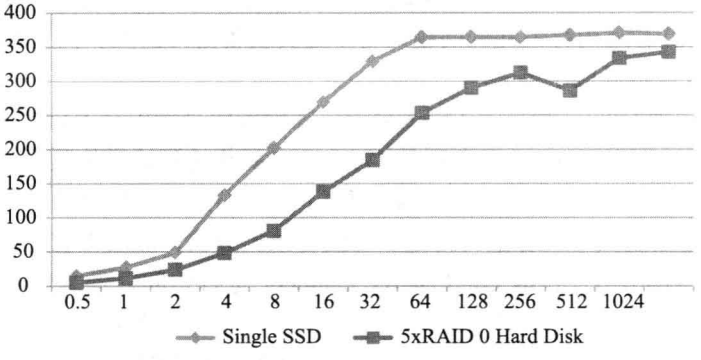

图 9-20　单个固态硬盘与由五块 RAID0 磁盘的带宽对比

从实验结果中我们看到，只有以大小为 64KB 的块或更大的块为单位读取数据时，单个固态硬盘的带宽才会到达峰值。对于 RAID0 磁盘系统，我们至少需要使用 1MB 大小的数据块才能充分利用多个磁盘。因此，当我们从硬盘系统读取适当大小的数据块时要保证使用的是 C 语言的 fread 函数。如果我们以大小为 1KB 的数据块为单位获取数据，传输速率能达到 24MB/s，比它读取数据的峰值带宽低 10%。RAID 系统中使用的磁盘越多，基本块的大小就可以越大。如果处理压缩的音乐或者图像文件，那么单个文件的大小可能只有几兆字节。

注意，数据是否可压缩对硬盘的性能有很大影响。服务器级别的硬盘，如 OCZ Vertex 3，对于不可压缩的数据可以提供更高的峰值和持续的带宽。因此，如果你的数据集已是压缩格式（MP3、MP4、WMV、H.264、JPG 等），那么你需要确保使用服务器硬盘。当处理不可压缩的数据流时，许多消费级的固态硬盘的带宽会下降到硬盘出厂时标注的峰值带宽的一半。

出现这种情况的原因在于使用高端服务器固态硬盘的同步 NAND 内存与使用消费级固态硬盘中的廉价而低性能的同步 NAND 存储器的不同。与处理未经压缩的数据一样，基于 NAND 的同步硬盘的性能优于同系列的异步硬盘，特别是硬盘开始保存数据的时候。OCZ 还提供了基于 PCI-E 的 RevoDrive R4 产品，其以占据 PCI-E 插槽为代价，使传输速度达到 2GB/s 以上。

遇到的另一个带宽限制是主机内存速度的限制。但如果在单个计算机节点引入多个 GPU，考虑到我们能够以 6GB/s 的速度通过 PCI-E 总线从高速固态硬盘系统获取数据，这

将不再是什么大问题。之后我们需要以6GB/s的速度从主机端内存发送数据到GPU端或从GPU端将数据写回到主机端。我们也可以再次以6GB/s的速度将数据写回到RAID控制器。这样，在CPU专门传输数据的情况下，我们将达到24GB/s的纯数据传输。现在，我们已经达到了大多数现代处理器的最大带宽限制并且已经突破了老一代CPU的带宽限制。实际上，如果我们想要解决缓慢的I/O设备问题，只有最新的4通道I7 Sandybridge-E CPU才有可能满足我们需求的带宽。

CUDA 4.0 SDK引入了点对点的GPU通信。CUDA 4.1 SDK同样在非NVIDIA硬件上也引入了点对点通信。因此，在适当的硬件上，GPU可以与任何支持的设备通信。而这也受到了InfiniBand以及其他高速网卡数量的限制。然而，原则上任何PCI-E设备都支持与GPU通信。因此，一个RAID控制器能够直接向GPU发送数据或从GPU接收数据。由于不存在主机端内存带宽、PCI-E或内存消耗的问题，对于这类设备潜力巨大。由于数据不必传入CPU然后再传回，因此延迟降低了很多。

一旦数据传输到GPU，对于GeForce显卡，在设备的全局内存上读/写数据时的带宽上限将达到190GB/s，而对于Tesla显卡则是177GB/s。要想达到这样的速度，你必须确保从线程读取数据时合并访问，并且保证程序100%利用了从内存传输到GPU上的数据。

最后，我们还需对共享内存进行讨论。即使将数据分块，然后将它们存入共享内存，接着以无存储体冲突的方式（bank conflict-free manner）进行访问，带宽也限制在1.3TB/s内。考查带有64KB一级缓存模块的AMD PhenomII与Nehalem I7处理器，其带宽最高可达330GB/s，只有同等大小一级缓存和共享内存的GPU最大带宽的25%左右。

以一个典型的4字节宽的浮点数或者整型数为例。全局内存带宽每秒最多可获取47.5G个元素（190GB/S÷4）。假定只读/写一个值，那么每秒处理的元素数减半为23.75G个，如果不存在数据重用，这将是应用程序的最大吞吐量。

费米架构设备的额定速率为每秒处理超过1T次的浮点运算，即每秒能够处理大约1000G次的浮点运算。开普勒架构设备的额定速率为每秒处理超过3T次的浮点运算。而实际的速率取决于你如何测量每秒执行的浮点运算次数。最快的测量方式是测FMADD指令（浮点数乘法、加法指令），即将两个浮点数相乘，然后将其结果与另外一个数相加。诸如这样，就要计算成两次浮点数运算而不是一次。实时指令流可将内存加载、整型计算、循环、分支等混合。因此，实际上，内核达不到这个峰值。

我们可以通过简单地调用之前开发的PCI-E带宽的可视化程序来测量实时速率。通过简单地执行全局内存间的内存复制操作，即可看到内核能够达到的最大读/写速度。

```
GTX 470: 8 bytes x 1 K (1x4x32)   0.060 ms, 489 MB/s
GTX 470: 8 bytes x 2 K (1x8x32)   0.059 ms, 988 MB/s
GTX 470: 8 bytes x 4 K (1x16x32)  0.060 ms, 1969 MB/s
GTX 470: 8 bytes x 8 K (1x32x32)  0.059 ms, 3948 MB/s
GTX 470: 8 bytes x 16 K (1x32x64)  0.059 ms, 7927 MB/s
GTX 470: 8 bytes x 32 K (1x64x64)  0.061 ms, 15444 MB/s
GTX 470: 8 bytes x 64 K (1x64x128) 0.065 ms, 28779 MB/s
GTX 470: 8 bytes x 128 K (1x64x256) 0.074 ms, 50468 MB/s
```

```
GTX 470: 8 bytes x 256 K (1x128x256) 0.090 ms, 83053 MB/s
GTX 470: 8 bytes x 512 K (1x256x256) 0.153 ms, 98147 MB/s
GTX 470: 8 bytes x 1 M (1x512x256) 0.30 ms, 98508 MB/s
GTX 470: 8 bytes x 2 M (1x1024x256) 0.56 ms, 105950 MB/s
GTX 470: 8 bytes x 4 M (1x2048x256) 1.10 ms, 108888 MB/s
GTX 470: 8 bytes x 8 M (1x4096x256) 2.19 ms, 112215 MB/s
GTX 470: 8 bytes x 16 M (1x8192x256) 4.26 ms, 112655 MB/s
GTX 470: 8 bytes x 32 M (1x16384x256) 8.48 ms, 113085 MB/s
GTX 470: 8 bytes x 64 M (1x32768x256) 16.9 ms, 113001 MB/s
GTX 470: 8 bytes x 128 M (2x32768x256) 33.9 ms, 112978 MB/s
GTX 470: 8 bytes x 256 M (4x32768x256) 67.7 ms, 113279 MB/s
```

注意，括号中的值表示线程网格数 × 线程块数 × 线程数。上述数字绘制在图 9-21 中。

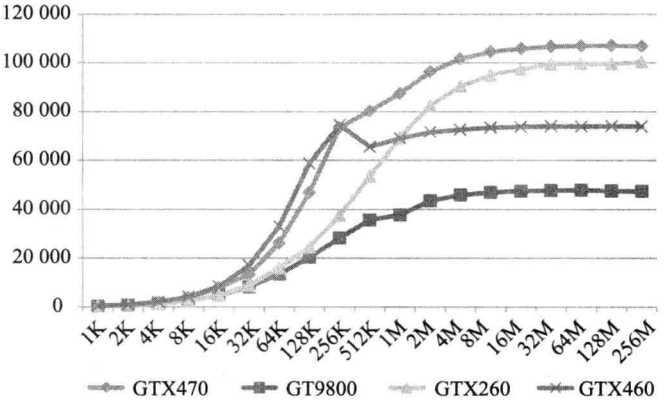

图 9-21 不同设备的全局内存带宽

这些结果是通过将 16 个内核压入到异步流中执行获得的。每个内核开始调用处有一个开始事件，调用结束后有一个停止事件。每个内核中的每个线程只完成将源存储空间内的单个元素复制到目标存储空间。每次批处理的第一个内核的执行时间忽略不计。通过剩下的内核计算得到总运行时间，再求平均运行时间。GTX 470 的额定带宽是 134GB/s，尽管内核很简单并且以最大传输规模进行数据传输，但仍达不到额定带宽。

从上表中可以看出，为达到内存性能峰值，线程的数量应该足够多。我们以每个线程块启动 32 个线程开始测试，直到一共启动 64 个线程块。这样做保证了每个 SM 都能分配到任务，而不是某个 SM 分到大量的线程和绝大部分的任务。接着我们增加每个线程块中的线程数量，最多 256 个线程，直到每个 SM 分配的线程块数合理为止。

将元素的类型从 uint1 转换成 uint2、uint3 和 uint4 将产生有趣的结果。当单个元素的大小增加，提交到内存子系统的存储事务次数就会减少。在 GTX470 上，将 4 字节的数据读取（单个整型数或浮点数）改为 8 字节（两个整型数或浮点数、单个双精度浮点数）的读取，将导致全局内存读 / 写数据的带宽峰值提高 23%（参见图 9-22）。然而，带宽的平均提高只有 7%，但是这仍然表明了通过简单地将 int1/float1 向量类型转变为 int2/float2 向量类型能够适当地减少运行时间。GTX460 上的变化与 GTX470 上的相似，但变化更加显著（参见图 9-23）。

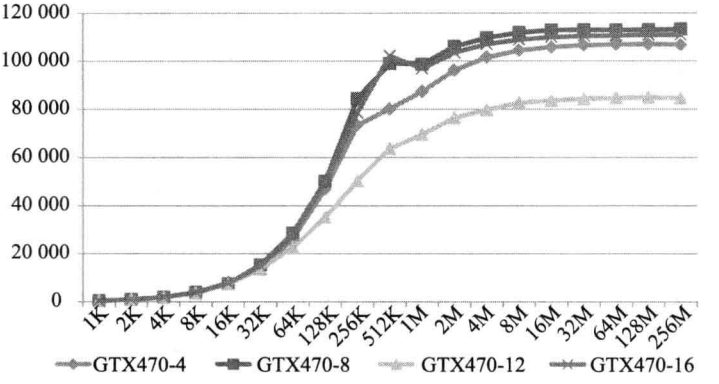

图 9-22 计算能力 2.0 的 GTX470 全局内存带宽（以字节为单位的存储事务）

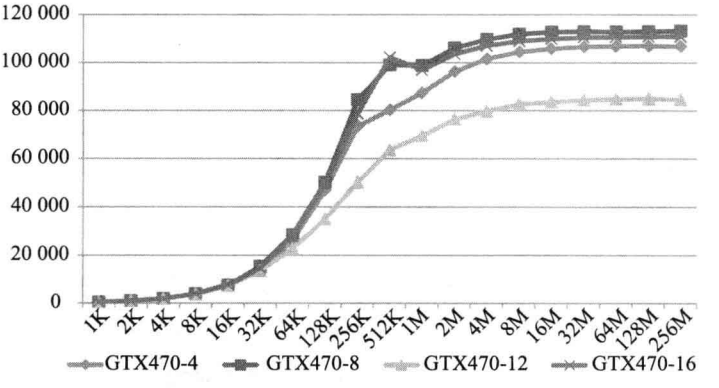

图 9-23 计算能力 2.1 的 GTX460 全局内存带宽（以字节为单位的存储事务）

为了获得最佳带宽，计算能力 2.1 设备上的 CUDA 代码将特殊编译。我们也发现当线程块内的线程数为 48 的倍数时效果最佳。鉴于这里每个 SM 中含有 3 组 16 个核而不是通常的两组，这种结果也就不足为奇了。将每个元素的大小从 4 字节增加为 8 字节或 16 字节时，带宽平均提高了 19%，最大提高了 38%。

单个线程束中每个线程处理 8 字节的存储事务将导致 256 字节的数据移向内存总线。我们使用的 GTX 460 与全局内存相连的总线位宽为 256 位。这清楚地表明，不考虑任何内存占用，在这样的设备上应该保证每个线程处理 8 字节或 16 字节（2 或 4 个元素）的数据。最有可能是由于 SM 中更高的 CUDA 代码比率从而导致单组 LSU（加载 / 存储单元）的竞争问题。

与 GTX 260 相比，计算能力 1.3 的设备与 Tesla C2050 设备类似，通过将每个元素的大小从 4 字节增加为 8 字节，可以使带宽平均增加 5%。然而，如果继续增加，程序的性能将大打折扣。9800 GT 并没有显示出明显的提升，这暗示了当处理的每个元素的大小为 4 字节时该设备的带宽已达到峰值。

最后，注意基于费米架构的 Tesla 设备实现了基于 ECC（错误检测与纠正）功能的内存协议。禁用该功能可以提升大约 10% 的速度，代价是失去了错误检测和纠正能力。只放置单台机器而不是放置于服务器中心，可能是一个挺好的折衷方案。

9.3.4 GPU 计时

1. 单核 GPU 计时

GPU 上对数据进行计时不是特别简单。由于使用 GPU 和 CPU 最好的方式是异步地进行操作（即 GPU 和 CPU 同时运行），因此使用基于 CPU 的时钟并不是一个很好的方案。当同时在 GPU 和 CPU 上进行顺序操作时，CPU 时钟不是特别精确。由于这并不是我们实际想要的，所以这是一个糟糕的方案。

由于 memcpy 操作隐式地进行同步，默认情况下，GPU 在同步模式下操作。程序开发人员希望将数据复制到设备、运行内核、再将数据从设备取回，然后将 CPU 内存中的运行结果存入磁盘或进一步处理。虽然这是一个容易理解的模式，但它同时也是一个缓慢的模式。这一模式意在使内核执行任务但它并没有考虑性能因素。

在第 8 章，我们详细地测试了流的使用。流是一种高效的任务队列。当没有向 CUDA API 定义流时，0 号流用于默认任务队列。然而，0 号流包含很多隐式的与主机同步的操作。你也许会期望一个异步操作，但实际上在使用 0 号流时，特定的 API 调用包含隐含的同步。

若要使用异步操作，我们需要先建立如下流：

```
// Create a new stream on the device
cudaStream_t stream;
CUDA_CALL(cudaStreamCreate(&stream));
```

为了便于带宽测试，我们创建了一个事件数组。

```
#define MAX_NUM_TESTS 16
cudaEvent_t kernel_start[MAX_NUM_TESTS];
cudaEvent_t kernel_stop[MAX_NUM_TESTS];
```

GPU 提供了一些由 GPU 硬件赋予时间戳的事件（参见图 9-24）。因此，想要在 GPU 上计算特定操作的执行时间，你需要向任务队列中先添加一个启动事件，然后添加想要计时的操作，最后添加停止事件。GPU 上执行的流是简单的 FIFO(先进先出）的操作队列。每个流代表一个独立的操作队列。

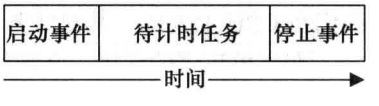

图 9-24 GPU 上计算任务的执行时间的流程

创建流后，你需要创建一个或多个事件。

```
for (u32 test=0; test < MAX_NUM_TESTS; test++)
{
 CUDA_CALL(cudaEventCreate(&kernel_start[test]));
 CUDA_CALL(cudaEventCreate(&kernel_stop[test]));
}
```

我们使用一个简单的循环来建立 MAX_NUM_TESTS 个事件——一个启动事件和一个停止事件。然后我们需要在待计时的任务任一端将这些事件插入流中。

```
// Start event
CUDA_CALL(cudaEventRecord(kernel_start[test],stream));

// Run the kernel
```

```
kernel_copy_single<data_T><<<num_blocks, num_threads, dynamic_shared_memory_usage,
stream>>>(s_data_in, s_data_out, num_elements);

// Stop event
CUDA_CALL(cudaEventRecord(kernel_stop[test],stream));
```

为了计算时间，要么每个CUDA调用要么全部一同调用cudaEventElapsedTime函数来获取两个带时间戳事件的时间差。

```
// Extract the total time
for (u32 test=0; test < MAX_NUM_TESTS; test++)
{
 float delta;

 // Wait for the event to complete
 CUDA_CALL(cudaEventSynchronize(kernel_stop[test]));

 // Get the time difference
 CUDA_CALL(cudaEventElapsedTime(&delta, kernel_start[test], kernel_stop[test]));

 kernel_time += delta;
}
```

你需要通过执行定时事件来实现，流之间的事件顺序并不能得到保证。CUDA运行时也许会在0号流中执行启动事件，然后切换到5号流中执行之前挂起的内核，有时稍后再返回0号流，启动0号流的内核，接着切换到其他流处理一些启动事件，最后返回0号流并把时间戳赋给结束事件。时间的改变量就是从启动阶段到结束阶段的时间消耗。

在这个例子中注意，只创建了一个流。我们创建了多个事件，但都是在同一个流中执行。在只有一个流的情况下，运行时只能顺序地执行事件，因此确保实现了正确的计时。

注意cudaEventSynchronize API调用。在事件未完成时，该调用使CPU线程阻塞。由于我们在CPU上未执行其他任务，因此上述操作完全符合我们的目的。

在主机程序的结尾，必须保证占用的资源都已释放。

```
// Free up all events
for (u32 test=0; test < MAX_NUM_TESTS; test++)
{
 CUDA_CALL(cudaEventDestroy(kernel_start[test]));
 CUDA_CALL(cudaEventDestroy(kernel_stop[test]));
}
```

在运行内核程序时，在事件执行前将其销毁将导致不确定的运行时错误。

最后，你应该明白使用事件不是免费的。运行时处理事件需要消耗资源。在这个例子中，我们特意为每个内核计时以确保没有显著的变化发生。多数情况下，一对分别位于任务队列开始和结束位置的启动事件和停止事件，足以计算整个任务队列的执行时间。

2. 多GPU计时

相比单GPU的计时，多GPU的计时更复杂一些，但幸运的是它们基于相同的原理。同

样，我们创建一些流并且将一些事件放入流中。

不幸的是，API 中并没有提供函数以从事件中获取绝对时间戳。只能获得两个事件的时间差。然而，通过在流的起始位置插入事件，你可以将其作为零时间点，并且用这种方式获得流开始的相关时间。然而，获取不同 GPU 上的两个事件的时间差将导致 API 返回一个错误。这使得用多 GPU 时，建立时间轴更复杂，因为你或许需要根据启动事件实际发生的时机来调整时间。从图 9-29 中可以看到向设备复制数据、执行内核、从设备复制数据、向设备复制数据、再次调用内核、最后从设备复制数据这一系列事件。

注意，对不同设备，复制时间大致相近，但是内核函数的执行时间有很大不同。在 GTX470 设备（CFD2）上，倒数第二次从设备复制数据的操作中，条状图变短了（258ms 与 290ms）。这是因为 GTX470 首先启动传输并且直到传输完成，其他设备才会启动传输。GT9800 作为较慢的设备，在 GTX470 完成传输后 GT9800 的内核函数还在执行。使用不同代的设备，可以得到这样的模式。传输速度大体相近，但是内核函数的运行时间导致传输启动的时刻改变。

图 9-25 是使用计时器产生的，但诸如 Parallel Nsight 和 Visual Profiler 的工具可以自动地画出时间轴以及 CPU 的时间轴，借助 CPU 的时间轴可以清楚地看到什么时间发生了什么事件。

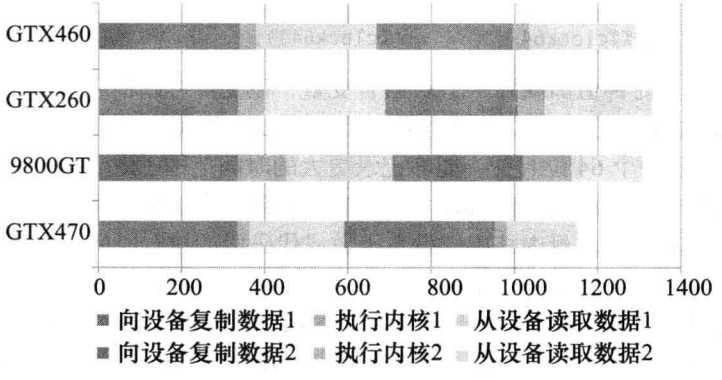

图 9-25 多 GPU 的时间轴

注意，通过调用 cudaEventQuery API 查询事件是否完成是可行的，而不会像调用 cudaEvent Synchronize 函数那样导致阻塞。这样，CPU 可以继续做有用的工作，或者跳转到下一个流以检查该流是否完成执行。

```
if (cudaEventQuery(memcpy_to_stop[device_num][complete_count_in_stop[device_num]])
== cudaSuccess)
{
 TIMER_T delta = 0.0F;

 CUDA_CALL( cudaEventElapsedTime( &delta, memcpy_to_start[device_num][0],
memcpy_to_stop[device_num][complete_count_in_stop[device_num]] ));

 printf("%sMemcpy to device test %d completed %.2f ms", device_prefix[device_num],
complete_count_in_stop[device_num], delta);
```

```
    complete_count_in_stop[device_num]++;
    event_completed = true;
}
```

这个特定例子，取自另一个程序。我们有一个事件数组 memcpy_to_stop，根据设备编号和测试编号进行索引。通过调用 cudaEventQuery 来检查事件是否完成，如果事件已经完成则该调用返回 cudaSuccess。如果成功，则从相同的设备上获取该事件和启动事件 memcpy_to_start 间的时间差。但是对于 0 号测试，我们得到了那块 GPU 上全部内核函数流的启动事件。通过调用 cudaEventElapsedTime 函数来获取时间差。

注意，如果事件未完成该调用将引起一个错误，因此调用 cudaEventQuery 来预防这种情况。如果我们想使用一个阻塞函数来等待事件完成，也可以调用 cudaEventSynchronize。

如果对绝对时间特别感兴趣，GPU 也提供了借助一些嵌入 PTX 代码的方法来访问低层计时器：

```
// Fetch the lower 32 bits of the clock (pre compute 2.x)
unsigned int clock;
asm("mov.u32 %0, %%clock ;" : "=r"(clock));

// Fetch the clock (req. compute 2.x)
unsigned long int clock64;
asm("mov.u64 %0, %%clock64 ;" : "=r"(clock64));
```

这段代码将原始时钟值加载到一个 C 语言变量，该变量可以稍后存储在历史缓冲中并传回主机。%clock 的值只是一个 32 位计数器，当超过 u32 最大值时会清零重新开始。计算能力 2.x 的硬件提供了一个 64 位时钟，能够记录更大的时间范围（该范围内的值可以被计时）。注意，CUDA API 提供了通过使用 clock 和 clock64 函数访问这些寄存器值的函数。

你可以用这个函数来测量内核中的设备函数或代码段的执行时间。这些测量值并不是通过 Visual Profiler 或 Parallel Nsight 显示出来的，因为它们的计时方案仅在全局级的内核函数中有效。你也可以以此来测量线程束到达栅栏点的时间。在调用诸如 syncthreads 栅栏原语之前，为每个线程束建立一个存储操作。这样你就可以看到在同步点的线程束分布情况。

非常重要的一点是，你必须明白一个内核函数中给定的线程束并不会一直运行。因此，与计时多个流一样，一个线程束也许会存储一个起始时间，之后挂起，然后恢复，最后遇到下一个计时器存储事件。时间变化量只是整体的实际时间差，并不是 SM 执行给定的线程束所耗费的时间。

你同样应该意识到，以这种方式插入代码也许会影响它的计时以及相对于其他线程束的执行顺序。你可以把数据存入全局内存并在随后将数据传输到能够分析它的主机上。因此，你的检测代码不仅影响了执行流，也影响了内存访问。这个影响可以通过孤立运行一个由 32 个线程构成的线程块（即一个单独的线程束），来降至最低。然而，这样将与 SM 中同时运行其他线程束以及 GPU 上启动其他 SM 的实际情况有出入了。

9.3.5 重叠 GPU 传输

下面有两种策略可以试图产生重叠的传输。第一种，用计算时间重叠传输时间。我们可以

在上一节看到这种方法的细节，上一节明确阐述了流的使用并且隐含了零复制内存的使用。

流在GPU计算中是一项非常有用的功能，通过建立独立工作队列我们能够以异步方式驱动GPU设备。也就是说，CPU可以将一系列的工作元素压入队列，然后离开，在再次服务GPU之前做别的事情。

从某种程度上说，用0号流同步操作GPU就像用一个单字符的缓冲区来访问串行设备。这种设备以前用于对设备原有串口的实现，如在RS232接口上操作的调制解调器。这些现如今已经过时，并且由USB1.0、USB2.0、USB3.0接口取代。原有的串行控制器UART，将向处理器发出一个中断请求，告诉处理器它已经收到足够的比特位去解码一个字符，并且其单一字符缓冲这已满。只有CPU应答中断请求时通信才继续。一次通信一个字符不够快并且CPU执行不够密集。这种设备很快被有16字符缓冲区的UART所替代。因此，该设备中断CPU的次数也减少了16倍。它可以处理输入的字符并积累到合适的大小再转移到CPU的内存中。

通过为GPU创建一个工作流，我们正在做类似的事情。取代了GPU和CPU同步工作的模式，取代了CPU不得不一直询问GPU来确认是否完成的模式，我们只是给它大量的工作去处理。只需定期去检查工作是否完成，如果完成，则可以将更多的工作压入流或工作队列中。

只要在试图访问它之前转换所需的设备，通过CUDA流接口我们就可以驱动多个GPU设备。对于异步操作，从GPU中传入/传出需要固定的或者是锁页的内存。

在单核处理器系统中，所有GPU将会连接到单个PCI-E交换机上。PCI-E交换机的目的是为了在PCI-E总线上连接各种高速设备。这样做还有其他的功能，PCI-E卡可以不需要访问主机内存就进行彼此传输。

尽管我们有了很多的PCI-E设备，以测试机器来说，4个GPU连接在4个独立的X8 PCI-E2.0链路上，同时它们也连接到同一个PCI-E控制器。另外，根据实现的不同，这个控制器可能实际就在CPU内部。这样，如果我们在同一时间及时地向多个GPU执行一系列传输，虽然各个设备的带宽在每个方向会达到5GB/S的速度，但是如果所有的设备都处于活跃状态，那么PCI-E交换机、CPU、内存还有其他部分的工作会达到这样的速度么？

在有4个GPU的系统上，会有怎样的扩展性？在I7 920的Nehalem系统上，对单卡使用PCI-E 2.0 X16链路时，我们测量的带宽大约是5GB/s。在AMD系统上，我们在PCI-E 2.0 X8链路上有2.5 ~ 3GB/S的带宽。由于AMD系统PCI-E通道的数量是I7系统的一半，因此这些数字大约就是你期望得到的。

我们修改了带宽测试程序，原有程序是我们早先在引入了更多的显卡和更多的并发传输时测试PCI-E带宽的程序。一旦我们开始给不同的GPU进行并发传输，任何事物都可能影响到传输。任何熟悉游戏行业多GPU应用的人会知道：简单插入第二块GPU不能保证产生双倍的性能。许多基准测试表明，大多数商业游戏能从两块GPU显卡中受益显著，增加第三块卡经常也会带来一些效果，但是没有插入第二块卡所产生的几乎两倍效果那么出色，而加入第四块卡常常会导致性能的恶化。

现在这种情况并不是很直观，即增加更多的硬件等价于更低的速度。然而，在CPU上，当核的数量对周围部件来说过多的时候，我们发现了同样的问题。一个典型的高端主板/

CPU 解决方案致力于将最多 32 个 PCI-E 设备连接到 PCI-E 总线上。这也意味着仅仅两块显卡就能满载 X16 PCI-E 2.0 的速度。任何比这一速度高的设备都是通过使用 PCI-E 转换芯片达到的，该芯片使得 PCI-E 链路多路传输。这会工作得很好，直到在 PCI-E 多路传输器上的两块显卡需要同时传输。

我们已经在 AMD 系统中运行了这本书中的大多数测试程序，该系统没有用多路传输，但是当 4 个 GPU 同时运行的时候，每个 GPU 的速度降低到 X8 链路的速度。这样，每个设备达到 2.5 ~ 3GB/S，我们能够获得的理论最大值为 10 ~ 12.5GB/S。此外，作为 AMD 解决方案，PCI-E 控制器内建在处理器中，该控制器也位于 PCI-E 系统和主存之间。主存的带宽约是 12.5GB/S。因而，可以发现系统将不太可能达到 4 个 GPU 的全部潜力。参见表 9-2 和表 9-3、图 9-26 和图 9-27。

你可以从表 9-2 和表 9-3 中看到，对于 3 个 GPU 传输的效果是非常可观的。我们看到几乎线性的增长。然而，当 4 个 GPU 一起竞争可用资源的时候（CPU、内存带宽、PCI-E 转换带宽），整体的速度就较慢了。

表 9-2 多 PCI-E 传输到设备的带宽变化

	1 个设备	2 个设备	3 个设备	4 个设备
传输到 GTX470	3151	3082	2495	1689
传输到 9800GT	0	3069	2490	1694
传输到 GTX260	0	0	2930	1792
传输到 GTX460	0	0	0	1822

表 9-3 多 PCI-E 从设备传输的带宽变化

	1 个设备	2 个设备	3 个设备	4 个设备
从 GTX470 传输	2615	2617	2245	1599
从 9800GT 传输	0	2616	2230	1596
从 GTX260 传输	0	0	2595	1522
从 GTX460 传输	0	0	0	1493

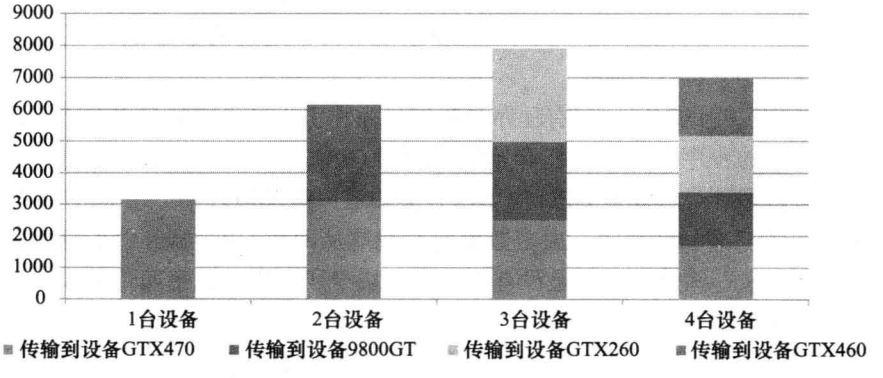

图 9-26 多 GPU 下传输到设备的 PCI-E 带宽（AMD 905e Phenom II）

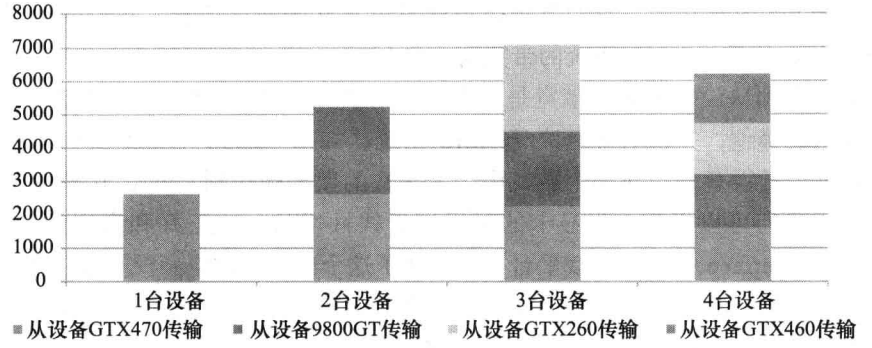

图 9-27 多 GPU 下从设备传输的 PCI-E 带宽（AMD 905e Phenom II）

另一个我们需要测试的多 GPU 平台是一个搭载了 Nehalem I7 平台的 6 芯 GPU 系统，采用华硕超级计算机主板（P6T7WS）并带有 3 块 GTX295 双 GPU 卡。它采用双 NF200 PCI-E 交换芯片，允许每个 PCI-E 卡可以有一个完整的 X16 链路工作。虽然这一点可能对 GPU 间通信（CUDA 4.x 支持的 P2P 模型）有帮助，但是如果两个显卡同时使用总线，它则不能扩展与主机通信的可用带宽。我们使用的 GTX290 显卡，是一个双 GPU 的设备。在内部，每个 GPU 必须共享 X16 PCI-E 2.0 链路。表 9-4 和图 9-28 展示了它的表现。

表 9-4 传输到设备时 I7 的带宽

	1 台设备	2 台设备	3 台设备	4 台设备	5 台设备	6 台设备
传输到设备 0	5026	3120	2846	2459	2136	2248
传输到设备 1	0	3117	3328	2123	1876	1660
传输到设备 2	0	0	2773	2277	2065	2021
传输到设备 3	0	0	0	2095	1844	1588
传输到设备 4	0	0	0	0	1803	1607
传输到设备 5	0	0	0	0	0	1579
合计	5026	6237	8947	8954	9724	10 703

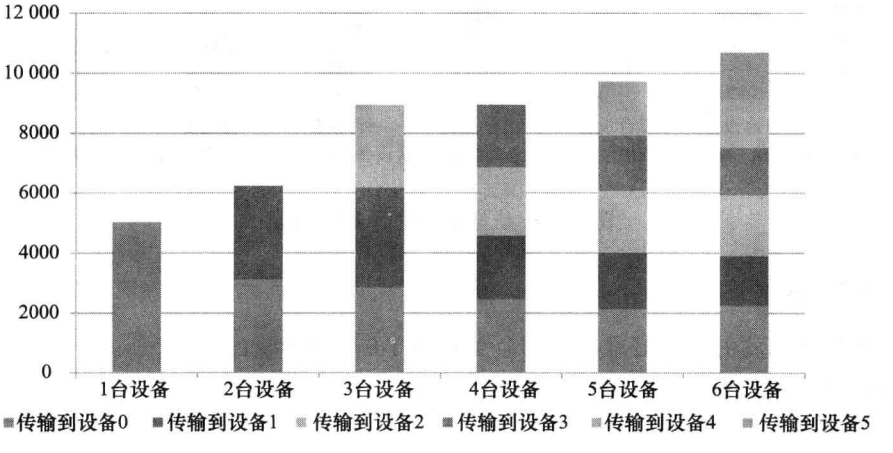

图 9-28 传输到设备时 I7 的带宽

正如在表 9-4 中看到的，我们发现设备的总带宽几乎线性增长。我们获得了仅仅超过 10GB/S 的峰值，比基于 AMD 系统的带宽高大约 20%。

我们可以看到从设备传输的带宽是一个不同的情况（参见表 9-5 和图 9-29）。用两个设备时，带宽达到峰值，并没有显著高于 AMD 系统。如果考虑到大多数 GPU 系统是面向游戏的，你将不会觉得意外。在一个游戏中，大多数的数据被送到 GPU，几乎没有返回 CPU 主机。因而，我们可以发现 3 张显卡的时候接近线性增长，这正好和高端的三重“速力”（Scalable Link Interface，SLI）游戏平台一致。除了这个配置，供应商没有动机去追求超出此设置的 PCI-E 带宽。由于 GTX290 实际上是一个双芯 GPU 显卡，我们可以看到内置 SLI 接口实际上并不能解除显卡的限制。我们能明显地看到一些资源竞争。

表 9-5 从设备传输时 I7 的带宽

	1 台设备	2 台设备	3 台设备	4 台设备	5 台设备	6 台设备
从设备 0 传输	4608	3997	2065	1582	1485	1546
从设备 1 传输	0	3976	3677	1704	1261	1024
从设备 2 传输	0	0	2085	1645	1498	1536
从设备 3 传输	0	0	0	1739	1410	1051
从设备 4 传输	0	0	0	0	1287	1035
从设备 5 传输	0	0	0	0	0	1049
合计	4608	7973	7827	6670	6941	7241

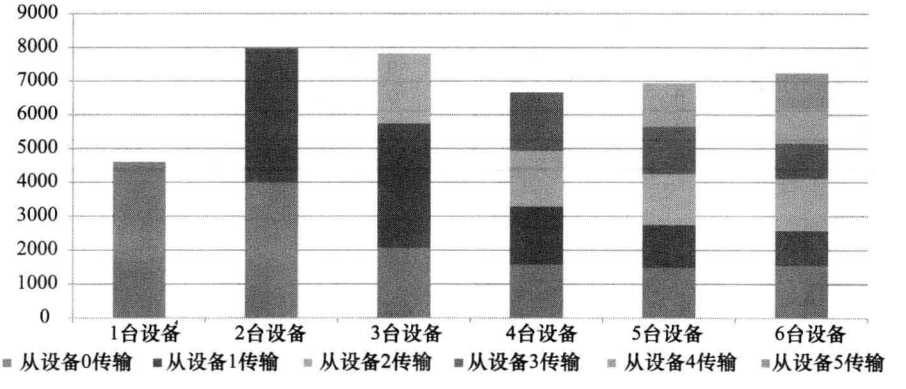

图 9-29 从设备传输时 I7 的带宽

9.3.6 本节小结

- 了解以下事实并且做好准备：你会受到 PCI-E 带宽容量的限制。
- 如可能，尽量使用锁页内存。
- 使用至少 2MB 的传输大小。
- 理解零复制内存的使用，它是流 API 的一种替代方法。

- 想想如何将内核执行时间与传输时间重叠。
- 使用多个 GPU 时不要期待线性的带宽增长。

9.4 策略 4：线程使用、计算和分支

9.4.1 线程内存模式

把应用程序分解成大小合适的网格、线程块和线程，是保证 CUDA 内核性能的关键环节之一。包括 GPU 在内的几乎所有计算机设计，内存都是瓶颈。线程布局的一个糟糕的选择通常也会导致一个明显影响性能的内存模式。

考虑第一个例子，比较一个 2×16 的线程布局（如图 9-30 所示）和一个 32×2 的线程布局。如果处理浮点值时需要考虑它们通常是如何覆盖内存的。在 2×16 的例子中，除了线程 1，线程 0 不会和其他线程合并。这种情况下硬件就需要 16 次内存读取。除非前半个线程束取得了所有需要的数据，否则，该线程束将无法继续运行。因此，在这些很耗时的内存交易中，至少有 8 个事务需要提前完成，SM 上的所有计算活动才能开展。由于大部分线程束将遵循相同模式，因此 SM 将发出的内存请求淹没而其计算部件几乎是空闲的。

线程0	线程1
线程2	线程3
线程4	线程5
线程6	线程7
线程8	线程9
线程10	线程11
线程12	线程13
线程14	线程15
线程16	线程17
线程18	线程19
线程20	线程21
线程22	线程23
线程24	线程25
线程26	线程27
线程28	线程29
线程30	线程31

图 9-30 2×16 线程布局的网格

从上一节的带宽分析中我们看到，SM 从线程束中获取的内存请求数量是有限制的。SM 对于任何一个线程束的数据请求，需要两个时钟周期才能完成。我们的例子中，请求被分成 16×8 字节的内存事务。

在费米架构中，第一次会导致一级缓存读失效（read miss）。一级缓存会从二级缓存请求最小大小 128 字节的数据（最少的数据），是此线程实际需要的 16 倍。因此当数据从二级缓存移到一级缓存时，只有 3.125% 的数据被线程 0 使用。由于线程 1 也需要相邻的内存地址，所以我们能提高到 6.25%，但这仍是很糟糕的。

第一次运行该代码时二级缓存不可能包含数据。它会发出从慢速全局内存读取一个 128 字节的操作。这个延迟较大的操作最终把 128 字节送到二级缓存。

二级缓存在 16 个 SM 的设备上大小是 768K。假设使用 GTX580，我们就有 16 个 SM。而每个 SM 只有最大为 48K 大小的一级缓存。使用 128 字节的缓存行，每个 SM 的缓存有 384 个单元。如果我们假设 SM 上满载 48 个线程束（开普勒架构支持 64 个），那么每个线程束发出 16 个独立的读操作，总共 768 个读操作。这意味着我们需要 768 个缓存行，而我们实际只拥有 384 个，这只是为了缓存需要的数据，以使每个线程束都可以在内存中读取一个单独的线程块。

有效缓存太小以至于在这个例子里无法用到时间局部性。时间局部性的含义是我们希望在缓存中的数据在下一次读取时仍然存在于缓存。在每个 SM 上，处理线程束的中途，缓存就会变满，硬件就会开始用新数据重新填充它。因此，在二级缓存中绝不会有数据重用，在填充全部缓存行时，会造成明显的开销。事实上，唯一的救命稻草是费米硬件，不像前几代，在我们的例子中它将把获取的数据转发给另一个线程。

高速缓存模型可能导致一个问题，使人们认为硬件能将他们从糟糕的编程中拯救出来。让我们假设，我们必须使用这个线程模式，然后我们可能把从内存取来的数据重复处理了若干次。读取数据的线程模式不一定是使用数据的线程模式。我们从一个 16×2 的共享内存中获取数据，接着同步线程，然后如果需要的话，切换到 2×16 的模式。尽管这可能产生共享内存体冲突，但它仍然比从全部内存中获取快一个数量级。我们可以简单地通过声明一块 33×2 的共享内存来添加一个用于补齐的元素，以确保当我们访问时，避免这些存储体冲突。

考虑一下处理内存系统的差异。我们发出一个 128 字节的合并读取代替 16 次分别读取。不仅在内存事务的数量和带宽利用率上有 16 ： 1 的改善。从二级缓存移到一级缓存的数据可以用一个事务而不是 16 个来完成。

SM 中的加载存储单元必须仅发出一个单独的获取指令而不是 16 个独立的获取指令，只占用 2 个时钟周期，而不是 32 个，然后释放加载存储单元，用于为来自其他线程束的别的任务提供服务。

每个线程束消耗一个单独的缓存行，每个 SM 最多 48 个。因此，对于二级缓存，每个 SM 有 384 个缓存行，然而我们只用了 48 个，也就是二级缓存的 12.5% 而不是之前的 200%。因此，即使是在费米架构的多层缓存结构下，要想达到接近全速的性能，也必须在每个线程束内按照 128 字节为单位的合并块读取数据。

现在，我们可以配置二级缓存大小。通过使用 -Xptxas-dlcm=cg 编译器标志指定获取单位为 32 字节而不是 128 字节。然而，这也改变了从全局内存到一级缓存的读取单位。这是一个简单却糟糕的办法，因为你在从全局内存读取数据时没有使用足够大的块数据。为了从一个给定的设备中获得最佳的性能，你要么需要弄明白内部是如何运行的，要么使用熟悉内部机理的专业人士编写好的函数库。

利用 Parallel Nsight，如果选择“Custom”（定制）实验并且添加一级和二级缓存计数器，我们可以很清楚地看到内存带宽的效果。我们感兴趣的特定计数器在表 9-6 中予以展示。它们可以在 Parallel Nsight 的“Custom”实验中设定，如图 9-31 所示。

表 9-6 Parallel Nsight 缓存计数器

Nsight 计数器	用途
全局内存加载的一级缓存命中数	可以在一级缓存找到的全局内存加载请求数量
全局内存加载的一级缓存失效数	无法在一级缓存找到的全局内存加载请求数量
分区 0 读取扇区到二级缓存失效数	二级缓存其中一半数量的失效数
分区 1 读取扇区到二级缓存失效数	二级缓存另一半数量的失效数
分区 0 读取扇区到二级缓存失效查询数	二级缓存其中一半数量的访问尝试数
分区 1 读取扇区到二级缓存失效查询数	二级缓存另一半数量的访问尝试数

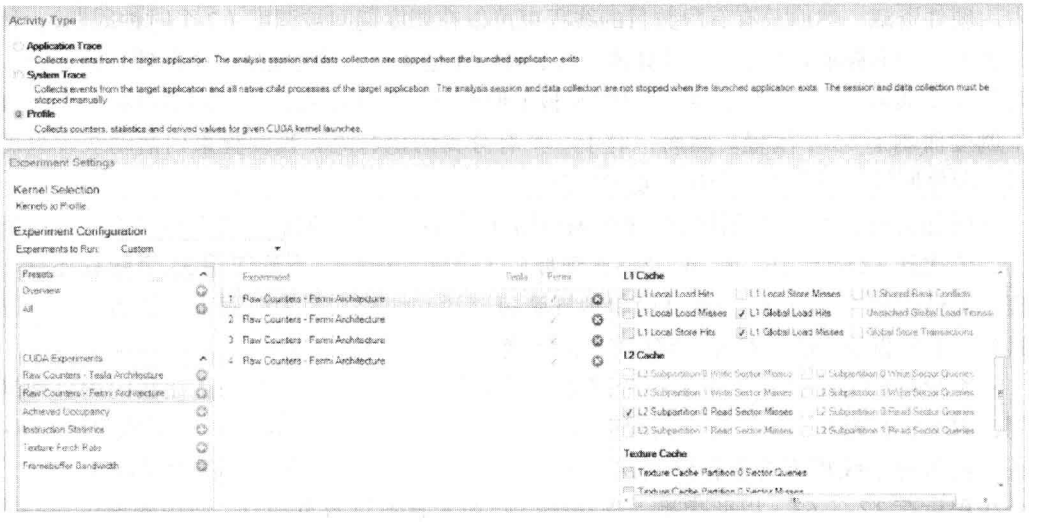

图 9-31 设置 Parallel Nsight 以获知缓存统计摘要

从这些计数器中，我们可以手动计算出一级缓存和二级缓存的命中率。命中率是读取（或写入）时能够直接在缓存找到的百分比。每一次缓存的访问都为我们节省了来自全局内存的数百个时钟周期的延迟。

表 9-7 样本排序示例中的缓存命中率

函数	耗时	占用率	活动线程束数	线程块数	线程数	一级缓存命中率（%）	二级缓存命中率（%）
sort_bins_gpu_kernel	28.8ms	0.17	8	512	32	87	69
sort_bins_gpu_kernel	33.5ms	0.33	16	256	64	86	60
sort_bins_gpu_kernel	44.4ms	0.67	32	128	128	80	36
sort_bins_gpu_kernel	47ms	0.83	40	64	256	78	32

当我们观察表 9-7 的样本排序算法的结果时，可以立即看到当内核超过 64 个线程时二级缓存的命中率急剧下降。占用率增加，但性能下降。考虑到产生前缀和数组中二级缓存的用途，这并不奇怪。如果每个线程处理一个桶，那么当扩展线程的数量时，内存区域中需要进行缓存的量就会增加。当它超过二级缓存的大小时，命中率就会急剧地下降。

解决问题的办法是把现有的使用一个线程处理一个桶的算法替换为所有线程同时处理一个桶的算法。这样，在每次迭代时，我们就使用合并方式访存并且大大增加了内存的局部访问性。一种替代的解决方案是，在线程的非合并访问和读 / 写全局内存时必须的合并访问之间，可以使用共享内存处理事务。

9.4.2 非活动线程

尽管有数以千计的线程是闲置的，但是它们并不是免费的。非活动线程的问题有两方面。首先，只要线程束中的一个线程是活跃的，那么对应线程束就保持活跃，可供调度，并

且占用硬件资源。然而只有有限数目的线程束可以在调度期间被调度（2个时钟周期）。计算能力2.0的硬件能够调度2个，计算能力2.1的硬件是4个，计算能力3.x的硬件是8个。以下两种方式都是无意义的：在多个CUDA核上调度只含一个线程的线程束或者在一个CUDA核上调度而剩下15个闲置。然而，对于一个有分支的执行流，当线程束内活动线程只剩一个时，硬件所做的正是属于上述无意义的事情。

有时你会看到不了解硬件的程序员编写的并行归约操作。他们在每个线程束内进行归约操作，依次从32降至16、8、4、2，直到最后1个活动线程。不论使用32个线程还是1个线程，硬件仍然会分配32个并且简单地屏蔽掉非活动线程。因为线程束仍然活跃，所以即使只有一个线程处于活动状态，它们仍然需要被调度并消耗硬件资源。

一个更好的方法是让每个块中全部的32个线程计算一组局部结果。让我们用求和操作为例，因为它很容易理解。每个线程束有32个线程，可以让线程束在一个周期内计算64个加法。这里每个线程都把值存储到共享内存中。因此，第一个线程束存储元素0 ~ 31，第二个线程束存32 ~ 63，以此类推。现在把待归约元素数目N除以2。使用阈值过滤线程if (threadIdx.x< (N/2))，不断重复，直至N为2。在这个过程中，有如下对应关系：

线程0 ~ 255读元素0 ~ 511（8个活动线程束）。

线程0 ~ 127读元素0 ~ 256（8个活动线程束）。

线程0 ~ 63读元素0 ~ 127（8个活动线程束）。

线程0 ~ 31读元素0 ~ 63（8个活动线程束）。

线程0 ~ 15读元素0 ~ 31（8个活动线程束）。

以此类推。

线程编号大于阈值的线程束不再被调度。编号小于N的线程，一直在工作，直到N小于32。在此我们可以简单地对所有剩余的元素做加法，或者继续归约迭代直至最后一层加合操作。

非活动的线程束本身也不是免费的。虽然SM内部关心的是线程束，而不是线程块，然而外部调度器只能向SM而不是线程束调度线程块。因此，如果每个块包含只有一个活动的线程束，那么仅有6 ~ 8个线程束以供SM从中选择进行调度。通常根据计算能力的版本和资源使用情况，在一个SM中容纳多达64个活跃的线程束。现在存在明显的一个问题，因为线程级的并行模型（TLP）依赖于大量的线程来隐藏内存和指令延迟。随着活跃线程束数量的减少，SM通过TLP隐藏延迟的能力也明显下降。一旦超过某个程度，就会伤害到性能，尤其是当线程束仍在访问全局内存的时候。

因此，诸如归约这类操作的最后一层或者其他线程束数量逐渐减少的操作中，我们需要引入一些指令级并行操作（ILP）。我们要尽可能地终止最后的线程束以使整个线程块都闲置出来，并替换为另一个包含一组更活跃线程束的线程块。

我们在本章的后面进一步讨论归约的细节。

9.4.3 算术运算密度

算术运算密度这个术语用来度量每次内存读取相应的算术运算的数目。因此，一个内

核，从存储器中取两个值，相乘，并将结果存储到内存，这种方法具有非常低的算术运算密度。

```
C[z] = A[y] * B[x];
```

读取和存储操作可能涉及一些索引计算。真正要做的工作是乘法。然而，每三个内存事务（两次读操作和一次写操作）只执行一次计算操作，所以内核属于内存密集型。

全部的执行时间是

T= 读取时间（A）+ 读取时间（B）+ 算术运算时间（M）+ 存储时间（C）

或者

T=A+B+M+C

请注意，我们在这里使用 A+B，而不是两倍的 A。单个读取操作的时间不容易预测。事实上，A、B 和 C 的读取时间都不是恒定的，因为它们是受其他 SM 在内存子系统执行的影响。获取 A 也可能会同时把 B 放入缓存，所以对于 B 的存取时间是比 A 少的。写入 C 则可能导致 A 和 B 被从缓存中换出。二级缓存中的更改可能来自于完全不同的 SM 的活动。因此，我们可以看到缓存让计时变得不可预知。

当考查算术运算密度时，我们的目标是提高有用的工作相对于内存读取和其他开销操作的比率。然而，我们必须考虑怎么界定内存读取。显然，当我们从全局内存中获取数据是符合要求的，但是共享内存或缓存的读取呢？由于处理器必须将数据从共享内存中完全移动到寄存器，因此我们必须将共享内存读存考虑为内存操作。类似的，如果数据来自一级缓存、二级缓存或常量缓存，它也必须在我们操作之前移动到一个寄存器里。

然而，在访问共享内存或一级缓存的情况下，相比于全局内存访问这样的操作成本降低一个数量级。因此，如果共享内存读取时间指定为 1，则一个全局内存读取的权值将是 10。

那么，如何才能提高此类指令流的算术运算密度呢？首先，我们要了解底层的指令集。指令的最大操作数是 128 字节，即一个 4 元素矢量的加载 / 存储操作。假设我们使用浮点数或整数，那么理想的数据块大小是 4 个元素；如果我们使用的是双精度浮点数，则最优数据块为两个元素。因此，我们采用前者的操作如下：

```
C[idx_C].x = A[idx_A].x * B[idx_b].x;
C[idx_C].y = A[idx_A].y * B[idx_b].y;
C[idx_C].z = A[idx_A].z * B[idx_b].z;
C[idx_C].w = A[idx_A].w * B[idx_b].w;
```

为了清晰地展示操作过程，上面给出了完整代码。如果扩展为矢量型矢量类，并提供一个乘法运算来执行此段代码，你可以简单地写成

```
C[idx_C] = A[idx_A] * B[idx_b];
```

不幸的是，目前的 GPU 硬件不支持这样的矢量操作，只能从标量型变量中加载、存储、移动和打包 / 解包。

有了这样基于矢量的操作，我们可以把相关操作的成本（加载 A、加载 B、存储 C，计算 idx_A、计算 idx_B 和计算 idx_C）分摊到 4 个乘法运算而不是 1 个。加载和存储操作时间

需要稍微长一些，因为我们要引进一个打包和解包操作，而在访问标量时无须这两个操作。我们将循环迭代减少 1/4，相应的内存请求的数目大幅下降，向内存系统发出的内存请求变少，每次请求的规模变大。这极大地提高了性能（约 20%），我们已经在本书中看到了一些这样的例子。

1. 超越函数操作

GPU 硬件主要为了加快游戏环境。通常，这些都需要处理数以十万计的多边形、以某种方式对现实世界建模。在 GPU 硬件内部建有一些加速器。它们是专门为特定目的而设计的硬件。GPU 具有以下加速器：

- 除法
- 平方根
- 平方根的倒数
- 正弦
- 余弦
- 以 2 为底的对数
- 以 2 为底的指数

这些不同的指令以 24 位精度执行操作，符合在很多游戏环境中常常使用的 24 位 RGB 设置。默认情况下，这些操作是没有启用的。计算能力 1.x 设备采取不同的简化方式，导致其单精度数学运算不兼容 IEEE 754 标准。对许多应用程序来说，这是无关紧要的。费米架构（计算能力 2.x）的硬件默认支持符合 IEEE 标准的浮点运算。

如果目标是更快，即使精度低些也没关系，则使用编译开关（-use_fast_math）或显式地使用内置操作。第一步仅需要在编译器中启用编译选项并检查现有应用程序的输出结果。这个问题的答案将会改变，至于会变化多少以及变化的重要程度，都是需要考虑的关键问题。在游戏界，如果全球飞行弹丸向左或向右偏离目标一个像素，那么没有人会注意到。而在以计算为目的的应用程序中，这种差异可能无法忽略。

单一操作可以选用 24 位数学运算，也可以通过使用显式的编译器内置指令来启用，如 __logf（x）等。对于这些内置指令的完整列表以及对使用它们所带来的弊端的解释，参见“CUDA C 编程指南”的附录 C.2。它们可以大大加快你的内核运算速度，所以如果这一选项适合你的特定代码，那么它是值得研究的。

2. 近似

在一定的搜索空间求解问题时，近似是一种有用的技术。双精度数学运算是特别昂贵的，至少比单精度浮点运算慢两倍。单精度数学运算使用 24 位尾数和 8 位指数。因此，在计算能力 1.x 的设备中，快速的基于 24 位整数的近似可以作为除单精度和双精度运算之外的第三种计算方式。请注意，在费米架构中，24 位本地整数支持替换为 32 位整数，所以在进行相同近似时，使用 24 位整数运算实际比 32 位的数学运算慢。

在支持本地双精度的所有计算硬件版本上，使用单精度近似是双精度计算速度的至少两

倍。有时单精度计算，因为占用更少的寄存器，可以达到高得多的加速，从而可能使更多线程块加载到硬件中。内存读取也只有之前的一半大小，每个元素的有效内存带宽增加一倍。消费级 GPU，在硬件中启用的双精度单元相比对应的 Tesla 系列要少一些，对这样的硬件，使单精度近似是更具吸引力的主张。

显然，使用近似你需要在速度和精度之间进行折衷，并会在程序中引入额外的复杂性。由于它可以带来显著的加速，因此这种折衷通常是值得探讨的。

一旦我们已经启用近似，内核就可以测试结果以查看它是否在一定容许范围内或符合一些准则，来保证进一步的分析是有理有据的。在数据集的一个子集上，分别进行单或双精度计算是必要的。

第一轮只为过滤数据。对于每一个兴趣准则之外的数据点，节省了昂贵的双精度运算时间。对于符合准则的每一个点，则增加了一个额外的 24 位或 32 位的过滤操作。因此，这种方法的收益取决于附加的过滤计算的相比成本相较于双精度运算的全部成本。如果过滤掉 90% 的双精度计算，则有一个巨大的加速。但是，如果那 90% 的计算部分仍需要双精度计算，那么这个策略是没有用的。

NVIDIA 声称费米架构的 Tesla 的双精度数学运算比之前计算能力 1.3 的实现（GT200 系列）快 8 倍。然而，消费级的费米架构显卡被人为限制为 Tesla 卡双精度性能的 1/4。因此，如果双精度对你的应用程序很关键，很显然采用 Tesla 是容易解决的办法。但是，有些人可能更喜欢使用多个消费级 GPU 的替代方案。两个拥有 3GB 内存的 GTX 580 将可能以更少的花费提供比单个费米架构的 Tesla 运行更快的解决方案。

如果双精度是次要的，或者你只是想在现有商用硬件上开发原型方案，那么采用 24 位单精度的过滤操作是很有吸引力的方案。另外，如果你有混合的 GPU，其中包含仍适合单精度运算的旧卡，可以使用较旧的卡扫描以定位感兴趣的部分空间，并用第二张卡在第一张卡的输出候选中详细考查问题空间。当然，用合适的 Tesla 卡，只需一张卡就能执行上述两遍过程。

3. 查找表

查找表是一个用于复杂算法的常见优化技术。对 CPU 端计算相当昂贵的算法，查找表一般能表现得相当好。其原理是，在数据空间中计算出数据中的代表点，然后应用插值方法根据与任一边缘点之间的相应距离生成中间点。这通常用于现实世界的建模中，因为线性插值的方法在拥有足够多数量的关键样本点时，可以提供一个实际信号的很好近似。

这种技术的一个变种，用在了对密文的暴力攻击。大多数系统上的密码存储为散列值即一串无明确意义的数字。散列表的特别设计使得它难以反向计算散列密码。相反，对于一个脆弱的散列表，很容易就能计算出原来的密码。

一种破解方法是基于常见短密码生成所有可能的置换。这一方法花费大量 CPU 时间。然后，攻击者只需使用刚计算好的散列值与目标散列值逐一对比，直到匹配。

在这两种情况下，查找表法用内存空间换计算时间。因此，通过简单的存储结果，就可以即时访问答案。很多人会在儿时就学习乘法表。同样的道理，由于存在大量重复乘法项，我们直接记忆结果，而不是现场繁琐计算 a * b。

当计算时间很长时，这种优化技术在CPU上，尤其是老式CPU上效果显著。然而，由于计算资源已变得越来越快，它可以花费更小地计算出结果，而不是从内存中查找它们。

平均算术指令的响应延时将会在18 ~ 24周期之间，而平均的存储器读取在400 ~ 600周期的级别。考虑到这些，你可以清楚地看到，在通过全局内存取来数据所花费的时间里，我们可以做很多计算工作。然而，这一点是假设我们将结果存储在全局内存而不是存储在共享内存或高速缓存。它也没有考虑到CPU与GPU的不同。在读取内存操作时，GPU不会闲置。事实上，GPU可能会切换到另一个线程，并执行一些其他的操作。当然，这取决于已调度到设备上的可用线程束的数量。

在许多情况下，查找方式可能战胜计算方式，尤其是当你实现了GPU的高占用率。而在低占用率处，计算方式往往胜出，当然取决于实际计算的复杂程度，假设当前的算术运算操作的延迟是20个周期，内存操作指令延迟是600个周期。显然，如果一次计算可在30次操作内完成，那么所用时间将大大快于低GPU占用下的内存查找方式，在这种情况下SM的行为就像一个串行处理器，因为它需要等待内存读取。如果具有合理的利用率，由于SM在执行其他的线程束，重叠的内存读取开销就被有效地隐藏掉了。

通常情况下，需要做些尝试以观后效，在找到可以有效增大GPU利用率的方法时，应该重新考量之前的方案。

注意，线性内插的情况下，在GPU硬件上包含基于低精度浮点运算的线性插值。这是纹理内存硬件的一个功能，并没有包括在本书中。纹理内存在计算能力1.x的硬件对于它的缓存功能（每SM 24 KB）是有用的，但这种在费米架构的用法在L1/L2高速缓推出后，在很大程度上是多余的。对于某些问题在硬件中的线性插值可能仍然是有用的。如果你对此感兴趣，请参阅“CUDA编程指南”中的“纹理和表面存储器”一章。

9.4.4 一些常见的编译器优化

我们将快速地浏览一下一些编译器的优化，看下它们是如何影响GPU的。这里强调一些优化者需要掌握的重要案例，也会让你理解优化是如何应用到源码级别的，而此处自动优化可能不适用。

有些编译器会以在某些目标机器上生成高效代码而闻名。毫无疑问，英特尔ICC编译器能够在Intel平台产生非常高效的代码。处理器的新特性迅速纳入以展示该技术。主流的编译器经常来自一个支持多种目标的代码库。这样就可以更有效地进行开发，但是编译器可能不那么容易为单一目标机器进行定制。

CUDA 4.1 SDK经历了从使用基于Open64编译器发展到使用更现代的基于LLVM编译器。从用户的角度来看最显著的好处是编译速度更快。NVIDIA还声称，代码速度提高了10%。我们看到这一改变明显地改进了代码的生成。但是，与任何新技术一样，仍存在改进的空间。可以肯定，随着时间的推移会出现这种情况。

编译器适用的优化是有据可查的。我们在这里只对一些常用优化进行宽泛的概述，对于大多数程序员来说，简单地设置优化级别就完全足够了。另一些人则喜欢知道究竟是怎么回

事，并检查输出。这些当然要在编程时间与潜在的收益和相对成本之间进行折衷。

1. 复杂运算简化

当访问数组的索引时，通常未优化的编译器代码将使用：

```
array_element_address = index * element size
```

这可以由两种技术之一更有效地取代。对第一种技术，首先，我们必须将数组的基址加载（第 0 个元素）到一个基址寄存器中的地址中。那么，我们的访问可表示成基址加偏移。我们也可以在每次循环迭代后，简单地对基址寄存器增加一个固定偏移大小（一个数组元素的字节数）。

C 语言下的示例分别如下：

```
{
 int i;
 int a[4];

 for (i=0;i<4;i++)
   a[i]=i;
}

vs.

{
 int i;

 int a[4];
 int *_ptr = a;

 for (i=0; i<4; i++)
   *_ptr++ = i;
}
```

在 GPU 使用中，这种优化依赖于这样一个事实，即某些指令（乘、除）比其他指令（加）计算花费更高的代价。而优化试图以更高效的（或更快）的操作取代高代价的操作。该技术同时适用于 CPU 和 GPU。特别地，在计算能力 2.1 设备上，整数加法是整数乘法吞吐量的 3 倍。

还要注意的是指针版本的代码创建了一个循环迭代之间的依赖关系。为了执行分配操作，ptr 的值必须是已知的。第一个例子更容易实现并行化，因为循环迭代间没有依赖并且地址 a[i] 可以静态计算。其实，简单地增加了 #pragma unroll 指令，会指示编译器展开全部的循环，因为在这个简单例子里边界是字面值。

这个例子演示了 CPU 优化中常使用的技术，以及为了让反向工程师恢复源代码所常用的循环并行技术，列在这里是为了加深你对 C 代码优化的理解，明白如何改变 C 代码可以让它在给定目标机器上运行得更快。同多数 C 源代码级的优化一样，它可能导致源代码的目的变得难懂。

2. 循环不变式分析

循环不变式分析查找在循环体内不变的表达式，并将其移动到循环体外。因此，例如，

```
for (int j=0;j<100;j++)
{
 for (int i=0; i<100; i++)
 {
   const int b = j * 200;
   q[i]= b;
 }
}
```

在这个例子中，这个参数j在参数i的循环体内是恒定的。因此，编译器可以很容易地检测到这一点，并将计算b的语句移动到内循环之外，产生如下代码：

```
for (int j=0;j<100;j++)
{
 const int b = j * 200;

 for (int i=0; i<100; i++)
 {
   q[i]= b;
 }
}
```

优化后的代码中，删除了数以千计的b和j的不必要计算（在内循环里j是恒定的，因此b也是恒定的）。然而，考虑另一种情况，b是一个函数外部的全局变量，而不是一个局部变量。例如：

```
int b;

void some_func(void)
{
 for (int j=0;j<100;j++)
 {
   for (int i=0; i<100; i++)
   {
    b = j * 200;
    q[i]= b;
   }
 }
}
```

编译器不能安全地进行优化，因为写q可能会影响b。即q和b的内存空间可以相交。它甚至不能安全地重复使用在向q赋值中的这一中间结果（j*200），而必须重新从内存中加载。因为b的内容可能在前面代码行赋值操作更改过。

如果单独考虑每行，那么这个问题会变得更清晰一些。任何内存事务，如读或写，如果该事务涉及访问当前不可用的数据，则可能会导致切换到另一个线程束。全局内存的该区域可以为任何SM的任何活动块的任何线程束上的任何线程所访问。从一个指令转移到下一个，很有可能出现任何可写的非寄存器中的数据已经改变的情况。

你可能会说，我已经将应用程序分割为N个不相交的块，所以上面的考虑是没有必要的。作为程序员，你可能知道这一点，但让编译器明白这一状况是非常困难的。因此，它会

选择安全的路线，并且不执行这种优化。许多程序员并不了解编译器的优化步骤，一旦因为优化过于激进，做了违背代码原意的事情，他们就会指责编译器。因此，在优化代码上，编译器往往是相当保守的。

作为程序员，理解这一点，可以让你做出源代码级的优化。记住把全局内存看作一个慢速 I/O 设备，从中读取一次数据，并重复使用这些数据。

3. 循环展开

循环展开是一种技术，旨在确保你在运行一个循环的开销内完成一个合理数量的数据操作。看下面的代码：

```
{
 for (i=0;i<100;i++)
   q[i]=i;
}
```

就汇编代码而言，这将产生：

- 在寄存器上加载 0，赋给参数 i。
- 在寄存器上测试 100。
- 一个分支，要么退出，要么执行循环。
- 对保存有循环计数器的寄存器加 1。
- 对下标为 i 的数组 q 计算地址。
- 将 i 存储到计算出的地址。

这些指令只在最后做了一些实在的工作。指令的其余部分都是开销。

我们可以重写这个 C 代码如下所示：

```
{
 for (i=0;i<25;i+=4)
   q[i]=i;
   q[i+1]=i+1;
   q[i+2]=i+2;
   q[i+3]=i+3;
}
```

因此，有用的工作与采用循环带来的开销之间的比例大大增加。然而，C 源代码量有所增加，而且现在所做的相对于第一个循环变得不太明显。

就 PTX 代码而言，我们可以看到每一个 C 语句都会翻译成 PTX 代码。对于每一个分支测试，现在有 4 个内存的复制操作。因此，GPU 比以前执行更多的指令，但是更高比例的内存复制操作是在做有用的工作。

在 CPU 领域，寄存器往往是有限的，因此在每个步骤中相同的寄存器会被重复使用。这样可以减少寄存器开销，但也意味着直到 q[i] 完成，q[i+1] 才能开始处理。我们会看到这种方法的 GPU 开销是相同的，每个指令都有 20 个周期的延迟，所以 GPU 把每个地址的计算分配到一个单独的寄存器。因此，我们有 4 个一组的并行指令，而不是 4 个顺序执行的指令。每组压入流水线，因此对应输出结果几乎是一个接着一个的。

使用这种方法限制的是寄存器的数目。由于GPU最多有64个（计算能力2.x和3.0）和128个（计算能力1.x），有相当大的余地可以展开小的循环体，同时实现良好的加速。

NVCC编译器支持#pragma unroll指令，它会自动展开全部的常量次循环。当循环次数不是常数，它将不会展开。如果程序员指定了循环展开，那么后者帮助不大。如果编泽器无法展开，它将报警直到代码修改或删除pragma语句。

你也可以指定#pragma unroll 4，其中的4可以替换为程序员想要的任意数值。通常情况下，4个或8个会工作得很好，但超出太多将使用过多寄存器，这会导致寄存器溢出。在计算能力1.x的硬件上，寄存器溢出到全局内存，将导致巨大的性能下降。从计算能力2.x的硬件起，寄存器先溢出到一级缓存，如果不够才会继续溢出到全局内存中。最好的解决办法是尝试一下，看看哪个值最适合每个循环。

4. 循环剥离

循环剥离是循环展开的增强技术，常用在循环次数不是循环展开大小的整数倍时。在这里，最后的数次循环分离出来，单独执行，然后展开循环的主体。

例如，如果我们有101次循环迭代，并计划使用4个层次的循环展开，前100次迭代采用循环展开，最后一次采用剥离迭代，以允许大部分的代码可以执行循环展开。与此类似，最后几次迭代要么单独作为一个循环，要么显式地执行。

循环剥离可以同样地应用到一个循环的开始。在这种情况下，它允许把一个未对齐的结构作为一个对准结构访问。例如，复制一个字节对齐的内存区到另一个字节对齐的内存是缓慢的，因为它每次只能执行一个字节的复制。前几个迭代可以剥离，以达到诸如32、64，或128字节对齐的目的。然后循环可以切换到更快的基于字、双字、四字的复制操作。在循环的后期可以再次使用基于字节的复制。

当使用#pragma loop unroll N指令时，编译器将展开循环，使得迭代次数不超过循环的边界，并在循环末端自动插入循环剥离代码。

5. 窥孔优化

这种优化寻找那些可以被同功能的、更复杂的指令代替的指令组。典型的例子是在乘法之后紧跟加法指令，常常在增益和偏移式计算中碰到。这种方式的构造可替换为更复杂的madd（乘法和加法）指令，从而将指令数目从两个减少到一个。

其他类型的窥孔优化包括控制流简化，代数运算简化和删除不会执行的代码。

6. 公共子表达式和折叠

许多程序员在编写代码时会重复一些操作，例如，

```
const int a = b[base + i] * c[base + i];
```

或者

```
const int a = b[NUM_ELEMENTS-1] * c[NUM_ELEMENTS-1];
```

在第一个例子中，数组b和c由参数base和i来索引。如果这些参数是在局部范围内起

作用，则编译器可以统一计算索引（base+i），并将该值增加到数组 b 和 c 的起始地址，同时增加到每个参数的工作地址。但是，如果任一个索引参数是全局变量，计算就必须重复，因为任何一个参数都可能已被其他同时运行的线程所改变。如果只有单个线程时，可以放心地删除第二步的计算。但使用多线程时，上述操作也可能是安全的，但编译器无法确切知道，所以通常会进行两次计算。

在第二个例子中，语句 NUM_ELEMENTS-1 被重复计算。如果我们假设 NUM_ELEMENTS 是一个宏定义，然后预处理器将其用实际值取代，所以我们可能得到 b[1024-1] * c[1024-1]。显然，这两种情况下，1024-1 都可以用 1023 代替。但如果 NUM_ELEMENTS 是一个形参，正如很多内核调用中出现的那样，这种类型的优化是不可用的。此时，我们又退回到公共子表达式优化的问题上。

因此，要注意，在函数中使用常量参数，或在全局内存中包含这样的参数，你可能会限制编译器对代码进行优化的能力。然后，你必须确保这些公共子表达式没有出现在源代码中。一般地，消除常见的子表达式，可使代码更容易理解并会提高性能。

9.4.5 分支

GPU 执行代码以线程块或线程束为单位。一条指令只被解码一次并被分发到一个线程束调度器中。在那里它将保存在一个队列中直到线程束调度器把它调度给一个由 32 个执行单元组成的集合，这个集合将执行该指令。

这个策略将指令读取和解码的时间分摊给了 N 个执行单元。其本身与旧式的向量机器非常相似。不过，主要的差异还在于 CUDA 并不需要每条指令都如此执行。如果代码中有一条分支并且只有几条指令处在该分支上，则这几条指令将会进入分支，而其他的指令在分支点等待。

这一读取 / 解码逻辑随后为那些分支线程读取指令流而其他的线程只无视它即可。实际上，线程束中的每个线程都有一个表示它执行与否的标志位。那些不在分支上的线程将清除标志位。相反，那些在此分支上的线程会设置标志位。

这种处理方式称为谓词法。当线程束中对应某个线程的标志位因为在分支上而被设置，就创造了一个谓词。大多数 PTX 运算码支持一个可选的谓词以便允许选中的线程执行指令。

因而，考虑如下的代码：

```
if (threadIdx.x < 32)
{
 if (threadIdx.x < 16)
 {
   if (threadIdx.x < 8)
    func_a1();
   else
    func_a2();
 }
 else
 {
   func_b();
 }
}
```

在代码的第一行程序就排除了当前线程块中除了第一线程束（即前32个线程）之外的其他线程束。这并不会产生该线程束中的任何分支。并没有调度该块中的其他线程束来执行这段代码。它们没有失速，只是跳过了这段代码并且继续执行之后的代码。

第一个线程束随后遇到了是否满足threadIdx.x<16的语句的测试，该判断恰好将线程束分为两半。这是一种特殊的场合，这时此线程束实际并没有产生分支。虽然线程束的大小是32，但是分支的准则实际是以半个线程束为单位的。如果你之前注意到，CUDA的核是安排成16个一组，而非32个一组。每次周期，调度器每周期将指令发送给两组或更多组核（每组为16个核）。如此一来，条件分支中的“true”（为真）和“false”（为假）路径就都能被执行到了。

在随后的步骤中，第16 ~ 31线程调用了func_b函数。然而，第0 ~ 15线程碰到了另一个分支条件。这次它不是基于一半线程束，而是1/4线程束。最小的调度数量是16个线程。因此，前8个线程组成的第一个集合跳转以调用func_a1函数，而剩下的8个线程（第8到第15）组成的第二个集合处于阻塞状态。

函数func_b和func_a1将继续独立地读取指令并把它们分发给两组均为半线程束的组。这在某种程度上比单个指令的读取低效，但是无论如何比顺序执行更好。最后func_a1会完成而随之func_a2将启动并阻塞线程0 ~ 7。与此同时，func_b可能也已经完成了。我们可以写一小段测试程序来展示这一点。

```
// All threads follow the same path
__global__ void cuda_test_kernel(
 u32 * const a,
 const u32 * const b,
 const u32 * const c,
 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < num_elements)
 {
   for (u32 iter=0; iter<MAX_ITER; iter++)
   {
    a[tid] += b[tid] * c[tid];
   }
 }
}

// Thread diverge by half warps
__global__ void cuda_test_kernel_branched_half(
 u32 * const a,
 const u32 * const b,
 const u32 * const c,
 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < num_elements)
```

```
 {
   for (u32 iter=0; iter<MAX_ITER; iter++)
   {
    if (threadIdx.x < 16)
     a[tid] += b[tid] * c[tid];
    else
     a[tid] -= b[tid] * c[tid];
   }
 }
}

// Thread diverge into one quarter group
__global__ void cuda_test_kernel_branched_quarter(
 u32 * const a,
 const u32 * const b,
 const u32 * const c,
 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < num_elements)
 {
   for (u32 iter=0; iter<MAX_ITER; iter++)
   {
    if (threadIdx.x < 16)
    {
     if (threadIdx.x < 8)
     {
       a[tid] += b[tid] * c[tid];
     }
     else
     {
       a[tid] -= b[tid] * c[tid];
     }
    }
    else
    {
     a[tid] += b[tid] * c[tid];
    }
   }
 }
}

// Thread diverge into one eighth group
__global__ void cuda_test_kernel_branched_eighth(
 u32 * const a,
 const u32 * const b,
 const u32 * const c,
 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;
```

```
if (tid < num_elements)
{
  for (u32 iter=0; iter<MAX_ITER; iter++)
  {
   if (threadIdx.x < 16)
   {
    if (threadIdx.x < 8)
    {
      if (threadIdx.x < 4)
       a[tid] += b[tid] * c[tid];
      else
       a[tid] -= b[tid] * c[tid];
    }
    else
    {
      if (threadIdx.x >= 8)
       a[tid] += b[tid] * c[tid];
      else
       a[tid] -= b[tid] * c[tid];
    }
   }
   else
   {
    a[tid] += b[tid] * c[tid];
   }
  }
}
```

这里我们建立了多个内核，每个内核会展示不同级别的分支。第一个是最优的，它没有分支。第二个基于半线程束进行分支。这些半线程束将会并发执行。之后我们进一步将第一个半线程束细分成了两组。它们将串行执行。之后我们再次将第一组细分成了一共4个串行的执行路径。我们看到的结果如下：

```
ID:0 GeForce GTX 470:Running 32768 blocks of 32 threads to calculate 1048576 elements
ID:0 GeForce GTX 470:All threads : 27.05 ms (100%)
ID:0 GeForce GTX 470:Half warps : 32.59 ms (121%)
ID:0 GeForce GTX 470:Quarter warps: 72.14 ms (267%)
ID:0 GeForce GTX 470:Eighth warps : 108.06 ms (400%)

ID:1 GeForce 9800 GT:Running 32768 blocks of 32 threads to calculate 1048576 elements
ID:1 GeForce 9800 GT:All threads : 240.67 ms (100%)
ID:1 GeForce 9800 GT:Half warps : 241.33 ms (100%)
ID:1 GeForce 9800 GT:Quarter warps: 252.77 ms (105%)
ID:1 GeForce 9800 GT:Eighth warps : 285.49 ms (119%)

ID:2 GeForce GTX 260:Running 32768 blocks of 32 threads to calculate 1048576 elements
ID:2 GeForce GTX 260:All threads : 120.36 ms (100%)
ID:2 GeForce GTX 260:Half warps : 122.44 ms (102%)
ID:2 GeForce GTX 260:Quarter warps: 149.60 ms (124%)
ID:2 GeForce GTX 260:Eighth warps : 174.50 ms (145%)
```

```
ID:3 GeForce GTX 460:Running 32768 blocks of 32 threads to calculate 1048576 elements
ID:3 GeForce GTX 460:All threads : 43.16 ms (100%)
ID:3 GeForce GTX 460:Half warps : 57.49 ms (133%)
ID:3 GeForce GTX 460:Quarter warps: 127.68 ms (296%)
ID:3 GeForce GTX 460:Eighth warps : 190.85 ms (442%)
```

我们用图 9-32 的形式更好地展示结果。

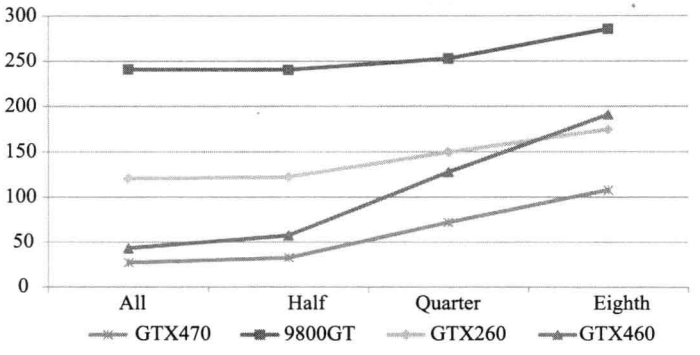

图 9-32 线程分支如何影响执行时间

注意，在计算能力 1.x 的设备上（9800GT 和 GTX260），线程是如何分支的并不是特别重要。它是有作用的，但花费的最大时间只是最优时间的 145%。通过比较，费米架构计算能力 2.x 的卡（GTX460、GTX470）在线程束中存在显著分支时会受到减速 4 倍以上的重创。GTX460 似乎对线程束分支尤为敏感。注意，当没有分支时，GTX470 几乎比 9800 GT 快了 10 倍，这是两代显卡之间的重大进步。

进一步考察在 32 路分支下的开销，在计算能力 1.x 的卡上会减速 27 倍，而在计算能力 2.x 的卡上会减速 125 ~ 134 倍。这个测试中的代码只是基于线程索引的 switch 语句，因此它无法直接与在这里用到的代码比较。然而，这种分支显然需要竭尽全力地予以避免。

在线程束中防止分支最早的方法是简单地将线程束中你不希望参与到结果中的区域用掩码标示出来。如何能办到呢？在线程束中的每个线程中都进行同样的运算，但选择一个不会对你要滤除的线程有任何贡献的值就行了。

例如，对于一个基于 32 位整数的 min 操作，为不应该参与的线程选择 0xFFFFFFFF 作为操作数。相反，对于 max、sum 以及其他许多算术型操作，只需在你不希望参与的线程中用 0 作为操作数即可。这通常将比在线程束中分支快很多。

9.4.6 理解底层汇编代码

GPU 将代码编译到一个叫做 PTX（并行线程执行指令集架构，Parallel Thread eXecution Instruction Set Architecture）的虚拟汇编系统中。这很像 Java 的字节码，它也是一种虚拟汇编语言。它既可以在编译时也可以在运行时解释成真实的、能够在设备中执行的二进制代码。编译时的解释仅插入了一些真实的二进制码到应用程序中，插入的二进制码依赖于你在命令行中选择的架构（-arch 开关）。

为了查看已生成的虚拟汇编码，只需要在编译命令行中添加 -keep 标识。对于 Visual Studio 的用户，默认的 NVIDIA 工程为保存 PTX 文件维持了一个选项（-keep）（如图 9-33 所示）。如果你偏好不让它们弄乱工程的目录结构，也可以使用“-keep–dir<directory>”选项来指明存储它们的位置。

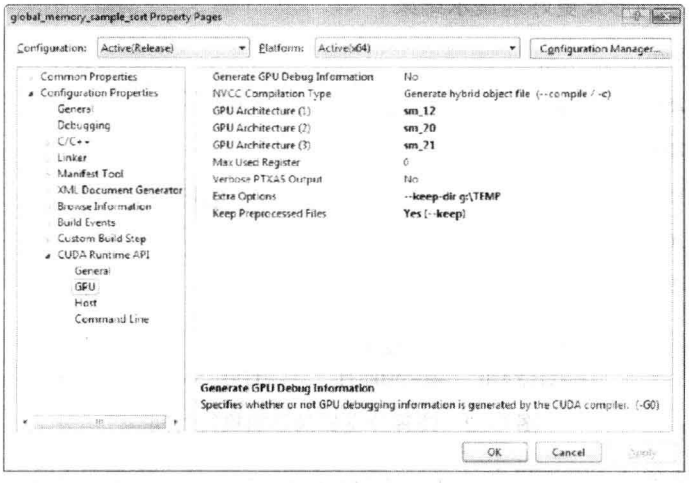

图 9-33 Visual C 选项：如何保存 PTX 文件

不过，PTX 并不是在硬件上执行的，因此它只在一定程度上有用。也可以使用如下的 cuobjdump 功能来查看经过翻译后的实际二进制码：

```
cuobjdump -sass global_mem_sample_sort.sm_20.cubin > out.txt
```

如果我们查看一个小型的设备函数，如下是在不同层次看到的东西：

```
__global__ void add_prefix_sum_total_kernel(
 u32 * const prefix_idx,
 const u32 * const total_count)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 prefix_idx[tid] += total_count[blockIdx.x];
}
```

在 PTX 中：

```
.entry _Z27add_prefix_sum_total_kernelPjPKj(
 .param .u64 _Z27add_prefix_sum_total_kernelPjPKj_param_0,
 .param .u64 _Z27add_prefix_sum_total_kernelPjPKj_param_1
)
{
 .reg .s32  %r<10>;
 .reg .s64  %rl<9>;

 ld.param.u64  %rl1, [_Z27add_prefix_sum_total_kernelPjPKj_param_0];
```

```
 ld.param.u64  %rl2, [_Z27add_prefix_sum_total_kernelPjPKj_param_1];
 cvta.to.global.u64  %rl3, %rl1;
 .loc 2 923 1
 mov.u32  %r1, %ntid.x;
 mov.u32  %r2, %ctaid.x;
 mov.u32  %r3, %tid.x;
 mad.lo.s32  %r4, %r1, %r2, %r3;
 cvta.to.global.u64  %rl4, %rl2;
 .loc 2 925 1
 mul.wide.u32  %rl5, %r2, 4;
 add.s64  %rl6, %rl4, %rl5;
 ldu.global.u32  %r5, [%rl6];
 .loc 2 925 1
 mul.wide.u32  %rl7, %r4, 4;
 add.s64  %rl8, %rl3, %rl7;
 ld.global.u32  %r6, [%rl8];
 add.s32  %r8, %r6, %r5;
 st.global.u32  [%rl8], %r8;
 .loc 2 926 2
 ret;
}
```

以及针对计算能力 2.0 设备实际生成的代码：

```
Function : _Z27add_prefix_sum_total_kernelPjPKj

/*0000*/ MOV R1, c [0x1] [0x100];
/*0008*/ S2R R0, SR_CTAid_X;
/*0010*/ S2R R2, SR_Tid_X;
/*0018*/ MOV32I R6, 0x4;
/*0020*/ IMAD R2, R0, c [0x0] [0x8], R2;
/*0028*/ IMUL.U32.U32.HI R3, R2, 0x4;
/*0030*/ IMAD.U32.U32 R4.CC, R2, R6, c [0x0] [0x20];
/*0038*/ IADD.X R5, R3, c [0x0] [0x24];
/*0040*/ IMAD.U32.U32 R6.CC, R0, R6, c [0x0] [0x28];
/*0048*/ IMUL.U32.U32.HI R0, R0, 0x4;
/*0050*/ IADD.X R7, R0, c [0x0] [0x2c];
/*0058*/ LD_LDU.E.E.32.32 R2, R0, [R4], [R6+0x0];
/*0060*/ IADD R0, R2, R0;
/*0068*/ ST.E [R4], R0;
/*0070*/ EXIT;
```

我已经把最后生成的实际原始十六进制代码移除了，因为它们在这里没有用。PTX 和目标汇编代码都使用如下的格式：

```
<instruction> <target_reg> <source_reg1> <source_reg2>
```

PTX 代码规范详尽地记录在 PTX 工业标准结构（ISA）中，可以在 NVIDIA GPU 计算工具包的“doc”目录下找到一个随 CUDA 4.1 SDK 发布的名为“ptx_isa_3.0.pdf”的文档。对于 GT200 和费米架构的二进制指令集可以在同目录下的“cuobjdump.pdf”文件中找到。对于 PTX 对应的实际的指令集还没有详细的解释，但是很容易就能看出指令对应 PTX ISA 中的位置。

虽然NVIDIA支持PTX ISA在不同硬件版本之间的前向兼容性，也就是说，针对计算能力1.X硬件的PTX可以运行在计算能力2.x的硬件上，但是二进制码却不兼容。对旧版本PTX的支持通常需要CUDA驱动程序为实际的目标硬件现场重新编译代码。

阅读下PTX ISA文档并好好理解它。它大量参考了CTA，也就是协作的线程数组。这就是在CUDA运行时层所谓的“线程块”。

在C代码中的变化将极大地影响最终生成的汇编代码。查看生成的代码并确保它在做预期的事通常是一个好的实践习惯。如果编译器导致内存的二次加载或是做一些你不希望的事情，则通常存在一个好的理由。一般地，可以在C源代码中查明原因，并解决这个问题。虽然大量底层的指令有等效的编译器固有函数可以被使用，但在某些情况下，也可以创建内联PTX以得到所需要的确切功能。

查看和理解底层的汇编函数最简单的方法之一是在Parallel Nsight中通过“View Disassembly”(查看反汇编）选项来查看源代码和汇编码的混合体。只需在CUDA代码中设置一个断点，通过Nsight菜单（“Start CUDA Debugging”，开始CUDA调试）运行代码，并等待断点执行到。然后在断点附近右键单击，出现的上下文菜单将会显示“View Disassembly”(查看反汇编)。这会弹出一个新的窗口，显示C、PTX和SASS等混合代码。例如：

```
 // 0..127 (warps 0..3)
 if (threadIdx.x < 128)
0x0002caa0 [3393] mov.u32  %r30, %tid.x;
0x0002caa0     S2R R0, SR_Tid_X;
0x0002caa8     MOV R0, R0;
0x0002cab0 [3395] setp.lt.u32  %p7, %r30, 128;
0x0002cab0     ISETP.LT.U32.AND P0, pt, R0, 0x80, pt;
0x0002cab8 [3397] not.pred  %p8, %p7;
0x0002cab8     PSETP.AND.AND P0, pt, pt, pt, !P0;
0x0002cac0 [3399] @%p8 bra  BB16_13;
0x0002cac0     NOP CC.T;
0x0002cac8     SSY 0x858;
0x0002cad0     @P0 BRA 0x850; # Target=0x0002cb50
 {
  // Accumulate into a register and then write out
  local_result += *(smem_ptr+128);
0x0002cad8 [3403] ld.u64  %rl28, [%rl7+1024];
0x0002cad8     IADD R8.CC, R2, 0x400;
0x0002cae0     IADD.X R9, R3, RZ;
0x0002cae8     MOV R10, R8;
0x0002caf0     MOV R11, R9;
0x0002caf8     LD.E.64 R8, [R10];
0x0002cb00 [3405] add.s64  %rl42, %rl42, %rl28;
0x0002cb00     IADD R6.CC, R6, R8;
0x0002cb08     IADD.X R7, R7, R9;
```

这里可以很容易地看出C语言源代码编写的对于threadIdx.x<128的测试是如何翻译成PTX的，以及每个PTX指令是如何翻译成一条或多条SASS指令的。

9.4.7 寄存器的使用

寄存器是 GPU 上最快的存储机制。它们是达到诸如设备峰值性能等指标的唯一途径。然而，它们数量非常有限。

要在 SM 上启动一个块，CUDA 运行时将会观察块对寄存器和共享内存情况的使用。如果有足够的资源，该块将启动。如果没有，那么块将不启动。驻留在 SM 中的块的数量会有所不同，但通常在相当复杂的内核上可以启动多达 6 个块，在简单内核上则可达到 8 个块（开普勒则多达 16 个块）。实际上，块数并不是主要的考虑因素。关键的因素是整体的线程数相对于最大支持数量的百分比。

我们在第 5 章列出了许多表，概述了每个块的寄存器使用数量如何影响可以被调度到一个 SM 的块的数目，以及相应地设备将会选择的线程的最终数量。

编译器提供了 -v 选项，它给出了一些关于目前分配情况更详细的输出。一个典型内核的例子如下：

```
ptxas info : Compiling entry function '_Z14functionTest' for 'sm_20'
ptxas info : Function properties for _Z14functionTest
40 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info : Used 26 registers, 8+0 bytes lmem, 80 bytes cmem[0], 144 bytes cmem[2], 52
bytes cmem[16]
```

只有当明白编译器所表达的含义时，上述输出才是有用的。第一项感兴趣的地方在于“ for sm_20”消息，它说明这里创建的代码针对计算能力 2.x 架构（费米架构）。如果专门在费米架构的目标设备上进行部署，那么要确保正确设置目标。除非另外指定，否则默认情况下，会生成计算能力 1.x 的代码，而这将限制可用的操作并且会生成对于费米架构效率不高的代码。

下一个感兴趣之处是“40 bytes of stack frame”，这通常意味着有一些可以进行取址的变量，或者声明了一个本地数组。C 语言中的术语“local”是指一个变量的范围，在 C++ 由关键字“private”取代，它能更准确地反映其含义。

CUDA 中，术语“local”是指一个给定的线程中一个变量的范围。因此，CUDA 文档还使用术语“local memory”，意思是线程的私有数据。不幸的是，“local”意味着靠近或接近，就内存而言这可能暗示数据靠近处理器。事实上，“local data”要么存储在计算能力 1.x 设备的全局内存中，要么在费米架构设备的一级缓存中。因此，仅在费米架构中它是真正靠近处理器的，而且即使在这种情况下，它的大小也是有限的。

通常会在计算能力 2.x 的设备代码中看到栈帧，尤其是在使用原子操作的时候。栈帧在一级缓存中也会存在，除非它太大了。在这些地方 CUDA 编译器可能简单地内联调用设备函数，从而移除了为被调用的函数传递形参的需求。如果创建栈帧仅仅是为了通过引用（例如指针）将值传递给设备函数，那么往往最好是取消此调用并手动为调用者编写内联函数。这将消除栈帧，并显著提高速度。

接下来的部分列出“8+0 bytes lmem”。编译器通过“ lmem”指向本地内存。如此，每次 8 个字节，若干浮点值或整数可能已存入本地内存。这通常不是一个好现象，尤其是在计算能力 1.x 设备中，因为是隐式的从慢速全局内存进行读 / 写操作。这表明需要考虑如何重新编写内核，也许该把这些值尽可能地存到共享内存或常量内存中去。

注意用在这里的“a+b”符号代表在那些区域声明变量的总量（第一个数字），然后是系统所用的总量（第二个数字）。此外，如果内核用到共享内存的话，smem（共享内存）的用量会如 lmem 一样被列出。

接下来我们看到“80 bytes cmem[0]”。这说明编译器已经使用了 80 个字节的常量内存。常量内存通常用于参数传递，因为多数形参在调用前后并不改变。方括号内的值是使用的常量内存存储体，我们并不关注。只需加和所有的 gmem 数量，就可以获得全部可用的常量内存。

寄存器的使用也可以通过在编译器中使用 -maxrregcount n 选项强制或控制。你可以使用它来指示编译器使用比现在更多或更少的寄存器。你或许希望使用更少的寄存器来允许 SM 额外调度一个块。另一种情况是可能已经被其他一些因素，如共享内存的用量，限制，因此不妨允许编译器使用更多的寄存器。通过使用更多的寄存器，编译器可以重复使用更多在寄存器中的值，而不是反复存储 / 读取它们。相反，使用更少的寄存器通常会导致更多的内存访问。

要求更少的寄存器以额外运行一个块是一种折衷的行为。寄存器数量越少，附加块将带来越高的占用率，但是这并不一定会使代码运行速度更快。这是一个大多数程序员开始学习 CUDA 时难以理解的思想。各种分析工具试图实现更高的占用率。在大多数情况下，这是一个好事，因为它允许硬件调度器可以有更多可选的线程束来运行。然而，只有当调度器实际运行的某个时刻，线程束不够用，因此 SM 阻塞的时候，增加更多的线程束才会有实际帮助。由于费米架构拥有双线程束分发器并且每 SM 中包含更多数量的 CUDA 核，可以比早期模型更高的频率执行线程束。效果随应用程序不同而异，但是减少寄存器的使用通常会导致代码执行得更慢。只有在特定应用程序中试试看才能决定。我们通过分析工具来考查 SM 是否处于阻塞状态。

一个更好的试图减少寄存器的方法是了解寄存器的使用和变量分配状况。要做到这一点，需要使用 -keep 编译标志来查阅 PTX 代码。PTX 是 CUDA 使用的虚拟汇编语言，定义了一些状态空间。变量就存在于这些状态空间中的某一个。如表 9-8 所示。如此，你总是可以在 PTX 代码中查询到每个变量的位置。

表 9-8 PTX 状态空间

状态名称	描述	速度	内核可访问性	主机可访问性	可见性
.reg	寄存器（最快）	最快	读 / 写	无	线程为单位
.const	常量内存	较快的一致访问	只读	读 / 写	上下文为单位
.global	全局内存	慢速（合并时）到特慢（无合并时）	读 / 写	读 / 写	上下文为单位
.local	私有内存	计算能力 1.x 设备上慢速；计算能力 2.x 设备上较快	读 / 写	无	线程为单位
.param（kernelcall）	主机端内核调用采用的形参	跟常量内存有关	只读	读 / 写	网格为单位
.param（devicecall）	全局函数调用设备函数时采用的形参	跟寄存器有关；通常设备函数采用内联	读 / 写	无	线程为单位
.shared	共享内存	在无存储体冲突时快速	读 / 写	无	线程块为单位

每个内核中的寄存器从 26 减少到 25 个，作用不大。然而，在寄存器临界数量（16、20、24 及 32）上过渡通常会允许调度更多的块。这将带来更多可选择的程线束，并且通常会提升性能。但情况并非总是如此，因为更多的块意味着对共享资源（共享内存、一级 / 二级缓存）更多的竞争。

减少寄存器的用量常常可以通过重新排列 C 语言源代码来实现。通过将变量的赋值和使用靠得更近，就可以使编译器重用寄存器。因此，可以在内核的开始处就声明 a、b 和 c。事实上，如果它们只是稍后才会在内核中使用，经常会发现通过把变量的创建和赋值移动到实际使用的附近就能减少寄存器的用量。然后，编译器就能够使用单个寄存器来处理这 3 个变量，因为它们处于内核中不同的而且没有联系的阶段。

9.4.8 本节小结

- 理解线程布局如何影响内存和缓存的存取模式。
- 内核启动时声明的线程数量只使用 32 的倍数。
- 思考如何增加实际每次内存读取时的工作量。
- 在优化代码和修改源代码以协助编译器时，至少应了解一些编译器的工作原理。
- 考虑如何避免线程束的分支。
- 查看 PTX 和最终的目标代码来确认编译器没有生成低效的代码。若存在低效代码，则分析原因并且修改源代码来解决。
- 了解并掌握数据被放在哪里以及编译器在表明什么。

9.5 策略 5：算法

在 GPU 上运行一个有效的算法是很具有挑战性的。虽然一个好的算法在 CPU 领域为最佳，但在 GPU 领域却未必。GPU 具有自己独特的特性。为了获得最佳性能，需要先了解 GPU 的硬件。因此，如果要考虑算法，我们需要先思考如下问题：

- 如何将问题分解成块或片，然后如何将这些块分解成线程；
- 线程如何访问数据以及产生什么样的内存模式；
- 如何分析数据重用性以及实现数据重用；
- 算法总共要执行多少工作以及与串行化的实现方法有何显著不同。

在由 Morgan Kaufman 出版的 800 多页的《GPU Computing Gems》一书中，详细介绍了以下各领域中的各类算法：

- 科学模拟
- 生命科学
- 统计模型
- 数据密集型应用程序
- 电子设计与自动化

- 光线追踪与渲染
- 计算机视觉
- 视频和图像处理
- 医学影像

本节的目的不是仅仅考虑适用于相关领域的算法，因为它们只限于少数人的兴趣。这里我们将一起看一些更为实际的通用算法，这样可以反过来有助于形成更加复杂的算法块。本书不仅仅为你提供了一系列的可供复制、粘贴的算法，同时也提供了有助于编写良好 CUDA 程序的一些概念和方法。

9.5.1 排序

可用的排序算法有很多，其中有一些可以轻松、高效地在 GPU 上实现，但也有很多不适合 GPU 实现。前几章中我们已经共同探讨了归并排序、基数排序以及更加特别的样本排序。此处我们即将一起探讨另一种并行排序算法，该算法对研究 GPU 上的算法实现很有意义。

奇偶排序

奇偶排序选择每个偶数数组索引，将其与比它大的相邻的奇数数组索引中的元素进行比较（参见图 9-34）。如果偶数索引中的元素大于奇数索引中的元素，就进行元素交换。之后重复该过程，本轮比较先将奇数索引中的元素与其相邻的更大的偶数索引中的元素进行比较。如此反复，直到不需要再进行交换，此时便完成了对列表的排序。

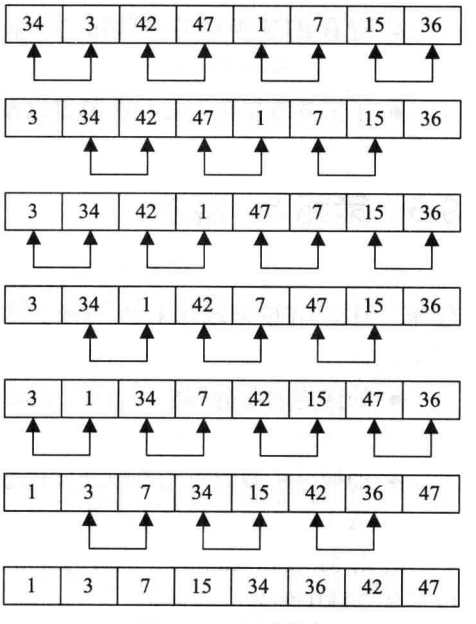

图 9-34 奇偶排序

奇偶排序是冒泡排序的改进。冒泡排序的工作原理是选择第一个索引中的元素，将其与右侧索引中的元素进行比较和交换，直至该数不再比其右侧的数字大。奇偶排序将该算法扩展为使用 P 个独立的线程分别进行比较操作，其中 P 的大小为列表中元素数目的一半。

如果定义一个大小为 N 的数组，使用 N/2 个线程进行处理似乎是一件很吸引人的事情。排序算法很容易理解，但当我们尝试在 GPU 上实现时却会产生许多有趣的问题，因此，这个例子值得我们仔细研究。

首先我们遇到的一个问题是，奇偶排序是专为并行系统设计，每个线程可以与它相邻的线程交换数据元素，并且只需要与它左边和右边相邻的两个线程建立连接即可。至于连接可通过共享内存实现。

如果 0 号线程访问 0 号和 1 号元素，1 号线程访问 2 号和 3 号元素，这将导致硬件合并访问的序列化问题。合并访问要求每个线程以连续的方式进行访问。因此，在计算能力 1.x

的硬件上，这种访问模式的效率非常差，其将导致对 32 字节的字进行多次获取。然而，在计算能力 2.x 的硬件上，大多数访问可以在两条缓存线上获取。偶数周期中获取到的数据可能在奇数周期中也使用到，反之亦然。另外，当数据明显多次重复利用且需要高度本地化时，缓存或共享内存是一个很好的选择。对于计算能力 1.x 的设备而言，共享内存是解决低聚合访问的唯一选择。

如果我们使用共享内存，则需要考虑存储体冲突。如果 0 号线程需要读取 0 号和 1 号存储体内的数据，然后相加将结果写回 0 号存储体。1 号线程读取 2 号和 3 号存储体，相加之后将结果写回 2 号存储体。在计算能力 1.x 的硬件上，系统有 16 组存储体，因此 8 号线程将折回再次读取 0 号和 1 号存储体中的内容。而在计算能力 2.x 硬件上，从 16 号线程开始我们将看到同样的效果。因此，当基于共享内存实现算法时，在计算能力 1.x 的硬件上会出现 4 次存储体冲突，而在计算能力 2.x 的硬件上会出现两次存储体冲突。

CPU 版的奇偶排序代码非常简单：

```
void odd_even_sort_cpu(u32 * const data,
                       const u32 num_elem)
{
 u32 offset = 0; // Start off with even, then odd
 u32 num_swaps; // Keep track of the number of swaps
 u32 run = 0;  // Keep track of the number of iterations

 printf("\nSorting %u elements using odd/even sort on cpu\n", num_elem);
 print_array(run, data, num_elem);

 do
 {
  run++;
  num_swaps = 0; // Reset number of swaps each iteration

  // Iterate over 0..num_elements OR
  // 1..(num_elements-1) in steps of two
  for (u32 i=offset; i<(num_elem-offset); i+=2)
  {
   // Read values into registers
   const u32 d0 = data[i];
   const u32 d1 = data[i+1];

   // Compare registers
   if ( d0 > d1 )
   {
    // Swap values if needed
    data[i] = d1;
    data[i+1] = d0;

    // Keep track that we did a swap
    num_swaps++;
   }
  }
```

```
    // Switch from even to odd, or odd to even
    if (offset == 0)
     offset = 1;
    else
     offset = 0;

    // If something swapped then print the array
    if (num_swaps > 0)
     print_array(run, data, num_elem);

   // While elements are still being swapped
   } while (num_swaps != 0);
}
```

这段代码首先对整个数据集的 0 号元素 ~ num_elem-1 号元素进行遍历，然后从数据集的 1 号元素 ~ num_elem-2 号元素进行遍历。每次循环将两个数据元素读到一个局部变量中然后进行比较，如果需要则进行交换。此外，我们使用 num_swaps 这个变量对本次遍历中交换操作的次数进行计数。当不再进行交换操作时，列表即已排好序。

对于一个几乎有序的列表，该算法十分适用。当列表初始是倒序排列时是最坏的情况，此时我们要将列表前面的元素移动到最后面。当列表为倒序排列时，排序的步骤如下所示。我们可以看到每个阶段中各单元之间的元素交换细节。

```
Run 000: 15 14 13 12 11 10 09 08 07 06 05 04 03 02 01 00
Run 001: 14 15 12 13 10 11 08 09 06 07 04 05 02 03 00 01
Run 002: 14 12 15 10 13 08 11 06 09 04 07 02 05 00 03 01
Run 003: 12 14 10 15 08 13 06 11 04 09 02 07 00 05 01 03
Run 004: 12 10 14 08 15 06 13 04 11 02 09 00 07 01 05 03
Run 005: 10 12 08 14 06 15 04 13 02 11 00 09 01 07 03 05
Run 006: 10 08 12 06 14 04 15 02 13 00 11 01 09 03 07 05
Run 007: 08 10 06 12 04 14 02 15 00 13 01 11 03 09 05 07
Run 008: 08 06 10 04 12 02 14 00 15 01 13 03 11 05 09 07
Run 009: 06 08 04 10 02 12 00 14 01 15 03 13 05 11 07 09
Run 010: 06 04 08 02 10 00 12 01 14 03 15 05 13 07 11 09
Run 011: 04 06 02 08 00 10 01 12 03 14 05 15 07 13 09 11
Run 012: 04 02 06 00 08 01 10 03 12 05 14 07 15 09 13 11
Run 013: 02 04 00 06 01 08 03 10 05 12 07 14 09 15 11 13
Run 014: 02 00 04 01 06 03 08 05 10 07 12 09 14 11 15 13
Run 015: 00 02 01 04 03 06 05 08 07 10 09 12 11 14 13 15
Run 016: 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15
```

对于该算法的 GPU 实现，我们采用了计算能力为 2.x 设备上的全局内存实现，具体代码如下：

```
__global__ void odd_even_sort_gpu_kernel_gmem(
 u32 * const data,
 const u32 num_elem)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 u32 tid_idx;
 u32 offset = 0; // Start off with even, then odd
 u32 num_swaps;
```

```
// Calculation maximum index for a given block
// Last block it is number of elements minus one
// Other blocks to end of block minus one
const u32 tid_idx_max = min((((blockIdx.x+1) * (blockDim.x*2))-1),(num_elem-1));

do
{
  // Reset number of swaps
  num_swaps = 0;

  // Work out index of data
  tid_idx = (tid * 2) + offset;

  // If no array or block overrun
  if (tid_idx < tid_idx_max)
  {
   // Read values into registers
   const u32 d0 = data[tid_idx];
   const u32 d1 = data[tid_idx+1];

   // Compare registers
   if ( d0 > d1 )
   {
    // Swap values if needed
    data[tid_idx] = d1;
    data[tid_idx+1] = d0;

    // Keep track that we did a swap
    num_swaps++;
   }
  }

  // Switch from even to off, or odd to even
  if (offset == 0)
   offset = 1;
  else
   offset = 0;

 } while (__syncthreads_count(num_swaps) != 0);
}
```

在循环结构上，CPU 代码使用了 do..while 循环结构，而没有使用传统的 for 循环结构。很明显，在该算法中，可以并行执行的操作就只有比较和交换操作，因此，当需要处理的数组大小为 N 时，我们需要使用 N/2 个线程进行处理。考虑到大多数在 GPU 上需要排序的列表都比较大，因此，这使得我们能够潜在地最大化利用一个设备上的线程（GTX580 上最多为 24 576 个线程）。

由于每个线程处理两个元素，我们不能简单地使用 tid 作为数组索引，因此，我们创建一个新的局部变量 tid_idx，保存该线程访问的数组的索引。另外，我们还创建了一个局部变量 tid_idx_max，用于设置数组的最后一个元素的值，如果当前有多个线程块时，则该值为该

线程块能访问数组元素的最大索引。

```
// Calculation maximum index for a given block
// Last block it is number of elements minus one
// Other blocks to end of block minus one
const u32 tid_idx_max = min( (((blockIdx.x+1) * (blockDim.x*2))-1),(num_elem-1));
```

循环结束条件的判断有些复杂。在串行版本的奇偶排序中，每次迭代只需对参数 num_swaps 执行一次写操作。而在并行版本奇偶排序中，我们需要知道是否有线程执行了交换操作。我们可以使用原子加法、增量、与以及或操作，但这些操作意味着每个线程的写操作必须顺序执行，这会造成序列化瓶颈。

我们可以通过使用基于共享内存的原子操作以在一定程度上缓解原子操作的开销。但需要注意的是，只有计算能力为 1.2 及以上（GT200 系列）的硬件才支持基于共享内存的原子操作。对于计算能力为 1.1 的硬件（9000 系列），我们只能使用基于全局内存的原子操作，与此同时变量 num_swaps 的声明也需要相应修改。

对于计算能力为 2.x 的硬件，我们可以使用另一个更快的解决方案，即在每轮循环结束时，使用新提供的原语 __syncthreads_count 进行等待，通过对其传递的参数判断来决定是否需要执行下一轮循环。只要所有线程中有一个传递的参数为非零，则所有线程中该函数的计算结果均为非零。因此，只要有一个线程执行了交换操作，所有线程将进行下一轮循环。

```
} while (__syncthreads_count(num_swaps) != 0);
```

主机端对内核进行调用的完整函数如下所示：

```
// Host function - copy to / from and invoke kernel
__host__ void odd_even_sort_gpu_gmem(
 u32 * const data_cpu,
 const u32 num_elem)
{
 const u32 size_in_bytes = (num_elem * sizeof(u32));
 u32 * data_gpu;

 // Allocate memory on the device
 CUDA_CALL(cudaMalloc((void **) &data_gpu,
                      size_in_bytes));

 // Copy data to GPU
 CUDA_CALL(cudaMemcpy(data_gpu, data_cpu, size_in_bytes, cudaMemcpyHostToDevice));

 // Use blocks of 256 threads
 const u32 num_threads = 256;
 const u32 num_blocks = (((num_elem/2) + (num_threads-1)) / num_threads);

 printf("\nInvoking Odd Even sort with %d blocks of
    %d threads (%u active)", num_blocks,
    num_threads, (num_elem / 2));

 // Invoke the kernel
```

```
odd_even_sort_gpu_kernel_gmem<<<num_blocks, num_threads>>>(data_gpu, num_elem);

cuda_error_check( "Error Invoking kernel",
      "odd_even_sort_gpu_kernel_gmem");

// Copy back to CPU memory space
CUDA_CALL(cudaMemcpy(data_cpu, data_gpu, size_in_bytes, cudaMemcpyDeviceToHost));

// Free memory on the device
CUDA_CALL(cudaFree(data_gpu));

print_array(0, data_cpu, num_elem);
}
```

对于这段代码，我们需要关心的是线程块的边界处理。对同一个较小的数据集，我们分别使用一个和两个线程块进行排序，得到的结果如下所示：

```
Invoking Odd Even sort with 1 blocks of 8 threads (8 active)
Run 000: 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15
Test passed
```

以及

```
Invoking Odd Even sort with 2 blocks of 4 threads (8 active)
Run 000: 08 09 10 11 12 13 14 15 00 01 02 03 04 05 06 07
Test failed
```

注意在第二个输出中我们看到排序只在块内执行，左边的值需要移动到右端，反之右端的值也一样。然而，因为块与块之间不重叠，因此无法这样做。一个显而易见的解决方案就是块与块之间进行重叠，但很明显这不是一个理想的解决方案。

CUDA 的设计模式允许线程块以任何顺序执行，当启动一个内核之后，块与块之间无法进行同步操作。我们可以启动多个内核进行线程块之间的同步，但这种方法只有当需要少量同步操作时才适用。而在该内核中，每轮迭代我们都需要重叠线程块之间的同步操作。由于两个 SM 需要共享相同的数据集，这将失去所有的局部性。我们需要通过基于全局内存的原子操作或归约操作来跟踪每个线程块是否发生了交换操作，判断是否需要继续调用内核进行下一轮迭代，中间涉及了许多主机端的交互操作。很明显，这并不是一个理想的解决方案。

至此，从绝大多数通过分块解决的排序算法中我们只找到两种方案可以解决这个问题。一种是像样本排序算法那样，将输入数据集预先分拣到多个列表中，列表 N_{-1} 中的元素值都比列表 N_0 中的小，列表 N_0 中的元素值都比列表 N_1 中的大，依此类推。第二种方案就是将这 N 个独立的列表合并，但这又会产生之前归并排序中出现的问题。

9.5.2 归约

归约是并行编程中一种常用的技巧。我们将通过不同的方法执行归约，找出在不同的计算平台上哪种方案才能得到最优的结果，并尝试解释其原因。

首先我们通过一个计算 N 个 32 位整数总和的例子来初步介绍归约。我们将使用一个

包含48 000 000个元素的数据集作为测试数据集，使用如此大的数据集使得我们的试验更加具有说服性。对这么大的数据集进行计算，第一个需要考虑到的就是溢出。如果对0xFFFFFFFF与0x00000001做加法操作，对于32位的整型数而言，就会产生溢出。因此，我们需要使用64位的数来进行累加操作。但这也存在一些问题。

由于溢出的问题，任何基于原子操作的累加都需要一个原子64位整型加法操作。然而不幸的是，只有计算能力为2.x的硬件才支持共享内存上64位的原子加法操作，而在全局内存上，只有计算能力为1.2及以上的硬件才支持64位的原子加法操作。

1. 基于全局内存的原子加法

让我们先来看看最简单的归约：

```
// Every thread does atomic add to the same
// address in GMEM
__global__ void reduce_gmem(const u32 * const data,
                            u64 * const result,
                            const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < num_elements)
  atomicAdd(result, (u64) data[tid]);
}
```

在这个例子中，每个线程从内存中读取一个元素，并把它累加到全局内存中的一个结果变量中。虽然这个方法很简单，但它可能是归约操作中最低效的一种形式。使用块与块间的原子操作意味着需要使用所有SM都能访问的共享变量。

在老式的硬件中，这意味着对全局内存进行写操作，而计算能力为2.x的硬件可以将结果先写到二级缓存上，实现所有SM间的共享，最终再将结果写回全局内存。以下是程序运行的结果：

```
Processing 48 MB of data, 12M elements
ID:0 GeForce GTX 470:GMEM   passed Time 197.84 ms
ID:3 GeForce GTX 460:GMEM   passed Time 164.28 ms
```

此处，我们只看支持64位整型原子操作的计算能力为2.x设备上的结果。

即使是针对二级缓存，原子写操作也一直存在一个问题，即其使线程之间的执行串行化。在这个问题中，每个SM处理6个线程块，每个线程块启动256个线程，每个SM一共处理1536个线程。在GTX470上一共有14个SM，因此每次有21 504个活跃线程。针对一个全局内存单元执行原子操作意味着会产生10K ~ 21K个串行执行的线程，在每个线程中的元素处理之前，该线程一直需要排队等待。很明显，这种解决方案极其低效，即使它非常简单。

2. 线程内归约

我们可以通过线程间的归约来改善当前算法。只需要对数据类型以及内核稍作修改，以确保不会数组越界。

```
// Every thread does atomic add to the same
// address in GMEM
__global__ void reduce_gmem_ILP2(const uint2 * const data,
                                 u64 * const result,
                                 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < (num_elements>>1))
 {
  uint2 element = data[tid];

  const u64 add_value = ((u64)element.x) +
                        ((u64)element.y);

  atomicAdd(result, add_value);
 }
}

// Every thread does atomic add to the same
// address in GMEM
__global__ void reduce_gmem_ILP4(const uint4 * const data,
                                 u64 * const result,
                                 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < (num_elements>>2))
 {
  uint4 element = data[tid];

  const u64 add_value = ((u64)element.x) +
                        ((u64)element.y) +
                        ((u64)element.z) +
                        ((u64)element.w);

  atomicAdd(result, add_value);
 }
}
```

在第一个例子中，我们通过使用内置向量类型 uint2 与 uint4，使每个线程分别处理两个元素和 4 个元素，运行时间的结果如下：

```
ID:0 GeForce GTX 470:GMEM ILP2 passed Time 98.96 ms
ID:3 GeForce GTX 460:GMEM ILP2 passed Time 82.37 ms

ID:0 GeForce GTX 470:GMEM ILP4 passed Time 49.53 ms
ID:3 GeForce GTX 460:GMEM ILP4 passed Time 41.47 ms
```

尽管执行时间明显减少，但我们并没有真正解决这个问题。我们所做的只是在线程块内归约使得每个线程需要排队的次数减少为原先的一半或者 1/4，据此减少总的执行时间。很

明显，两个方案的执行时间分别也对应减少为原先的一半以及1/4。然而，仍然有5K的线程需要排队等待对全局内存执行写操作。

注意，在执行局部加法时，我们减少了对全局内存的写操作，减少的倍数跟ILP的级别有关。但我们仍需要注意加法操作是如何执行的。具体代码我们按以下方式书写：

```
const u64 add_value = ((u64)element.x) + element.y + element.z + element.w;
```

在C语言中，一个表达式通常从左到右依次计算。对操作符左边的值进行强制类型转换会隐式地导致操作符右边的值也强制类型转换。因此，我们可能期望通过将element.x强制转换成无符号的64位整型，使需要与其做加法操作的element.y也强制转换成无符号的64位整型数，接着element.z与element.w也发生强制转换，与之前结果相加。但这意味着这4个数的加法操作是顺序执行的。然而，元素z与w的计算根本不需要依赖于元素x与y。事实上，PTX代码就是这样做的。由于元素z与w并没有被强制转换成64位的整型数，其仍是32位的整型数，因此，计算可能出现溢出。

这个问题出现的根本原因是C语言中，当操作符可交换时，其允许计算以任何顺序执行。然而通常看到的是从左到右的计算方式，因此我们自认为编译器就是这样执行的。事实上这是C语言编译器的一个可移植性问题。当在如GPU超标量处理器上时，编译器独立运行两套加法操作，以最大限度地利用流水线。我们一点也不希望在等待18 ~ 22个加法周期完成第一个加法操作后再顺序执行第二个加法操作。

因此，正确的加法操作应该按如下方式书写：

```
const u64 add_value = ((u64)element.x) + ((u64)element.y) + ((u64)element.z) + ((u64)element.w);
```

在每次加法操作执行之前，我们就强制将每个值转换成64位整型数。之后加法操作无论以怎样的顺序执行都是对64位的整型数计算。然而，对于浮点数，这样简单地强制转换成双精度型是远远不够的。由于浮点型并没有期望的精度，根据浮点型的工作方式，若把一个非常小的数加到一个大数上，会造成小数被丢弃掉。对于这类问题，最好的解决方案就是首先对浮点值进行排序，然后从小到大依次处理。

我们可以通过使用多个uint4类型数据对内核进行简单地修改，从而更进一步利用ILP技术。

```
// Every thread does atomic add to the same
// address in GMEM
__global__ void reduce_gmem_ILP8(const uint4 * const data,
                                 u64 * const result,
                                 const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < (num_elements>>3))
 {
  const u32 idx = (tid * 2);

  uint4 element = data[idx];
  u64 value = ((u64)element.x) +
```

```
            ((u64)element.y) +
            ((u64)element.z) +
            ((u64)element.w);

  element = data[idx+1];
  value += ((u64)element.x) +
            ((u64)element.y) +
            ((u64)element.z) +
            ((u64)element.w);

  atomicAdd(result, value);
 }
}

// Every thread does atomic add to the same
// address in GMEM
__global__ void reduce_gmem_ILP16(const uint4 * const data,
                                  u64 * const result,
                                  const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < (num_elements>>4))
 {
  const u32 idx = (tid * 4);

  uint4 element = data[idx];
  u64 value = ((u64)element.x) +
              ((u64)element.y) +
              ((u64)element.z) +
              ((u64)element.w);

  element = data[idx+1];
  value += ((u64)element.x) +
            ((u64)element.y) +
            ((u64)element.z) +
            ((u64)element.w);

  element = data[idx+2];
  value += ((u64)element.x) +
            ((u64)element.y) +
            ((u64)element.z) +
            ((u64)element.w);

  element = data[idx+3];
  value += ((u64)element.x) +
            ((u64)element.y) +
        ((u64)element.z) +
        ((u64)element.w);

  atomicAdd(result, value);
 }
}
```

```
// Every thread does atomic add to the same
// address in GMEM
__global__ void reduce_gmem_ILP32(const uint4 * const data,
                                  u64 * const result,
                                  const u32 num_elements)
{
 const u32 tid = (blockIdx.x * blockDim.x) + threadIdx.x;

 if (tid < (num_elements>>5))
 {
  const u32 idx = (tid * 8);

  uint4 element = data[idx];
  u64 value = ((u64)element.x) +
              ((u64)element.y) +
              ((u64)element.z) +
              ((u64)element.w);

  element = data[idx+1];
  value += ((u64)element.x) +
           ((u64)element.y) +
           ((u64)element.z) +
           ((u64)element.w);

  element = data[idx+2];
  value += ((u64)element.x) +
           ((u64)element.y) +
           ((u64)element.z) +
           ((u64)element.w);

  element = data[idx+3];
  value += ((u64)element.x) +
          ((u64)element.y) +
           ((u64)element.z) +
           ((u64)element.w);

  element = data[idx+4];
  value += ((u64)element.x) +
           ((u64)element.y) +
           ((u64)element.z) +
           ((u64)element.w);
 element = data[idx+5];
 value += ((u64)element.x) +
          ((u64)element.y) +
          ((u64)element.z) +
          ((u64)element.w);

 element = data[idx+6];
 value += ((u64)element.x) +
          ((u64)element.y) +
          ((u64)element.z) +
```

```
            ((u64)element.w);

    element = data[idx+7];
    value += ((u64)element.x) +
            ((u64)element.y) +
            ((u64)element.z) +
            ((u64)element.w);

    atomicAdd(result, value);
  }
}
```

注意，此处我们将数据加载与加法操作结合到一起。我们可以将所有的数据加载移到函数的开始处执行。然而，考虑到每个 uint4 数据类型需要 4 个寄存器，这对于 ILP32 内核而言意味着需要 32 个寄存器来保存一次迭代读取出的数据值。此外，这些值做完加法后只有在最后才执行一次写操作。因此，如果我们使用过多的寄存器将导致可调度线程块的数量缩减，或者将寄存器中的值溢出到本地内存中。在计算能力为 2.x 的设备上，本地内存即一级缓存，而在计算能力为 1.x 的设备上，本地内存即全局内存。以下是这几个 ILP 内核函数的运行结果：

```
ID:0 GeForce GTX 470:GMEM ILP8 passed Time 24.83 ms
ID:3 GeForce GTX 460:GMEM ILP8 passed Time 20.97 ms

ID:0 GeForce GTX 470:GMEM ILP16 passed Time 12.49 ms
ID:3 GeForce GTX 460:GMEM ILP16 passed Time 10.75 ms

ID:0 GeForce GTX 470:GMEM ILP32 passed Time 13.18 ms
ID:3 GeForce GTX 460:GMEM ILP32 passed Time 15.94 ms
```

通过运行结果我们看到，使用 ILP 内核的运行时间明显减少很多。注意，ILP32 解决方案的运行时间比 ILP16 更长。尽管相对于原先最简单的版本，现在的内核实现了 20 倍以上的加速比，但我们仍没有解决原子排队写问题，我们所做的只是减少了所有需要排队的元素数目。目前为止仍有大量活跃线程（10K ~ 20K）需要排队等待对同一个累加值执行写操作。

3. 基于块数目的归约

当前，我们使用了 N 块，其中 N 是问题规模大小，(具体计算过程为：1200 万个元素数（48MB）除以每块包含的线程数再除以每线程处理的元素数)。最终我们需要执行 N 次原子写操作，所有线程顺序执行，造成程序性能瓶颈。

我们可以通过创建更少的块以及大量增加每个块的执行任务来减少竞争。然而，这样做的前提是必须同 ILP32 的例子中那样不增加寄存器的使用量。否则，由于寄存器资源的缺乏导致对本地内存进行读 / 写操作，最终使程序的性能明显下降。

目前为止，我们一共开启了 48K 个块，但其数目可以减少至 16、32、64、128 或 256 块。然后我们可以让每个线程对内存进行扫描，将结果累加到寄存器中，当每个块完成之后，再将结果写回全局内存。基于块的数量，这样做会在 SM 之间产生高效的本地内存引用，这样就充分利用了内存带宽，如果存在 L2 缓存，其也能得到充分利用。

```
// Every thread does atomic add to the same
// address in GMEM after internal accumulation
__global__ void reduce_gmem_loop(const u32 * const data,
                                 u64 * const result,
                                 const u32 num_elements)
{
 // Divide the num. elements by the number of blocks launched
 // ( 4096 elements / 256 threads) / 16 blocks = 1 iter
 // ( 8192 elements / 256 threads) / 16 blocks = 2 iter
 // (16384 elements / 256 threads) / 16 blocks = 4 iter
 // (32768 elements / 256 threads) / 16 blocks = 8 iter
 const u32 num_elements_per_block = ((num_elements / blockDim.x) / gridDim.x);

 const u32 increment = (blockDim.x * gridDim.x);

 // Work out the initial index
 u32 idx = (blockIdx.x * blockDim.x) + threadIdx.x;

 // Accumulate into this register parameter
 u64 local_result = 0;

 // Loop N times depending on the number of
 // blocks launched
 for (u32 i=0; i<num_elements_per_block; i++)
 {
  // If still within bounds, add into result
  if (idx < num_elements)
   local_result += (data[idx]);

  // Move to the next element in the list
  idx += increment;
 }

 // Add the final result to the GMEM accumulator
 atomicAdd(result, local_result);
}
```

这段代码首先计算出对该数据集每个线程应该执行迭代多少次。参数 gridDim.x 记录启动的线程块的数目。每个线程块由 blockDim.x 个线程组成。因此，我们可以计算出每个线程需要累加的元素数目。然后，将每个线程读取到的数据累加到 local_result 中，当线程完成其对应块的处理之后再一次性地将结果写回全局内存。

这样我们就将线程级的竞争转换成了块级的竞争。由于我们仅仅使用了几百个块，因此它们同时执行写操作的概率相对较低。很明显，当我们增加块的数量，竞争也会变得更加激烈。一旦所有 SM 都最大程度地加载线程块进行处理，考虑到负载平衡，就没有必要再增加块的数目了。

由于 GTX460 只有 7 个 SM，每个 SM 最多处理 6 个线程块，设备每次最多只能加载处理 42 个线程块，因此在它上面运行该程序得到的结果可能是最糟的。因此，我们尝试将块的数目从 49 152 个降低到 16 的平方个，更少的块可以更加充分地利用 SM。运行得到的结果如下所示：

从运行结果中我们发现，无论是在 GTX470 还是 GTX460 上，随着每轮减少一半的块，

```
ID:0 GeForce GTX 470:GMEM loop1 49152 passed Time 197.82 ms
ID:0 GeForce GTX 470:GMEM loop1 24576 passed Time 98.96 ms
ID:0 GeForce GTX 470:GMEM loop1 12288 passed Time 49.56 ms
ID:0 GeForce GTX 470:GMEM loop1 6144 passed Time 24.83 ms
ID:0 GeForce GTX 470:GMEM loop1 3072 passed Time 12.48 ms
ID:0 GeForce GTX 470:GMEM loop1 1536 passed Time 6.33 ms
ID:0 GeForce GTX 470:GMEM loop1 768 passed Time 3.35 ms
ID:0 GeForce GTX 470:GMEM loop1 384 passed Time 2.26 ms
ID:0 GeForce GTX 470:GMEM loop1 192 passed Time 1.92 ms
ID:0 GeForce GTX 470:GMEM loop1 96 passed Time 1.87 ms
ID:0 GeForce GTX 470:GMEM loop1 64 passed Time 1.48 ms
ID:0 GeForce GTX 470:GMEM loop1 48 passed Time 1.50 ms
ID:0 GeForce GTX 470:GMEM loop1 32 passed Time 1.75 ms
ID:0 GeForce GTX 470:GMEM loop1 16 passed Time 2.98 ms

ID:3 GeForce GTX 460:GMEM loop1 49152 passed Time 164.25 ms
ID:3 GeForce GTX 460:GMEM loop1 24576 passed Time 82.45 ms
ID:3 GeForce GTX 460:GMEM loop1 12288 passed Time 41.52 ms
ID:3 GeForce GTX 460:GMEM loop1 6144 passed Time 21.01 ms
ID:3 GeForce GTX 460:GMEM loop1 3072 passed Time 10.77 ms
ID:3 GeForce GTX 460:GMEM loop1 1536 passed Time 5.60 ms
ID:3 GeForce GTX 460:GMEM loop1 768 passed Time 3.16 ms
ID:3 GeForce GTX 460:GMEM loop1 384 passed Time 2.51 ms
ID:3 GeForce GTX 460:GMEM loop1 192 passed Time 2.19 ms
ID:3 GeForce GTX 460:GMEM loop1 96 passed Time 2.12 ms
ID:3 GeForce GTX 460:GMEM loop1 64 passed Time 2.05 ms
ID:3 GeForce GTX 460:GMEM loop1 48 passed Time 2.41 ms
ID:3 GeForce GTX 460:GMEM loop1 32 passed Time 1.96 ms
ID:3 GeForce GTX 460:GMEM loop1 16 passed Time 2.70 ms
```

执行时间也几乎线性递减。我们通过增加每个线程的计算任务，而不是通过增加 ILP 的数目（此处即 loop1），来减少块的数目。

注意，在之前的例子中 GTX460 执行效果一直比 GTX470 好。然而，在本例中，当块的数目减少到一定程度时，即块的数目减少为 384 时，GTX460 的执行效果却不如 GTX470 的执行效果（参见图 9-35）。此时，GTX470 拥有更多 SM（尽管每个 SM 中的 SP 数目比 GTX460 的少，GTX470 有 32 个 CUDA 核，GTX460 有 48 个 CUDA 核）以及更大缓存的特点开始逐渐影响程序的性能。

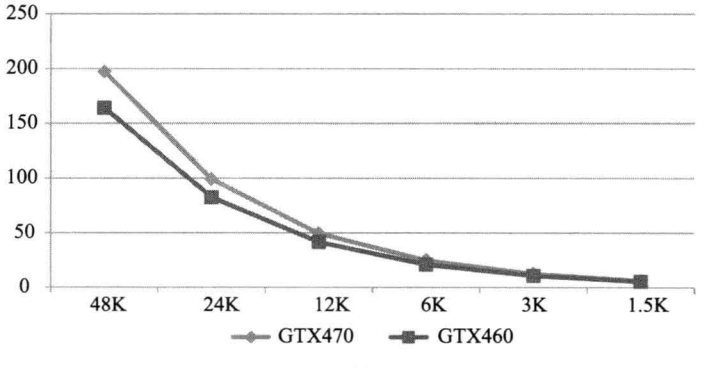

图 9-35 执行时间与线程块数目（线程块数目多的情况）

我们进一步观察当块的数目较少时的运行情况。可以看出，在SM调度/占用问题起作用之前，执行时间的最小值出现在64块附近（如图9-36所示）。此处，我们将该图按横坐标分成两部分，一部分显示了具有较多块时运行的时间趋势，另一部分显示了具有较少块时的运行时间趋势，这样，就方便我们观察块数较小时的运行效果。

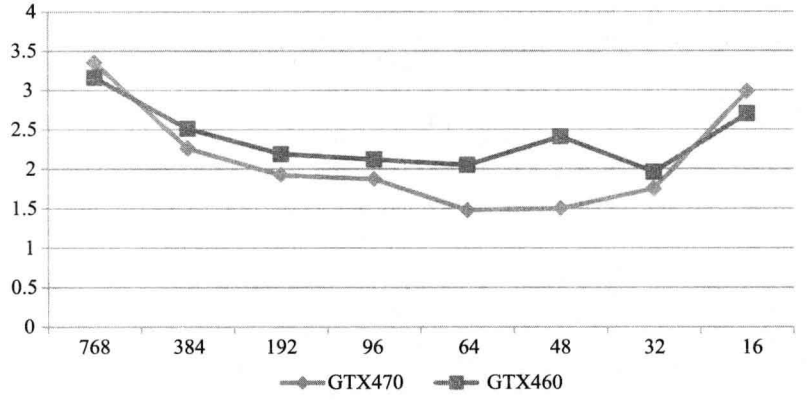

图9-36 执行时间与线程块数目（线程块数目少的情况）

注意，到目前为止我们并没有使用ILP（指令级并行）。但我们知道引入ILP可以让我们获得更少的运行时间，特别是当块的数目很少时。之前，当块的数目为64时运行时间最优。GTX470的每个SM只需要处理4个块，32个线程束。由于是32位的内存访问，我们需要完全负载每个SM，即每个SM处理48个线程束，才能达到全局内存的最大存储带宽。当我们保持线程束的数目不变时，只有通过利用ILP技术才能做到这一点。

```
// Every thread does atomic add to the same
// address in GMEM after internal accumulation
__launch_bounds__(256)
__global__ void reduce_gmem_loop_ILP2(
 const uint2 * const data,
 u64 * const result,
 const u32 num_elements)
{
 const u32 num_elements_per_block = (( (num_elements / 2) / blockDim.x) /gridDim.x);
 const u32 increment = (blockDim.x * gridDim.x);

 // Work out the initial index
 u32 idx = (blockIdx.x * blockDim.x) + threadIdx.x;

 // Accumulate into this register parameter
 u64 local_result = 0;

 // Loop N times depending on the number
 // of blocks launched
 for (u32 i=0; i<num_elements_per_block; i++)
 {
```

```
  // If still within bounds, add into result
  if (idx < (num_elements>>1))
  {
   const uint2 elem = data[idx];

   local_result += ((u64)elem.x) + ((u64)elem.y);

   // Move to the next element in the list
   idx += increment;
  }
 }

 // Add the final result to the GMEM accumulator
 atomicAdd(result, local_result);
}

// Every thread does atomic add to the same
// address in GMEM after internal accumulation
__launch_bounds__(256)
__global__ void reduce_gmem_loop_ILP4(
 const uint4 * const data,
 u64 * const result,
 const u32 num_elements)
{
 const u32 num_elements_per_block = (( (num_elements/4) / blockDim.x) / gridDim.x);
 const u32 increment = (blockDim.x * gridDim.x);

 // Work out the initial index
 u32 idx = (blockIdx.x * blockDim.x) + threadIdx.x;

 // Accumulate into this register parameter
 u64 local_result = 0;

 // Loop N times depending on the number
 // of blocks launched
 for (u32 i=0; i<num_elements_per_block; i++)
 {
  // If still within bounds, add into result
  if (idx < (num_elements>>2))
  {
   const uint4 elem = data[idx];

   local_result += ((u64)elem.x) + ((u64)elem.y);
   local_result += ((u64)elem.z) + ((u64)elem.w);

   // Move to the next element in the list
   idx += increment;
  }
 }

 // Add the final result to the GMEM accumulator
 atomicAdd(result, local_result);
}
```

使用ILP会带来一个额外的好处：执行循环所花费的时间（开销）由更多有用的指令（内存获取，加法操作）分摊。以下是运行结果：

```
ID:0 GeForce GTX 470:GMEM loop1 64 passed Time 1.48 ms
ID:3 GeForce GTX 460:GMEM loop1 64 passed Time 2.05 ms
ID:0 GeForce GTX 470:GMEM loop2 64 passed Time 1.16 ms
ID:3 GeForce GTX 460:GMEM loop2 64 passed Time 1.49 ms
ID:0 GeForce GTX 470:GMEM loop4 64 passed Time 1.14 ms
ID:3 GeForce GTX 460:GMEM loop4 64 passed Time 1.38 ms
```

在loop1中我们对单个32位的元素处理，而在loop2中我们使用了uint2类型对两个元素处理，相应的，在loop4中，使用uint4类型处理4个元素。无论哪种情况，我们都使用了从之前结果得到的最优块数目64。从运行结果中我们看到，每个线程处理的元素从32位增加到64位时，运行时间减少了20% ~ 25%，而从64位增加到128位时，GTX470上的运行时间几乎没发生变化，GTX460的运行时间只减少了8%。这与设备的带宽有着直接关系，从之前程序的结果我们可以看到，当每次内存读取从64位增加到128位，GTX460（计算能力为2.1）设备明显能达到更高的存储带宽。

4. 使用共享内存的归约

如果我们看了当前内核的最后一条指令会发现，仍有一个问题需要解决，即通过原子加法操作将结果写回全局内存。到目前为止，我们开启了64个线程块，每个线程块包含256个线程，一共有16K个线程试图对最终的累加值执行写操作。然而，我们真正需要的是减少线程块内的线程，从而减少最终写操作的次数。事实上，这样做可以将写操作的次数从16K减少到64，其实质是在移除串行化的瓶颈，因此，运行的总时间能够显著减少。

然而，回到本章的第一节所讲的内容，我们知道当程序的执行速度足够快之后，如果还想继续提高程序的执行速度则需要一些额外的付出。注意，由于速度的提升，内核可能变得越来越复杂。

共享内存是一组包含32个（计算能力为1.x只有16个）存储体的存储体开关。假设每个线程唯一对应共享内存的一个存储体索引（0 ~ 31），则每个线程每次时钟可以处理一个元素。此时对于单个线程束而言，性能是最优的。然而，线程束的数量并不只有一个，如果多个线程束同时访问共享内存，由于竞争的线程束之间必须共享LSU，因此，单个线程束使用共享内存的全部带宽的能力将减小。一旦LSU以100%的占用率执行，程序性能将受到SM上由一级缓存和共享内存联合组成的64K存储区的带宽限制。

我们可以简单地将块级归约的结果写到SM共享内存的某个变量中。因此，对于256个线程，我们将实现一个256 ：1的归约比。然而，由于这256个线程中仍是串行执行的，因此，这并不会带来很大的性能提升。

SM中的执行单元可以基于每半个线程束执行一次，即每次处理16个线程。因此，针对半个线程束进行归约是有意义的。我们可以对另外半个线程束的线程执行额外的归约操作，也可以将结果写到一组共享内存变量中。事实证明，这两种方案在执行时间上并没有很大区别。

然而，在共享内存中对线程块每个线程进行归约的关键问题在于在哪存放执行归约的共

享内存变量。如果我们将该值存放在线程束执行归约所占用的一组64字节的变量之后，则会造成线程块内的下一个线程束无法达到64字节对齐访问的效果。在共享内存中，不同块进行交互将导致存储体冲突。

我们选择将其直接写到全局内存，因为这种解决方案相对比较简单，并且在性能分析时能明显显示出边界情况。因此，相对于将写操作冲突次数从16K次减少为64次，我们决定只减少到512次，归约比为32。

```
__global__ void reduce_gmem_loop_block(
 const uint4 * const data,
 u64 * const result,
 const u32 num_elements)
{
 const u32 num_elements_per_block = (( (num_elements/4) / blockDim.x) / gridDim.x);
 const u32 increment = (blockDim.x * gridDim.x);
 const u32 num_u4_elements = (num_elements>>2);
 // Work out the initial index
 u32 idx = (blockIdx.x * blockDim.x) + threadIdx.x;

 // Accumulate into this register parameter
 u64 local_result = 0;

 // Loop N times depending on the
 // number of blocks launched
 for (u32 i=0; i<num_elements_per_block; i++)
 {
  // If still within bounds, add into result
  if (idx < num_u4_elements)
  {
   const uint4 elem = data[idx];

   local_result += ((u64)elem.x) + ((u64)elem.y);
   local_result += ((u64)elem.z) + ((u64)elem.w);

   // Move to the next element in the list
   idx += increment;
  }
 }

 const u32 num_half_warps = blockDim.x >> 4;
 const u32 half_warp = threadIdx.x >> 4;

 // Have first N threads clear the half warps
 if (threadIdx.x < num_half_warps)
  intra_half_warp_reduce[threadIdx.x] = 0;

 // Wait for threads to zero SMEM
 __syncthreads();

 // Reduce first by half warp into SMEM
```

```
    // 256 -> 16 (32 banks)
    atomicAdd( &intra_half_warp_reduce[half_warp],
        local_result );

    // Wait for all threads to complete
    __syncthreads();

    // Write up to 16 values out to GMEM
    if (threadIdx.x < num_half_warps)
     atomicAdd(result,
        intra_half_warp_reduce[threadIdx.x]);
}
```

以下是运行结果：

```
ID:0 GeForce GTX 470:GMEM loopB 64 passed Time 0.93 ms
ID:0 GeForce GTX 470:GMEM loopC 64 passed Time 0.93 ms

ID:3 GeForce GTX 460:GMEM loopB 64 passed Time 1.34 ms
ID:3 GeForce GTX 460:GMEM loopC 64 passed Time 1.33 ms
```

在本例中，loopB 对全局内存执行了 512 次原子写操作。而在第二个内核 loopC 中，在对全局内存执行 64 次原子写操作之前线程块内就已进行了一次额外的块内归约操作。从运行结果中我们可以看出，这两种方案在性能上并没有很大的区别，这证明额外的归约操作并没有为程序带来很大的性能提升，因此，在最终的解决方案中，我们将移除该操作。这一点也不令人惊讶，因为 512 次内存写操作延迟已经由大量的计算很好地隐藏，因此，将原子写操作次数减少到 64 次并不会带来很大的性能提升。

如果与之前的最优方案对比，即在 GTX470（计算能力为 2.0）上通过累加到寄存器最后将 16K 值的总和结果写回全局内存，则需要耗费 1.14ms。而通过在共享内存中进一步进行归约操作则可以将总运行时间减少到 0.93ms，节约了 19% 的时间开销。由于 GTX470 有 14 个 SM，SM 内的归约操作可以明显减少最终对全局内存原子写操作的次数，但在 SM 之间需要协调。

相比之下，GTX460（计算能力为 2.1）设备的运行时间则从原先的 1.38ms 减少到了 1.33ms，只减少了 4%。造成这种明显差异的根本原因在于 GTX470 的存储总线位宽为 320 位，而 GTX460 的只有 256 位。因此导致不同加速比，这一点是值得我们深入研究的。

如此小的加速比意味着在 GTX470 上，全局内存的多次原子写操作并不是造成性能瓶颈的主要原因。它可能也暗示着 LSU 的使用率可能达到了 100%。在计算能力为 2.1 的设备上，LSU 与 CUDA 核的比值比计算能力为 2.0 设备上的要小得多。无论是全局内存访问还是共享内存访问都需要 LSU。

因此，基于共享内存上的半个线程束归约相对于前一节基于全局内存的纯原子写的解决方案明显使运行时间减少很多。

5. 另一种解决方案

同任何算法实现一样，我们需要经常回头看看哪些工作是我们先前已经做过的以及如何

运用它们来完善当前的设计。在早期基于G80设备进行GPU编程的时代，Mark Harris写了一篇关于并行归约的研究报告[⊖]。相对于执行一个比率为512：16的归约，其将整个数值集合全部写到共享内存中，然后利用共享内存执行一系列的部分归约，累加得到的结果仍然保存到共享内存中。

运行的结果让人印象深刻。该方案对4 000 000个无符号的整型数进行处理一共花了0.268ms。将数据集规模增加到12 000 000个元素（48MB数据）则需要1.14ms，与之前在GTX470上的0.93ms相差不多。

然而，相比于G80上128个CUDA核，GTX470拥有448个CUDA核，其计算能力提高了3.5倍。同样，内存带宽也从G80的86GB/s增加到了GTX470的134GB/s，提高了1.5倍。此外，Mark的内核将值累加到一个32位的整型数中，而我们为了避免溢出将值累加到了64位整型数中。因此，这两个内核是无法直接进行比较的。

尽管如此，该方法得到的效果却很不错。显然，将值累加到寄存器中比累加到共享内存中更快。由于硬件并不支持直接对共享内存执行的操作，因此执行任何操作之前我们都需要将数据从共享内存中移入或移出。之所以选择基于寄存器的累加，其中一个原因就是减少这部分操作的开销。然而，这并不代表是该部分归约操作的最佳方案。

从本章开始书写到现在为止，已经过去一段时间了，在这段时间里CUDA SDK版本已经从4.0升级到了4.1，编译器也从之前的Open64编译器改为了现在的基于LLVM。这些改变也为我们的程序性能带来了一定的提升，程序执行时间从原先的0.93ms减少到了0.74ms，单单只是改变了编译器，就带来了如此大的性能提升。

然而，到最终的代码为止，究竟有多少时间是由归约而减少的呢？我们可以简单地对最终归约添加注释来计算出该时间。当添加注释之后，我们发现运行时间降到了0.58ms，减少了0.16ms，约占总时间的21%。通过进一步分析，发现其中有0.1ms是用来执行原子加法操作的。

使用Parallel Nsight 2.1版本，从其分析得到的数据中我们可以提取到很多有用的信息：

- 一共有48个可调度的线程束，平均活跃的线程束数目为32；
- SM之间的任务量分配不均匀；
- 大多数问题都是由于简短的类产生；
- 分支很少；
- 总的运行时间中有8%的时间SM处于阻塞状态，产生阻塞的原因要么是取指令要么是指令依赖。

该占用情况具有一定的误导性，造成这种现象的主要原因是任务分配不均而不是运行时问题。事实上问题的关键在于开启的线程块的数目。对于14个SM，每个SM处理6个线程块，则一共可调度84个线程块。不幸的是，我们一共只开启了64个线程块，因此，部分SM并不是最大负载的工作，从而造成平均每个SM执行线程束的数目下降，部分SM闲置并意味着在执行工作负载的后期，部分SM闲置。

我们选择启动64个线程块主要是因为在之前的试验中我们发现当线程块数目为64时得

⊖ Mark Harris，“Optimizing Parallel Reduction in CUDA”，NVIDIA Developer Technology，2007.

到的性能最优。然而，这些方案都是基于16K次全局内存原子写操作的。后来，我们通过基于SM共享内存将原子写操作减少到了512次。一旦移除了全局内存的瓶颈，64个线程块就不一定是最佳选择了。通过执行一个样例可以看到：

```
ID:0 GeForce GTX 470:GMEM loopC 6144 passed Time 2.42 ms
ID:0 GeForce GTX 470:GMEM loopC 3072 passed Time 1.54 ms
ID:0 GeForce GTX 470:GMEM loopC 1536 passed Time 1.11 ms
ID:0 GeForce GTX 470:GMEM loopC 768 passed Time 0.89 ms
ID:0 GeForce GTX 470:GMEM loopC 384 passed Time 0.80 ms
ID:0 GeForce GTX 470:GMEM loopC 192 passed Time 0.82 ms
ID:0 GeForce GTX 470:GMEM loopC 96 passed Time 0.83 ms
ID:0 GeForce GTX 470:GMEM loopC 64 passed Time 0.77 ms
ID:0 GeForce GTX 470:GMEM loopC 48 passed Time 0.82 ms
ID:0 GeForce GTX 470:GMEM loopC 32 passed Time 0.95 ms
ID:0 GeForce GTX 470:GMEM loopC 16 passed Time 1.40 ms
ID:3 GeForce GTX 460:GMEM loopC 6144 passed Time 3.53 ms
ID:3 GeForce GTX 460:GMEM loopC 3072 passed Time 2.04 ms
ID:3 GeForce GTX 460:GMEM loopC 1536 passed Time 1.41 ms
ID:3 GeForce GTX 460:GMEM loopC 768 passed Time 1.11 ms
ID:3 GeForce GTX 460:GMEM loopC 384 passed Time 0.97 ms
ID:3 GeForce GTX 460:GMEM loopC 192 passed Time 0.92 ms
ID:3 GeForce GTX 460:GMEM loopC 96 passed Time 0.91 ms
ID:3 GeForce GTX 460:GMEM loopC 64 passed Time 0.95 ms
ID:3 GeForce GTX 460:GMEM loopC 48 passed Time 1.00 ms
ID:3 GeForce GTX 460:GMEM loopC 32 passed Time 1.02 ms
ID:3 GeForce GTX 460:GMEM loopC 16 passed Time 1.29 ms
```

注意，在GTX470上，启动线程块的最佳数目为384，而在GTX460上则是96。当启动192个线程块时，两个设备上的运行效果都还不错。但很明显，64个线程块的运行效果并不好。

然而，出现最后一个问题SM有8%的时间是空闲的原因是什么呢？如果此时还有其他线程块，则能帮助我们将该时间能够减少7%，但导致问题产生的根本原因是什么呢？内核的输出为我们提供一个线索：

```
1>ptxas info : Used 18 registers, 1032+0 bytes smem, 52 bytes cmem[0]
1>ptxas info : Compiling entry function '_Z27reduce_gmem_loop_block_256tPK5uint4Pyj'
for 'sm_20'
1>ptxas info : Function properties for _Z27reduce_gmem_loop_block_256tPK5uint4Pyj
1> 16 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
```

注意，与CUDA 4.0 SDK中的编译器不同，4.1版本的编译器会将uint4类型存储到本地内存。在费米架构的设备上，该本地内存即一级缓存。我们可以通过一个uint4类型的指针对uint4进行访问。由于uint4类型是128位对齐的（4×32位的字），因此必须保证它们位于同一缓存行上而且还需对其执行存储事务边界保护。这样，当线程访问uint4类型的第一个元素时，其剩余的三个元素将同时读取到一级缓存中。现在，我们得到了两个版本的内核，一个是基于一级本地内存访问的，另一个是基于一级缓存直接访问的。理论上它们应该没有什么不同，以下是执行结果：

```
ID:0 GeForce GTX 470:GMEM loopD 384 passed Time 0.68 ms
ID:0 GeForce GTX 470:GMEM loopD 192 passed Time 0.72 ms
ID:0 GeForce GTX 470:GMEM loopD 96 passed Time 0.73 ms

ID:3 GeForce GTX 460:GMEM loopD 384 passed Time 0.85 ms
ID:3 GeForce GTX 460:GMEM loopD 192 passed Time 0.81 ms
ID:3 GeForce GTX 460:GMEM loopD 96 passed Time 0.80 ms
```

无论是在GTX470还是GTX460设备上，执行时间都明显减少。查看缓存利用率统计信息发现，当我们将原先的本地内存版本（loopC）更改为指针版本（loopD）之后，一级缓存的命中率从61.1%提高到了74.5%。另外，SM阻塞的百分比也下降了5%。针对这项数据，GTX460上的变化显然更明显，其最初为9%，比GTX470略高一点。出现这种现象，可能是由于一级缓存数据能够在线程之间共享，数据不再是线程私有的了。

你可能会有疑问：为什么不能像之前那样使用84个线程块计算呢？其中一个原因就是凑整问题。包含12 000 000个元素的数据集无法平均分配到84个线程块中，即部分线程块处理的元素数目要比其他线程块多。这意味着逻辑结构将更加复杂，同时每个执行的线程块也将更加复杂。如果在不考虑该问题的前提下使用84个线程块执行则需要0.62ms，比使用384个线程块的版本多出了0.06ms。这表明使用384个线程块将每块划分得很小，现有的负载平衡机制能很好地对其进行处理。将代码复杂化的代价远远超过了其带来的好处，事实上，只有在输入数据集的大小不确定的情况下才有必要这样做。

回到共享内存与原子操作版本的内核谁更快的问题上。我们可以将基于原子操作的归约替换成以下这段代码：

```
// Write initial result to smem - 256 threads
smem_data[threadIdx.x] = local_result;
__syncthreads();

// 0..127
if (threadIdx.x < 128)
 smem_data[threadIdx.x] += smem_data[(threadIdx.x)+128];
__syncthreads();

// 0..63
if (threadIdx.x < 64)
 smem_data[threadIdx.x] += smem_data[(threadIdx.x)+64];
__syncthreads();

// 0..31 - A single warp
if (threadIdx.x < 32)
{
 smem_data[threadIdx.x] += smem_data[(threadIdx.x)+32]; // 0..31
 smem_data[threadIdx.x] += smem_data[(threadIdx.x)+16]; // 0..15
 smem_data[threadIdx.x] += smem_data[(threadIdx.x)+8];  // 0..7
 smem_data[threadIdx.x] += smem_data[(threadIdx.x)+4];  // 0..3
 smem_data[threadIdx.x] += smem_data[(threadIdx.x)+2];  // 0..1
```

```
 // Have thread zero write out the result to GMEM
 if (threadIdx.x == 0)
  atomicAdd(result, smem_data[0] + smem_data[1]);
}
```

注意这段代码的执行原理。首先，所有256个线程（0号线程束～6号线程束）将其当前的local_result值写入到共享内存中保存64位整型值的一个256维数组中。然后将线程索引为128到255（4号线程束～7号线程束）对应的数组元素值相应累加到线程索引为0到127（0号线程束～3号线程束）对应的数组元素值中。由于同一线程块中不同线程束进行合作，因此我们需要添加 __syncthreads() 调用以保证每个线程束都执行完毕。

我们接着归约直到需要归约的线程数目达到一个线程束的大小，即32个。当达到这个临界点后，线程束内的所有线程就是同步的，因此，就不再需要进行同步操作了。事实上，单个线程块内的线程同步操作是对线程束的同步操作。

到目前为止，有很多选择。我们可以继续使用 if threadIdx.x < threshold 的操作，或者直接忽略线程束中的多余线程执行一个无用操作。额外的测试意味着将产生相当多的额外指令，因此，我们将在一个线程束内计算出所有的值。注意，这与执行多个线程束不同，如使用128或64个线程进行测试时。在单个线程束内，减少线程的数量并不会带来很大改变。相比之下，原先的测试删除了整个线程束。

那么，与基于原子操作的归约相比，这样做究竟带来了多少性能提升呢？

```
ID:0 GeForce GTX 470:GMEM loopE 384 passed Time 0.64 ms
ID:3 GeForce GTX 460:GMEM loopE 192 passed Time 0.79 ms
```

与上一个版本相比，GTX470上的执行时间从原先的0.68ms减少到了0.64ms，而GTX460上的执行时间从0.8ms减少到了0.79ms。性能的提升并不明显，但尽管如此，执行速度确实加快了。在继续之前，我们还可以对这段代码进行最后一个优化。

通常，当数组索引的值不是一个常数值时，编译器针对数组索引生成的优化代码并不是特别理想。CUDA编译器也不例外。我们可以用指针代替数组代码，这样，在某些情况下可以执行得更快。我们也可以减少对共享内存区的读/写次数。然而，与大多数优化解决方案一样，代码将变得更复杂，难于理解，也不易维护和调试。

```
// Create a pointer to the smem data area
u64 * const smem_ptr = &smem_data[(threadIdx.x)];

// Store results - 128..255 (warps 4..7)
if (threadIdx.x >= 128)
{
 *(smem_ptr) = local_result;
}
__syncthreads();

// 0..127 (warps 0..3)
if (threadIdx.x < 128)
```

```
{
 // Accumulate into a register and then write out
 local_result += *(smem_ptr+128);

 if (threadIdx.x >= 64)   // Warps 2 and 3
  *smem_ptr = local_result;
}
__syncthreads();

// 0..63 (warps 0 and 1)
if (threadIdx.x < 64)
{
 // Accumulate into a register and then write out
 local_result += *(smem_ptr+64);
 *smem_ptr = local_result;

 if (threadIdx.x >= 32)   // Warp 1
  *smem_ptr = local_result;
}
__syncthreads();

// 0..31 - A single warp
if (threadIdx.x < 32)
{
 local_result += *(smem_ptr+32);
 *(smem_ptr) = local_result;

 local_result += *(smem_ptr+16);
 *(smem_ptr) = local_result;

 local_result += *(smem_ptr+8);
 *(smem_ptr) = local_result;

 local_result += *(smem_ptr+4);
 *(smem_ptr) = local_result;

 local_result += *(smem_ptr+2);
 *(smem_ptr) = local_result;

 local_result += *(smem_ptr+1);

 // Have thread zero write out the result to GMEM
 if (threadIdx.x == 0)
  atomicAdd(result, local_result );
}
```

该方案首先将当前线程的结果存储到 local_result 中，这是由于在共享内存中进行累加操作的意义并不大。共享内存只需存储后半部分线程传递到前半部分的结果值即可。因此，在每次执行归约操作时，只有前半部分线程才会对共享内存进行写操作。当归约到单个线程束时，这段代码的执行时间比通过共享内存节省的读写操作时间还要长，因此我们终止了测试

和写操作。另外，为了避免地址计算，我们使用了简单的指针加法操作，共享内存的地址在代码开始处就已经存放到了一个指针中。修改之后的运行时间如下所示：

```
ID:0 GeForce GTX 470:GMEM loopE 384 passed Time 0.62 ms
ID:3 GeForce GTX 460:GMEM loopE 192 passed Time 0.77 ms
```

从运行结果中我们可以看到 GTX470 和 GTX460 的运行时间均减少了 0.2ms。此外，我们已在很大程度上消除了基于共享内存的原子归约操作，这使得程序也能在旧版本的硬件上运行。为了移除最终将结果归约到全局内存的操作，我们可以以 blockIdx.x 为索引将结果写到一个数组中，然后通过调用另一个内核将各自的结果加起来。

6. 一个可供选择的 CPU 版本

为供参考，方便我们在 CPU 端也能看到同样归约，CPU 的串行和并行实现代码如下所示：

```
u64 reduce_cpu_serial(const u32 * data,
                      const u32 num_elements)
{
 u64 result = 0;

 for (u32 i=0; i< num_elements; i++)
      result += data[i];

 return result;
}

u64 reduce_cpu_parallel(const u32 * data,
                        const int num_elements)
{
 u64 result = 0;
 int i=0;

#pragma omp paralle l for reduction(+:result)
 for (i=0; i< num_elements; i++)
   result += data[i];

 return result;
}
```

在 AMD 的 Phenom II X4 处理器（4 核）上，运行频率为 2.5MHz，串行版本的执行时间为 10.65ms，并行版本的执行时间为 5.25ms。并行版本是通过使用 OpenMP 和一些归约原语实现的。为了使这些编译指示在 NVCC 编译器下也可用，我们只需在编译时使用 -Xcompiler-openmp 标志即可使用 OpenMP 指令实施 CPU 级的线程并行。

这段代码产生了 N 个线程，其中 N 为核的数目。然后，线程可以在任何可供使用的核上运行。任务分解成 N 块，最终再将结果合并。

与大多 CPU 上的并行程序一样，该程序的时间也随着核数的增加亚线性减少。我们看到当核数从 1 个增加到两个时，程序性能过渡得很好，当使用两个核计算时，执行时间减少了 35%。当使用 3 个核心计算时，执行时间减少了 50%。然而，当核数增加到 4 个时，执行

时间只减少了2%，性能的提升并不明显（参见图9-37）。如果继续增加核数，我们将看到性能的变化曲线呈U型。

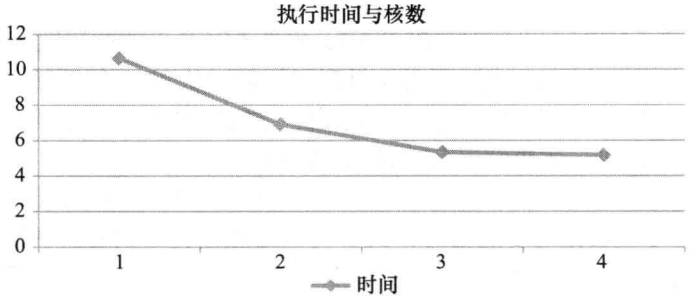

图9-37 4核处理器上OpenMP执行时间比例

出现这种现象的原因主要是，计算性能随着核数的增加而提升的同时，卡槽的内存带宽也是在众多核之间共享的。以本测试系统为例，AMD 905e处理器的内存带宽为12.5MB，不做任何计算操作仅读取48MB的数据就需要3.8s，几乎占了整个执行时间的一大半。因此，这里不再是OpenMP与CUDA的比较问题了，而是CPU与GPU上每个核可用的内存带宽的比较问题。

7. 并行归约总结

最初最简单的GPU实现在GTX470上执行需要197ms，在GTX460上需要164ms。与CPU上4核并行得到的结果5.25ms相比，这样的性能实在很糟糕。即便再快的设备，如果编程功底很差也会导致产生程序性能非常糟糕。

在最终的GPU版本中，已经尽可能最少在SM外使用原子操作。最终，从纯计算的角度来看，与4核的CPU版本相比，GPU版本实现在GTX460上获得了6.8倍的加速比，而在GTX470上获得了8.4倍的加速比。然而，0.62ms的计算时间实在太短，无法隐藏任何传输时间。5GB/s的PCI-E 2.0内存带宽大约为测试平台主内存带宽（12GB/s）的40%。5GB/s的传输速度使得每毫秒能够传输5MB的数据。因此，传输48MB的数据需要9.6ms。我们可以将少于10%的传输时间与计算时间重叠，但也意味着总的执行速度不会快于PCI-E 2.0的传输速度。

这种问题在GPU编程中经常会遇到。只有当问题十分复杂时，GPU巨大计算能力的优势才能充分体现。在这种情况下，其优势才会彻底地超过CPU。而对于执行sum、min、max等这类及简单操作的程序任务，任务的执行时间根本无法隐藏PCI-E的传输时间，除非我们通过将数据长期保存在设备上，从而使传输时间大打折扣。这也是6GB的Tesla设备比那些便宜的最大容量为4GB的消费级显卡更具吸引力的原因之一。

为了增加GPU内存空间中总的数据存储容量，我们可以在一个系统中安装多块显卡，例如，每个机器节点最多安装4个显卡，或者使用极端冷却方案安装更多显卡。这样，在单个节点上安装4个Tesla显卡就能最多存储24GB数据。如果设备支持UVA（通用虚拟地址）技术（支持条件：计算能力在2.x以上的设备，64位操作系统，Linux系统或者在Windows

上使用 TCC 驱动，SDK 必须为 CUDA 4.x 运行时），主机端内存空间可以通过 GPU 内存空间使用 UVA 技术直接扩增。Intel 处理器与 GPU 之间的通信（点对点，P2P）也可不必通过 CPU 路由数据直接执行，节省了大量的 PCI-E 带宽。

随着 PCI-E 2.0（5GB/s）升级到 PCI-E 3.0，每个 PCI-E 卡槽的带宽也得到有效的加倍，通过支持新的 PCI-E 3.0 标准，该问题明显得到了缓解。从 2012 年开始，搭载 Ivybridge 与 Ivybridge-E 处理器的主板开始支持 PCI-E 3.0 标准。之后，PCI-E 图形显卡也开始逐渐出现。除了 PCI-E 带宽增长之外，主机端内存带宽也增加不少。

这也突出了贯穿本书的另一个要点。CPU 可以作为 GPU 的得力搭档共同处理所有的简单问题。例如，当多块数据需要通信时，CPU 可以处理重叠区域的共享数据而 GPU 处理其他绝大部分的数据。通常，这种情况下会产生许多分支，而 GPU 并不擅长处理分支，因此，可以采取更好、基于合作的解决方案。

9.5.3 本节小结

- 目前有许多文档以及充分的资源详细介绍了许多专业领域的算法，并且大多都以插件库的形式可供使用。
- 注意，并不是所有的并行算法都能在 GPU 上实现。在实现算法时要考虑合并与通信等方面的因素。
- 与大多 API 的开发目的一样，诸如 __syncthreads_count 这类新函数的引入可能是用来处理一些特定类型的问题。仔细阅读与 API 有关的各种信息以了解其使用方法。
- 每个线程尽可能多地处理元素数据。然而，每个线程处理过多的元素也可能会对性能带来副作用。
- 正如在归约例子中展示的那样，最简单的内核往往是执行速度最慢的。为了获取更高的性能，通常需要花费很多时间在编程上，并且也需要对底层硬件有很好的认识和理解。
- 一个多核 CPU 虽然能够很好地处理计算负载，但通常会受到内存带宽的限制，这会限制我们充分利用所有核的能力。
- OpenMP 可以为 CPU 线程提供一个易用的多线程接口，并且它也是标准 CUDA 编译器 SDK 的一部分。

9.6 策略 6：资源竞争

9.6.1 识别瓶颈

如果一个程序某方面出了问题，该怎么办，对于一个程序员来说，这通常是不清晰的。对于大多数 GPU 程序而言，如果分配给它们合理规模的任务去执行，GPU 带来的性能提升要比 CPU 显著得多。现在的问题是有多显著？这个问题引出了另一个问题，即 GPU 在处理繁多、确定的任务时具有很大的优势。当任务包含大量算法时，将它划分成多个独立的问题进行处理效果会很好。

通常包含大量分支或基本顺序执行的算法并不适合GPU以及绝大多数并行结构系统。在减少一个程序并行度的过程中，我们经常会看到单线程与多线程之间性能的一个均衡点。通常GPU的主频最高可达1000MHz，这相当于CPU主频的1/3或者更小。另外，GPU也不具有一般大型流水线系统中必备的、昂贵的分支预测逻辑。

CPU已有数十年的发展，基于单线程的性能提升几乎已经发展到了尽头。因此，相对于CPU，大多数串行代码并不能在GPU上高效执行。这可能会随着未来的混合架构而有所改变，尤其是我们看到专用CPU推荐纳入NVIDIA的“丹佛项目”中。此举旨在将基于ARM架构的CPU核嵌入GPU体系中。我们已经看到在常见的CPU平台中嵌入GPU元素，因此无论是在未来的CPU世界还是GPU世界中，很可能出现一种混合架构，使各设备的利用率达到最大。

然而，我们仅考虑那些可以在GPU上运行良好的、具有数据并行性质的问题。对于我们的内核，怎样才是一个好内核的标准呢？是否应该通过对比来进行选择？怎样才是一个实际的目标呢？

目前，许多领域都在使用CUDA进行加速处理。英伟达官方网页http://www.nvidia.com/object/cuda_app_tesla.html提供了一些实现想法，如果已经有了解决方案，还可以通过购买的方式获得该解决方案。以下列出了CUDA在各领域的应用：

- 政府管理与国防
- 分子动力学、计算化学
- 生命科学、生物信息学
- 电动力学和电磁学
- 医疗成像、CR、MRI
- 石油和天然气
- 金融计算与期权定价
- Matlab、Labview等数学软件
- 电子自动化设计
- 天气和海洋建模
- 视频、图像和视觉应用

因此，如果所在的领域是期权定价，可以到相关区域，浏览一些网站，根据特定的供应商分析，可以看到Monte Carlo定价模型相对于单核的CPU加速比可提升30 ~ 50倍。当然，你一定会问CPU的型号是多少，时钟频率是多少，用到了几个核心这些问题以作比较。此外，你还必须记住供应商试图售卖的产品的一些相关数据。这样，你就可以通过这些数据来判定哪款设备最适合，哪些设备在处理该领域问题时存在缺陷，从而选择购买哪款设备。一些研究可以告诉我们在特定领域哪些数据才是关键数据，根据这些研究做出正确的购买选择。

但是，并不要因为GPU最初得到的结果不及其他程序而感到沮丧。通常，这需要花费多年的努力，才能产生巨大优势，但同时也意味着这需要支持大量的陈旧代码。原先在向量机时代被遗忘的那些解决方案可能在现在看来都是较优的解决方案。

此外，还需要记住的一点是，尽管CUDA已成为主流，许多项目都是从创业公司发起的，而且它们的发行产品也越来越多。通常这些创业公司都是有一些能干的博士生创建的，

他们想在自己的领域进行进一步的研究或让他们的论文实现商业化。他们往往由一些小团体组成，每个人都很擅长各自领域的问题，但并不是所有人都有计算机的教育背景。因此，对于那些既了解 CUDA 又了解相应应用领域的人而言，往往能做到更好、更具有竞争力。

分析工具

（1）Visual Profiler

对已实现的代码进行分析，我们首先可以考虑的就是 SDK 中提供的那些分析工具。第一个介绍的工具就是 NVIDIA 的 Visual Profile。这是一款多平台的工具。它能指出在我们的内核中哪些是错误的，并告诉我们该如何改进。

使用该工具，只需先编译 CUDA 内核，然后单击“File->New Session”（文件→新会话），选择刚生成的可执行文件。如果需要，还可以输入一些工作目录以及命令行参数。最后，告诉分析器应用程序执行多久，这样就不用担心内核在崩溃之后还一直等待执行出最终结果。使用 Windows 系统的用户请注意，使用 Visual Profile 进行分析时需禁用默认的 Aero 桌面主题，改为标准的桌面主题。

也许不会仔细看图 9-38 中显示的时间轴，但必须弄清哪些是主要部分。从图 9-38 中可以看到，内核执行的计算非常少（图中央绿色条状部分），主要是一系列的内核使用默认流在多个 GPU 中顺序执行。我们看到，使用默认流会导致隐式同步，对整体运行时间产生巨大影响。

如果使用另一个流，得到的结果将有所不同。关于流的使用，只需在将数据拷入至设备或从设备复制回主机时指定特定的流，即将复制操作压入到某个流中。此时尽管我们使用了两个 GPU，但工具仍提醒我们内核的内存数据传输基本没有重叠。这种现象是正常的，主要是由标准内核往往既有输入数据又有输出数据造成的。虽然费米架构的设备在物理硬件层有两个存储复制引擎，但在诸如此处使用的 GTX470 和 GTX460 这类消费级显卡上只有一个是可用的。因此，所有数据传输必须通过相同的存储复制流并按顺序执行。当第一个流完成从将数据复制至设备后接着执行将数据从设备复制回的操作，然后是下一个流将数据复制至设备。

然而，对于 Tesla 设备而言，并不会出现该类问题，因为它的两个存储复制引擎均是可用的。只有对于消费级硬件，我们才需要采取不同的方式。对于 Tesla 设备，在所有的复制至操作和内核调用完成之前，我们不需要将任何复制回操作压入到流中。在完成之后，我们再将一系列“复制回”（copy back）命令压入到流中执行拷回操作。这中间可能会出现一些内核的执行会与最后一个内核或复制回操作产生重叠的现象，但不会产生很大影响。

此外，该分析还呈现了另一个问题，即数据的传输带宽并没有充分利用（查看“Low Memcpy/Compute Overlap”（低内存复制 / 计算重叠）消息）。在本例中，我们使用的是 32MB 的数据块。如果回顾一下本章前面的章节，我们会发现这足以达到 PCI-E 总线带宽的峰值。然而，此处的问题是计算部分占用了绝大部分的时间。即使将数据传输和内核执行重叠执行，带来的性能提升也是微不足道的。因此，只有真正理解了分析工具提供的信息，然后再从相关方面做出改进才能减少程序执行时间。

整体而言，Visual Profile 是一个非常有用的工具，安装使用都很简单，并且它能快速产生合理的分析结果，支持多种平台（参见图 9-39）。

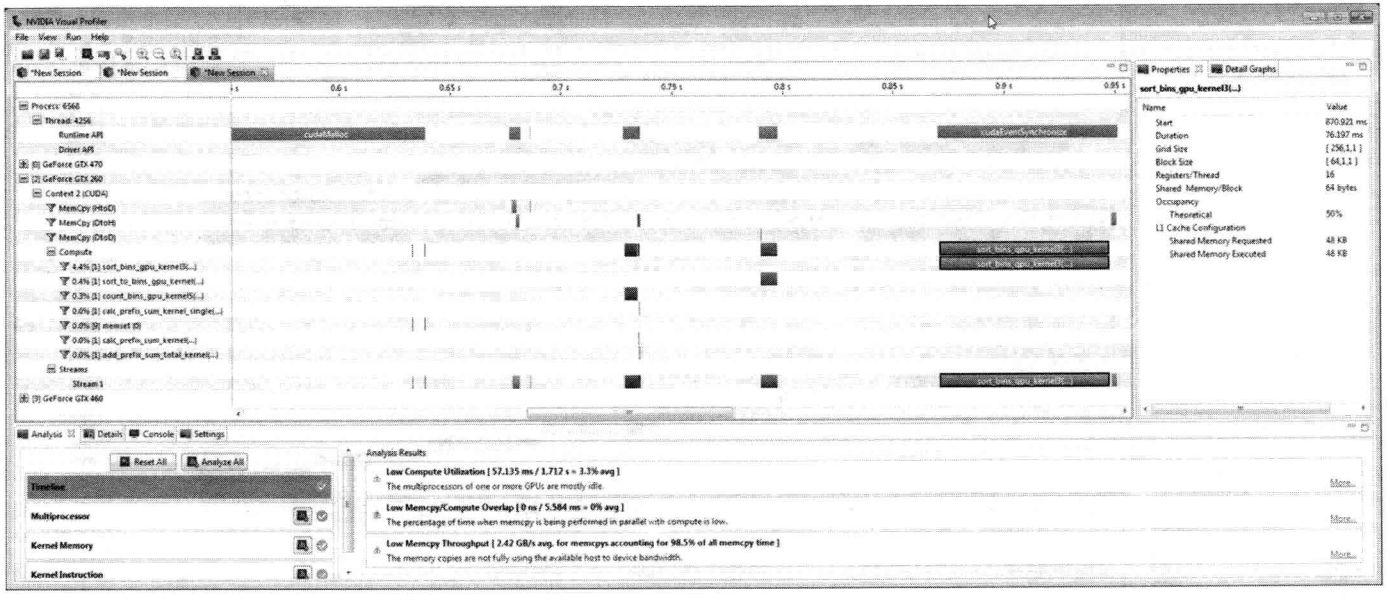

图 9-38　Visual Profile 时间轴

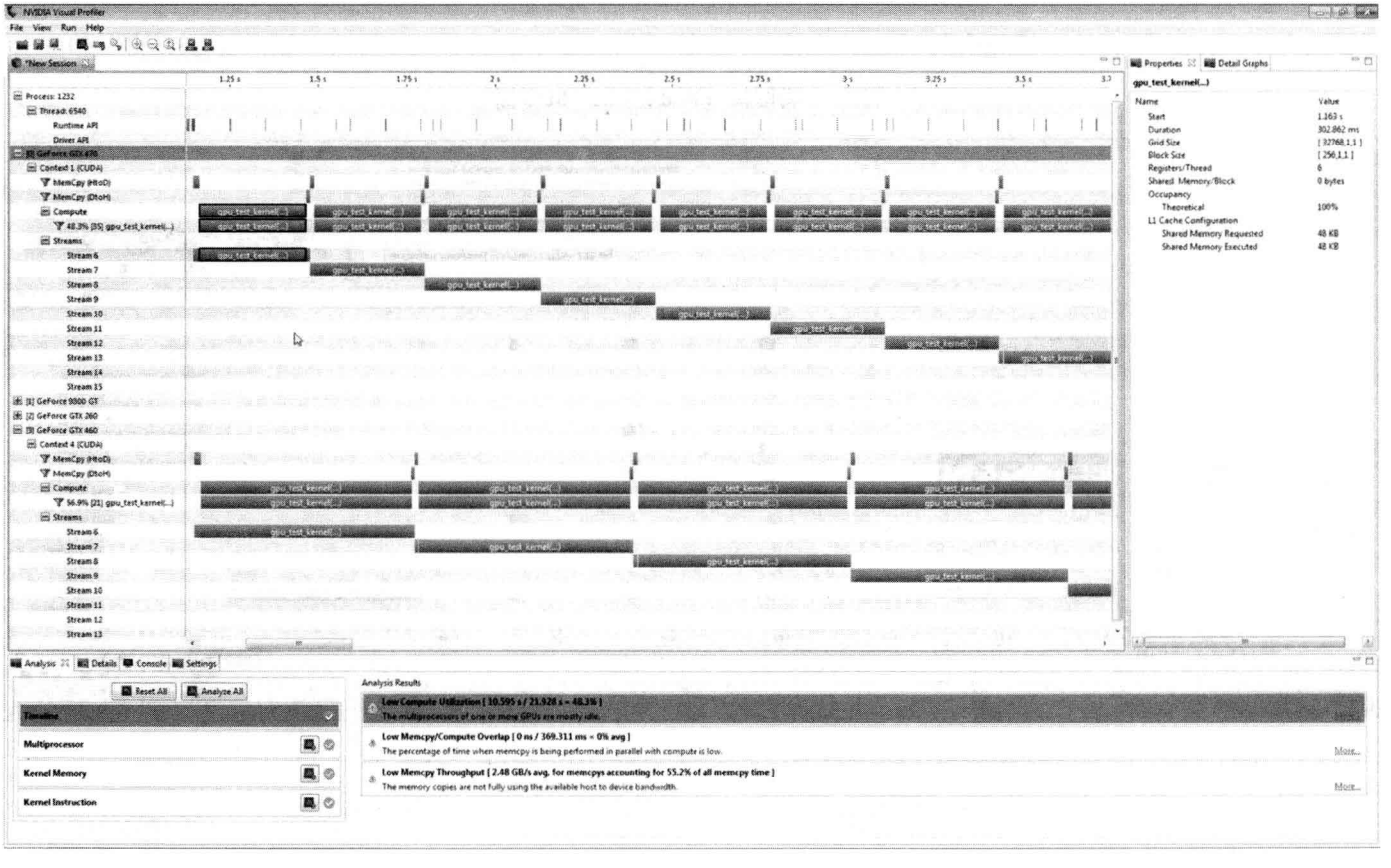

图 9-39 Visual Profile 多 GPU 分析

（2）Parallel Nsight

尽管 Visual Profiler 很好用，但遗憾的是，它只能为我们提供这么多的信息。如果想进一步得到一些分析信息，则需要更高级的分析工具，例如 Parallel Nsight。Parallel Nsight 是一款仅支持 Windows 平台的可视化分析器与调试器。即使主要开发环境并不是 Windows 平台，Parallel Nsight 也值得专门准备一台空闲的 PC 尝试它那丰富的分析功能。

Parallel Nsight 是一款比 Visual Profiler 更为深入的分析工具。它会展示更多有关内核的信息以及内核做了哪些工作。然而，与大多数复杂工具一样，它需要花费一定时间来学习如何使用。相对而言，Visual Profiler 的安装和使用就简单得多，是一款适合初学者的分析工具，而 Parallel Nsight 则是一款中级或高级的工具。

Parallel Nsight 最好在一个拥有计算能力为 2.x（费米）图形卡以上的个人电脑上安装使用。Parallel Nsight 也可以通过两台具有 NVIDIA 显卡的个人电脑进行远程调试。但一般而言，使用一台电脑进行分析调试更加简单，因为使用远程调试时还需等待两台电脑之间的数据传输。

Parallel Nsight 在调试和性能分析方面提供了许多选项。其中两个比较主要的选项就是“Application Trace”应用程序跟踪和“Profile”（概括分析）。“Application Trace”应用程序跟踪功能可以像 Visual Profiler 一样生成一个时间轴，并且可以显示出 CPU 与 GPU 是如何交互的以及主机端和设备端的交互时间。使用时间轴还可以验证流操作以及内核执行和内存复制操作重叠执行的正确性。

此外，该功能还支持多个并发 GPU 的时间轴显示。例如，在图 9-40 的时间轴中显示出没有足够的任务使所有 GPU 处于忙碌状态。图中仅显示了计算部分。红色条的第一个和最后一个上下文表示费米架构 GPU，中间两个绿色条代表旧式 GPU。每个红色方块代表给定流的一次内核调用。可以看到每一组内核都是顺序执行。此外，我们还看到第一个 GPU 有很长一段时间处于空闲状态。诸如此类的问题，只有使用如 Parallel Nsight 这样的工具才能看到。只单独使用主机端以及 GPU 端计时器是很难办到的。

另一个比较有用的功能是“Profile”（概括分析），该选项位于“Activity Type”（活动类型）菜单下（参见图 9-41），它可以概括分析整个 CUDA 内核。然而，由于多项测试需要运行多个内核，因此当选择该选项时不会生成时间轴。

选择“Experiments to Run”（运行实验）下拉列表中的“All”（所有），这是最简单的一个选项。选择之后可以看到如图 9-42 中显示的一个测试列表，测试的内容非常广泛。如果要开始收集数据，只需单击“Application Control”（应用程序控制）面板中的“Launch”（启动）按钮即可，如图 9-43 所示。注意，面板中的圆形图标呈绿色时表示“连接”状态，即 Parallel Nsight 监视器已成功地与目标设备连接。在执行其他任何选项之前都必须保证该图标呈绿色状态。查看帮助选项可进一步了解 Parallel Nsight 监视器的配置信息。

一旦单击了“Launch”（启动）按钮，应用程序将开始运行直到退出。然后在屏幕上方有一个包含很多选项的下拉列表，其中最后一个选项为“GPU Devices”（GPU 设备）（参见图 9-44）。选择该选项则可以查看系统中的 GPU 设备信息。

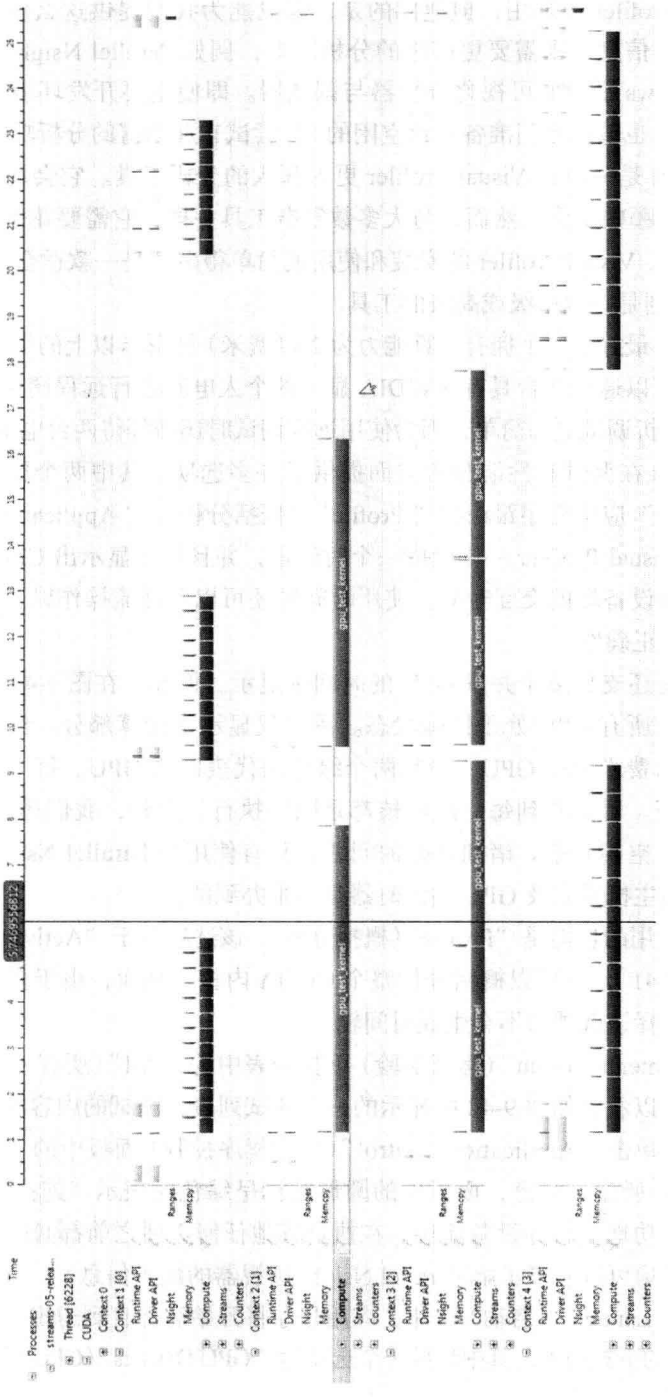

图 9-40 Parallel Nsight 多 GPU 时间轴

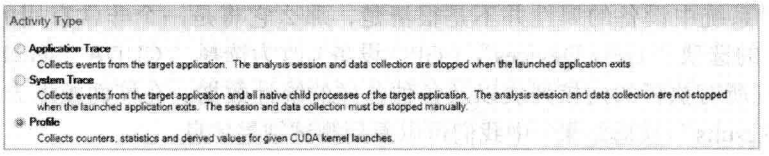

图 9-41 Parallel Nsight 活动类型选项

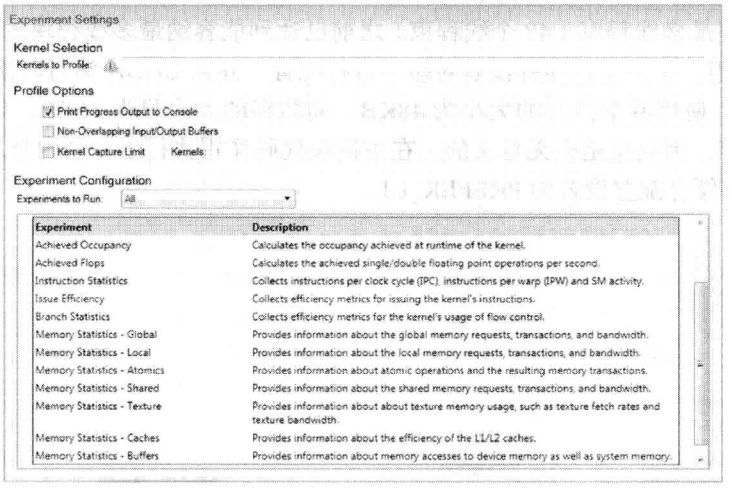

图 9-42 Parallel Nsight 实验

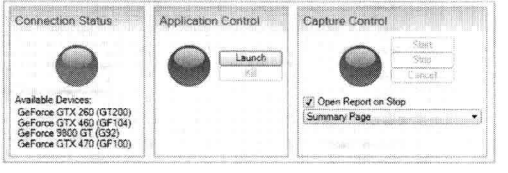

图 9-43 Parallel Nsight 应用程序启动控制

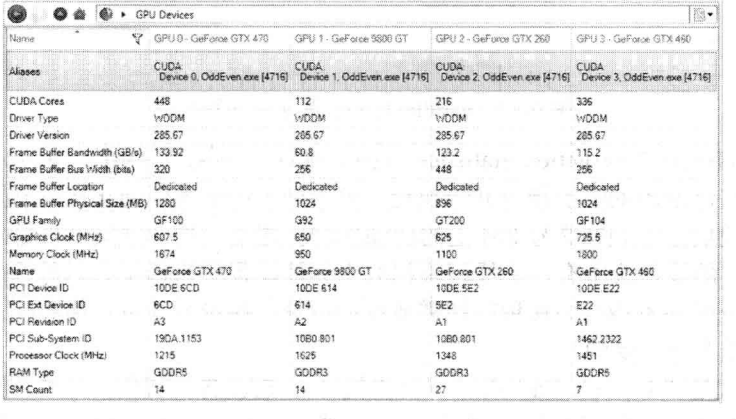

图 9-44 Parallel Nsight 显示 GPU 设备信息

如果你对系统中设备的属性并不是很清楚，那么它将是一个非常有用的功能。接着，将下列列表中的选项“GPU Devices”（GPU 设备）改为选择“CUDA Launch”（CUDA 启动），我们看到所有执行的内核列表以及各种各样的统计数据。在该面板下方的可扩展列表“Experiment Results”（实验结果）中我们可以看到测试结果信息。

在这个具体例子中，执行了 6 个内核。从显示结果中我们可以看到一些问题。首先，没有任何内核的占用率达到理论占用率的 33% 以上（参见图 9-45）。对于第一内核，这是由于在设备实现处理最多线程束（48 个线程束）之前已达到能容纳最多线程块的限制（8 个线程块）。另外请注意，第一个内核并没有对缓存进行配置，并且 CUDA 运行时使用了 PREFER_SHARED 选项，使得共享内存的大小为 48KB，而缓存的大小只为 16KB。但由于内核中并没使用共享内存，因此这是毫无意义的。在主机端代码调用执行第一个内核之前，我们应该调用相关函数将缓存配置设置为 PREFER_L1。

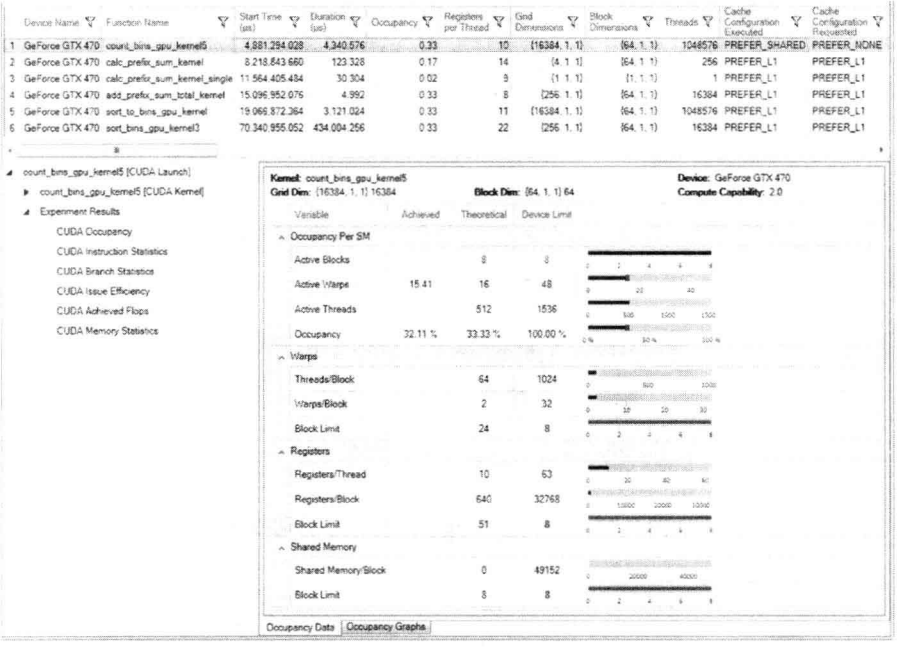

图 9-45 Parallel Nsight 显示占用率信息

接下来的测试是“Intruction Statistics”（指令统计）（参见图 9-46）。此处，我们也发现了一些问题。一些已发出的高层指令并未执行。这表明 SM 必须序列化执行并重新发出相同的指令。另外，我们还看到在 2 号 SM 上的活动剧增。这是一种非常糟糕的情形，因为它意味着分配给该 SM 的线程块中有一个线程块相对于其他线程块执行了大量的额外工作，即线程块之间的任务分配并不均匀。这种问题需要我们在算法层进行改进才能解决。线程块之间的任务均衡分配是非常必要的。

下一项测试是“Branch Statistics”（分支统计），它会告诉我们一个线程束内执行的分支情况（参见图 9-47）。理想情况下，在分支不可能为零的前提下我们通常希望分支的数量很

少。此处分支的比例为 16%，这造成了在之前“Intruction Statistics”（指令统计）测试中我们看到的指令重新发出。此外，这种现象的产生也源于算法中每个线程中工作量的不同。因此，平衡工作线程块之间的负载是很有必要的。

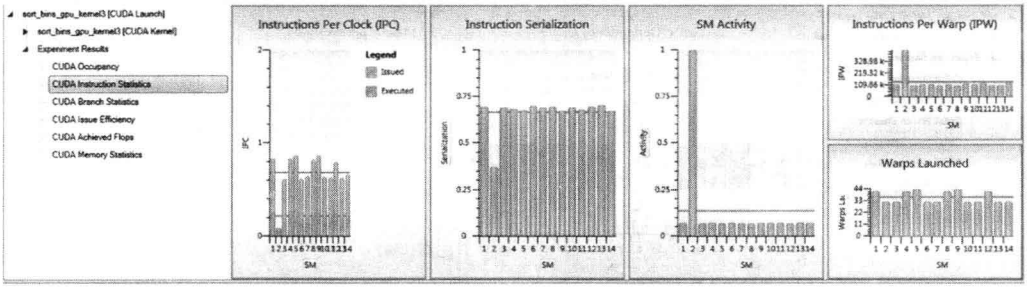

图 9-46 Parallel Nsight 的“Intruction Statistics”（指令统计）

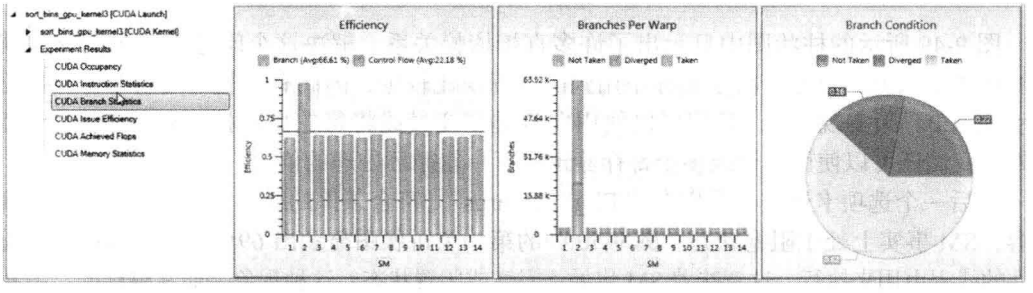

图 9-47 Parallel Nsight 的“Branch Statistics”（分支统计）

接着是对 SM 发出和执行指令的能力的测试。我们希望在“Active Warps per Cycle”（每周期活跃线程束）图中看到每个 SM 中指令的发出与执行都非常平均。在之前得到的结果中我们看到 2 号 SM 的执行时间非常长，然而，实际分配给它执行的线程束却很少。这表明在分配给该 SM 处理的线程块中有一个线程块执行的任务比其他线程块要多许多。另外，测试结果还显示了一张非常底层的“Eligible Warps per Active Cycle”（每活跃周期符合条件的线程束）图，依次显示出 SM 在某一时刻的阻塞情况（参见图 9-48）。

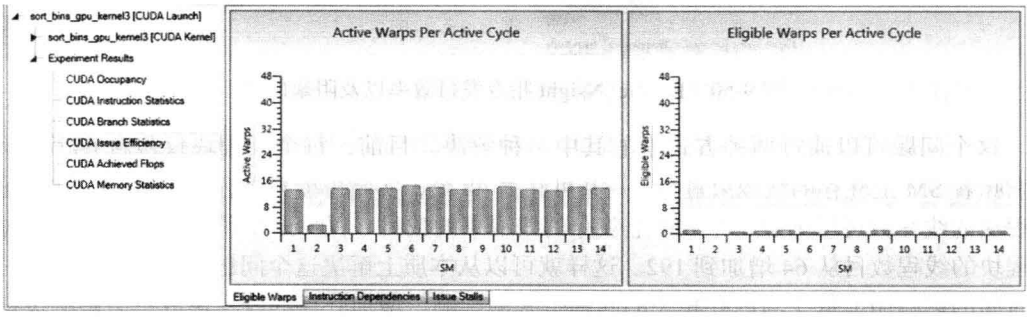

图 9-48 Parallel Nsight 指令发出效率以及符合条件的线程束

单击图下方选项卡的下一个选项，查看指令依赖性的分布（参见图 9-49）。指令依赖是由一个操作的输出成为下一个操作的输入所造成的。由于 GPU 采用的是惰性计算模型，因此，GPU 采取超长指令依赖操作最佳。

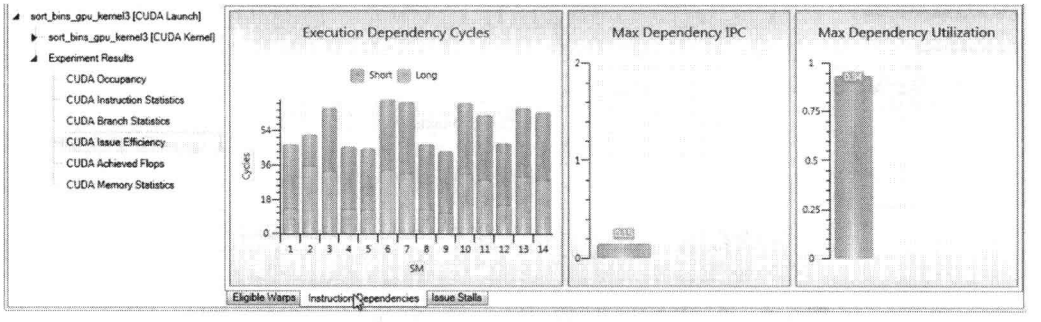

图 9-49 Parallel Nsight 指令发出效率以及指令依赖

图 9-49 所示的柱状图中显示出了许多直接依赖关系。解决这个问题最简单的方法就是在线程级引入 ILP 技术。由于实际中用到的线程块比较少，因此有大量未使用的寄存器可用来引入 ILP。为实现 ILP，我们可以使用向量类型变量或将每轮循环扩展到处理 N 个元素。此外，我们还可以使用一个或多个寄存器预取下一轮循环处理的值。

最后一个选项卡证实了我们在“EligibleWarps”（符合条件的线程束）选项卡中所看到的内容，SM 事实上处于阻塞状态。图 9-50 中的第一个饼状图显示出 69%的时间 SM 没有符合条件的线程束用来执行，这意味着 SM 将处于阻塞或闲置状态，这种现象是非常糟糕的。图 9-50 中的第二个饼状图显示了阻塞的原因，我们看到其中有 85%的时间是由执行依赖所造成的。

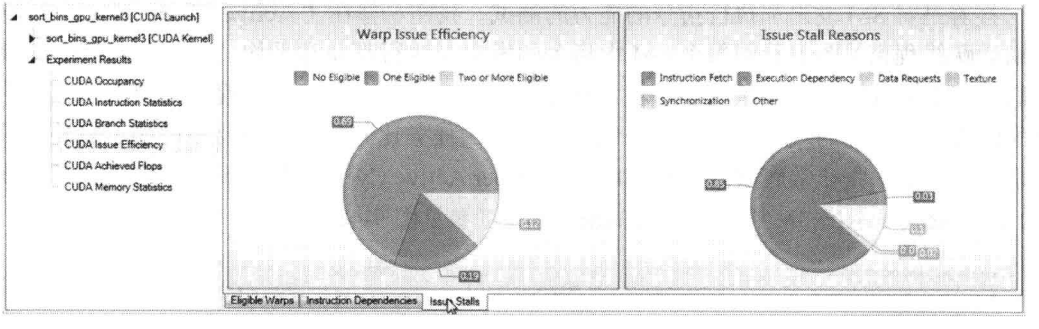

图 9-50 Parallel Nsight 指令发射效率以及阻塞问题

这个问题可以通过两种方式中的其中一种解决。目前，每个线程块仅包含 64 个线程，这意味着 SM 上驻存的线程束过少（一共可驻存 48 个，实际可能只驻存了 16 个）。增加每个线程块的线程数可以增加每个 SM 上驻存的线程束的数目。仅从这个角度看，我们需要每个线程块的线程数目从 64 增加到 192。这样就可以从本质上解决这个问题。然而，该问题对整体时间的影响明显低于与内存相关的问题带来的影响。增加驻存的线程块数目会影响缓存的使用，因此可能会对整体时间产生更大的影响。

我们可以生成两个不同版本的内核观察从全局内存获取数据的总量，以此从实际的角度来看这个问题。其中一个内核每个线程块使用 128 个线程，另一个使用 64 个线程。由于寄存器有剩余，我们也可以将 16 个元素获取到 64 位版本的寄存器中，将 12 个元素获取到 128 位版本的寄存器中。最大限度地使用寄存器，同时每个 SM 仍能维持同时处理 8 个线程块。

果然，线程束发射效率得到了提升，不满足条件的线程束从 75% 减少到了 25%。理论上每个 SM 驻存的线程束数目从 16 个增加到了 32 个（实际上是从 13.25 个增加到了 26.96 个）。占用率从 27%提升至 56%。所有都得到了改善，但这些都是次要影响。内核执行的是排序算法，因此同绝大多数排序算法一样，很可能出现存储限制。

事实上，当我们通过 CUDA 内存统计信息来比较这两个内核时发现两者是有差别的。每个 SM 处理的线程块数量增加，意味着分配给每个线程块的一级缓存的比例减少。反过来，这将导致全局内存的数据获取操作增加一倍，以获取那些未缓存在一级或二级缓存中的数据。

在第一个内核中，每个线程块使用 64 个线程，缓存命中率高达 93.7%（参见图 9-51）。其中有 6.3% 的数据在一级缓存中未命中，但在二级缓存中可命中 6.3% 数据中的 30% 数据，即 1/3 的数据。因此，只有很少部分的读取事务需要从全局内存获取数据，大多数数据都在缓存中。

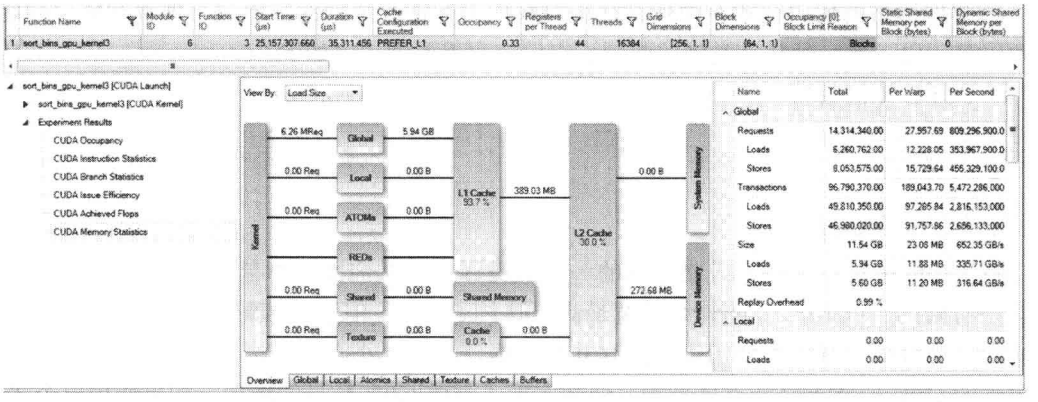

图 9-51 内存统计信息一览图（256 个线程块 ×64 个线程）

当我们将每个线程块的线程数目增加到 128，线程块的总数将减半至 128(参见图 9-52)。但这并不会造成影响，因为每个设备有 14 个 SM，每个 SM 能最多处理 8 个线程块，任何时间内我们都可以最多同时调度处理 112 个线程块。因此，我们可以在 SM 处理线程块能力的最大范围内增加驻存的线程束数目。

注意缓存命中率的问题。无论是一级缓存还是二级缓存命中率都比之前低。全局内存获取的数据量从 272MB 增加到 449MB，几乎增加了一倍。尽管这表明 SM 的利用率得到了改善，但执行时间却从 35ms 增加到了 46ms。主要是因为每个线程分配了一个样本块处理，许多存储访问并未合并，因此代价非常高。

另外，还需注意的是线程块中的线程对单个样本块合作排序的设计问题，这种影响可能不太明显。之前的分析已经向我们展示了相关的依赖性。通过在排序阶段将线程与任务以其

他的方式映射关联、平衡，或调整桶的边界可以明显提高吞吐量。

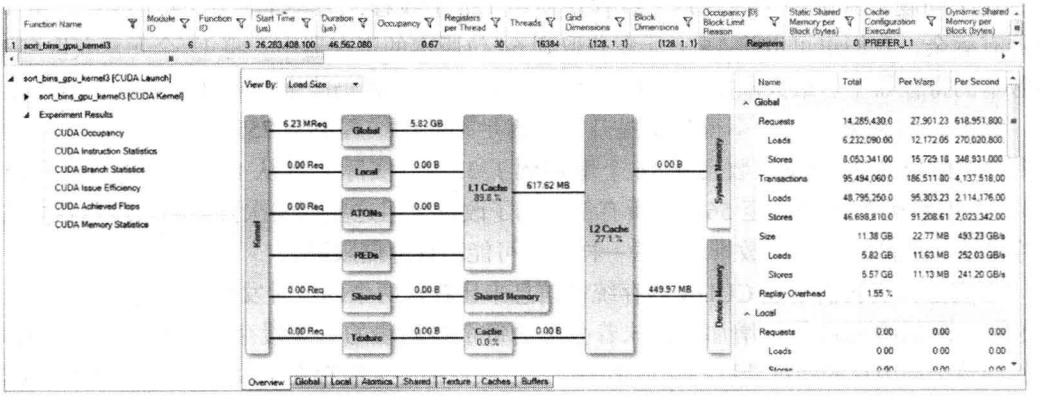

图 9-52 内存统计信息一览图（128 个线程块 ×128 个线程）

9.6.2 解析瓶颈

了解你运行的代码究竟在做什么往往对我们是非常有帮助的，但这与理解和解决问题却是两码事。以下是三种常见类型的瓶颈，它们按照重要性排序如下：

- PCI-E 传输瓶颈
- 内存带宽瓶颈
- 计算瓶颈

1. PCI-E 传输瓶颈

PCI-E 传输瓶颈往往是需要考虑的一个关键因素。从前面的章节我们看到，PCI-E 总线带宽是有限的，PCI-E 2.0 的总线带宽最高可达约 5GB/s，但这也依赖于主机端硬件。然而，要实现这个峰值，则需要使用锁页内存并且数据传输的大小必须适当。在一个计算机节点增加更多的 GPU 通常会降低总体带宽，但却实现了整体 GPU 数量的增加。如果使用单个 Tesla GPU 或使用多个 GPU 将所有数据都存储到 GPU 内存空间中，传输开销就会消除。通常增加 GPU 卡所造成的带宽缩减的范围很大程度上取决于主机端硬件。因此，了解传输的数据量大小以及其用法是很有必要的。

压缩技术是一种可以明显增加 PCI-E 传输速率硬限制的技术。当我们传输数据时，真地需要传送所有数据吗？以图像数据为例，往往图像数据包含一个用于描述透明度的阿尔法通道。如果在 GPU 上不会使用该数据，则在传输数据时我们可以只传输 RGB（红、绿和蓝）三种成分的数据，从而将总的需要传输的数据减少 25%。尽管这样传输数据之后意味着每个像素的数据有 24 位，可能造成非对齐的访存方式，从而导致执行时间增加，但由传输节省的时间明显可以隐藏该影响。

但是数据压缩之后存在一个问题，即通过压缩数据恢复出源数据。但这通常与问题相关，你也许会使用简单的诸如行程长度编码算法对数据进行压缩，将一长串相同的字母用数

字和字母表示，数字表示有几个该字母，然后在GPU端短时间内通过计数重新构造源字符串。从一个传感器可能传入很多活动，然后在一段时间里并没有出现需要关注的活动。很明显，我们可以将需要关注的数据全部传输到GPU端，而对不需要关注的数据直接在主机端丢弃，或以压缩的形式进行传输。

使用流使计算与数据传输重叠进行或使用我们已介绍的另一种很关键的技术：零复制内存。当PCI-E的传输时间超过内核执行时间时，使用这种技术就可以完全隐藏计算时间。如果没有将这两部分操作重叠进行，则总的执行时间将是这两部分各自花费的时间之和，最终在没有计算执行时会产生巨大间隙。参见第8章更多关于流用法的内容。

PCI-E并不是我们需要考虑的唯一传输瓶颈。主机端对内存带宽也存在一定限制。例如，英特尔的SandyBridge-E处理器的主机使用4存储体内存，这意味着相对于其他解决方案，它可以达到更高的主机端内存带宽。当然，在问题允许的情况下通过使用P2P（点对点）传输方式也可以节省主机端内存带宽。不幸的是，在执行写操作时，为了支持使用P2P函数，不可以使用Windows 7操作系统。因此除了那些使用Tesla卡以及TCC（Tesla计算集群）驱动程序的操作系统，Windows 7是唯一目前不支持该特性的主流操作系统。

每个计算机节点加载以及保存数据到诸如本地存储设备或网络存储设备的速度也是一个限制因素。以RAID 0模式对高速SSD驱动器进行连接可以帮助解决该问题。这些都是选择主机硬件需要考虑的，详情见第11章。

2. 内存瓶颈

假设数据从GPU端传输的问题已解决，那么接下来需要考虑的问题是全局内存的内存带宽。从时间和功率消耗的角度来看，移动数据的开销是非常大的。因此，考虑高效的数据存取以及数据重用是在选择一个合适算法时的基本标准。GPU拥有大量的计算资源，因此一个低效但GPU访存模式友好（合并、分片、高度本地化）的算法优于一个计算密集但GPU访存模式不友好的算法。

当考虑内存时，也需要考虑线程间的合作问题，而且线程合作最好限制在单一的线程块内。假设线程通信局限于小范围的通用算法比假设每个线程可以与所有其他线程对话的通用算法更有用。一般地，为旧式向量机设计的算法比为当前集群式计算机中N个独立处理节点设计的分布式算法高效得多。

现代GPU中，一级缓存和二级缓存有时能出乎意料地对内核执行时间产生巨大影响。当需要实现数据重用、得到一个可预测的结果或在计算能力为1.x的硬件上开发时，我们可以用到共享内存。即使一级缓存分配了48KB大小，每个SM中仍有16KB可用的本地共享内存存储空间。

费米架构的GPU拥有16个SM，这相当于拥有256KB共享内存以及768KB一级缓存。程序员通过设置，也可以将其改为768KB共享内存以及256KB一级缓存。通过一种或两种机制实现数据重用是获得高吞吐量的关键。典型地，可以通过确保计算的局部性来实现这一点。我们可以通过将较大数据集划分成若干个小块，重复多次传输来代替之前一整块的传输方式。这样，无论做出怎样的转换，都无须多次对全局内存发出读/写操作，数据可一直存储在片上。

尽管需要大量的内存事务，内存合并还是实现高内存吞吐量的关键。在费米和开普勒架构的设备上，当使用 32 位数值（例如浮点数或整型值）时获取全部带宽需要整个 GPU 几乎完充满线程，不存在任何空闲（每个 SM 驻存 48 ~ 64 个线程束，即处理 1536 ~ 2048 个线程）。通过使用各种向量类型变量增加事务处理规模，从而优化指令级并行以及内存带宽。对于许多程序而言，每个线程处理 4 个元素比处理一个元素的效果好。

3. 计算瓶颈

（1）复杂性

尽管 GPU 有巨大的计算吞吐量，但仍有许多问题超出了 GPU 的计算能力范围。

这令人感到惊讶。这些问题通常都是整体数据量庞大，例如，各种形式的医疗图像扫描或数据处理，来自能够产生大量样本数据的设备。这类问题以前通常是通过计算机集群处理。然而现在，随着具有超强计算能力的多 GPU 计算机的出现，许多问题可直接通过单个计算机处理。

包含大量计算任务的算法在 GPU 上的执行效果比在 CPU 上的执行效果好。然而，算法通常也包含了大量的复杂逻辑部分。例如典型的分块算法中边界单元的处理。如果每个单元从与它相邻的单元收集收据，那么在块拐角处的单元只能从与它相邻的其他 3 个单元收集数据。

从图 9-53 中我们看到，整个块中间绿色部分的单元没有边界限制条件。它们可以直接从当前块中与它们相邻的单元收集数据。然而不幸的是，一些程序员在写程序时通常首先处理特殊情形。他们的内核代码如下：

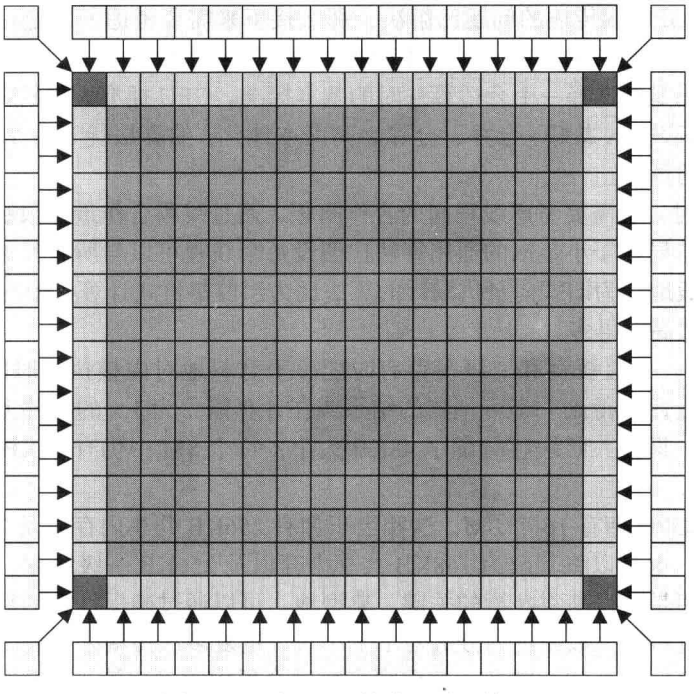

图 9-53　需要处理的边界阴影单元

```
if (top left corner cell)
else if (top right corner cell)
else if (bottom right corner cell)
else if (bottom left corner cell)
else if (top row)
else if (right row)
else if (bottom row)
else if (left row)
else (must be centre element)
```

控制逻辑复杂的算法极其不适合在GPU实现。如果每个线程执行相同的内核，在处理中间部分的元素之前需要做9次条件判断。如果颠倒条件判断的顺序，改成第一个判断中间部分的元素，这意味着我们需要4个边界测试。这可能带来一定改善，但仍不是最有效的方案。最好的解决方案就是为每种特殊情况单独写一个处理内核，或者让CPU处理这些复杂的条件判断。

此种类型的问题是一种模板型的问题，每个单元从周围N层的单元获取数据并以某种方式计算得到结果。在这个简单的例子中，N的值为1，即直接从与该单元相邻的单元获取数据来计算。随着N增大，通过使用某个因子，可以使离中心单元越远的单元对最终的结果影响越小。

由于每个单元的值需要通过周围单元的值计算得到，因此，会多次读取每个单元的值。对于这类问题一种常见的解决方法是，使用多线程将数据分块读入到共享内存中。无论是在读取数据还是写回数据时，允许对全局内存合并访问，从而达到性能提升的效果。然而，共享内存在线程块之间是不可见的，即共享内存只能在同一线程块中的线程之间共享，并且线程块与线程块之间也没有直接传输共享内存数据的机制。这主要是由CUDA的设计造成的，每次执行时，所有线程块中只有一部分线程块能够执行，因此，共享内存会在旧线程块撤出、新线程块调度之后重复利用。

在加载边界单元时，对于超出标准分块范围之外的单元，我们可以将其从全局内存中读出也可以将其一同读入共享内存。以行的方式从全局内存中读取数据可以使访存合并，高效迅速。然而，按列加载每个单元则会造成一系列独立的内存事务，效率很低。由于这些单元可能多次读取，按列读取可能会造成内存密集操作，从而限制性能。因此，至少列数据应通常存储于共享内存中。

通常，书写多个内核可以很好地消除控制流复杂性问题。我们可以用一个内核处理拐角处的元素，一个内核处理边缘行的元素，另一个内核处理边缘列的元素，最后一个内核处理中间部分的元素。如果合适，这些内核还可以调用一个通用的程序来处理一系列的数据值，这样，数据获取的复杂性就移除了。

注意，对于计算能力为1.x与计算能力为2.x的硬件应使用不同的解决方案。由于计算能力为1. x硬件没有为全局内存而设的高速缓存，每次内存事务将产生相当严重的延迟。因此，对于这类设备而言，我们只能手动将每个分块中的必要数据缓存到共享内存，或者将这部分计算任务交给CPU执行。

然而，计算能力为2.x的设备既有一级缓存也有二级缓存。每个分块必须处理自己本地的元素，而对于上方、左上方以及左方的元素，很有可能已由之前活跃的其他线程块加载到缓存中了。而对于分块右方、右下方和下方的元素，通常此时不存在于缓存中，除非对这一小块数据集进行了多次传输。对于这部分元素，通常由接下来的线程块从全局内存访问而读取到缓存中。此外，我们也可以使用预读取PTX指令（参见PTX ISA）显式地请求缓存行将这些数据读取到缓存中。

由于使用了缓存，通过将一级缓存的大小设置为48KB并且不使用共享内存，成功地消除了大量复杂的控制复杂性。复杂性的消除通常有助于对涉及计算边界的内核进行加速优化。

（2）指令吞吐量

与大多数处理器一样，并不是所有指令在每个设备上的执行时间都是相同的。对于一个给定的处理器，选择正确的指令进行混合是编译器需要认真执行的工作，但程序员也需要在这方面有一定的知识储备。

首先，我们需要确保当前硬件生成了正确的二进制文件。理想情况下，我们需要为每一个目标硬件平台设置计算能力规范。在Visual Studio中，只需在项目选项中设定即可，这一点在之前的章节中我们已经介绍过。此外，我们也可以通过命令行的方式使用-arch标志来指定。

对于单精度浮点型操作而言，所有计算能力的计算机几乎都能实现每个线程、每时钟周期执行一条指令的吞吐量。记住，是每个线程。若按绝对数目计算，则需要用每个线程束包含的线程数乘以每个SM同时处理的线程束的数目再乘以GPU上SM的数目。因此，在开普勒GTX680上，每个时钟周期可执行32个线程 ×8个线程束 ×8个SM = 2048个指令。现在，吞吐量与指令延迟并不等价。在计算出当前结果以供之后的操作使用之前，可能需要花费20个时钟周期或者更多。指令流水线中一系列的浮点操作可能在20个周期之后才开始执行，每条指令执行一个周期。因此，吞吐量是每个线程每个周期执行一条指令，但指令延迟则是20个时钟周期。

然而，对于双精度的浮点型硬件而言，却达不到该效果。对于计算能力为2.0的硬件，其执行速度为单精度的一半。而对于计算能力为2.1的硬件，其执行速度只有单精度的三分之一。计算能力为2.1的硬件（GTX460/560）以及计算能力为3.0的硬件（GTX680）瞄准的更多为游戏市场，因此，它们处理双精度浮点型的能力差一些。

在使用32位整型值时，我们也遇到了类似的问题。只有加法和逻辑指令能够全速运行。其他所有整型数指令（乘法、乘加法、移位、比较指令等）在计算能力为2.0的硬件上运行速度只达到一半，而在计算能力为2.1硬件上运行速度只有的1/3。照例，除法和求模运算没有涵盖在内。这两种操作无论在哪种计算能力的硬件上，执行开销都非常昂贵，在计算能力为1.x的硬件上执行这两条指令中的一条指令相当于执行数十条其他指令，而在计算能力为2.x的硬件上则相当于执行少于20条的其他指令（参见NVIDIA“CUDA C编程指南”，4.1版，第5章）。

在计算能力为2.0的设备上执行类型转换指令的速度只达一半，而在计算能力为2.1的设备上执行速度也只有1/3。由于硬件只支持原生整型数类型（在计算能力为2.x的设备上为

32位，计算能力为1.x的设备上为24位），当使用到8位或16位整型数时，这些类型转换指令就会发挥作用。因此，两个单字节的值做加法会导致先将这两个值的精度升为整型值，然后再将之后计算得到的结果精度降为单字节的值。同样，单精度浮点值与双精度浮点值之间的类型转换也将导致额外的类型转换指令的调用。

在C语言中，所有整数都默认为有符号整型数。而所有带小数点的数都将视作双精度浮点数，除非在数值后面跟有一个F后缀。因此，

```
#define PI (3.14)
```

表示创建一个双精度宏定义，而

```
#define PI (3.14F)
```

表示创建一个单精度宏定义。

在浮点表达式中，使用无后缀的常量将导致在计算过程中数据类型隐式转换为双精度浮点型。而当将结果赋给单精度变量时，也会执行一次单精度的隐式转换。因此，忘记使用F后缀将导致创建不必要的转换指令。

（3）同步和原子操作

在许多算法中都需要同步点。一个线程块执行同步的开销并不大，但却会潜在地影响性能。除非每个线程块包含的线程数特别多，否则CUDA调度程序会试图使每个SM最大限度地调度更多线程块（参见第5章），即每个SM调度处理16个线程块。随着每个线程块线程数量的增加，SM能够调度的线程块数量也相应减少。这并不会对程序造成很糟的影响，但如果结合同步，则可导致SM阻塞。

当线程块执行同步时，大量可供调度的线程束（计算能力为1.x设备上可供调度24个线程束，计算能力为2.x上可供调度48个线程束，计算能力为3.x上可供调度64个线程束）变得不再可供调度，直到除最后一个线程束外的其他线程束到达同步点之后才能再次调度。在每个线程块包含1024个线程的极端情况下（每个SM处理两个线程块），多达一半的SM上驻存的线程束将在同步点汇合。由于没有ILP，SM通过执行多线程隐藏内存延迟的效果将变得非常差。SM的执行不再是峰值效率。显然，我们希望能尽可能长时间的让SM的吞吐量达到最大。

解决同步问题的方案就是不使用包含大量线程的线程块。我们需要做的只是尽可能地完全填充SM，使其不闲置，因此每个线程块启动192个线程是最佳的，这样在计算能力为2.x的设备上每个SM就能调度处理8个线程块，而在计算能力为3.x的硬件上每个线程块启动256个线程更佳。

然而不幸的是，有时候我们不仅需要线程间的同步，也需要线程块间的同步。相对于线程块间的数据同步，线程间的同步更加高效。鉴于线程间的同步可通过共享内存实现，因此基于线程块的同步则可通过使用全局内存实现。这样，我们就可以对这两种方案简单地执行并对比，观察哪种方案更快。

原子操作的执行方式与同步操作非常类似，每个线程块中的线程每32个一组形成线程束，线程束中的所有线程排队依次执行相应的原子操作，因此整个时间开销比较大。然而，

与同步操作不同的是，当每个线程执行完原子操作之后即可全速执行之后的指令。这样有助于增加可供调度的线程束的数量，但无法改善总的线程块执行时间。当线程块内所有线程没有完成原子操作之前，线程块不能从 SM 中撤出。因此，单个原子操作实际上是串行展开给定线程块的每个线程束，其执行时间即线程块中每个线程束执行时间的总和。只有线程块内每个线程都完成原子操作，整个线程块才能结束。

同步和原子操作对内核带来的影响可以在 Parallel Nsight 的“CUDA Issue Efficiency”（CUDA 事件效率）实验中看到。

（4）控制流

正如我们之前看到的，分支产生的每种可能的情况都需要单独执行，因此会对执行时间产生严重影响。编译器意识到这一点后，于是使用了一种名为分支断定的技术。

大多数 PTX 指令可以通过使用 PTX ISA 的 .p 符号进行断定，例如：

```
setp.eq.s32 %p16, %r295, 1;
@%p16 bra  BB9_31;
```

此处我们在每个线程中建立一个断定寄存器，对 295 号虚拟寄存器中的值 1 进行检测并对 16 号断定寄存器进行设置。第二条指令中 16 号断定寄存器用来断定 bra 这条分支指令。这样，只有满足之前 setp.eq.s32 测试条件的线程将沿着该分支执行。我们可以用一条 mov 指令或者类似的指令取代分支。具有代表性的就是编译器对小型 if-else 结构的处理。例如：

```
if (local_idx >= 12)
local_idx = 0;
```

这段代码解释为：

```
setp.gt.u32 %p18, %r136, 11;
selp.b32 %r295, 0, %r136, %p18;
```

由于实际中每个线程束都执行了断定指令，而对于那些没有断定标记位的线程则简单忽略，因此这样能有效地避免分支。即使其他的解决方案可能更好，编译器也会偏重心于分支断定。至于偏向的程度标准则是简单地基于 if 结构的大小。考虑下面的例子：

```
// Fetch the test data element
switch(local_idx)
{
 case 0: elem = local_elem_00; break;
 case 1: elem = local_elem_01; break;
 case 2: elem = local_elem_02; break;
 case 3: elem = local_elem_03; break;

 case 4: elem = local_elem_04; break;
 case 5: elem = local_elem_05; break;
 case 6: elem = local_elem_06; break;
 case 7: elem = local_elem_07; break;

 case 8: elem = local_elem_08; break;
 case 9: elem = local_elem_09; break;
```

```
    case 10: elem = local_elem_10; break;
    case 11: elem = local_elem_11; break;

    case 12: elem = local_elem_12; break;
    case 13: elem = local_elem_13; break;
    case 14: elem = local_elem_14; break;
    case 15: elem = local_elem_15; break;
}
```

这段代码简单地根据索引选择 N 个局部变量中的一个。局部变量是单独命名的，因为若创建一个数组，编译器会将其存储于本地内存中。不幸的是，编译器实现的是一系列 if-else-if 类型的语句，这意味着在对 16 号元素操作之前，必须执行之前的 15 次测试。而我们则期望实现一张跳转表，在每一跳的目标处创建一个任务。这可能需要两条指令，一条用来将 local_idx 加载到寄存器，然后根据基地址加上寄存器中的值实现直接跳转。而跳转表是在编译时建立的。

这样，我们需要确保在编译时产生的控制流即期望产生的控制流。可以通过相对简单的方式实现这一点，例如，检查 PTX 编码，如果仍不确定的话还可检查实际目标代码。在绝大多数的程序案例中预测得到的效果都非常好，但也有部分程序例外。

9.6.3 本节小结

- 使用性能分析工具深入挖掘实际结果与预期不同的原因。
- 通过生成普通情况和特殊情况的内核来避免复杂逻辑的内核，或通过缓存的特性完全消除复杂的内核。
- 了解流控制中预测的工作机制。
- 不要假设编译器将会提供与其他更成熟的编译器相同级别的优化措施。CUDA 仍比较新，一切还有待完善。

9.7 策略 7：自调优应用程序

GPU 优化并不像 CPU 优化一样。很多技术交织在一起，而其他技术又伴随着一些不良的影响。在前面的章节中，我一直在努力涵盖优化的主要领域。但是，优化从来都不是一门精确的科学，无论如何也不是从程序员们的练习得出来的。在设计 GPU 的代码时有太多需要考虑的因素。获取最佳的解决方案并不容易，这需要相当长的时间熟悉什么在起作用，尝试不同的解决方案以及理解为什么一个可以运行，而另一个不可以。

考虑如下一些主要的因素：

- 主机到 GPU 的数据传入 / 传出。
- 内存合并。
- 启动配置。
- 理论上和实际的占有率。

- 缓存利用率。
- 共享内存使用率以及冲突。
- 分支。
- 指令级并行。
- 设备计算能力。

对于刚开始使用 CUDA 的人来说，有很多值得思考的东西，需要一段时间才能精通这些领域。然而，最具挑战性的是可以在一台设备上工作的方法可能无法在另外的设备上运行。在这本书中，我们使用了可用设备的整个系列并为展示差异使用了多种主机平台。

不同的 CPU 提供了不同层次的性能和功能，类似地，GPU 也是如此。CPU 受制于 x86 架构，使得一个系统的设计目标是设计成串行程序。而现在已经有很多扩展提供了额外的功能，如 MMX、SSE、AVX 等。x86 指令集如今已经以微指令的形式实现在硬件中，从而适用于任何目标硬件。沙桥架构也许是最好的例子，其微指令是缓存的，而不是实际的 x86 汇编代码指令。

自从 GTX8800 这样的第一代支持 CUDA 的设备发布以来，GPU 硬件从未止步不前，正在发生显著的改变。CUDA 会编译成 PTX，这是一种虚拟的汇编代码，面向并行处理器类型的架构。与合作线程阵列（Cooperative Thread Array，CTA）的概念类似，可以实现在大多数并行硬件上，PTX 本身可以编译成很多目标，包括 CPU。然而，据我们所知，对于不同的英伟达 GPU，它都会编译到一个特定的计算能力上。因此，需要熟悉一个给定的计算能力可以提供什么，也就是需要了解在什么样的硬件上编写代码。这一直都是好的优化的基础。如今，抽象、分层和隐藏架构等发展趋势，都是为了提高程序员的工作效率，但这往往都是以牺牲性能为代价的。

并不是每一个程序员都对错综复杂的硬件机理感兴趣。即使考虑了上述列表的问题，如果没有深思熟虑以及大量的试验和错误，你不可能在第 1 次、第 2 次，甚至第 N 次得到一个最佳的解决方案。因此，这个问题行之有效的方法是只需制定出程序针对给定问题最佳利用硬件的方法。这在一个小的问题集合或者真实问题上也都是可以完成的。

9.7.1 识别硬件

在任何优化过程中的第一步都要知道什么硬件是可用的以及它是什么。要找出有多少 GPU，只需调用

```
cudaError_t cudaGetDeviceCount(int * count);
```

这将把传入的 count 变量设置为可用的设备数量。如果没有可用的 CUDA 硬件，该函数返回 cudaErrorNoDevice。

然后为每台发现的设备，我们需要知道它的能力上限是什么。因此，我们调用

```
cudaError_t cudaGetDeviceProperties (struct cudaDeviceProp * prop, int device);
```

我们在第 8 章中详细介绍了设备的属性，所以这里不再赘述。然而，至少应该关注以下细节：

- 成员变量 major 和 minor 一起使用时，可以提供该设备的计算能力。
- 标志变量 integrated，尤其是和 canMapHostMemory 标志一起使用时，允许使用零复制内存（策略 3 中讲述的）以及避免对于那些 GPU 内存，事实上已经在主机的设备间进行内存复制。
- totalGlobalMem 可以最大程度地利用 GPU 内存并保证不会试图在 GPU 上分配过多内存空间。
- sharedMemPerBlock 可以让你知道每一个 SM 有多少共享内存可用。
- multiProcessorBlock 代表设备中目前的 SM 数量。用该值乘以一个 SM 上可运行线程块的数量就是整个设备可容纳的线程块数。占用率计数器（occupancy calculator），例如 Visual Profiler 和 Parallel Nsight，都可以说明一个给定的内核可以运行多少个线程块。一般来说是 8 个，在开普勒架构可以是 16 个。这是你需要在 GPU 上分配最小线程块数量。

这些信息给了我们一些定义问题空间的界限。那么我们就有两种选择：要么分析离线的最佳解决方案，要么尝试找出运行时的解决方案。通常，离线的方法会有更好的结果，可以极大地提高对所涉及问题的理解，并可能使你重新设计程序的某些方面。对于获得最佳性能，运行时的做法是必要的，即使是已经进行了重要的分析。

因此在开发阶段，优化的第一步是离线。如果针对多个计算能力，则需要一个合适的显卡测试你的应用程序。总的来说，对于消费级显卡，最流行的英伟达显卡有 9800（计算能力 1.1），8800（计算能力 1.0），GTX260（计算能力 1.3）和 GTX460（计算能力 2.1）。对于更现代的 DirectX11 显卡，460/560 显卡占主导地位，对比于用户选择的更昂贵的 470/570 显卡，其能耗更小。本书中我们选择的硬件很好地反映了市场的发展趋势，提出的数据尽可能地帮助面向大众市场的应用开发人员。

自从在 8800 系列显卡上发布以来，由于我们一直在使用 CUDA，我们已经有了一些可用的消费级显卡。显然，其中的一些已经停产，但仍可以很容易地在 eBay 或其他地方买到。所需要的是一个带有 4 个双槽间距的 PCI-E 插槽的主板，这些槽全插满时可以以相同的速度运行。尽管现在已经被 MSI 890FXX-G70 取代，本书在开发时用到的主板是 AMD MSI 790FX-GD70。请注意，最新的 990FX 主板系列中不再提供 4 个双槽间距的插槽。

9.7.2 设备的利用

在确定我们拥有什么样的硬件后，必须利用它。如果系统中有多个 GPU，这是通常的情况，一定要充分利用它们。自从 CUDA 4.x SDK 起，多 GPU 编程比以前更加容易。因为你可能只使用了一个 GPU，所以要确保没有错过成倍的性能提升。更多信息请参阅第 8 章。

所有的应用程序都是不同的，所以影响性能的主要因素可能并不总是相同的。然而，许多情况下却都是相同的，主要因素都是启动配置。第一部分是确保在生成的过程中建立了多个目标，并为打算支持的每个计算能力建立一个目标。根据内核运行在哪一个 GPU 上，会自动选择目标代码。同时确保运行任何性能测试前，选择了发布模式作为生成的目标，这可

以提供多达 2 倍的性能提升。因为不会发布调试模式，所以不要选择这个作为生成目标，除非是为了测试。

接下来，我们需要检查以确保正确性。我建议对比运行 GPU 和 CPU 代码，然后对于两个相同的测试输出做一个内存比较（memcmp）。请注意，这将检测到任何错误，即使错误并不是那么显著。尤其是在浮点运算时，操作的顺序会造成精度的误差。在这种情况下，需要反复检查两个结果，看看对于你的特定问题，差异是否在某个显著水平上（例如 0.01、0.001、0.0001 等显著水平）显著。

根据启动配置，我们要尽量优化以下方面：

- 每个块的线程数；
- 全部的块数量；
- 每个线程执行的任务（指令级并行）。

不同的计算能力其结果都会不同。一个简单的 for 循环就需要遍历所有可能的组合并记录每一个对应的时间。然后在最后只需打印出对结果的总结。

每个线程块的线程数，从 1 开始，以 2 的幂递增，直到达到 16。然后以 16 为补偿增加线程计数直到达到每个块 512 个线程。根据不同的内核资源使用情况（寄存器、共享内存），由于在早期计算设备上不可能达到 512 个线程，因此需要根据这些设备相应地缩减线程数。

请注意，我们此处选择了 16 作为增量值，而不是线程束的大小——32。这是因为线程束的条件分支是以半个线程束为单位的。某些设备如 GTX460 是 3 组，每组 16 个 CUDA 核，而其他的计算能力通常是两组。因此对于这些设备，使用 48 的倍数作为线程数目会更好地运行。

作为一般规则，编写得很好的内核代码通常在每块 128、192 或者 256 个线程时，运行得最好。需要从每个线程块只含一个线程不断增加线程数直到一个峰值，此时的性能会趋于平稳，如果继续增加，性能反而下降。当每个 SM 中同时驻留的线程数达到最大值时，性能会达到一个顶峰，因为指令和内存延迟的隐藏在峰值发挥作用。

如果能增加每个 SM 驻留的块的数目，使用稍少的线程数（例如，192 而不是 256）通常是可取的。这通常会提供一个更好的指令混合，随着块数的增长，它们更有可能不会同时遇到对同一个资源的竞争。

如果在 16、32 或者 64 个线程时就达到了最大性能，那么通常表明存在资源竞争，或内核是高度面向指令级并行的，并且每个线程中使用了大量的寄存器。

对于每个块的理想线程数，一旦有了一个基准数据，试着使用各种 vector_N 类型（例如，int2、int4、float2、float4 等）的两个或 4 个元素增加每个线程的工作量，通常会看到进一步地性能提升。最容易的方法是通过创建具有相同名称的新函数，重载内核函数。CUDA 将根据运行时不同类型的传入参数调用相应的内核代码。

使用向量类型将增加寄存器的使用，反过来又可能会降低每个 SM 中线程块的驻留数量，也会进一步提高缓存的利用率。内存吞吐量也将可能随着内存事务总数的下降而增加。然而，有同步点的内核可能会受到影响，因为随着驻留块数的下降，SM 中可供调度的线程

束变少了。

正如许多的优化方法一样，其结果是难以预料的，因为有些因素会符合你的想法而另一些没有。最好的解决办法就是尝试和检查。然后回退，以理解哪个或者哪些因素是主要的，哪些是次要的。不要浪费时间在次要因素上，除非主要因素已经解决。

9.7.3 性能采样

自调优程序的最后一部分是采样。虽然可以建立一个围绕计算能力以及 SM 数量的良好的性能模型，还有许多其他因素需要考虑。相同的显卡型号可能分别使用 GDD3 和 GDD5 内存，后者具有更大的全局内存带宽。同一个显卡的时钟可以设定为 600MHz，也可能是 900MHz。一个有 16 个 SM 的显卡的优化策略可能在只有其一半数量 SM 的显卡上不起作用，反之亦然。一台笔记本电脑的移动处理器可能放在一个 PCI-E X1 插槽上，并且主机端可能拥有专用内存或者共享内存。

收集每一种显卡并且阐述每一个产品可能要解决的变化是不可能的。即使能做到这一点，下周 NVIDIA 将仍会发布另一个显卡。对于消费级应用程序的开发者来说，这无疑是一个主要的问题，而对于有较少种类的 Telsa 显卡的开发者则不是。然而，当一个新的显卡发布时，人们首先想到的是在上面运行他们已有的应用程序，其次，如果他们进行了升级，也希望看到相应的性能提升。

采样便是这个问题的答案。根据其特点，设置最适合的启动配置，每个显卡都将有一个峰值。正如我们所看到的，本书中的一些测试，不同的显卡有不同的设置。费米显卡的每个线程块适合有 192 或 256 个线程，然而之前的 GPU 适合于设置每个块 128 和 192 个线程。计算能力 2.1 的显卡，在混合使用 64 或 128 字节的内存读取和基于指令的并行时，性能最佳；而使用 32 字节的内存读取单位和每个线程单个元素时是无法与之比较的。在合并读取时，早期的显卡对于线程、内存顺序高度敏感。这些卡上的全局内存带宽尽管远远赶不上较新的模型，但在某些任务上如果使用共享内存，它们也会有类似的性能。如果数据随后会完整地加载到缓存中，费米显卡的缓存会发挥很大的作用，使得非常小的线程数（32 或 64）能表现出更高的占用率。

在安装程序时，请运行一个很小的测试集作为安装过程的一部分。在所有可行的线程数上运行一个循环。尝试每个线程的指令级并行数量，取从一个到 4 个元素。尝试启用和禁用共享内存。运行一定数量的实验，每个重复一定次数，得到平均结果，将 GPU 相关的理想值存储在数据文件或程序配置文件中。如果用户以后升级 CPU 或 GPU，重新运行实验，并更新配置。只要不在每次启动时都这样做，用户将很高兴你将应用程序调整至最适合他们硬件的状态。

9.7.4 本节小结

- 如果没有尝试过，有太多的因素让你无法断定改变会带来什么效果。
- 在开发时，需要进行一些实验以获取最好的解决方案。

- 不同的硬件平台上的最佳解决方案是不同的。
- 编写应用程序的时候，要意识到会碰到各种硬件，并要知道每个平台上哪些能（静态的或者动态的）工作得最好。

9.8 本章小结

在本章中，通过不同的例子，我们已经详细地看到了一些提升内核吞吐量的策略。应该知道那些影响性能的因素和它们的相对重要性（主要的是传输，内存 / 数据模式，最后是 SM 利用率）。

优化代码中最关键的是正确性。没有自动地回归测试，不可能可靠地优化代码。这并不一定是非常复杂的。利用一些已知的数据集，在已知的版本上对比测试是完全足够的。应该保证任何程序在离开办公桌前，95% 以上的错误已经发现了。测试不只是一些测试团队的工作，也是作为一名生产可靠运行代码的专业程序员的责任。优化往往会一遍又一遍地重写代码。只需运行一分钟却得到错误答案，与需要运行一小时而能得到正确答案相比，是没有用的。在每次更改后务必测试一下正确性，这样在错误出现时就能及时发现。

应该知道，优化是一个耗费时间和反复的过程，它会增加对代码以及硬件是如何起作用的理解。反过来，因为更熟悉什么可以或者不能很好地工作在 GPU 上，从而从一开始就能设计和编写出更好的代码。

问题和优化

1. 利用一个现有的、包含一个或多个 GPU 内核的程序。运行 Visual Profiler 和 Parallel Nsight 分析内核。哪些是需要查看的关键指标？如何优化这个程序？

2. 一个同事打印出来一份 GPU 内核代码，想咨询如何运行得更快，有什么建议呢？

3. 另一个同事提出用 CUDA 实现一个 Web 服务器。你觉得这是一个好主意吗？对于这个程序，你觉得会遇到什么问题？

4. 实现一个共享内存版本的奇偶排序，输出一个单一的排序列表，你觉得可能会遇到什么问题？

答案

1. 首先，应该观察每个内核的执行时间。如果一个或更多的内核占据了大部分时间。那么在优化它之前，优化其他的内核便是浪费时间。

其次，应该观察一下时间线，尤其是在与传输相关的时候。它们是不是与内核操作进行了重叠，它们是不是使用了锁页内存？ GPU 是不是一直占用或者周期性地由主机分配任务？

对最耗时的两个内核，是什么引起它们占用那么多的时间？是否有足够多的线程总量？所有的 SM 上是否有足够多的线程块？是否有一个 SM 出现了峰值，如果有，为什么？什么是线程的内存模式，可不可以由硬件进行合并？是不是有串行化点，例如，共享内存的存储

体冲突、原子操作和同步点？

2. 首先，需要在分析细节之前，理解这个问题。“观察代码”的优化策略可能有效也可能无效。当然，也许可以以某种方式在纸面上优化代码，但当你的同事是要真正寻求这个问题好的答案时，就需要更多信息了。

也许最好的答案是告诉你的同事分析应用程序，包括主机的时间线，然后回来分析结果。在这个过程中，他们有可能会看到问题出在哪里，甚至有可能与原来的内核打印输出无关。

3. 数据高度并行化的应用程序非常适合运行在 GPU 上。而含有许多分支线程的任务高度并行化的应用程序却不适合。在 CPU 上，典型的实现 Web 服务器是每 N 个连接分配一个线程，并将其动态地分配在服务器集群中，来防止任意一个节点服务器过载。

GPU 在许多组由 32 个线程构成的线程束上执行代码，实际上是一个在必要时可以执行单线程控制流的向量处理器，但是这会在性能上有很大的损失。构建实时的动态网页需要大量的控制流，其中有很多要为每个用户产生分支。PCI-E 的传输会比较小，而且效率不高。

GPU 不会是一个好的选择，而 CPU 主机方式则会好很多。然而，GPU 可以在服务器的后端操作上用到，执行某些分析工作，使用户产生的数据变得有意义。

4. 这对于思考如何解决某些开放式问题是一个有益的锻炼。首先，这个问题并没有指出如何合并 N 个块的输出。

对于最大的数据集，最快的解决办法应该是样本排序，因为它完全消除了归并排序的步骤。样本排序的框架已经在本文提供了，但仍然是一个相当复杂的排序。然而，它需要每个桶里有可变数目的元素，这会带来损失。用前缀求和方法将这些桶补齐为一个 128 字节大小的区域会有很大的帮助。

归并排序更容易实现，它允许固定的块大小，这也是我认为最合适的实现方式。

在奇偶排序中，全局内存的合并访问问题在很大程度上由费米架构的缓存所掩盖，因为局部性会非常明显。在计算能力 1.x 上实现排序，会使用共享内存或者寄存器。在载入和写回数据时，需要以合并的方式访问全局内存。

第 10 章

函数库和 SDK

10.1 简介

为了利用 GPU 加速任务，直接使用 CUDA 编写程序并不是唯一可选的过程。有三种常见的开发 CUDA 应用程序的方式：

- 使用函数库
- 基于指令的编程
- 直接编写 CUDA 内核

接下来，我们会依次考察它们并且理解什么时候应该使用它们。

10.2 函数库

函数库是十分有用的组件，因为它可以帮你在开发过程中节省数周甚至数月的精力。我们应该尽可能地使用函数库，因为它们基本上都是由特定领域的专家开发的，所以它们是可靠而高效的。一些常见的、免费的函数库如下：

- Thrust——一个 C++ STL（Standard Template Library）实现的函数库
- NVPP——NVIDIA 性能原语（和 Intel 的 MKK 类似）
- CuBLAS——BLAS（基本线性代数）函数库的 GPU 版本
- cuFFT——GPU 加速的快速傅里叶变换函数库
- cuSparse——稀疏矩阵数据的线性代数和矩阵操作
- Magma——LAPACK 和 BLAS 函数库
- GPU AI——基于 GPU 的路径规划和碰撞避免
- CUDA Math Lib——支持 C99 标准的数学函数

除此之外，还有很多商业化的产品，它们也会提供一些限制性的免费功能或试用版本：

- Jacket——对 .m 代码可选的、基于 GPU 的 Matlab 引擎
- Array Fire——类似于 IPP、MKL 和 Eigen 的矩阵、信号和图像处理库
- CULA 工具——线性代数库

- IMSL——Fortran IMSL 数值函数库的实现

当然，还有很多其他的函数库我们并没有在此列出。我们在 www.cudalibs.com 维护了一个 CUDA 函数库的列表，其中还包括一些我们自己的函数库，它们可为个人或学术用途免费使用，在得到许可和支持的前提下可用于商业用途。

10.2.1 函数库通用规范

作为一般原则，NVIDIA 提供的函数库，不对调用者进行内存管理。而是希望调用者提供指向设备中被分配的内存区域的指针。这就使得很多设备上的函数可以一个接一个地执行而无须在两个函数调用间使用不必要的设备 / 主机进行传输操作。

由于它们不执行内存操作，因此分配和释放内存变成了调用者的职责。这甚至扩大到为函数库用到的暂存空间或缓冲区提供内存。

虽然对程序员来说这可能是一笔开销，但实际上这是一种很好的设计原则并且在设计函数库时也应该遵循。内存分配是代价很高的操作。资源是很有限的。让函数库在后台持续不断地分配、释放内存，不如在启动时执行一次分配操作，然后在程序退出时再执行一次释放操作。

10.2.2 NPP

NPP 函数库提供了一系列图像和通用信号处理的函数。它支持所有的 CUDA 平台。为了将 NPP 包含到工程中，只需包含相关的头文件并链接到预编译库。

对于信号处理函数，库函数接收一个或多个源指针（pSrc1、pSrc2 等）、一个或多个目的指针（pDst1、pDst2 等）；对于 in-place 操作接收一个或多个混合指针（pSrcDst1、pSrcDst2 等）。函数库根据待处理的数据类型对函数进行命名。该函数库并不支持 C++ 函数名重载（支持不同数据类型作为参数的通用函数只使用一个函数名）。

信号处理支持的数据类型有 Npp8u、Npp8s、Npp16u、Npp32u、Npp32s、Npp64u、Npp64s、Npp32f 和 Npp64f。这些等同于 signed 和 unsigned 版本的 8、16、32、64 字节类型以及 32 位和 64 位单精度和双精度浮点类型。

该函数库图像处理部分也遵循相似的命名规则以使函数名反映出用途和数据类型。图像数据可以被组织成数字的形式，因此有一些关键的字母可以让你从名字中看出其功能和数据类型。它们是：

- A——当图像中包含一个不应该被处理的 alpha 通道时使用。
- Cn——当数据被设计成 n 个通道的包装或者是交错格式时使用，例如 {R、G、B}，{R、G、B}，{R、G、B} 等是 C3。
- Pn——颜色数据被划分成平面时使用，例如来自于同一种颜色的所有数据都是相邻的，因此 {R、R、R}，{G、G、G}，{B、B、B} 等是 P3。

命名除了表明数据是如何组织的之外还能告诉你函数是如何操作数据的。

- I——图像数据被就地（in-place）处理时使用。也就是说，源图像数据将会被在其上

执行的操作覆写。

- M——表明将会使用非零的掩码决定哪些像素满足标准。只有满足标准的那些像素会被处理。这是很有用的，例如将一张图像覆盖到另一张上面。
- R——表明函数通过调用一个指定的感兴趣区域（Region Of Interest，ROI）对图像的一个子部分进行操作。
- Sfs——表明函数对输出数据执行固定的比例缩放和饱和操作。

使用这么短的函数名后缀可能导致对于函数名普通读者很难理解。然而，一旦你记住了函数名每个属性的涵义并且使用一些 NPP，将会很快地识别出某个指定的函数执行什么操作。

图像数据函数也使用额外的参数 pSrcStep 或 pDstStep，它们是指向图像行或列大小且以字节的单位的指针，其中也包括所有为了确保对齐加入到行宽度的填充字节。许多图像处理函数在每一行的结尾处添加填充字节以确保下一行在合适的边缘处开始。因此，一个 460 像素宽的图像每行可能添加至 512 字节。行宽度值为 128 的倍数是很好的选择，因为这能够从内存子系统中读取整个缓存行。

我们来看信号处理函数库里一个简单的例子。我们取出两个随机数据集合，进行异或操作。我们同时在主机和设备端进行，然后将结果进行对比。

```
#include <stdlib.h>
#include <stdio.h>
#include <iostream>

#include "cuda.h"
#include "cuda_helper.h"
#include "common_types.h"
#include "timer.h"

// NPP Library
#include "npp.h"
#include "nppcore.h"
#include "nppdefs.h"
#include "nppi.h"
#include "npps.h"
#include "nppversion.h"

#define NPP_CALL(x) {const NppStatus a = (x); if (a != NPP_SUCCESS) { printf("\nNPP
Error: (err_num=%d) \n", a); cudaDeviceReset(); ASSERT(0);} }

int main(int argc, char *argv[])
{
 const int num_bytes = (1024u * 255u) * sizeof(Npp8u);
 // Declare and allocate memory on the host
 Npp8u * host_src_ptr1 = (u8 *) malloc(num_bytes);
 Npp8u * host_src_ptr2 = (u8 *) malloc(num_bytes);
 Npp8u * host_dst_ptr1 = (u8 *) malloc(num_bytes);
 Npp8u * host_dst_ptr2 = (u8 *) malloc(num_bytes);
```

```
// Check memory allocation worked
if ( (host_src_ptr1 == NULL) || (host_src_ptr2 == NULL) ||
     (host_dst_ptr1 == NULL) || (host_dst_ptr2 == NULL) )
{
 printf("\nError Allocating host memory");
 exit(0);
}

// Declare and allocate memory on the device
Npp8u * device_src_ptr1;
Npp8u * device_src_ptr2;
Npp8u * device_dst_ptr1;
Npp8u * device_dst_ptr2;
CUDA_CALL(cudaMalloc((void **) &device_src_ptr1, num_bytes));
CUDA_CALL(cudaMalloc((void **) &device_src_ptr2, num_bytes));
CUDA_CALL(cudaMalloc((void **) &device_dst_ptr1, num_bytes));
CUDA_CALL(cudaMalloc((void **) &device_dst_ptr2, num_bytes));

// Fill host src memory with random data
for (u32 i=0; i< num_bytes; i++)
{
 host_src_ptr1[i] = (rand() % 255);
 host_src_ptr2[i] = (rand() % 255);
}

// Copy the random data to the device
CUDA_CALL(cudaMemcpy(device_src_ptr1, host_src_ptr1, num_bytes,
cudaMemcpyHostToDevice));
CUDA_CALL(cudaMemcpy(device_src_ptr2, host_src_ptr2, num_bytes,
cudaMemcpyHostToDevice));

// Call NPP library to perform the XOR operation on the device
TIMER_T start_time_device = get_time();
NPP_CALL(nppsXor_8u(device_src_ptr1, device_src_ptr2, device_dst_ptr1, num_bytes));
NPP_CALL(nppsAnd_8u(device_src_ptr1, device_dst_ptr1, device_dst_ptr2, num_bytes));
TIMER_T delta_time_device = get_time() - start_time_device;

// Copy the XOR'd data on the device back to the host
CUDA_CALL(cudaMemcpy(host_dst_ptr1, device_dst_ptr2, num_bytes,
cudaMemcpyDeviceToHost));
// Perform the same XOR followed by AND on the host
TIMER_T start_time_cpu = get_time();
for (u32 i=0; i< num_bytes; i++)
{
 host_dst_ptr2[i] = host_src_ptr1[i] ^ host_src_ptr2[i];
 host_dst_ptr2[i] &= host_src_ptr1[i];
}
TIMER_T delta_time_cpu = get_time() - start_time_cpu;

// Compare the device data with the host calculated version
```

```
    printf("\nComparison between CPU and GPU processing: ");
    if (memcmp(host_dst_ptr1, host_dst_ptr2, num_bytes) == 0)
    {
     printf("Passed");
    }
    else
    {
     printf("**** FAILED ****");
    }
    printf("\nCPU Time: %f, GPU Time: %f", delta_time_cpu, delta_time_device);

    // Free host and device memory
    CUDA_CALL(cudaFree(device_src_ptr1));
    CUDA_CALL(cudaFree(device_src_ptr2));
    CUDA_CALL(cudaFree(device_dst_ptr1));
    CUDA_CALL(cudaFree(device_dst_ptr2));
    free(host_src_ptr1);
    free(host_src_ptr2);
    free(host_dst_ptr1);
    free(host_dst_ptr2);

    // Reset the device so it's clear for next time
    CUDA_CALL(cudaDeviceReset());
}
```

注意，在对 NPP 函数库调用的地方我们使用了 NPP_CALL 这个宏。它与全书中一直使用的 CUDA_CALL 类似。它检查调用者的返回值是否一直为 NPP_SUCCESS（0），若不是，则会打印与返回值相关的错误代码。负值代表错误，正值代表警告。不幸的是，并没有函数可以将错误代码转换成错误消息，因此你必须在 NPP 文档中查找错误值（参见 7.2 节）。

```
NPP_CALL(nppsXor_8u(device_src_ptr1, device_src_ptr2, device_dst_ptr1, num_bytes));
NPP_CALL(nppsAnd_8u(device_src_ptr1, device_dst_ptr1, device_dst_ptr2, num_bytes));
```

每一个 NPP 调用都会在设备端启动一个内核。默认情况下，NPP 用默认的流 0 以同步模式进行操作。然而，你将经常一个接一个地执行大量的操作。然后，你可能希望接着在 CPU 端执行一些其他工作，因此随后你会回过来再检查 GPU 任务的进展。

为了指定 NPP 使用某个特定的、已定义好的流，使用下面的 API 调用：

```
void nppSetStream (cudaStream_t hStream);
```

和本书中看到的其他一些例子一样，如果有许多连续内核调用，将它们放入非默认的流中能够获得更好的整体性能。很大程度上是因为这种方式允许异步内存传输从而将计算和传输工作重叠执行。然而，为了实现这种效果，我们需要稍微改写一下程序：

```
// Max for compute 2.x devices is 16
#define NUM_STREAMS 4

int main(int argc, char *argv[])
```

```
{
 // 64MB
 const int num_bytes = (1024u * 255u * 256) * sizeof(Npp8u);

 // Select the GTX470 in our test setup
 CUDA_CALL(cudaSetDevice(0));

 printf("\nXOR'ing with %d MB", (num_bytes / 1024) / 1024);

 // Declare and allocate pinned memory on the host
 Npp8u * host_src_ptr1;
 Npp8u * host_src_ptr2;
 Npp8u * host_dst_ptr1[NUM_STREAMS];
 Npp8u * host_dst_ptr2;

 CUDA_CALL(cudaMallocHost((void **) &host_src_ptr1, num_bytes));
 CUDA_CALL(cudaMallocHost((void **) &host_src_ptr2, num_bytes));
 CUDA_CALL(cudaMallocHost((void **) &host_dst_ptr2, num_bytes));
 for (u32 i=0; i< NUM_STREAMS; i++)
 {
  CUDA_CALL(cudaMallocHost((void **) &(host_dst_ptr1[i]), num_bytes));
 }

 // Declare and allocate memory on the device
 Npp8u * device_src_ptr1[NUM_STREAMS];
 Npp8u * device_src_ptr2[NUM_STREAMS];
 Npp8u * device_dst_ptr1[NUM_STREAMS];
 Npp8u * device_dst_ptr2[NUM_STREAMS];
 for (u32 i=0; i< NUM_STREAMS; i++)
 {
  CUDA_CALL(cudaMalloc((void **) &(device_src_ptr1[i]), num_bytes));
  CUDA_CALL(cudaMalloc((void **) &(device_src_ptr2[i]), num_bytes));
  CUDA_CALL(cudaMalloc((void **) &(device_dst_ptr1[i]), num_bytes));
  CUDA_CALL(cudaMalloc((void **) &(device_dst_ptr2[i]), num_bytes));
 }

 // Fill host src memory with random data
 for (u32 i=0; i< num_bytes; i++)
 {
  host_src_ptr1[i] = (rand() % 255);
  host_src_ptr2[i] = (rand() % 255);
 }

 TIMER_T start_time_device = get_time();

 printf("\nRunning Device Synchronous version");

 for (u32 i=0; i< NUM_STREAMS; i++)
 {
  // Copy the random data to the device
```

```
 CUDA_CALL(cudaMemcpy(device_src_ptr1[i], host_src_ptr1,
                   num_bytes, cudaMemcpyHostToDevice));
 CUDA_CALL(cudaMemcpy(device_src_ptr2[i], host_src_ptr2,
             num_bytes, cudaMemcpyHostToDevice));

 // Call NPP library to perform the XOR operation on the device
 NPP_CALL(nppsXor_8u(device_src_ptr1[i], device_src_ptr2[i],
            device_dst_ptr1[i], num_bytes));

 // Copy the XOR'd data on the device back to the host
 CUDA_CALL(cudaMemcpy(host_dst_ptr1[i], device_dst_ptr1[i],
            num_bytes, cudaMemcpyDeviceToHost));
}

// Grab the end time
// Last memcpy is synchronous, so CPU time is fine
TIMER_T delta_time_device = get_time() - start_time_device;

printf("\nRunning Host version");

// Perform the same XOR on the host
TIMER_T start_time_cpu = get_time();
for (u32 i=0; i< NUM_STREAMS; i++)
{
 for (u32 i=0; i< num_bytes; i++)
 {
  host_dst_ptr2[i] = host_src_ptr1[i] ^ host_src_ptr2[i];
 }
}
TIMER_T delta_time_cpu = get_time() - start_time_cpu;

// Compare the device data with the host calculated version
for (u32 i=0; i< NUM_STREAMS; i++)
{
 compare_results(host_dst_ptr1[i], host_dst_ptr2, num_bytes,
            "\nSingle Stream Comparison between CPU and GPU processing: ");
}

printf("\nRunning Device Asynchronous version");

// Now run and alternate streamed version
// Create a stream to work in

cudaStream_t async_stream[NUM_STREAMS];
for (u32 i=0; i< NUM_STREAMS; i++)
{
 CUDA_CALL(cudaStreamCreate(&async_stream[i]));
}

// Grab the CPU time again
```

```
start_time_device = get_time();

for (u32 i=0; i< NUM_STREAMS; i++)
{
 // Tell NPP to use the correct stream
 NPP_CALL(nppSetStream(async_stream[i]));

 // Copy the random data to the device using async transfers
 CUDA_CALL(cudaMemcpyAsync(device_src_ptr1[i], host_src_ptr1, num_bytes,
           cudaMemcpyHostToDevice, async_stream[i]));
 CUDA_CALL(cudaMemcpyAsync(device_src_ptr2[i], host_src_ptr2, num_bytes,
          cudaMemcpyHostToDevice, async_stream[i]));

 // Call NPP library to perform the XOR operation on the device
 NPP_CALL(nppsXor_8u(device_src_ptr1[i], device_src_ptr2[i],
           device_dst_ptr1[i], num_bytes));
}

for (u32 i=0; i< NUM_STREAMS; i++)
{
 // Tell NPP to use the correct stream
 NPP_CALL(nppSetStream(async_stream[i]));

 // Copy the XOR'd data on the device back to the host using async mode
 CUDA_CALL(cudaMemcpyAsync(host_dst_ptr1[i], device_dst_ptr1[i], num_bytes,
          cudaMemcpyDeviceToHost, async_stream[i]));
}
// Wait for everything to complete
for (u32 i=0; i< NUM_STREAMS; i++)
{
 CUDA_CALL(cudaStreamSynchronize(async_stream[i]));
}

// Grab the end time
TIMER_T delta_time_device_async = get_time() - start_time_device;

// Compare the device data with the host calculated version
for (u32 i=0; i< NUM_STREAMS; i++)
{
 compare_results(host_dst_ptr1[i], host_dst_ptr2, num_bytes, "\nMulti Stream
Comparison between CPU and GPU processing: ");
}

printf("\nCPU Time: %.1f, GPU Sync Time: %.1f, GPU Async Time: %.1f", delta_time_cpu,
delta_time_device, delta_time_device_async);

// Free host and device memory
for (u32 i=0; i< NUM_STREAMS; i++)
{
 CUDA_CALL(cudaFree(device_src_ptr1[i]));
```

```
    CUDA_CALL(cudaFree(device_src_ptr2[i]));
    CUDA_CALL(cudaFree(device_dst_ptr1[i]));
    CUDA_CALL(cudaFree(device_dst_ptr2[i]));
    CUDA_CALL(cudaFreeHost(host_dst_ptr1[i]));
    CUDA_CALL(cudaStreamDestroy(async_stream[i]));
  }

  CUDA_CALL(cudaFreeHost(host_src_ptr1));
  CUDA_CALL(cudaFreeHost(host_src_ptr2));
  CUDA_CALL(cudaFreeHost(host_dst_ptr2));

  // Reset the device so it's clear for next time
  CUDA_CALL(cudaDeviceReset());
}
```

我们在流版本中看到的主要不同是，现在我们需要在主机端有多个输出数据块，同时在设备端要有多个副本。因此，所有的设备阵列可以通过 [NUM_STREAMS] 索引，这就使每个流可以完全独立于其他流进行操作。

为了使用异步模型我们需要像锁页内存一样分配主机内存，因此我们必须使用 cudaHostMalloc 而不是 malloc。相应的内存执行操作使用 cudaFreeHost 而不是 free。我们还需要在处理流的数据之前等待流的完成。在这个例子中，我们等待了所有的 4 个流，但是实际上，如果一个流完成了，会被提供更多的工作。分别看一看第 8 章和第 9 章中关于多 GPU 编程和优化的内容，了解这是如何工作的。

从 Parallel Nsight 中我们可以看到，这确实在新的流版本的代码中发生了（参见图 10-1）。请注意 Parallel Nsight 的输出在两次大规模的到设备的传输后紧接着是少部分的内核操作。同时也要注意流 2 中的内核仍在执行时（Memory 和 Compute 两行）流 3 中的传输已经开始了。最后，注意传回主机的传输都是一个接一个进行的。

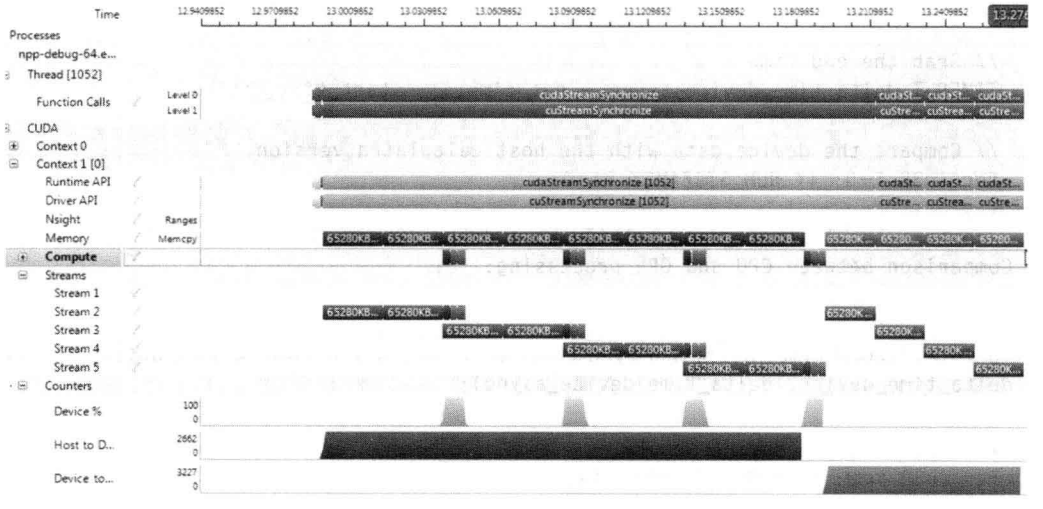

图 10-1 NPP 流调用

在这个例子中，传输决定了总体的时间帧（经常都是这样的）。这主要取决于你在 GPU 执行的处理规模以及你是否需要持续传输数据。把数据留在 GPU 是一个很好的解决方案，尤其是如果以后你想将它可视化或者不需要在主机端保留一份副本。

在这个特定的例子中，由于和传输时间相比这些内核执行时间都比较少，因此同步时间为 300ms 而异步时间为 280ms。我们会遇到一个很短的内核 / 传输重叠期，因此我们仅在整体时间帧中节省了这段时间。为了从能够并行的独立工作负载中显著地受益，我们需要加入更多 GPU，其中传输和内核可以在 N 个 GPU 上并行地执行。

综合考虑事件、内存复制和内核，使用异步模式会使你的程序获得巨大的提升。这是因为当 CPU 安排更多的任务时，CUDA 设备会继续执行工作集而不是闲置。通过对相互独立的工作单元使用多个流，除了常规的数据级并行外，你还可以定义任务级的并行。在某种程度上，这在费米级的 GPU（计算能力 2.x）上是可以利用的特性，因为它以前经常通过并排的和并发的内核来填充 GPU。随着 GPU 中的 SM 设备变得越来越大（开普勒架构就是这种情况），这也就变得越来越重要。

请注意，这里的设置是针对消费级显卡中的单 DMA 传输引擎的。Tesla 设备启用了 2 个 DMA 引擎，这使得从设备发起的传输和到设备的传输也可以重叠执行。在之前的时间轴中这可能会导致（随着一些程序的变更）“复制到设备”传输发生的同时发生“复制回主机”传输。事实上，我们可以消除“复制回主机”的传输时间，这样可以节省大量的时间。由于 Tesla 设备上的两个 DMA 引擎都启用了，因此启用流可以为这个平台带来很多好处。

同样注意对于这个测试，我们使用了 PCI-E 2.0 X8 链路。如果使用 PCI-E 3.0 X16 链路可以将传输时间减少为现在的 1/4，这就使传输不再是一个严重的问题。

一旦你让内核工作了（而且仅在你让它工作时），就切换到异步操作模式。然而，异步操作会让调试工作变得更加复杂，因此最好只在所有部分都正确工作的情况下才这样做。

SDK 样例：Grabcut、Histogram Equalization、BoxFilter、Image Segmentation、Interoperability with the FreeImage library。

10.2.3 Thrust

熟悉 C++ 的人可能使用过 C++ STL，特别是 BOOST 函数库。模板实际上是 C++ 中非常有用的一个特性。在传统的 C 语言中，如果你有一个要执行某一操作的简单函数，例如求和操作，你需要明确地指定操作类型。因此，你可能需要下面这样的代码：

```
int sum(const int x, const int y)
{
  return x+y;
}
```

如果要调用这个函数并且传递一个浮点数，那么就需要一个新的函数。由于 C 语言不允许两个函数有相同的函数名，我们需要使用像 sum_i32、sum_u8、sum_u32、sum_f32 等这样的函数名。然而，提供或者使用类型特定的函数库是很枯燥的。

C++ 试图通过函数重载的方式来解决这个问题。这就使多个函数能够使用相同的名字。

根据传递参数的类型，系统会调用合适的函数。然而，即使现在可以使用相同的名字调用各个函数，函数库的提供者仍然需要编写函数体来解决 int8、int16、f32 等不同参数类型。

C++ 模板系统解决了这个问题。我们得到一个通用的函数并且只有参数的类型会发生改变。那么程序员就不需要多次复制、粘贴代码以支持所有他们想象得到的可能情况。模板（正如它的名字表明的）意味着以类型无关的方式为想要执行的操作提供一个模板。如果函数被调用了（并且只有在函数被调用时），编译器会在编译时生成相应版本的函数。

因此，STL 出现了，它是 C++ 中一些与常见函数的类型无关的定义。如果你碰巧使用的是 int32 版本而不是 float32 版本，那么只有 int32 版本会最终在你的应用程序中生成实际的代码。与函数库相反，模板的缺点是它会在每次编译运行时被编译，因此这就增加了编译时间。

NVCC 编译器实际上是 C++ 的前端而不是 C 的前端。和主要的 Linux 和 MAC 开发环境一样，Microsoft Visual Studio 作为 Windows 平台上的标准开发包，也支持 C++ 语言。

Thrust 函数库支持很多 STL 容器，因此这就使其能支持大规模的并行处理器。最简单的结构往往是最好的，因此数组（STL 所说的向量）在 Thrust 中被很好地支持。然而，并不是所有容器都适用，例如 list 容器就有未知的访问时间以及可变的长度。

在探究 Thrust 之前，我们需要提到的另一个 C++ 中的概念是 C++ 是命名空间。如果在 C 语言中声明了两个名字相同的函数，会在链接阶段遇到错误（如果它在编译时没有检测到的话）。在 C++ 中，假如两个相同函数名属于不同命名空间，是非常有效的做法。命名空间有点类似于为函数指定一个函数库前缀以使编译器辨别应该在哪个函数库中寻找函数。C++ 命名空间实际上是一个类选择器。从高层次来说，C++ 的类仅仅是一种将相关函数和数据聚集起来定义的一种方式。因此，假如我们使用了不同的命名空间，那么我们的同一个函数可以有不同调用。例如：

```
ClassNameA::my_func();
ClassNameB::my_func();
```

由于有了命名空间前缀，编译器能够识别出哪个函数将会调用。如果把类定义看作是既包含数据又有许多函数指针的结构体定义的话，那么 ::（两个冒号）就等价于→操作符。

最后，为了使用 Thrust 函数库我们还需要另一个 C++ 中的概念，那就是仿函数（functor）。仿函数可以认为是一个可以存储状态和数据的函数指针。它实际上是基于某个给定类定义的对象实例的指针。

如果曾经使用过诸如 qsort（Quicksort）这样标准 C 函数库中的函数的话，可能会对“用户提供的函数”这个概念比较熟悉。在 qsort 中，要提供一个比较函数指出一个不透明的数据对象与另一个不透明的数据对象相比是大还是小。可能会有某个特定的或多个标准对数据记录进行排序。qsort 函数库的提供者无法预想到太多种可能性，因此他们提供了一个正规的参数而你必须要将自己的对比函数传递给它。

Thrust 函数库提供了两个向量容器。STL 中的向量仅是能够改变大小的数组，它能够容纳任何类型的元素。Thrust 分别为主机和设备提供了向量类并且分别驻留在主机和设备的全局内存中。

向量可以使用数组下标（[and] 两个符号）进行读取或修改。然而，请注意如果向量在设备上，那么对于每个这样的访问，Thrust 通过 PCI-E 总线在后台执行单独的传输。因此，把这样一个结构放在循环里是一个很糟糕的主意。

使用 Thrust 的第一步是从 Thrust 向量获取需要的数据。因此，我们首先要包含必需的包含文件。Thrust 提供了以下大量的函数类型集合：

- 转换（transformation）
- 归约（reduction）
- 前缀求和（prefix sum）
- 再排序（reordering）
- 排序（sorting）

Thrust 并不是传统意义上的函数库，因为它的所有内容都在所包含的头文件中。因此，要避免包含所有的文件，只要包含需要的头文件就行了。

Thrust 提供了两种向量对象：host_vector 和 device_vector，通过 C++ 语法我们可以创建这些对象的实例。例如：

```
thrust::host_vector <float> my_host_float_vector(200);
thrust::device_vector <int> my_device_int_vector(500);
```

在这段代码中，我们声明了两个对象。一个物理上存在于主机，另一个存在于设备上。

thrust:: 这部分指定了类命名空间，基本上可以认为它像 C 语言中的函数库标识符一样。host_vector 和 device_vector 是对象提供的函数（构造函数）。<int> 和 <float> 标识符传递给构造函数（初始化函数）以构造对象。然后，构造函数使用它们并与传入函数的参数值一起分别分配 200 个大小为 sizeof（float）的元素和 500 个大小为 sizeof（int）的元素。在内部可能会有其他的数据结构，但是从高的层次来说，使用这样的构造器是很有效的。

C++ 中的对象也有一个析构函数，当对象超出域的范围时该函数会调用。该函数负责回收所有的构造函数分配的或对象在运行时使用的资源。因此，和 C 语言有所不同，对于 Thrust 向量，我们没有必要调用 free 或 cudaFree。

定义好一个向量之后现在需要对向量进行数据存取。Thrust 向量从概念上来说仅仅是一个可以改变大小的数组。因此，向量对象提供了 size() 函数，它返回向量中元素的个数。这样就可以使用标准循环结构了，例如：

```
for (int i=0; i < my_device_int_vector.size(); i++)
{
  int x = my_device_int_vector[i];
  ...
}
```

在这个例子中，数组 [ ] 操作符由类定义提供。因此，在每次迭代中会调用一个函数将 i 转换成物理上的数据。如果这份数据碰巧在设备上，Thrust 将会在后台产生一个从设备到主机的传输，这对程序员是完全透明的。size() 函数意味着只有在运行时我们才会知道迭代的次数。由于元素个数可能在循环体中改变，因此该函数必须在每次迭代中调用一次。同时，

这也防止了编译器静态地展开循环。

是否喜欢这样做取决于自己的想法。喜欢它的阵营中的人更喜欢抽象化因为它可以使编程变得更加简单并且不需要关注硬件。这个阵营主要由一些没有经验的程序员和那些仅仅想尽快完成计算的人组成。讨厌它的人想要弄清楚到底发生了什么并且强烈地想提升已有硬件的性能。他们不希望使用简单的数组解除引用来初始化一个非常低效的 4 字节的 PCI-E 传输。在进行调度时，他们更喜欢为数兆字节的主机和设备之间的传输，因为这样更加有效。

事实上，Thrust 同时满足了这两个阵营的人。大规模的传输可以简单地通过使用设备向量初始化主机向量或相反的方式实现。例如：

```
thrust::host_vector <float> my_host_float_out_vector
(my_device_float_results_vector.begin(), my_device_float_results_vector.end() );
```

或

```
thrust::copy(my_host_float_out_vector.begin(), my_host_float_out_vector.end(),
my_device_float_results_vector.begin(), my_device_float_results_vector.begin() );
```

在第一个例子中，我们在主机端创建了一个新的向量并且用设备向量初始化了该主机向量。请注意，我们并没有像之前创建主机向量时那样只在构造函数中指定一个参数值，这里我们指定了两个值。在这种情况下，Thrust 通过减法计算出需要复制的元素个数并且相应地在主机上分配存储空间。

在第二个例子中，我们使用了显式复制的方法。这种方法（函数）需要 3 个参数：目标区域的起点和结尾以及源区域的起点。由于 Thrust 知道使用的向量是什么类型的，因此该复制方法对主机和设备都适用。我们没有必要指定额外的参数，比如 cudaMemcpyDeviceToHost 或 cudaMemcpyHostToDevice 或根据传递的值的类型调用不同的函数。Thrust 仅通过使用 C++ 模板重载命名空间，从而可以根据传递的参数调用一些函数。由于这些是在编译时完成的，因此从强类型检查中受益并且没有运行时的开销。模板是 C++ 与 C 相比的主要好处之一。

1. 使用 Thrust 提供的函数

一旦数据在 Thrust 设备向量或主机向量容器中，我们就可以使用大量 Thrust 提供的标准函数。Thrust 提供了一个简单的排序函数，该函数只需要提供向量开头和结尾的索引。它把任务分配到不同的线程块上并且执行任何归约和线程块间的通信操作。经常会有 CUDA 新手在这样的代码中犯错误。有了类似使得排序函数的标准函数，利用 GPU 就像使用常见的 C 语言 qsort 函数库例程一样简单。

```
thrust::sort(device_array.begin(), device_array.end());
```

我们可以从一个小的程序中看到这一点确实存在。

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/generate.h>
#include <thrust/sort.h>
#include <thrust/copy.h>
```

```
#include <cstdlib>

// 1M Elements = 4MB Data
#define NUM_ELEM (1024*1024)

int main(void)
{
 // Declare an array on the host
 printf("\nAllocating memory on host");
 thrust::host_vector<int> host_array(NUM_ELEM);

 // Populate this array with random numbers
 printf("\nGenerating random numbers on host");
 thrust::generate(host_array.begin(), host_array.end(), rand);
 // Create a device array and populate it with the host values
 // A PCI-E transfer to device happens here
 printf("\nTransferring to device");
 thrust::device_vector<int> device_array = host_array;

 // Sort the array on the device
 printf("\nSorting on device");
 thrust::sort(device_array.begin(), device_array.end());

 // Sort the array on the host
 printf("\nSorting on host");
 thrust::sort(host_array.begin(), host_array.end());

 // Create a host array and populate it with the sorted device values
 // A PCI-E transfer from the device happens here
 printf("\nTransfering back to host");
 thrust::host_vector<int> host_array_sorted = device_array;

 printf("\nSorting Complete");

 return 0;
}
```

2. 关于 Thrust 的问题

有趣的是，基于 GPU 和 CPU 的排序可能在相同的时间执行也可能在不同的时间执行，这取决于如何安排传输。不幸的是，Thrust 总是使用默认的流并且无法像 NPP 函数库那样改变它。因为没有流参数可以传递，也没有函数可以设置当前选择的流。

使用默认的流会产生一些严重的问题。排序操作实际上只是一系列运行在 0 号流上的内核。这些内核（和其他内核一样）异步地启动。然而，内存传输操作（除非明确的异步完成）却是同步的。因此，任何从 Thrust 调用的函数会返回一个值（例如 reduce），任何复制回主机的操作都会引发一个隐式的同步。在样例代码中，在从设备复制回的代码后加入主机数组排序调用，这样就会使 GPU 和 CPU 排序串行化。

3. 多 CPU/GPU 的思考

在 CPU 端，Thrust 自动地产生 N 个线程，N 为物理处理器核的个数。实际上，Thrust 在 CPU 端使用 OpenMP，并且 OpenMP 在默认情况下会使用 CPU 上物理处理器核的个数。

如果 GPU 版本也像这样将一个任务划分到 N 个 GPU 上工作就好了。主机端实现了一个基于 NUMA（非统一内存访问）的内存系统。这意味着任何 CPU 套接字和 CPU 核可以访问所有的内存地址。因此，即使在双 CPU 系统中，8、12 或 16 个 CPU 核可以针对某一问题并行地工作。

多 GPU 更像是一群连接到 PCI-E 总线上的分布式内存机器。如果有正确的硬件和操作系统，GPU 可以使用对等（p2p）功能通过总线直接和其他 GPU 交流。多 GPU 共同工作以完成排序操作是有些复杂的。

然而，像常规的多 GPU 编程一样，Thrust 支持单线程 / 多 GPU 和多线程 / 多 GPU 模型。它并不会隐式地利用多 GPU，而是让程序员选择创建多线程或是在合适的地方使用 cudaSetDevice 调用以选择可以正确工作的设备。

使用多处理器进行排序有两种基本方法。第一种是使用归并排序。在这种方法中，数据划分成多个大小相同的块，每一块单独排序，最后再采用归并操作。第二种是使用像我们之前在样例程序中看到的算法，对数据块进行部分排序或是预排序。然后，排好序的数据块，可以单独地排序或者是简单地将它们组合在一起以形成排好序的输出结果。

由于内存访问时间比比较时间要多很多，因此传递最少数据（无论是读还是写）的算法往往是最快的。产生竞争的操作（例如归并）和那些能够在整个过程中都维持高并行性的算法相比最终会限制算法的可扩展性。

对于基本类型（u8、u16、u32、s8、s16、s32、f32、f64），Thrust 使用一种非常快速的基数排序，在之前的样例中我们已经看到了。对于其他类型和用户自定义的类型 Thrust 使用归并排序。Thrust 根据数据类型和范围，会自动调整基数排序使用的位数。因此，一个最大数据范围仅为 256 的 32 位排序，比使用全部数据范围的排序快很多。

4. 排序时间

为了看到程序中的时间，我们在 Thrust 的样例程序中加入定时器并且看看获取的数据。

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/generate.h>
#include <thrust/sort.h>
#include <thrust/copy.h>
#include <cstdlib>

#include "cuda_helper.h"
#include "timer.h"

void display_gpu_name(void)
{
 int device_num;
 struct cudaDeviceProp prop;
```

```
 CUDA_CALL(cudaGetDevice(&device_num));

 // Get the device name
 CUDA_CALL( cudaGetDeviceProperties( &prop, device_num ) );
 // Print device name and logical to physical mapping
 printf("\n\nUsing CUDA Device %u. Device ID: %s on PCI-E %d",
  device_num, prop.name, prop.pciBusID);
}

// 4M Elements = 16MB Data
#define NUM_ELEM (1024*1024*4)

int main(void)
{
 int num_devices;
 CUDA_CALL(cudaGetDeviceCount(&num_devices));
 for (int device_num = 0; device_num < num_devices; device_num++)
 {
  CUDA_CALL(cudaSetDevice(device_num));
  display_gpu_name();

  const size_t size_in_bytes = NUM_ELEM * sizeof(int);
  printf("\nSorting %lu data items (%lu MB)", NUM_ELEM, (size_in_bytes/1024/1024));

  // Allocate timer events to track time
  float c2d_t, sort_d_t, sort_h_t, c2h_t;
  cudaEvent_t c2d_start, c2d_stop;
  cudaEvent_t sort_d_start, sort_d_stop;
  cudaEvent_t c2h_start, c2h_stop;

  CUDA_CALL(cudaEventCreate(&c2d_start));
  CUDA_CALL(cudaEventCreate(&c2d_stop));
  CUDA_CALL(cudaEventCreate(&sort_d_start));
  CUDA_CALL(cudaEventCreate(&sort_d_stop));
  CUDA_CALL(cudaEventCreate(&c2h_start));
  CUDA_CALL(cudaEventCreate(&c2h_stop));

  // Declare an array on the host
  printf("\nAllocating memory on host");
  thrust::host_vector<int> host_array(NUM_ELEM);

  // Populate this array with random numbers
  printf("\nGenerating random numbers on host");
  thrust::generate(host_array.begin(), host_array.end(), rand);

  // Create a device array and populate it with the host values
  // A PCI-E transfer to device happens here
  printf("\nTransferring to device");
  CUDA_CALL(cudaEventRecord(c2d_start, 0));
  thrust::device_vector<int> device_array = host_array;
```

```
    CUDA_CALL(cudaEventRecord(c2d_stop, 0));

    // Sort the array on the device
    printf("\nSorting on device");
    CUDA_CALL(cudaEventRecord(sort_d_start, 0));
    thrust::sort(device_array.begin(), device_array.end());
    CUDA_CALL(cudaEventRecord(sort_d_stop, 0));
    CUDA_CALL(cudaEventSynchronize(sort_d_stop));

    // Sort the array on the host
    printf("\nSorting on host");
    sort_h_t = get_time();
    thrust::sort(host_array.begin(), host_array.end());
    sort_h_t = (get_time() - sort_h_t);

    // Create a host array and populate it with the sorted device values
    // A PCI-E transfer from the device happens here
    printf("\nTransfering back to host");
    CUDA_CALL(cudaEventRecord(c2h_start, 0));
    thrust::host_vector<int> host_array_sorted = device_array;
    CUDA_CALL(cudaEventRecord(c2h_stop, 0));

    // Wait for last event to be recorded
    CUDA_CALL(cudaEventSynchronize(c2h_stop));

    printf("\nSorting Complete");

    // Calculate time for each aspect
    CUDA_CALL(cudaEventElapsedTime(&c2d_t, c2d_start, c2d_stop));
    CUDA_CALL(cudaEventElapsedTime(&sort_d_t, sort_d_start, sort_d_stop));
    CUDA_CALL(cudaEventElapsedTime(&c2h_t, c2h_start, c2h_stop));

    printf("\nCopy To Device : %.2fms", c2d_t);
    printf("\nSort On Device : %.2fms", sort_d_t);
    printf("\nCopy From Device : %.2fms", c2h_t);
    printf("\nTotal Device Time: %.2fms", c2d_t + sort_d_t + c2h_t);
    printf("\n\nSort On Host : %.2fms", sort_h_t);

    CUDA_CALL(cudaEventDestroy(c2d_start));
    CUDA_CALL(cudaEventDestroy(c2d_stop));
    CUDA_CALL(cudaEventDestroy(sort_d_start));
    CUDA_CALL(cudaEventDestroy(sort_d_stop));
    CUDA_CALL(cudaEventDestroy(c2h_start));
    CUDA_CALL(cudaEventDestroy(c2h_stop));
  }
  return 0;
}
```

由于 Thrust 对所有的调用使用默认的流，因此，为了对设备代码进行计时，我们简单地

加入一些事件并且获取不同事件间的时间差。然而请注意，我们需要在排序后和最后一个事件完成后对流进行同步。cudaEventRecord 函数（即使设备现在没有做任何事情）会立刻返回并且不会设置事件。因此，在设备排序后遗漏了同步调用会大量增加实际报告的时间。

我们从 4 个不同设备上获取的对 16MB 随机数据进行排序的时间如表 10-1 和图 10-2 所示。正如在表格中看到的，当我们按照时间轴负方向看各代的 GPU 时，发现执行速度会线性地减少。从计算能力 1.1 的 9800GT 之后，我们看到了执行时间上巨大的飞跃。9800GT 的内存带宽比 GTX 260 的一半还要小并且处理能力最多是它的 2/3。

表 10-1 Thrust 在不同设备上的排序时间

Device	Time	Device	Time
GTX470	67.45	GTX260	109.02
GTX460	85.18	9800GT	234.83

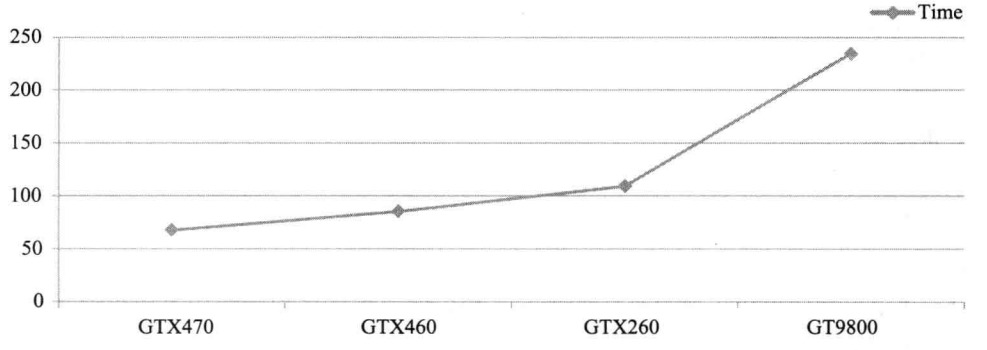

图 10-2 不同设备的 Thrust 排序时间（16MB 数据）

通过比较，2.5GHz AMD Phenom II X4 的主机时间是很糟糕的。它的排序时间（大约 2400ms）甚至比 9800GT 还要慢十多倍。然而，这个比较真的公平吗？这取决于 Thrust 在 CPU 上实现的排序效率有多高，以及使用的特定的 CPU 和主机内存带宽。Parallel Nsight 和任务管理器都表明在我们的测试系统中 Thrust 没有占用超过 25% 的 CPU 负载。这表明它远远没有充分地利用 CPU 资源。因此，这样的对比，对于 CPU 是不公平的并且会人为地拉高 GPU 的性能参数。

```
Using CUDA Device 0. Device ID: GeForce GTX 470 on PCI-E 8
Sorting 4194304 data items (16 MB)
Allocating memory on host
Generating random numbers on host
Transferring to device
Sorting on device
Sorting on host
Transfering back to host
Extracting data from Thrust vector
Sorted arrays Match
```

```
Running single core qsort comparison
Sorting Complete
Copy To Device : 10.00ms
Sort On Device : 58.55ms
Copy From Device : 12.17ms
Total Device Time : 80.73ms


Thrust Sort On Host: 2398.00ms
QSort On Host : 949.00ms
```

正如看到的一样，单核的qsort的性能能够轻易地超过CPU端的Thrust排序，并且使用了接近100%的单核负载。如果我们假设并行版本有着和之前OpenMP归约相似的加速比（通常的CPU的数据是这里显示的一半），比如说475ms。即使这样，基于GPU的Thrust排序的速度仍然超过了CPU，大约是它的6倍（即使算上PCI-E总线上的传输）。

Thrust也有许多其他有用的函数：

- 二分查找
- 归约
- 归并
- 重排序
- 前缀求和
- 集合操作
- 变换

Thrust用户指南提供了所有这些函数的文档说明。这些函数的使用方法类似于我们这里使用的排序的例子。

很显然，我们可以写出很多关于Thrust的内容，但是这一章总体上是关于函数库的，因此，我们只会再介绍一个例子，这就是归约。

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/sequence.h>
#include <thrust/sort.h>
#include <thrust/copy.h>
#include <cstdlib>
#include "cuda_helper.h"
#include "timer.h"

void display_gpu_name(void)
{
 int device_num;
 struct cudaDeviceProp prop;

 CUDA_CALL(cudaGetDevice(&device_num));

 // Get the device name
```

```
 CUDA_CALL( cudaGetDeviceProperties( &prop, device_num ) );

 // Print device name and logical to physical mapping
 printf("\n\nUsing CUDA Device %u. Device ID: %s on PCI-E %d",
  device_num, prop.name, prop.pciBusID);
}

long int reduce_serial(const int * __restrict__ const host_raw_ptr,
                       const int num_elements)
{
 long int sum = 0;

 for (int i=0; i < num_elements; i++)
  sum += host_raw_ptr[i];

 return sum;
}

long int reduce_openmp(const int * __restrict__ const host_raw_ptr,
            const int num_elements)
{
 long int sum = 0;

#pragma omp parallel for reduction(+:sum) num_threads(4)
 for (int i=0; i < num_elements; i++)
  sum += host_raw_ptr[i];

 return sum;
}

// 1M Elements = 4MB Data
#define NUM_ELEM_START (1024*1024)
#define NUM_ELEM_END (1024*1024*256)
int main(void)
{
 int num_devices;
 CUDA_CALL(cudaGetDeviceCount(&num_devices));

 for (unsigned long num_elem = NUM_ELEM_START; num_elem < NUM_ELEM_END; num_elem *=2)
 {
  const size_t size_in_bytes = num_elem * sizeof(int);

  for (int device_num = 0; device_num < num_devices; device_num++)
  {
   CUDA_CALL(cudaSetDevice(device_num));
   display_gpu_name();

   printf("\nReducing %lu data items (%lu MB)", num_elem, (size_in_bytes/1024/1024));
   // Allocate timer events to track time
```

```
    float c2d_t, reduce_d_t, reduce_h_t, reduce_h_mp_t, reduce_h_serial_t;
    cudaEvent_t c2d_start, c2d_stop;
    cudaEvent_t sort_d_start, sort_d_stop;

    CUDA_CALL(cudaEventCreate(&c2d_start));
    CUDA_CALL(cudaEventCreate(&c2d_stop));
    CUDA_CALL(cudaEventCreate(&sort_d_start));
    CUDA_CALL(cudaEventCreate(&sort_d_stop));
    // Declare an array on the host
    thrust::host_vector<int> host_array(num_elem);

    // Populate this array with random numbers
    thrust::sequence(host_array.begin(), host_array.end());

    // Create a device array and populate it with the host values
    // A PCI-E transfer to device happens here
    CUDA_CALL(cudaEventRecord(c2d_start, 0));
    thrust::device_vector<int> device_array = host_array;
    CUDA_CALL(cudaEventRecord(c2d_stop, 0));

    // Sort the array on the device
    CUDA_CALL(cudaEventRecord(sort_d_start, 0));
    const long int sum_device = thrust::reduce(device_array.begin(),
device_array.end());
    CUDA_CALL(cudaEventRecord(sort_d_stop, 0));
    CUDA_CALL(cudaEventSynchronize(sort_d_stop));

    // Sort the array on the host
    reduce_h_t = get_time();
    const long int sum_host = thrust::reduce(host_array.begin(), host_array.end());
    reduce_h_t = (get_time() - reduce_h_t);
   // Allocate host memory
   int * const host_raw_ptr_2 = (int *) malloc(size_in_bytes);
   int *p2 = host_raw_ptr_2;

   if ( (host_raw_ptr_2 == NULL) )
   {
    printf("\nError allocating host memory for extraction of thrust data");
    exit(0);
   }

   // Extract data from Thrust vector to normal memory block
   for (int i=0; i<num_elem; i++)
   {
    *p2++ = host_array[i];
   }

   reduce_h_mp_t = get_time();
   const long int sum_host_openmp = reduce_openmp(host_raw_ptr_2, num_elem);
   reduce_h_mp_t = (get_time() - reduce_h_mp_t);

   reduce_h_serial_t = get_time();
```

```
    const long int sum_host_serial = reduce_serial(host_raw_ptr_2, num_elem);
    reduce_h_serial_t = (get_time() - reduce_h_serial_t);

    // Free memory
    free(host_raw_ptr_2);

    if ( (sum_host == sum_device) && (sum_host == sum_host_openmp) )
     printf("\nReduction Matched");
    else
     printf("\n**** FAILED ****");

    // Calculate time for each aspect
    CUDA_CALL(cudaEventElapsedTime(&c2d_t, c2d_start, c2d_stop));
    CUDA_CALL(cudaEventElapsedTime(&reduce_d_t, sort_d_start, sort_d_stop));

    printf("\nCopy To Device : %.2fms", c2d_t);
    printf("\nReduce On Device : %.2fms", reduce_d_t);
    printf("\nTotal Device Time : %.2fms", c2d_t + reduce_d_t);
    printf("\n\nThrust Reduce On Host: %.2fms", reduce_h_t);
    printf("\nSerial Reduce On Host: %.2fms", reduce_h_serial_t);
    printf("\nOpenMP Reduce On Host: %.2fms", reduce_h_mp_t);

    CUDA_CALL(cudaEventDestroy(c2d_start));
    CUDA_CALL(cudaEventDestroy(c2d_stop));
    CUDA_CALL(cudaEventDestroy(sort_d_start));
    CUDA_CALL(cudaEventDestroy(sort_d_stop));
   }
  }

  return 0;
  }
```

归约是一个非常有趣的问题（像我们之前见到的一样），考虑到数据传输到 GPU 的时间，我们很难编写出一个比 OpenMP 更快的版本。我们以绝对优势获胜，因为只有一个明显的到设备的传输，但是它却决定了整体的时间。然而，如果数据已经在 GPU 上了，那么这个传输就和归约步骤无关了。

接下来我们使用大规模的数据，对基于 Thrust 的归约操作进行计时（同时在主机和设备上）并和标准单核的串行归约操作、基于 OpenMP 的 4 核归约操作以及 4 个 GPU 测试设备进行对比，结果如表 10-2 和图 10-3 所示。表中缺少了 CPU 上的 Thrust 归约时间这一项，这是因为它要比其他的时间多很多。对于所有大小的线程块我们都得到了一致的结果：CPU Thrust 排序花费的时间大约是单核串行版本执行时间的 10 倍。

表 10-2 不同数据规模以及不同 GPU 上的归约操作时间（ms）

Device	4MB	8MB	16MB	32MB	64MB	128MB	256MB	512MB
GTX470	1.21	2.14	4.12	7.71	15.29	30.17	59.87	119.56
GTX460	1.7	3.34	6.26	12.13	23.9	47.33	97.09	188.39
GTX260	2.38	1.85	3.53	6.8	13.46	26.64	55.02	106.28
9800 GT	1.75	3.24	6.4	12.59	26.18	51.39	110.82	202.98
Copy	2.97	6.51	9.34	17.69	38.15	68.55	134.96	278.63
Serial	6	11	22	43	91	174	348	696
OpenMP	1	4	5	9	33	37	91	164

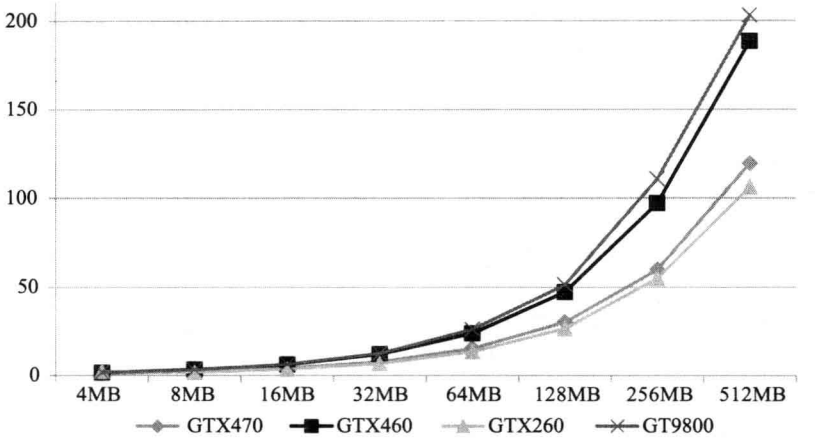

图 10-3 随数据规模变化的 GPU 归约时间（ms）

请注意图 10-3（和表 10-2），y 轴是以毫秒为单位的时间。因此，该值越小越好。令人惊讶的是，GTX260（获得了最好的归约时间）略微超过了之后版本的 GTX470。

这个结果很不错，但是与 CPU 版本相比如何呢？所有的显卡都比单核串行实现要快，加速大约为 3.5 ~ 7.0 倍。GTX470 和 GTX260 显卡和 OpenMP 版本相比，执行得也不错，其时间大约为并行 CPU 版本的 2/3。GTX460 和 CPU 版本接近，9800GT 相对更慢一些。然而，如果我们考虑到 PCI-E 2.0 x8 的传输时间，对于 512MB 数据是 278ms，即使是 GTX260 的 106ms，再加上 278ms 也要比 OpenMP 版本的 164ms 要慢。

对于使用 CUDA 实现的归约操作或支持流的 Thrust 异步版本，可以减少内核时间，因为我们使连续的传输和内核操作重叠执行。当然，这是假设我们有多个归约操作要执行或者将一个单独的归约操作分解成一系列小的归约操作。即使这样，预计的最好情况是 178ms，这仍然要比 OpenMP 版本慢。很明确的是，我们要在适当的地方利用 OpenMP 和 CPU。如果数据已经在 GPU 上了，那么就在 GPU 上执行归约操作。否则，使用 CPU 会更加有效，如图 10-4 所示。

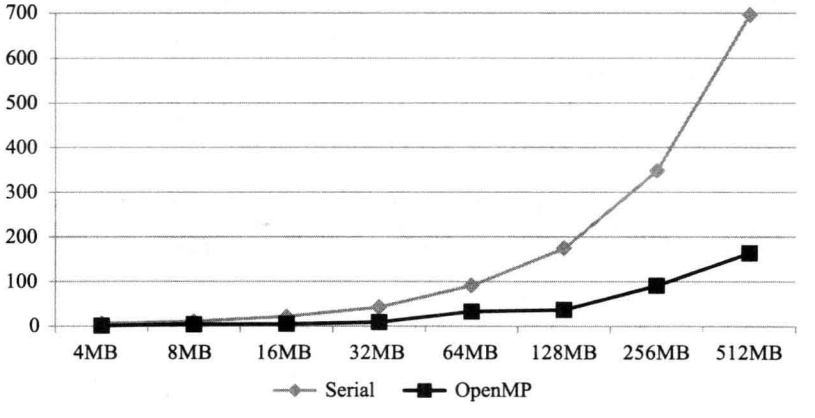

图 10-4 OpenMP 和串行归约时间（ms）

5. 使用 Thrust 和 CUDA

因此，我们希望在常规的 CUDA 代码中使用 Thrust 的这些特性。可以完全使用 Thrust 提供的操作编写自己的应用程序。然而，函数库永远不会包含所有东西，对于某些事情它们可以完全胜任，然而对于其他的却不是这样。因此，我们不希望仅仅为了利用函数库而强制使用某种特定的思考方式。

Thrust 并不提供用简单的方式从 host_vector 结构复制数据的机制。它仅仅提供了读取单独元素的方法。因此，一个复制操作一次只能执行一个元素，这是非常慢的。然而，使用 device_vectors，我们可以有另一种方法。

首先，需要自己在设备上分配存储空间，这样就会得到一个指向数据的指针而不是 Thrust 迭代器。然后需要将常规的设备指针转换成 Thrust 设备指针。我们使用 device_ptr 构造函数完成这一步骤。然后可能将该 Thrust 设备指针传给不同的 Thrust 函数。现在 Thrust 工作在你提供的基本数据上，因此它对你是可见的而不是隐藏在 Thrust 函数库中的。

我们可以改写排序程序以充分利用这一点。

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/sort.h>
#include <cstdlib>

#include "cuda_helper.h"
#include "timer.h"
#include "common_types.h"

void display_gpu_name(void)
{
 int device_num;
 struct cudaDeviceProp prop;

 CUDA_CALL(cudaGetDevice(&device_num));

 // Get the device name
 CUDA_CALL( cudaGetDeviceProperties( &prop, device_num ) );

 // Print device name and logical to physical mapping
 printf("\n\nUsing CUDA Device %u. Device ID: %s on PCI-E %d",
  device_num, prop.name, prop.pciBusID);
}

__global__ void fill_memory(int * const __restrict__ data,
            const int num_elements)
{
 const int tid = (blockIdx.x * blockDim.x) + threadIdx.x;
 if (tid < num_elements)
  data[tid] = (num_elements - tid);
}

// 4M Elements = 16MB Data
```

```
#define NUM_ELEM (1024*1024*4)

int main(void)
{
 const size_t size_in_bytes = NUM_ELEM * sizeof(int);
 display_gpu_name();

 printf("\nSorting %lu data items (%lu MB)", NUM_ELEM, (size_in_bytes/1024/1024));
 // Declare an array on the device
 printf("\nAllocating memory on device");
 int * device_mem_ptr;
 CUDA_CALL(cudaMalloc((void **) &device_mem_ptr, size_in_bytes));

 const u32 num_threads = 256;
 const u32 num_blocks = (NUM_ELEM + (num_threads-1)) / num_threads;

 printf("\nFilling memory with pattern");
 fill_memory<<<num_threads, num_blocks>>>(device_mem_ptr, NUM_ELEM);

 // Convert the array to
 printf("\nConverting regular device pointer to thrust device pointer");
 thrust::device_ptr<int> thrust_dev_ptr(device_mem_ptr);

 // Sort the array on the device
 printf("\nSorting on device");
 thrust::sort(thrust_dev_ptr, thrust_dev_ptr + NUM_ELEM);

 printf("\nFreeing memory on device");
 CUDA_CALL(cudaFree(device_mem_ptr));

 return 0;
}
```

请注意 thrust::device_ptr 这个构造函数，它创建了 thrust_dev_ptr 这个对象并且之后能够传递给 thrust::sort 函数。和传统的迭代不同的是，Thrust 设备指针并没有“begin”和“end”函数，因此我们只能使用基地址加上长度来获得排序的最后一个元素。

这允许在简单的设备内核上实现主机初始化的 Thrust 调用。然而，要注意对于（截至 SDK 4.2）Thrust 并没有设备级的接口，因此无法在设备或全局函数中调用 Thrust 函数。诸如 sort 这样的函数，会自己产生多个内核。由于 GPU 自己无法产生其他的工作（至少在 Kepler K20 发布前），因此我们限制在了基于主机的控制。

SDK 样例程序：Line of Sight、Radix Sort、Particles、Marching Cubes、Smoke Particles。

10.2.4 CuRAND

CuRAND 函数库提供了 GPU 端不同类型的随机数字的生成。在 C 语言中，可能习惯在主机端调用标准库函数 rand()。和其他的标准库函数一样，rand() 无法在设备代码中调用。因此，唯一的选择是在主机端创建一组随机数字，然后将它们复制到设备端。但这会导致很多问题：

- 增加主机启动时间
- 增加 PCI-E 带宽
- 实际上，数据集的随机分布并不好

标准库函数 rand() 并不是设计来产生真正随机的数据的。它（像很多随机数生成算法一样）通过创建一组伪随机数字并且每次简单地从列表中选择下一个元素，从而实现随机数生成。因此，任何知道生成随机数所使用的种子的人都可以准确地预测出给定序列中的下一个随机数。

这往往会带来一些后果，尤其是在安全领域。很多算法利用随机性（以某种方式），使模仿对等方变得很困难。假如两个对等方交换随机数生成器的种子。对等方 A 将一个随机数解码成消息帧。使用相同的种子和相同的随机数生成器的对等方 B 知道要从对等方 A 收到哪一个数据标识符。假如捕获到了对等方 A 和 B 之间的标识符序列，那么攻击方（C）就有可能计算出下一个数字并且假装是对等方 A 或 B。

这是很有可能的，因为随机数字通常只是小规模的、重复的伪随机数字序列。如果随机数字集合很小，那么就很容易进行攻击。这个种子要么不是程序员设置的（设置成某个“秘密”数字），要么是基于当前时间设置的。启动时间并不是非常随机的，因此基于时间的种子事实上是在很小的窗口范围内。秘密很少能够保持神秘。

另外一个例子是密码生成。如果要在系统中创建几百个用户，那么他们经常会提交“随机”的密码，这些密码是第一次登录时填写的。这些密码可能是很长的字符串，并且让人产生一种很安全的感觉。然而，如果它们实际上是从使用小规模的伪随机数字集合的随机数生成器中选择出来的，那么实际暴力攻击的搜索空间是非常小的。

因此，在任何对序列进行预测而产生问题的情况中，我们需要的随机数要比大多数 rand() 标准库提供的更好。

为了使用 CuRAND 函数库，需要包含下列头文件：

```
#include <curand_kernel.h>
```

此外，需要确保链接了下列函数库：

```
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v4.1\lib\x64\curand.lib
```

很显然，要用当前使用的 CUDA 工具包的版本替代以上路径。接下来，让我们看看生成随机数的例子：

```
#include <stdio.h>
#include <stdlib.h>
#include <curand_kernel.h>
#include "cuda_helper.h"
#include "cuda.h"

#define CURAND_CALL(x){const curandStatus_t a = (x);if (a != CURAND_STATUS_SUCCESS){
printf("\nCuRand Error: (err_num=%d) \n", a); cudaDeviceReset(); ASSERT(0);} }
__host__ void print_array(const float * __restrict__ const data, const int num_elem)
{
 for (int i=0; i<num_elem; i++)
```

```
 {
  if ( i% 4 == 0)
   printf("\n");
  printf("%2d: %f ", i, data[i]);
 }
}

__host__ int main(int argc, char *argv[])
{
 const int num_elem = 32;
 const size_t size_in_bytes = (num_elem * sizeof(float));

 curandGenerator_t rand_generator_device, rand_generator_host;
 const unsigned long int seed = 987654321;
 const curandRngType_t generator_type = CURAND_RNG_PSEUDO_DEFAULT;

 // Allocate memory on the device
 float * device_ptr;
 CUDA_CALL( cudaMalloc( (void **) &device_ptr, size_in_bytes ));

 // Allocate memory on the host for the device copy
 float * host_ptr;
 CUDA_CALL( cudaMallocHost( (void **) &host_ptr, size_in_bytes ));

 // Allocate memory on the host for the host version
 float * host_gen_ptr = (float *) malloc(size_in_bytes);
 if (host_gen_ptr == NULL)
 {
  printf("\nFailed to allocation memory on host");
  exit(0);
 }

 // Print library version number
 int version;
 CURAND_CALL(curandGetVersion(&version));
 printf("\nUsing CuRand Version: %d and generator: CURAND_RNG_PSEUDO_DEFAULT",
version);

 // Register the generator - note the different function calls
 CURAND_CALL(curandCreateGenerator(&rand_generator_device, generator_type));
 CURAND_CALL(curandCreateGeneratorHost(&rand_generator_host, generator_type));

 // Set the seed for the random number generators
 CURAND_CALL(curandSetPseudoRandomGeneratorSeed(rand_generator_device, seed));
 CURAND_CALL(curandSetPseudoRandomGeneratorSeed(rand_generator_host, seed));

 // Create a set of random numbers on the device and host
 CURAND_CALL(curandGenerateUniform(rand_generator_device, device_ptr, num_elem));
 CURAND_CALL(curandGenerateUniform(rand_generator_host, host_gen_ptr, num_elem));

 // Copy the set of device generated data to the host
 CUDA_CALL(cudaMemcpy(host_ptr, device_ptr, size_in_bytes, cudaMemcpyDeviceToHost));
```

```
    printf("\n\nRandom numbers from GPU");
    print_array(host_ptr, num_elem);

    printf("\n\nRandom numbers from Host");
    print_array(host_gen_ptr, num_elem);
    printf("\n");

    // Free device resources
    CURAND_CALL(curandDestroyGenerator(rand_generator_device));
    CUDA_CALL(cudaFree(device_ptr));
    CUDA_CALL(cudaFreeHost(host_ptr));
    CUDA_CALL(cudaDeviceReset());

    // Free host resources
    CURAND_CALL(curandDestroyGenerator(rand_generator_host));
    free(host_gen_ptr);
}
```

这个程序使用 CuRand API 在设备端和主机端生成 num_elem 个随机数。然后，它会打印这两组随机数。输出结果如下：

```
Using CuRand Version: 4010 and generator: CURAND_RNG_PSEUDO_DEFAULT

Random numbers from GPU
 0: 0.468090  1: 0.660579  2: 0.351722  3: 0.891716
 4: 0.624544  5: 0.861485  6: 0.662096  7: 0.007847
 8: 0.179364  9: 0.260115 10: 0.453508 11: 0.711956
12: 0.973453 13: 0.152303 14: 0.784318 15: 0.948965
16: 0.214159 17: 0.236516 18: 0.020540 19: 0.175973
20: 0.085989 21: 0.863053 22: 0.908001 23: 0.539129
24: 0.849580 25: 0.496193 26: 0.588651 27: 0.361609
28: 0.025815 29: 0.778294 30: 0.194206 31: 0.478006

Random numbers from Host
 0: 0.468090  1: 0.660579  2: 0.351722  3: 0.891716
 4: 0.624544  5: 0.861485  6: 0.662096  7: 0.007847
 8: 0.179364  9: 0.260115 10: 0.453508 11: 0.711956
12: 0.973453 13: 0.152303 14: 0.784318 15: 0.948965
16: 0.214159 17: 0.236516 18: 0.020540 19: 0.175973
20: 0.085989 21: 0.863053 22: 0.908001 23: 0.539129
24: 0.849580 25: 0.496193 26: 0.588651 27: 0.361609
28: 0.025815 29: 0.778294 30: 0.194206 31: 0.478006
```

在样例程序中，我们需要注意一个非常重要的问题：除了初始化随机数生成器外，API 调用对于设备函数和主机函数是相同的。设备版本必须使用 curandCreateGenerator，主机版本必须使用 curandCreateGeneratorHost。此外，请注意，curandGenerateUniform 函数必须使用相应的基于主机或设备的指针才能调用。如果将两者混合使用则很有可能导致 CUDA “未知错误”或是程序崩溃。不幸的是，由于主机端和设备端内存分配为常规的 C 指针，因此函数库无法告知我们传入的指针是设备端的还是主机端的。

我们同样要注意 CuRand(像 NPP 一样）支持多流。因此，对 curandSetStream(generator，stream）的调用会将函数库切换到流中的一个异步操作。默认情况下，函数库会使用 0 号流（默认流）。

使用 CuRand 库，可以选择很多类型的随机数生成器，包括一种使用 Mersenne Twister 算法进行 Monte Carlo 模拟的方法。

SDK 样例程序：Monte Carlo、Random Fog、Mersenne Twister、Sobol。

10.2.5 CuBLAS 库

我们要提到的最后一个函数库是 CuBLAS（CUDA 基本线性代数库）。CuBLAS 库是为了移植 Fortran 科学应用程序中普遍使用的 Fortran BLAS 库。为了允许简单地移植已存在的 Fortran BLAS 代码，CuBLAS 库维护了一个按列排列的布局，这与标准的 C 语言的按行排列的布局相反。同样，当访问数组元素时，它使用 1 ~ N 作为下标而不是 C 语言标准中的 0 ~ (N-1)。

因此，如果想将 Fortran 中遗留的代码移植到 CUDA 中，使用 CuBLAS 库是很理想的。在过去的 10 年中，有很多代码库是使用 Fortran 编写的。这个库的最大好处之一是能够让这些遗留的代码在现代 GPU 加速的硬件上运行，而无须大量的代码修改。然而，这也是它的缺陷，因为它并不会吸引那些学习使用现代计算编程语言的人。

CuBLAS 库文档提供了一些样例宏可以将旧式的 Fortran 数组索引转化成大多数程序员认为的“常规”数组索引。然而，即使用宏或是内联函数实现了，对于那些试图使用“非 Fortran”索引的人来说，这会增加执行时间开销。这样来看使用该函数库对 C 程序员来说成为了一种痛苦。C 程序员更希望看到一种本身支持 C 数组索引的、独立的 C 风格的 CuBLAS 实现。

该函数库的第 4 个版本抛弃了以前的 API。现在它需要所有的调用者通过在其他任何调用之前调用 cublasCreate 函数创建一个句柄。该句柄用于随后的调用中，并且允许 CuBLAS 可重入且使用多异步流支持多 GPU 从而达到最佳性能。注意，尽管提供了这么多特性，程序员仍然要自己处理多设备。就像很多其他函数库提供的一样，CuBLAS 库不会自动地将负载分配到多 GPU 设备上。

通过包含 cublas_v2.h 文件而不是以前的 cublas.h 文件可以使用最新的 API。我们应该使用最新的 API 替代以前的 API。像 NPP 库一样，我们更希望对已经存在于 GPU 上的数据进行操作，因此调用者需要负责数据在设备间的传输。许多“ helper”函数提供了这些方面的帮助。

新的 CuBLAS 接口本身是完全异步的，这意味着即使函数有返回值也要把该返回值当作是不可用的，除非程序员特意等待异步 GPU 操作完成。这种向异步流的转移在 Kepler K20 发布后变得十分重要。

现在我们看一个简单的例子，在主机端声明一个矩阵 x，将它复制到设备上，执行一些操作，然后再将数据复制回主机并打印矩阵：

```
#include <stdio.h>
#include <stdlib.h>
#include <cublas_v2.h>
#include "cuda_helper.h"
#include "cuda.h"

#define CUBLAS_CALL(x){const cublasStatus_t a = (x); if(a != CUBLAS_STATUS_SUCCESS){
printf("\nCUBLAS Error: (err_num=%d) \n", a); cudaDeviceReset(); ASSERT(0);} }

__host__ void print_array(const float * __restrict__ const data1,
                          const float * __restrict__ const data2,
                          const float * __restrict__ const data3,
                          const int num_elem,
                          const char * const prefix)

{
 printf("\n%s", prefix);
 for (int i=0; i<num_elem; i++)
 {
  printf("\n%2d: %2.4f %2.4f %2.4f ", i+1, data1[i], data2[i], data3[i]);
 }
}

__host__ int main(int argc, char *argv[])
{
 const int num_elem = 8;
 const size_t size_in_bytes = (num_elem * sizeof(float));

 // Allocate memory on the device
 float * device_src_ptr_A;
 CUDA_CALL( cudaMalloc( (void **) &device_src_ptr_A, size_in_bytes ));
 float * device_src_ptr_B;
 CUDA_CALL( cudaMalloc( (void **) &device_src_ptr_B, size_in_bytes ));
 float * device_dest_ptr;
 CUDA_CALL( cudaMalloc( (void **) &device_dest_ptr, size_in_bytes ));

 // Allocate memory on the host for the device copy
 float * host_src_ptr_A;
 CUDA_CALL( cudaMallocHost( (void **) &host_src_ptr_A, size_in_bytes ));
 float * host_dest_ptr;
 CUDA_CALL( cudaMallocHost( (void **) &host_dest_ptr, size_in_bytes ));
 float * host_dest_ptr_A;
 CUDA_CALL( cudaMallocHost( (void **) &host_dest_ptr_A, size_in_bytes ));
 float * host_dest_ptr_B;
 CUDA_CALL( cudaMallocHost( (void **) &host_dest_ptr_B, size_in_bytes ));

// Clear destination memory
memset(host_dest_ptr_A, 0, size_in_bytes);
memset(host_dest_ptr_B, 0, size_in_bytes);
memset(host_dest_ptr, 0, size_in_bytes);
```

```
// Init the CUBLAS library
cublasHandle_t cublas_handle;
CUBLAS_CALL(cublasCreate(&cublas_handle));

// Print library version number
int version;
CUBLAS_CALL(cublasGetVersion(cublas_handle, &version));
printf("\nUsing CUBLAS Version: %d", version);

// Fill the first host array with known values
for (int i=0; i < num_elem; i++)
{
 host_src_ptr_A[i] = (float) i;
}
print_array(host_src_ptr_A, host_dest_ptr_B, host_dest_ptr, num_elem, "Before Set");

const int num_rows = num_elem;
const int num_cols = 1;
const size_t elem_size = sizeof(float);

// Copy a matrix one cell wide by num_elem rows from the CPU to the device
CUBLAS_CALL(cublasSetMatrix(num_rows, num_cols, elem_size, host_src_ptr_A,
          num_rows, device_src_ptr_A, num_rows));

// Clear the memory in the other two
CUDA_CALL(cudaMemset(device_src_ptr_B, 0, size_in_bytes));
CUDA_CALL(cudaMemset(device_dest_ptr, 0, size_in_bytes));

// SAXPY on device based on copied matrix and alpha
const int stride = 1;
float alpha = 2.0F;
CUBLAS_CALL(cublasSaxpy(cublas_handle, num_elem, &alpha, device_src_ptr_A,
           stride, device_src_ptr_B, stride));

alpha = 3.0F;
CUBLAS_CALL(cublasSaxpy(cublas_handle, num_elem, &alpha, device_src_ptr_A,
          stride, device_dest_ptr, stride));

// Calculate the index of the max of each maxtrix, writing the result
// directly to host memory
int host_max_idx_A, host_max_idx_B, host_max_idx_dest;
CUBLAS_CALL(cublasIsamax(cublas_handle, num_elem, device_src_ptr_A,
          stride, &host_max_idx_A));
CUBLAS_CALL(cublasIsamax(cublas_handle, num_elem, device_src_ptr_B,
          stride, &host_max_idx_B));
CUBLAS_CALL(cublasIsamax(cublas_handle, num_elem, device_dest_ptr,
          stride, &host_max_idx_dest));

// Calculate the sum of each maxtrix, writing the result directly to host memory
float host_sum_A, host_sum_B, host_sum_dest;
```

```
CUBLAS_CALL(cublasSasum(cublas_handle, num_elem, device_src_ptr_A,
            stride, &host_sum_A));
CUBLAS_CALL(cublasSasum(cublas_handle, num_elem, device_src_ptr_B,
            stride, &host_sum_B));
CUBLAS_CALL(cublasSasum(cublas_handle, num_elem, device_dest_ptr,
            stride, &host_sum_dest));

// Copy device versions back to host to print out
CUBLAS_CALL(cublasGetMatrix(num_rows, num_cols, elem_size, device_src_ptr_A,
            num_rows, host_dest_ptr_A, num_rows));
CUBLAS_CALL(cublasGetMatrix(num_rows, num_cols, elem_size, device_src_ptr_B,
            num_rows, host_dest_ptr_B, num_rows));
CUBLAS_CALL(cublasGetMatrix(num_rows, num_cols, elem_size, device_dest_ptr,
            num_rows, host_dest_ptr, num_rows));

// Make sure any async calls above are complete before we use the host data
const int default_stream = 0;
CUDA_CALL(cudaStreamSynchronize(default_stream));

// Print out the arrays
print_array(host_dest_ptr_A, host_dest_ptr_B, host_dest_ptr, num_elem, "After Set");

// Print some stats from the arrays
printf("\nIDX of max values : %d, %d, %d", host_max_idx_A,
 host_max_idx_B, host_max_idx_dest);
printf("\nSUM of values : %2.2f, %2.2f, %2.2f", host_sum_A,
 host_sum_B, host_sum_dest);

// Free device resources
CUBLAS_CALL(cublasDestroy(cublas_handle));
CUDA_CALL(cudaFree(device_src_ptr_A));
CUDA_CALL(cudaFree(device_src_ptr_B));
CUDA_CALL(cudaFree(device_dest_ptr));

// Free host resources
CUDA_CALL(cudaFreeHost(host_src_ptr_A));
CUDA_CALL(cudaFreeHost(host_dest_ptr_A));
CUDA_CALL(cudaFreeHost(host_dest_ptr_B));
CUDA_CALL(cudaFreeHost(host_dest_ptr));

// Reset ready for next GPU program
CUDA_CALL(cudaDeviceReset());
}
```

该程序的基本步骤如下：

- 使用 cublasCreate 函数创建一个 CuBLAS 句柄。
- 在主机和设备上分配资源。
- 直接根据主机上的矩阵在设备上设置一个矩阵。

- 在设备端运行 Saxpy。
- 在设备端运行 max 和 sum 函数。
- 将结果矩阵复制回主机并显示。
- 释放所有分配的资源。

实际上，真正的程序要比这复杂得多。在这里，我们只试图展示为了使一些简单的 CuBLAS 函数在 GPU 上工作所需的基本模板。

SDK 样例程序：Matrix Multiplication。

10.3 CUDA 运算 SDK

CUDA SDK 是和常规的工具箱、驱动分开下载的（尽管现在它捆绑到了 CUDA 5 发布版本中且对于 Windows 用户是可选的），因此未来它可能需要单独下载。它包含大量样例程序，还提供了可以找到所有 CUDA 文档的接口。

SDK 大约包含 200 个样例程序，因此我们只选择很少的几个样例程序进行详细介绍。我们将考查一些通用的应用程序，因为本书的读者范围比较广泛，介绍通用的例子比工具箱中特定领域的例子更容易理解。

这些运算的例子对刚开始进行 GPU 编程的人以及一些想要了解程序实现细节的高级程序员来说是很有用的。不幸的是，大量的底层 API 是对人们透明的。当我们学习一个新的 API 时，最不希望看到的是除此之外还要学习另一个更上层的 API，因为这会使我们对它的理解变得更加复杂。

很多 SDK 样例使用 cutil 或其他标准 CUDA 发行版之外的包。因此，当你看到下面这行代码的时候

```
cutilSafeCall(cudaGetDeviceProperties(&deviceProps, devID));
```

你可能希望它能够在你的代码中正常运行。然而，为了实现这个目的，还需要从 SDK 中包含相关的 cutil 头文件。NVIDIA 并没有保证各个版本函数库之间的通用性。由于这些并不是官方 CUDA 发行版中的，因此并不支持。

CUDA API 总是以 cuda 开头的。因此，如果看到了除此之外的 API，应该知道如果要使用这些调用，即需要从 SDK 样例中引入额外的代码。

那么 cutilSafeCall 做了些什么呢？它基本上和我们在本书中一直使用的 CUDA_CALL 宏类似。如果从调用者返回了一个错误，它会打印文件和行号然后退出。那么为什么不直接使用 cutil 这个包呢？主要是因为这个库中有很多函数而且实际上只会使用其中很少的一些。

然而，这个包中也有很多有用的函数，例如 gpuGetMaxGflopsDeviceId 函数可以识别出系统中速度最快的 GPU 设备。在深入研究这些样例代码之前，应该浏览 SDK 提供的函数库，这样会有助于对样例代码的理解。

10.3.1 设备查询

设备查询是一个很有趣的应用程序，因为它很简单而且可以看到 GPU 能够做些什么。我们可以在“C:\ProgramData\NVIDIA Corporation\NVIDIA GPU Computing SDK 4.1\C\bin\win64\Release”找到它并且从命令行而不是 Windows 接口运行它。

很显然，我们使用的是 4.1 版本工具包，且运行在 64 位 Windows 系统上，这可能会和你的系统有所不同。该程序的输出如下：

```
Found 4 CUDA Capable device(s)

Device 0: "GeForce GTX 470"
 CUDA Driver Version / Runtime Version          4.1 / 4.1
 CUDA Capability Major/Minor version number:    2.0
 Total amount of global memory:                 1280 MBytes (1342177280 bytes)
 (14) Multiprocessors x (32) CUDA Cores/MP:     448 CUDA Cores
 GPU Clock Speed:                               1.22 GHz
 Memory Clock rate:                             1674.00 Mhz
 Memory Bus Width:                              320-bit
 L2 Cache Size:                                 655360 bytes
 Max Texture Dimension Size (x,y,z)             1D=(65536), 2D=(65536,65535),
                                                3D=(2048,2048,2048)
 Max Layered Texture Size (dim) x layers        1D=(16384)x 2048,2D=(16384,16384)x
                                                2048
 Total amount of constant memory:               65536 bytes
 Total amount of shared memory per block:       49152 bytes
 Total number of registers available per block: 32768
 Warp size:                                     32
 Maximum number of threads per block:           1024
 Maximum sizes of each dimension of a block:    1024 x 1024 x 64
 Maximum sizes of each dimension of a grid:     65535 x 65535 x 65535
 Maximum memory pitch:                          2147483647 bytes
 Texture alignment:                             512 bytes
 Concurrent copy and execution:                 Yes with 1 copy engine(s)
 Run time limit on kernels:                     No
 Integrated GPU sharing Host Memory:            No
 Support host page-locked memory mapping:       Yes
 Concurrent kernel execution:                   Yes
 Alignment requirement for Surfaces:            Yes
 Device has ECC support enabled:                No
 Device is using TCC driver mode:               No
 Device supports Unified Addressing (UVA):      No
 Device PCI Bus ID / PCI location ID:           8 / 0
 Compute Mode:
  < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously)>

Device 1: "GeForce 9800 GT"
 CUDA Capability Major/Minor version number:    1.1
 Total amount of global memory:                 1024 MBytes (1073741824 bytes)
 (14) Multiprocessors x ( 8) CUDA Cores/MP:     112 CUDA Cores
 GPU Clock Speed:                               1.63 GHz
```

```
 Memory Clock rate:                             950.00 Mhz
 Memory Bus Width:                              256-bit
 Total amount of shared memory per block:       16384 bytes
 Total number of registers available per block: 8192
 Maximum number of threads per block:           512
 Device PCI Bus ID / PCI location ID:           7 / 0

Device 2: "GeForce GTX 260"
 CUDA Capability Major/Minor version number:    1.3
 Total amount of global memory:                 896 MBytes (939524096 bytes)
 (27) Multiprocessors x ( 8) CUDA Cores/MP:     216 CUDA Cores
 GPU Clock Speed:                               1.35 GHz
 Memory Clock rate:                             1100.00 Mhz
 Memory Bus Width:                              448-bit
 Total amount of shared memory per block:       16384 bytes
 Total number of registers available per block: 16384
 Maximum number of threads per block:           512
 Device PCI Bus ID / PCI location ID:           1 / 0

Device 3: "GeForce GTX 460"
 CUDA Capability Major/Minor version number:    2.1
 Total amount of global memory:                 1024 MBytes (1073741824 bytes)
 ( 7) Multiprocessors x (48) CUDA Cores/MP:     336 CUDA Cores
 GPU Clock Speed:                               1.45 GHz
 Memory Clock rate:                             1800.00 Mhz
 Memory Bus Width:                              256-bit
 L2 Cache Size:                                 524288 bytes
 Total amount of shared memory per block:       49152 bytes
 Total number of registers available per block: 32768
 Maximum number of threads per block:           1024
 Device PCI Bus ID / PCI location ID:           2 / 0
```

该程序会对所有的 GPU 进行迭代以寻找并列出每个设备的各种细节信息。为了简洁起见，我们只完整地列出了 4 个设备中的 1 个并且从其他设备信息中提取了一些有趣的部分。Kepler GK104 的相关细节信息如下：

```
Device 0: "GeForce GTX 680"
 CUDA Capability Major/Minor version number:    3.0
 Total amount of global memory:                 2048 MBytes (2146762752 bytes)
 ( 8) Multiprocessors x (192) CUDA Cores/MP:    1536 CUDA Cores
 GPU Clock Speed:                               1006 MHz
 Memory Clock rate:                             3004.00 Mhz
 Memory Bus Width:                              256-bit
 L2 Cache Size:                                 524288 bytes
 Total amount of shared memory per block:       49152 bytes
 Total number of registers available per block: 65536
 Warp size: 32
 Maximum number of threads per block:           1024
 Concurrent copy and execution:                 Yes with 1 copy engine(s)
```

值得注意的内容是，当前的驱动程序和运行时的版本号，两者应该一致。计算能力定义

了对于给定编号的设备的类型。同时，这里详细记录了每个设备的核 /SM 数量、设备的速度、内存速度和带宽。因此，我们可以计算给定设备的峰值带宽。提到带宽，我们接下来介绍一个 SDK 中非常有用的应用程序。

10.3.2 带宽测试

SDK 提供的带宽样例程序提供了以下关于特定设备 / 主机设置的有用统计信息：

- 主机到设备带宽（换页内存和锁页内存）
- 设备到主机带宽（换页内存和锁页内存）
- 设备到设备带宽

在 x8 PCI-E 2.0 链路上的 GTX470 的实际输出如下：

```
>> bandwidthtest --device=0 --memory=pageable
Device 0: GeForce GTX 470
 Quick Mode

 Host to Device Bandwidth, 1 Device(s), Paged memory
  Transfer Size (Bytes) Bandwidth(MB/s)
  33554432 1833.6

 Device to Host Bandwidth, 1 Device(s), Paged memory
  Transfer Size (Bytes) Bandwidth(MB/s)
  33554432 1700.5

 Device to Device Bandwidth, 1 Device(s)
  Transfer Size (Bytes) Bandwidth(MB/s)
  33554432 113259.3

>> bandwidthtest --device=0 --memory=pinned
Host to Device Bandwidth, 1 Device(s), Pinned memory
  Transfer Size (Bytes) Bandwidth(MB/s)
  33554432 2663.6

 Device to Host Bandwidth, 1 Device(s), Pinned memory
  Transfer Size (Bytes) Bandwidth(MB/s)
  33554432 3225.8

 Device to Device Bandwidth, 1 Device(s)
  Transfer Size (Bytes) Bandwidth(MB/s)
  33554432 113232.3
```

使用这个示例程序的时候，会看到使用锁页内存到底会使系统有多大的收益。我们在第 9 章讲到过这个，然而没有什么比亲自在系统上看到这些数据更能让我们确定锁页内存会带来更快的内存传输。即使是 PCI-E 2.0 x8 链路上，现代的 Sandybridge-E 处理器使用锁页内存也能够达到 3GB/s 的速度，而与之相比，在类似的链路上使用换页内存只能达到 2.3GB/s 的速度。

典型的消费级 GPU 显卡上的内存大小都是从 512MB（88/9800 系列）~ 2GB（GTX680）。

关于为什么不能在和 GPU 进行传输时使用锁页系统内存，这实在没有什么原因。即使是在有 4GB 内存限制的 32 位系统中，CUDA 仍然能够在后台使用锁页传输方式。因此，可能需要自己锁定内存并且避免在驱动程序中进行隐式的换页内存到页锁定内存的复制。

由于现在内存很便宜，可以完全地满载机器，特别当是有多 GPU 显卡或使用 Tesla 显卡时。用少于 100 欧元 / 美元 / 英镑的钱就能买到 16GB 的主机内存。

GTX470 内存时钟为 1674MHz，总线宽度为 320 位。因此，我们将总线宽度除以 8 就能获得其字节数（40 字节）。接下来我们乘以时钟速度（66 960MB/s）。然后对于 GDDR5 还需要再乘以 2（133 920MB/s）。最后再除以 1000 获得官方给出的内存带宽（133.9GB/s）或是除以 1024（130.8GB/s）获得实际带宽。

那么为什么我们得到的设备到设备的带宽是 113 232MB/s 而不是 113 920MB/s 呢？缺少的那 20MB/s 或者 15% 的带宽去哪了呢？ GPU 永远都不会达到这个理论上的峰值带宽。这就是为什么与计算峰值带宽相比运行带宽测试是很有用的。接下来，根据 PCI-E 的布局、主机 CPU、CPU 芯片组、主机内存等，就能清楚知道从系统中能得到多大的带宽。通过了解这些，就能知道对于某个特定的目标，该应用程序能够达到何种程度并且因此知道还有多少潜力可以利用。

请注意，使用基于 Tesla 的 Fermi 设备可以通过使用 nvidia_smi 工具禁用 ECC 内存选项从而获得显著的带宽提升。错误检查和修正（ECC）使用汉明码分配位模式。这（实际上）意味着你需要更大的内存空间存储相同的数据块。这个额外的存储需求意味着对于 ECC 带来的额外冗余，需要在空间和速度上付出代价。NVIDIA 声称在 Kepler K20（GK110）上已经解决了该问题，K20 使用 ECC 带来的影响大约是 Fermi 的 1/3。

10.3.3 SimpleP2P

SimpleP2P 示例程序展现了如何使用计算能力 2.x 的设备（Fermi）中引入的 P2P 内存传输能力。P2P 内存传输的原理是避免通过主机内存传输（参见图 10-5）。主机内存直接可以访问 PCI-E I/O 集线器（北桥），在 Intel 基于 QPI 的系统上经常就是这样的。它也可能访问处理器的另一端，比如 Intel 的 DMI 和 AMD 基于超传输的系统。

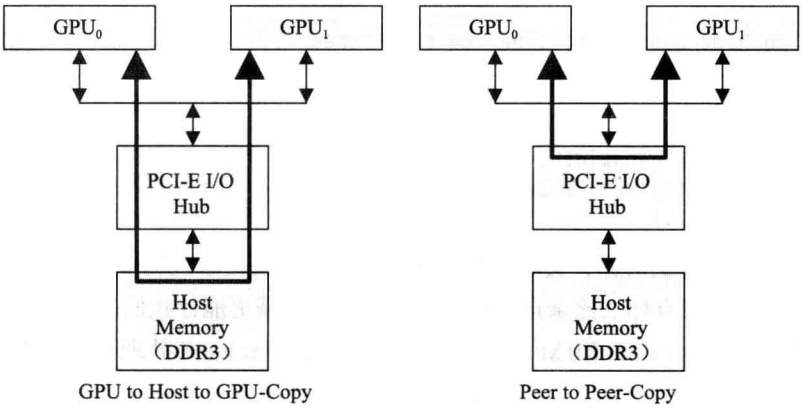

图 10-5 P2P 传输

从其本身速度的角度来说，主机内存可能代表了传输的瓶颈，这也取决于系统中 GPU 的数量。PCI-20 链路上最大 PCI-E 传输速度接近 6GB/s。对于 PCI-E3.0（GTX680、GTX690、Tesla K10）实际带宽几乎翻了一倍，每个方向略小于 12GB/s。为了最大化带宽，通常需要定义两个锁页内存区域并且使用双缓冲策略传输到某一块并从另外一块传出。特别是对于那些比较老的处理器，仅仅通过主机内存在 GPU 间执行一些传输就能很快地用尽全部主机内存带宽。这将严重地限制那些试图使用 CPU 以获得额外处理能力的行为，因为它将和 GPU 传输一起竞争主机内存带宽。

P2P 的想法是将数据保存在主机内存空间之外并且直接在 GPU 间进行传输。尽管这是一个非常有用的特性，但是主流的 Windows 7 操作系统对它的支持却很少。因此在本书之前的部分我们并没有见到它。我们要在这提及它，因为这个技术有很多用处。使用 P2P 功能的要求是：

- 64 位系统（因此 UVA（统一虚拟寻址）启用），这是 P2P 可用的一个要求。
- 两个或更多支持该特性的计算能力 2.x 的设备。
- 在相同 PCI-E I/O 集线器上的 GPU 设备。
- 适当的驱动程序级支持。

为了在 Windows 7 下使用这个特性，需要使用 64 位操作系统并且 TCC（Tesla 计算集群）驱动是激活的。由于 TCC 驱动只有 Tesla 显卡才能激活，因此实际上没有主流的消费级显卡在 Windows 7 中支持该特性。因此，它应该是适用于集群、高性能计算（HPC）和其他一些以计算为中心的应用程序的一个特性。这并不是可以使用的，比如说消费级 PC 上的视频转码应用程序。

为了支持 P2P，应该检查 UVA 是否已启用：

```
struct cudaDevice device_prop;
CUDA_CALL(cudaGetDeviceProperties(&device_prop));
if (device_prop.unifiedAddressing == 1) // If unified addressing is enabled
```

UVA 只有在 64 位操作系统或者是使用 TCC 驱动的 Windows 7 系统中才能启用。

接下来需要检查设备 A 是否可以和设备 B 对话。注意，仅仅通过了这个测试并不意味着设备 B 能够和设备 A 对话。在启用特定方向上的 P2P 访问时要消耗资源。可以使用以下代码执行该测试：

```
int peer_access_avail;
int src_device = 0;
int peer_device = 1;

CUDA_CALL(cudaDeviceCanAccessPeer( &peer_access_avail, src_device, peer_device));

if (peer_access_avail == 1) // If peer access from device 0 to device 1 is available
{
  int flags = 0;
  CUDA_CALL(cudaSetDevice(peer_device));
  CUDA_CALL(cudaEnablePeerAccess(peer_device, flags);
}
```

一旦启用对等访问，在设备内核或主机初始化的内存副本中按照对等访问的方向对内存进行访问。因此，设备 0 可以启用来自设备 1 的对等访问（像之前的示例一样）。接下来可以在设备 1 中调用一个内核，传给它一个指向设备 0 全局内存空间的指针。接下来内核会像处理指向主机内存的零复制设备指针一样对该指针进行解引用。每当有一个通过该指针的访问，设备 1 将会从设备 0 通过 PCI-E 总线启动一个读取操作。当然，零复制内存使用时的其他注意事项这里也适用。特别地，应该避免再次读取这样的内存并且为了更好的性能尝试实现合并访问模式。

当然可以对设备初始化的内存副本使用这样的特性。为了从设备实现，需要让一个设备内核通过设备指针获取数据，然后将它存储到本地设备的全局内存中。同样，可以在设备之间拉取 / 推送数据。然而，如果想要双向访问，需要记住在两个方向都启用 P2P 访问。

第二种方法是显式内存复制，我们必须从主机初始化。这种方法有两种标准形式：同步版本和异步流版本：

```
cudaMemcpyPeer(dest_device_ptr, dst_device_num, src_device_ptr,
src_device_num,num_bytes);
```

和

```
cudaMemcpyPeerAsync( dest_device_ptr, dst_device_num, src_device_ptr, src_device_num,
num_bytes, stream );
```

最后，一旦我们完成了，需要使用以下调用关掉为 P2P 访问供应的资源：

```
cudaDeviceDisablePeerAccess(device_num);
```

10.3.4 asyncAPI 和 cudaOpenMP

asyncAPI SDK 示例程序提供了一个使用异步 API 的例子，但是事实上对于刚接触 CUDA 的人来说它并不很容易理解。在本书中我们已经介绍了流和异步操作。这些对于设置多 GPU 从而使其在 CPU 使用的同时进行工作是很重要的。因此，我们将看看这个程序到底做了些什么。

asyncAPI 示例程序的基本前提是创建一个异步的流，然后依次将复制到设备操作、内核操作和最终复制回主机操作填充到流中。在这个阶段它会在 CPU 上执行一些代码，这些代码仅仅是在 GPU 运行异步内核的时候进行求和。

cudaOpenMP 示例程序表明了如何在 CUDA 中使用 OpenMP。它识别出 CPU 线程的个数和每个附属 CUDA 设备的个数及名称。接下来它试图在每个 GPU 设备上产生一个线程并且在不同设备间共享工作。

在这里，我们将提供一个类似的示例程序。它将两种 SDK 结合起来，做了适当简化并且可能更加有用，它可以作为模板用在自己的工作中。

```
#include <stdio.h>
#include <omp.h>

#include "cuda_helper.h"
#include "cuda.h"

__global__ void increment_kernel(int * __restrict__ const data,
```

```
                                    const int inc_value,
                                    const int num_elem)
{
 const int idx = blockIdx.x * blockDim.x + threadIdx.x;

 // Check array index does not overflow the array
 if (idx < num_elem)
 {
  // Repeat N times - just to make the kernel take some time
  const int repeat = 512;

  for (int i=0; i < repeat; i++)
   data[idx] += inc_value;
 }
}

// Max number of devices on any single node is, usually at most, eight
#define MAX_NUM_DEVICES 8

__host__ int main(int argc, char *argv[])
{
 const int num_elem = 1024 * 1024 * 16;
 const int size_in_bytes = num_elem * sizeof(int);
 const int increment_value = 1;
 const int loop_iteration_check = 1000000;
 const int shared_mem = 0;

 // Define the number of threads/blocks needed
 const int num_threads = 512;
 const int num_blocks = ((num_elem + (num_threads-1)) / num_threads);

 // One array element per CPU thread
 int host_counter[MAX_NUM_DEVICES];
 float delta_device_time[MAX_NUM_DEVICES];
 cudaDeviceProp device_prop[MAX_NUM_DEVICES];

 int num_devices;
 CUDA_CALL(cudaGetDeviceCount(&num_devices));
 printf("\nIdentified %d devices. Spawning %d threads to calculate %d MB using (%dx%d)",
num_devices, num_devices, ((size_in_bytes/1024)/1024), num_blocks, num_threads );

 // Declare thread private, per thread variables
 int * device_ptr[MAX_NUM_DEVICES];
 int * host_ptr[MAX_NUM_DEVICES];
 cudaEvent_t start_event[MAX_NUM_DEVICES], stop_event[MAX_NUM_DEVICES];
 cudaStream_t async_stream[MAX_NUM_DEVICES];

 // Create all allocations outside of OpenMP in series
 for (int device_num=0; device_num < num_devices; device_num++)
```

```
  {
   // Set the device to a unique device per CPU thread
   CUDA_CALL(cudaSetDevice(device_num));

   // Get the current device properties
   CUDA_CALL(cudaGetDeviceProperties(&device_prop[device_num], device_num));
   // Allocate the resources necessary
   CUDA_CALL(cudaMalloc((void **) &device_ptr[device_num], size_in_bytes));
   CUDA_CALL(cudaMallocHost((void **) &host_ptr[device_num], size_in_bytes));
   CUDA_CALL(cudaEventCreate(&start_event[device_num]));
   CUDA_CALL(cudaEventCreate(&stop_event[device_num]));
   CUDA_CALL(cudaStreamCreate(&async_stream[device_num]));
  }

  // Spawn one CPU thread for each device
 #pragma omp parallel num_threads(num_devices)
  {
   // Variables declared within the OpenMP block are thread private and per thread
   // Variables outside OpenMP block exist once in memory and are shared between
   // threads.

   // Get our current thread number and use this as the device number
   const int device_num = omp_get_thread_num();

   // Set the device to a unique device per CPU thread
   CUDA_CALL(cudaSetDevice(device_num));

   // Push start timer, memset, kernel, copy back and stop timer into device queue
   CUDA_CALL(cudaEventRecord(start_event[device_num], async_stream[device_num]));

   // Copy the data to the device
   CUDA_CALL(cudaMemsetAsync(device_ptr[device_num], 0, size_in_bytes,
             async_stream[device_num]));

   // Invoke the kernel
   increment_kernel<<<num_blocks, num_threads, shared_mem, async_stream[device_num]
 >>>(device_ptr[device_num], increment_value, num_elem);

   // Copy data back from the device
   CUDA_CALL(cudaMemcpyAsync(host_ptr[device_num], device_ptr[device_num],
            size_in_bytes, cudaMemcpyDeviceToHost,
            async_stream[device_num]));

   // Record the end of the GPU work
   CUDA_CALL(cudaEventRecord(stop_event[device_num], async_stream[device_num]));

   // Device work has now been sent to the GPU, so do some CPU work
   // whilst we're waiting for the device to complete its work queue

   // Reset host counter
```

```
int host_counter_local = 0;
int complete = 0;

// Do some work on the CPU until all the device kernels complete
do
{
 // Insert useful CPU work here
 host_counter_local++;

 // Check device completion status every loop_iteration_check iterations
 if ( (host_counter_local % loop_iteration_check) == 0 )
 {
  // Assume everything is now complete
  complete = 1;

  // Check if all GPU streams have completed. Continue to do more CPU
  // work if one of more devices have pending work.
  for ( int device_check_num=0; device_check_num < num_devices;
   device_check_num++)
 {
   if ( cudaEventQuery(stop_event[device_check_num]) == cudaErrorNotReady )
    complete = 0;
  }
 }

} while( complete == 0 );

// Write out final result
host_counter[device_num] = host_counter_local;

// Calculate elapsed GPU time
CUDA_CALL(cudaEventElapsedTime(&delta_device_time[device_num],
          start_event[device_num],
          stop_event[device_num]));
} // End parallel region

// Now running as a single CPU thread again

// Free allocated resources
// Create all allocations outside of OpenMP in series
for (int device_num=0; device_num < num_devices; device_num++)
{
 // Set the device to a unique device per CPU thread
 CUDA_CALL(cudaSetDevice(device_num));

 CUDA_CALL(cudaStreamDestroy(async_stream[device_num]));
 CUDA_CALL(cudaEventDestroy(stop_event[device_num]));
 CUDA_CALL(cudaEventDestroy(start_event[device_num]));
 CUDA_CALL(cudaFreeHost(host_ptr[device_num]));
 CUDA_CALL(cudaFree(device_ptr[device_num]));
```

```
    // Reset the device for later use
    CUDA_CALL(cudaDeviceReset());
  }

  // Print a summary of the results
  for (int device=0; device < num_devices; device++)
  {
    printf("\n\nKernel Time for device %s id:%d: %.2fms",
           device_prop[device].name, device, delta_device_time[device]);
    printf("\nCPU count for thread %d: %d", device, host_counter[device]);
  }
}
```

SDK 示例程序中还有几点需要更深入地讨论。首先，在 asyncAPI 示例程序中，使用了 0 号流（默认流）。不幸的是，很多情况下默认流会引发流之间隐式的同步。很可能最终使用双缓冲或三缓冲方法，上述隐式的同步将会困扰着你。当使用异步操作的时候，一定要创建自己的流：

```
cudaStream_t async_stream[MAX_NUM_DEVICES];
CUDA_CALL(cudaSetDevice(device_num));
CUDA_CALL(cudaStreamCreate(&async_stream[device_num]));
```

asyncAPI 流示例程序中的第二点是：它通过将元素的个数 N 直接除以线程的个数得到线程网格中线程块的个数。这是因为 N 是线程数的倍数，但如果不是呢？实际上，数组中最后的元素将不会由 GPU 内核处理。对于刚开始学习 CUDA 的人来说，这个问题并不很明显。如果在 N 不是线程数倍数的情况下，一定要使用下面的公式计算线程块的个数：

```
const int num_elem = 1024 * 1024 * 16;
const int num_threads = 512;
const int num_blocks = ((num_elem + (num_threads-1)) / num_threads);
```

然后在内核中添加对数组越界的检查：

```
// Check array index does not overflow the array
if (idx < num_elem)
```

现在，程序增加了将 num_elem 传递到内核并且在内核中对它进行检查的开销。如果能保证使用的总是线程数倍数，则可以不使用这段代码并且坚持使用更简单的 num_blocks = num_elem / num_threads 方法。大多数情况下，这是正确的，因为我们经常会控制数据块的大小。

我们现在看看 cudaOpenMP 示例程序，多个 CPU 线程如何启动呢？它对 omp_set_num_thread 进行了一次调用：

```
omp_set_num_threads(num_gpus);
//omp_set_num_threads(2*num_gpus);
#pragma omp parallel
{
}
```

这里有两种方法：每个 GPU 设置一个线程或每个 GPU 设置多个线程（参见图 10-6）。当 CPU 核数比 GPU 个数多时，第二种方法会更有效。一种更简单的、更可靠工作的 OpenMP 指令形式是我们之前在示例程序中使用的：

```
// Spawn one CPU thread for each device
#pragma omp parallel num_threads(num_devices)
{
}
```

使用这种方法我们无须关注 OpenMP 是否已配置或环境变量是否设置，它会产生指定个数的线程。注意，当前线程也是用来执行工作的线程之一：

```
Identified 4 devices. Spawning 4 threads to calculate 64 MB using (32768x512)

Kernel Time for device GeForce GTX 470 id:0: 427.74ms
CPU count for thread 0: 1239000000

Kernel Time for device GeForce 9800 GT id:1: 3300.55ms
CPU count for thread 1: 1180000000

Kernel Time for device GeForce GTX 285 id:2: 1693.63ms
CPU count for thread 2: 1229000000

Kernel Time for device GeForce GTX 460 id:3: 662.61ms
CPU count for thread 3: 1254000000
```

从程序的输出结果可以看到（通过使用不同的 GPU）线程在不同的时间完成，如图 10-6 所示，有 4 个线程在运行（包括初始线程）。如果在屏幕上观察能在顶部看到深绿色的条形，它们表明线程基本上一直在运行（~ 95%）。其中有偶尔的停顿，它们用浅绿色标志。下方是 4 个 GPU 任务，其中每一个都执行内存分配、内核启动和复制回主机的操作。这些条形的底部一行表明 CPU 在当前时间帧的使用情况。可以看到，CPU 几乎在整个时间内都是忙碌的。

随着 4 个 GPU 的完成，CPU 线程继续工作直到集合中的 GPU 都完成。如果我们的 GPU 确实性能特性迥异，我们当然可以为某些 GPU 分配更多的工作。然而，现在大多数 GPU 系统都是用相同的 GPU，因此在它们都完成之前我们不需要关心再次提交工作。给定相似的任务，它们的工作量等同，它们都会在几乎相同的时间范围内完成。

我们要解决使用 OpenMP 过程中的下一个问题是在什么地方进行资源分配和资源释放。在给定设备上进行内存分配和资源创建是一个很耗时的过程。通常这需要对线程间内存的分配和共用的数据结构有普遍的了解。为了在线程间共享数据结构需要进行加锁操作，这反过来也会导致串行化。当我们把资源分配 / 释放放到 OpenMP 并行区域时，就可以清楚地看到这一点。因此，在 OpenMP 并行区域之前或之后进行分配 / 释放能够在该区域达到最佳的 CPU 利用率。

和这相关的是为了检查设备是否完成而调用 CUDA API，特别是对 cudaEventQuery 调用。这些调用的开销并不低。如果我们将 loop_iteration_check 常量的值从 1 000 000 变到仅仅为 1，我们会看到 CPU 计数从 1 239 000 000 减少到 16 136。实际上，每个线程在每个循环迭代中都会请求设备的状态。因此，相对于做其他事情，CPU 会在与驱动程序打交道上花费更多的时间。不幸的是，asyncAPI 正是这样实现的，这也是我们在这里强调它的原因。要留意在循环中使用的每一个 API 调用。这要花费些时间，因此不要让 CPU 在每个周期内仅仅对设备进行查询。在两次查询中间让 CPU 做些有用的事。

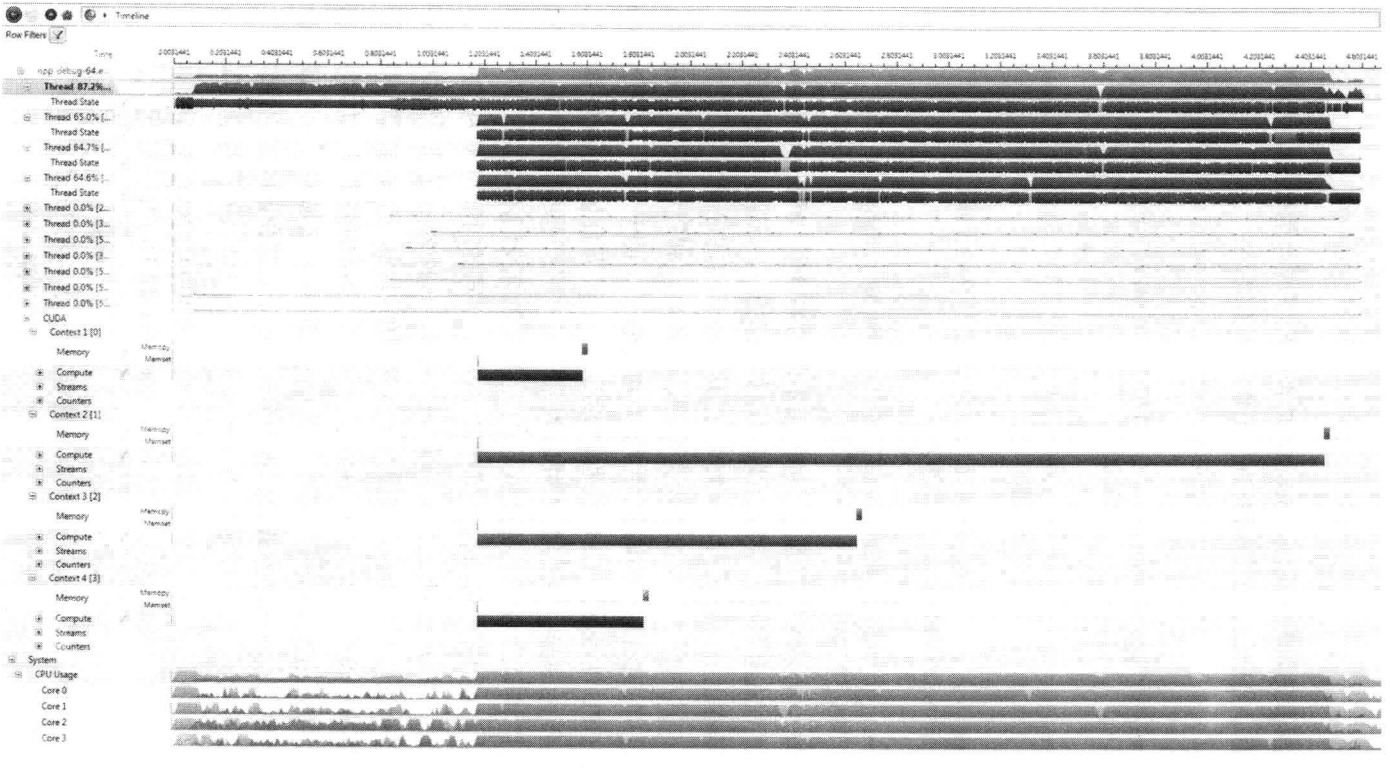

图 10-6 使用 OpenMP 的多 GPU

10.3.5 对齐类型

对齐类型的示例程序向我们演示使用 __align__（n）指令的效果。例如：

```
typedef struct __align__(8)
{
  unsigned int l, a;
} LA32;
```

这里 8 是字节个数，所有元素的起始地址应该是这个数字的倍数。这个示例说明，align 指令允许编译器为每个线程分配比实际更大的读取单位。在前面 LA32 的情况中，编译器可以使用一个 64 位的读取而不是两个 32 位的读取。正如我们在第 9 章看到的一样，更少的内存事务等价于更多的带宽。在这个例子中，我们使用向量类型，定义它们的时候也使用了 align 指令。

在之前示例程序中我们看到：为了实现峰值带宽等目标，在此过程中必须产生数量足够多的内存事务。不幸的是，这个 SDK 示例程序并没有按照这样的想法进行编写。它使用了 64 个包含 256 个线程的线程块，每个 SM 调度 32 个线程束。为了完全加载计算能力 2.x 的设备，我们需要 48 个线程束（对于 Kepler 是 64）。该示例程序使用的线程块太少了。因此我们将其扩展到 1024 个线程块并且每个块使用 192 个线程，这个数字对于所有计算能力的设备都适用。

我们也将基本类型输出加到了测试中，这样就可以看到基准结果。此外，每个运行专门编译以生成与当前设备的计算水平对应的代码。请注意这个 SDK 示例程序（即使有变化），仅仅达到了内存传输峰值的 50%。然而，我们真正感兴趣的是相对内存带宽。

初始时，我们看到了如表 10-3 和图 10-7 所示的不同设备的基准结果。我们可以使用这个基准性能表，对对齐和未对齐类型的表现进行评估。

表 10-3 不同设备的基线准性能表

Type	GT9800	GTX285	GTX460	GTX470	Size in Bytes
u8	0.6	18	20	32	1
u16	1	36	22	48	2
u32	19	48	42	49	4
u64	23	59	43	51	8

从图 10-7 我们可以看到，在 u32（即 4 字节）时它们都达到了最大合并内存大小。这等价于 32 个线程乘以 4 字节（也即总共 128 个字节）。在 Fermi 上，这是一个缓存行的大小。因此，在计算能力 2.x 设备上我们截止（flatline）到这个大小。

GTX285 设备（计算能力 1.3）执行 16 个线程的合并内存读取而不是像计算能力 2.x 的设备那样执行 32 个线程的。因此，它会受益于连续（back-to-back）的读取并且会充分利用每个线程 64 位（8 字节）的读取。此外，由于 SM 的个数多于 Fermi 级显卡的两倍（内存总线比 GTX470 更宽），在这个特定的内核中，它可以在性能上超过 GTX470。

在 9800GT（计算能力 1.1）上，我们看到了和 GTX285 相似的模式。然而，这里的主要不同是物理内存带宽仅仅是 GTX285 的一半左右。因此，在每线程 32 位和 64 位访问中我们获得的收益是很小的，这比我们在 GTX285 看到的要小很多。通过运行示例程序，我们可以

看到对齐和未对齐访问的百分比变化，如表 10-4 所示。在表 10-5 中 100% 代表着没有变化。

因此，如果我们按照计算能力向前回顾，可以看到（特别是对于早期的计算水平），对齐访问带来的收益很大。当在数据结构中加入这个指令时，我们所看到的最好情况是 31 倍的速度提升。即使转移到现代 GPU，我们仍然能看到 2 倍的性能提升。很显然，除非会导致更多的内存从主存移动到 GPU，在其他任何情况下使用这样的指令都是很有效的。

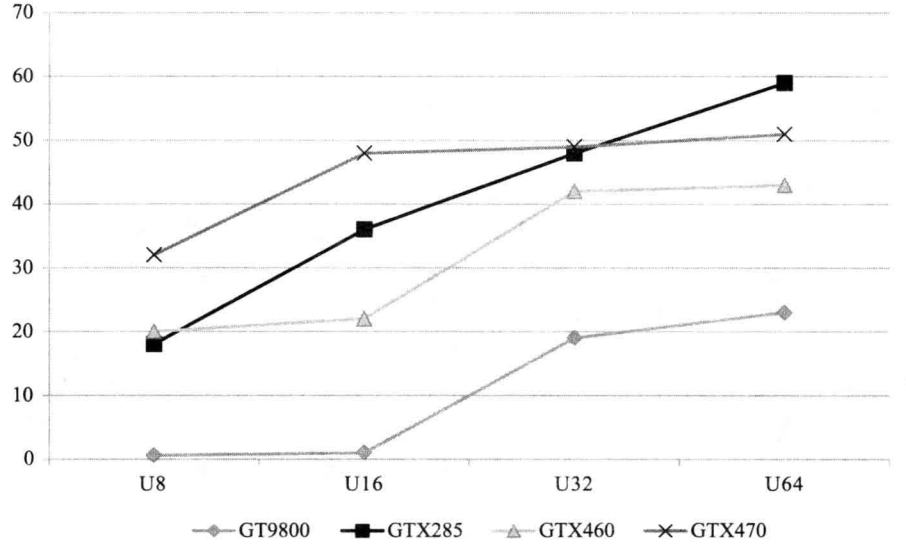

图 10-7 不同设备的基准性能图（MB/s 与传输大小）

表 10-4 不同设备在对齐 / 未对齐情况下的速度（MB/s）

Type	GT9800 Nonaligned	9800 Aligned	GTX285 Nonaligned	GTX285 Aligned	GTX460 Nonaligned	GTX460 Aligned	GTX470 Nonaligned	GTX470 Aligned
RBGA8	0.6	18.7	11	48	21	41	40	49
LA32	2.4	23.3	30	59	42	42	47	51
RGB32	2.6	2	20	9	33	30	32	29
RGBA32	2.7	23.6	15	51	25	43	24	51
RGBA32_2	10.7	10.6	25	25	34	34	32	32

表 10-5 对齐和未对齐访问模式的百分比变化

Type	GTX470	GTX460	GTX285	GT9800
RBGA8	123	195	436	3117
LA32	109	100	197	971
RGB32	91	91	45	77
RGBA32	213	172	340	874
RGBA32_2	100	100	100	99

注意 RGB32 的情况。它实际上是一个 96 位大小的结构（3 个 u32），其中 float3 被 int3 有效地替换。加入 align 指令会在结构结尾插入 4 个字节。尽管这允许合并访问，但是从内存系统传输的 25% 的数据被弃了。在未对齐情况下，费米设备上每次从缓存行启动的“过读取”（overfetch）节省了 33% 的后续内存读取操作。

从这个示例程序我们可以得出的结论是：如果使用了结构体，则需要考虑它的合并影响并且（至少）使用 align 指令。一个更好的解决方案是创建数组结构体而不是结构体数组。例如，使用分开的红、绿和蓝（RGB）的色彩平面而不是交错的 RGB 值。

10.4 基于指令的编程

本书主要关注直接编写 CUDA 程序。如果你很喜欢编写程序并且拥有 CS（Computer Science 计算机科学）背景的话，这很好。然而，现在有许多编写 CUDA 程序的人并不属于这一类。很多人最关心的是解决他们自己的计算问题，而不太在乎如何编写 CUDA 或从计算机科学角度来说解决方案是否优美。

OpenMP 的巨大成功之一在于它相对来说比较容易学习。它需要在 C 源代码中加入指令修饰符，告诉编译器当前编译代码的各种并行特性。因此，这需要程序员明确地识别出代码中的并行。编译器则负责充分利用该并行性完成更加艰难的任务。总体来说，这个过程执行得相当合理。

因此，使 GPU 编程变得更容易的显著的方法是将 OpenMP 模型扩展到 GPU 上。不幸的是，对此出现了两种标准：OpenMP4ACC 和 OpenACC 标准。这里，我们将关注 OpenACC 标准，因为它是 NVIDIA 明确支持的。通常你会发现赞助商的规模和程序员的普及面很大程度决定了特定的软件开发革新是成功还是失败。大多数的标准（不考虑谁开发了程序）涉及的主要内容是一致的。因此，大多数情况下，学会其中一个会使学习其他的变得更容易。

如果对使用指令编写 GPU 代码感兴趣，则很有可能对 CPU 上的 OpenMP 指令有适当的理解。我们发现像 OpenACC 类似的标准带来的主要不同在于它们（即使直接编程的程序员也是）需要管理数据的位置。有多个 CPU 物理插槽的 OpenMP 系统中存在 NUMA（非统一内存访问）系统。

从图 10-8 我们可以看出多个 CPU 的系统内存直接附加到指定的 CPU 上。因此驻留在 CPU_0 上的进程访问驻留在 CPU_0 上的内存所花费的时间明显比访问 CPU_1 本地的内存所需时间长。我们假设有 6 个进程运行在 2 个 CPU 插槽上，其中每个 CPU 有 4 个核。执行需要多对多通信方式的数据交换，这意味着吞吐量被最慢链路限制了。这可能是处理器之间的 QPI/ 超传输链路，它必须通过该链路才能将内存传输到另一个处理器的内存总线。OpenMP 模型简单地忽略了这个影响并且缺少许多基于加速器的解决方案需要的数据概念。

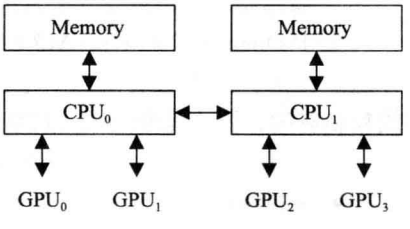

图 10-8 多 GPU 数据通道

OpenACC

OpenACC 是一个继 OpenMP 之后、面向指令编程方向的改进，这对独立的单个机器来说是很成功的。

OpenACC 的目标是：

- 独立的、基于循环的并行。
- 尚未接触过 CUDA 或认为它很复杂的那些程序员。
- 不希望学习 CUDA 并且乐于将特定目标体系结构的细节提取到编译器的那些程序员。
- 希望快速地为已存在于 GPU 上的串行应用程序创建原型的那些程序员。

OpenACC（和 OpenMP 一样）试图对硬件进行抽象，并让程序员编写标准的串行代码，然后编译器将其转换成运行在加速器上的代码。和 OpenMP 一样，它涉及到将一系列的编译指示语句添加到循环附近，以通知编译器在特定的循环处并行地运行。

优点：

- 和 OpenMP 看起来很相似，因此对于使用过 OpenMP 的人来说很容易学习。
- 现存的串行源代码保持不变只是简单地使用指令对其进行修饰。
- CPU 和 GPU 加速版本使用一套源代码集。
- 加速器供应商无关。有应用于多种硬件平台的潜力（和 OpenCL 相似），包括基于 CPU 的 AVX 加速。
- 考虑了很多的“细节”，如将用户指定的数据在共享内存移入 / 移出时，应该缓存数据。
- 供应商给出的数据表明，非 CUDA 程序员的学习曲线是很平滑的。
- 除了 C 语言外还支持 Fortran。这就允许很多现存的 Fortran 程序无须大量重写就能受益于 GPU 加速。

缺点：

- 当前在 Visual Studio 下并不被支持，因此实际上它是 Linux 专用的解决方案。
- 目前它是 PGI、CAPS 和 Cray 支持的商业产品，因此它并不是免费 CUDA SDK 产品套件的一部分。
- 为了使重要的程序达到与使用 OpenMP 相当或更好的性能，用户必须额外指定各种各样的简单数据子句（data clause）以最小化基于 PCI-E 的传输。
- 目标是单 CPU/ 单 GPU 解决方案。添加额外的 GPU 后不会自动调整。多 GPU 的使用需要使用多 CPU 线程 / 进程。这可能会在未来发生变化。
- CUDA 工具箱或硬件的新特性可能需要编译器供应商明确的支持。目前，OpenACC 编译器的支持需要几个月的时间才能切换到 CUDA SDK 新版本或支持新的硬件版本。

关于 OpenACC 和 OpenMP 的主要问题是 OpenMP 没有不同等级的内存或不同位置的内存这些概念，因为这些概念不存在于传统的 CPU 编程模型中。在 OpenMP 中，数据要么是线程私有的，要么是全局的（共享的）。

通过对比，GPU 系统是更加复杂的。因为我们有：

- 主机内存
- GPU 全局内存

- GPU 常量内存
- GPU 线程块私有内存（CUDA 中的共享内存）
- GPU 线程私有内存（CUDA 中的本地内存）

OpenACC 模型（为了简单起见），工作的基础是数据驻留在主机上且在并行区域开始时传输到加速器内存空间，然后在在并行区域结束时传输回主机。因此，每一个并行区域（默认的）以这些 PCI-E 总线上的隐式内存复制为分界。

尽管这是一种简单的思考方式，从概念上看，这是有可能以性能为代价来确保正确性的一种较为容易的方法。如果只有一次计算并且不会再次使用数据，那么无论以何种方式使用 CUDA，都会是很有效的。然而，如果打算对数据进行大量的传输，那么则需要通过在指令中加入数据限定符来显式指定哪些数据要保留在设备端。

因此我们通过一个简单的程序，展示它是如何转换到 OpenMP/OpenACC 的。如果我们使用经典的归约，通常会看到下面的代码：

```
long int reduce_serial(const int * __restrict__ const host_ptr,
                       const int num_elements)
{
 long int sum = 0;

 for (int i=0; i < num_elements; i++)
      sum += host_ptr[i];

 return sum;
}

long int reduce_openmp(const int * __restrict__ const host_ptr,
                       const int num_elements)
{
 long int sum = 0;

#pragma omp parallel for reduction(+:sum)
 for (int i=0; i < num_elements; i++)
 {
  sum += host_ptr[i];
 }

 return sum;
}

long int reduce_openacc(const int * __restrict__ const host_ptr,
                        const int num_elements)
{
 long int sum = 0;

#pragma acc kernels
 for (int i=0; i < num_elements; i++)
```

```
{
 sum += host_ptr[i];
}

return sum;
}
```

正如看到的一样，我们所做的是将 OpenMP 指令替换成 OpenACC 指令。然后我们使用供应商提供的 OpenACC 编译器进行编译。这可能生成介于高级的 CUDA 与原始 PTX 之间的代码。通常，接下来会调用 NVCC 编译器生成目标 GPU 代码。一些供应商还支持 NVIDIA GPU 之外的其他目标代码。

在编译阶段，大多数供应商编译器提供关于他们如何将串行代码转换成设备代码的数据。然而，这有点像 NCC 中的 -v 选项，因为需要理解编译器的提示信息。这里我们看一个 PGI 编译器输出的例子。

```
Accelerator kernel generated
60, #pragma acc loop gang, vector /* blockIdx.x threadIdx.x */
CC 1.3 : 21 registers; 1024 shared, 20 constant, 0 local memory bytes; 100% occupancy
CC 2.0 : 23 registers; 1048 shared, 40 constant, 0 local memory bytes; 100% occupancy
```

为了理解该输出结果，需要理解 OpenACC 术语与 CUDA 术语是如何对应的（参见表 10-6）。

表 10-6 OpenACC 和 CUDA 术语

OpenACC	CUDA
Gangs	Blocks
Workers	Warps
Vectors	Threads

第一行说明内核占用了 60 个 gang（CUDA 术语中的线程块）。然后指出它分别为“CC 1.3”和“CC 2.0”产生了输出（计算能力 1.3 和 2.0 的设备）。它同样告诉我们所使用的寄存器个数、每个线程块使用的共享内存的字节数、每个线程块内常量内存的字节数以及溢出到本地内存的那些寄存器。

最后，它会基于内核正在使用的寄存器个数和共享内存量，计算达到接近 100% 占用率时理想的线程数（OpenACC 调用这些向量）。然而，对于给定的内核 / 数据模式，它不一定总是选择最佳值。指定这些可以让我们覆写或部分覆写这样的选择。

它会查看你的数据，然后决定最佳的启动参数（线程数、线程块数、线程网格数等）。同样，它会自动地尝试将数据分配到常量内存和（或）全局内存。如果你愿意，可以自由地覆写这些选择。

为了覆写主机上的镜像全局数据的默认行为（自动后台更新命令），你需要指定如何对数据进行管理。可以使用下列代码实现：

```
#pragma acc data <directives>
```

其中 <directives> 可以是下面之一，另外还有一些其他我们没有提到的却更复杂的形式：

copy（data、data2、…）——通过在内核开始时复制回、在内核结束时复制出（默认行为）

来维护一个等功能的 CPU 版本。

copyin（data1、dara2、…）——仅仅将数据复制到 GPU 且不将其复制回来，也就是说丢弃 GPU 数据。对于 GPU 将要处理的只读数据，这是很有用的。

copyout（data1、data2、…）——仅从 GPU 将数据复制回 CPU。对于在 GPU 声明输出数据是很有用的。

create（data1、data2、…）——在 GPU 上分配临时存储而无须在任何方向上执行复制操作。

present（data1、data2、…）——数据已经存在于 GPU，因此不需要再复制或分配新的。

请注意，OpenACC 模型希望使用 C99 标准，特别是使用 C 语言中的 __restrict__ 关键字，来指定任何已使用的指针不是其他指针的别名。如果不这样做很可能导致你的代码无法向量化。

可以通过使用 PGI_ACC_TIME=1（供应商特定的）选项添加数据指令帮助。（在 PGI 编译器的情况下）这会启用分析操作。然后它会告诉你内核调用的频率、内核的线程块维度和花费的时间以及最终传输数据所花费的时间。其中后面一部分经常是最重要的，并且是数据子句可以帮上忙的地方。同样可以使用 Linux 上可用的标准分析工具（例如 Visual Profiler）查看 OpenACC 编译器实际上是怎么工作的。通过这样做你可能会发现一些原本没有意识到的错误。

能够参于线程块大小的选择，那么我们也能对其执行特定的优化。例如，可以制定比数据元素更少的线程块和线程。默认情况下，虽然标准上没有任何的说明，但 OpenACC 编译器试图为每个元素选择一个线程。因此，如果想让每个线程处理 4 个元素（之前我们看到这样效果很好），可以通过指定较少的线程块和线程实现：

```
#define NUM_ELEM 32768
#pragma acc kernels loop gang(64), vector(128)
for( int i = 0; i < NUM_ELEM; i++ )
{
 x[i] += y[i];
}
```

这里我们指定该循环应该使用 64 个线程块（gang）且每个线程块有 128 个线程（vector）。因此，我们在设备端有 8192 个活动线程。假设一个诸如 GTX580 这样有 16 个 SM 的设备，那么每个 SM 有 4 个线程块，每个线程块有 128 个线程。这等价于每个 SM 有 16 个线程束，对于达到 GTX580 的理想占用率仍是太少了。为了解决这个问题，我们需要增加线程块（gang）个数或线程（vector）个数。

你可能希望每个线程处理多个数据元素，然而这取决于特定的算法而不是增加线程块或线程的个数。只要编译器知道了元素的个数（就像之前的例子一样），它就会让每个线程处理多个元素，这里是 4 个。

同样要记住（和常规的 CUDA 一样），线程实际上是以线程束为单位运行的，也就是每 32 个线程一组。分配 33 个线程实际会在硬件上分配 64 个线程，其中 31 个线程仅仅浪费设备的空间资源而不会做任何事。一定要以 32 的倍数为一组分配线程块（OpenACC 中的 vector）。

CUDA 同样适用，如果指定了 gang 或 vector（线程块或线程），虽然很多情况下不需要这么做，那么常规的内核启动规则也适用。因此一个线程块能够支持的线程数是有限制的，

这个数目根据目标硬件的计算能力而变化。通常，我们发现 64、128、192、256 在计算能力 1.x 的设备中运行效果很好。而 128、192、256、284、512 在计算能力 2.x 的设备上运行效果很好。对于计算能力 3.x 的平台来说，256 是最佳值。

然而，当添加一些限定符时，我们要考虑未来硬件的可能影响以及该影响会如何限制其他加速器的使用。如果不指定任何东西，那么编译器就会选择它认为最佳的值。当每线程块有更多线程、每 SM 有更多线程块的新 GPU 出现的时候，只要供应商更新了编译器来适应它，仍会工作正常。如果确实指定了这些参数，则应该将其指定为当前最大值的某个倍数，这样代码在未来的设备上运行时就不会用尽线程块。

默认情况下，OpenACC 模型使用同步内核调用。也就是说，主机处理器将会等待 GPU 完成，然后在 GPU 内核调用返回后才能继续执行。这类似于在 C 中进行函数调用，而不是生成一个工作从线程，随后进行汇合。

现在你应该意识到，尽管使用这一方法编写初始程序很好，但只要有可能（应用程序运行得很好），我们就应该使用异步模型替代它。机器中很可能有一个合适的多核 CPU，那么在 GPU 进行后台计算的时候，可以充分地利用该 CPU。应优先分配到 CPU 的是类似内存载入 / 载出这样不需太多计算的操作。

GPU 的内存传输瓶颈是 CPU 上归约操作的执行速度能够比 GPU 更好（或者至少是相当的）的原因之一。为了计算 GPU 上的数据，我们可以直接在 GPU 上生成或者是通过 PCI-E 总线传入。如果通过总线传输了两个数据项，而仅仅是为了执行某些简单的操作（比如相加操作），那么就算了吧，在 CPU 上执行就行了。PCI-E 传输的代价，远远超过了此场合下能考虑到的其他任何代价。对 GPU 来说，最佳的任务是那些计算密集的区域或者 GPU 额外的内存带宽能够起很大作用的地方。

因此，OpenACC 为内核和数据提供了 async 子句以允许它们在主机端异步地运行并使传输与主机端操作异步执行。

```
#pragma acc kernels loop async
for (i=0; i< num_elem; i++)
{
...
}
```

异步传输需要锁页内存，也就是说不会交换到磁盘上的内存。在 OpenACC 中，不需要像在 CUDA 中那样特别关心这些。指定 async 子句将会导致 OpenACC 编译器在传输的底层使用锁页内存。当然，使用异步操作时要记住的一点是，在异步操作完成之前，不能交换传输到内核或内核操作的数据。

一旦人们掌握了异步通信并且能够在一个单核 CPU/ 单 GPU 上获得他们所能达到的最佳性能，那么一个明显的问题是能否使用多 GPU 对应用程序加速。答案当然是肯定的。通常情况下，如果是在一个节点内使用多 GPU，会看到接近线性的扩展性。

OpenACC 标准仅支持“每个 GPU 一个 CPU 线程”形式的单个节点上的多 GPU。如果打算在 CPU 上执行某些工作（这是很合理的），这可以充分利用多核 CPU 的潜力。因此，使

用 OpenACC 只需要简单地使用 OpenMP 指令启动一些线程就行了。

```
#pragma omp parallel num_thread(4)
```

假设使用一个 4 核 CPU 和 4GPU 的显卡，那么可以向 OpenACC 指定希望当前的线程使用某一特定的 GPU。

```
#pragma omp parallel num_thread(4)
{
   const int cpu_thread_id = omp_get_thread_num();
   acc_set_device_num( cpu_thread_id, acc_device_nvidia );
}
```

如果你的系统中只有两个 GPU，那么最好为 OpenMP 指定两个线程。如果希望使用 4 个线程（但是只有两个由 GPU 使用），则可以这样做：

```
const int num_gpus = acc_get_num_devices( acc_device_nvidia );

#pragma omp parallel num_thread(4)
{
  const int cpu_thread_id = omp_get_thread_num();
  if (cpu_thread_id < num_gpus)
  {
    // Do CPU and GPU work
    acc_set_device_num( cpu_thread_id, acc_device_nvidia );
  }
  else
  {
       // Do CPU only work
  }
}
```

我们可以使用 MPI，通过下列代码实现相同的目的：

```
const int num_gpus = acc_get_num_devices( acc_device_nvidia );

// Get my MPI virtual process id (rank)
int my_rank;
MPI_Comm_rank( MPI_COMM_WORLD, &my_rank );

if ( my_rank < num_gpus)
{
// Do CPU and GPU work e.g. workers
   acc_set_device_num( my_rank, acc_device_nvidia );
}
else
{
  // Do CPU only work, e.g. master
}
```

在这里要注意一个问题，acc_set_device_num API 调用是每个主机线程执行一次的事件。这就是 CUDA 4.x SDK 之前 cudaSetDevice 调用的方式。无法在单独的主机线程上选择上下

文从而达到控制多 GPU 的目的。唯一支持的模型是每个 GPU 上下文对应一个主机线程。

注意，1 : 1 的 GPU 和 CPU 核比率的情况下，这对于大量使用的系统是很理想的。然而，较大的 GPU 和 CPU 核比率也可能很有效，因为事实上 GPU 程序很少会使 GPU 饱和。因此，可能存在 GPU 未充分利用的某些点，特别是在同步点或两个内核调用之间。在主 / 从布局的情况下（在 MPI 中是很典型的），专门留出一个不含 GPU 的 CPU 核作为主控机（master）是很有好处的。

我要在这里谈到的另外一点是内存模式。当 OpenACC 在执行全局内存结合的加速器上实现的时候，它像 CUDA 程序一样会被较差的内存布局严重影响。这里没有自动的变换。需要考虑内存布局并且创建一个对于 GPU(数据每 32 个元素一列而不是顺序的行）来说最佳的。

总的来说，OpenACC 代表着 GPU 编程中一个非常有趣的发展并且很可能为很多非 GPU 程序员打开 GPU 编程的舞台。这些人中很多会进一步学习 CUDA，因为将 OpenACC 和 CUDA 结合起来是很有可能的。因此，可以从 OpenACC 开始，如果发现某个特定的领域需要那些额外的控制，那就直接切换到 CUDA。这样，应用程序的大部分代码都不需要改变。

10.5 编写自己的内核

在本章我们已经展示了许多其他的设置选项，从在高层指定并行性让编译器执行繁重的 lifting 到使用更擅长利用硬件的人开发的函数库。你永远不会（实际上也不应该试图这样）在每个方面都是最好的。诸如编译器和函数库这样的工具能够让你在别人的帮助下达到自己的目标。你的知识只停留在自己感兴趣的领域。

作为一名专业的开发人员（即使是一名学生），也应该关注开发一个解决方案所花费的时间。开发一个最有效的并行快速排序可能是技术上的一个挑战，但是一些聪明的计算机专业的毕业生已经写出了相关的论文。如果你在进行招聘，那么很显然，应该把他们录用过来。购买知识（以人或软件的形式）可以给在竞争中带来巨大的优势。

选择一些涉及并不精通的领域内容的函数库也是很重要的。如果在开发一个图像模糊系统，那么从磁盘载入 / 保存图像就不是你所关心的。有很多开源的或是商用的函数库会涉及开发中的这些方面。

使用函数库时可能遇到的常见的问题是内存分配。大多数基于 GPU 的解决方案（如果分配内存）并不分配锁页内存。因此，一个返回载入图像的指针的图像库在你将图像数据传输到 GPU 上时会导致程序执行变慢。因此，找找那些可以让用户控制内存管理或者是支持锁页内存的函数库。

我们遇到的下一个关于指令和函数库的问题是：一般地，除非是特意编写，否则它们对多 GPU 不敏感。由于通常可以采用多达 4 个 GPU 组建成工作站，这种方法类似于只使用标准 4 核 CPU 的一个核。为了支持多 GPU 配置而进行的编程不是无关紧要的，但也不是难度很大的事。我们在 CudaDeveloper 内部使用的函数库支持多 GPU 设置。它使数据处理变得复杂了并且需要更多地思考，但是确实也更加可行。

问题是需要自己编写多少代码，通常有关于性能。使用指令时，付出了一定百分比的性

能以实现更快的程序开发。通过对比，函数库可能会在减少开发精力的情况下带来显著的加速，但是其可能的副作用是减少程序的灵活性，以及执照问题。很多在商业使用的角度上被限制了，这表明如果不打算自己开发而使用函数库的话，则应该为特权功能付费。至于学术用途，只感谢其所做贡献通常就足够了。

因此，为什么要使用编写自己的内核是有很多原因的。本书提供对使用 CUDA 编写内核时遇到问题的很好见解。无论是自己编写内核还是抽象为别人的问题，这些基本原则（合并内存访问、充分利用硬件、避免资源竞争、了解硬件局限，数据局部性原理）都是适用的。

在本章，我们已经提到了一些 NVIDIA 提供的函数库。如果工作在这些提到的领域，那么为什么不选择使用这样的函数库呢？它们是由生产商开发的并且在他们的硬件上运行得很好。它们设计为构建更复杂的算法模块的基础。NVIDIA 的授权条款是很慷慨的，因为他们希望用户使用函数库并创建 CUDA 应用程序。如果你认为 CUDA 的更广泛使用意味着更多的 GPU 售出这也是毫不奇怪的，当然，关于 CUDA 的知识也就变得更有价值了。

问题是这真的能够带来足够程度的性能吗？大多数人使用高级语言进行编程，是因为这比使用汇编语言的生产率更高。更优秀的程序员对于 C 语言和汇编语言的细节都很了解。他们知道什么时候使用 C 语言以获得生产收益，也知道什么时候要使用汇编编写一些函数。使用指令 / 函数库的问题大体上是相似的。当然可以所有代码都自己编写，但是除非不得不这样做，否则为什么要自找麻烦呢？

当为 GPU 编写应用程序时，一个很好的方法是首先获得 CPU 端可以运行的原型。考虑一下如何让 CPU 版本利用多核以及这是否会给应用程序带来好处。CPU/GPU 工作的平衡是怎样的呢？在需要的时候会如何在 CPU 上创建线程？然而，至少开始时，要坚持使用一个 CPU 线程和一个 GPU，但是要想一想最终想要实现怎样的目标。

现在考虑一下主机 / 设备传输。传输 – 计算 – 传输模型通常（取决于比例）无法充分利用 GPU。某种程度上，我们可以将传输和计算重叠执行，然而这也取决于需要针对的硬件。

接下来考虑一下 GPU 的内存层次结构。打算利用 GPU 的哪个位置（寄存器、共享内存、缓存、常量内存、纹理内存）？对于这种类型的数据存储你需要怎样的数据格式？

现在我们看看内核设计。分解为线程和线程块的方式会影响线程间 / 线程块间的通信量以及资源使用情况。可能会遇到怎样的串行和竞争问题呢？

一旦有一个可以工作的 CPU/GPU 应用程序，分析它并让它尽可能高效地工作。在这个阶段要特别注意正确性，最好进行一些对比自动测试。

然后，这就给我们带来了内核实现效率的问题，其中要考虑 CUDA/ 函数库 / 指令的选择。假如已经计划好了如何使用 GPU，那么你的选择会如何影响执行的效率呢？关于使用 CUDA/ 函数库 / 指令的选择会对性能带来正面的影响还是负面的影响？影响的百分比是多少？

我们考虑一下共享内存的例子。OpenACC 有一个 cache 修饰符，它告诉编译器要将该数据存放在共享内存（否则会忽视共享内存这个资源，这也可能取决于编译器提供商）。函数库很少会直接显示出共享内存的使用，但是却经常在内部高效地使用它并且通常会在文档中记录下来。新一些的硬件可能会有不同的实现。例如，Kepler 硬件可能会将共享内存配置成 32 位或 64 位宽度的，这意味着许多金融类和其他的应用程序会从这个优化操作中显著受益。

能利用这种重要的优化手段吗？如果依赖指令提供商或是函数库开发者做这个，那么他们会提供哪一个层次的支持呢？这会花费多长时间呢？如果这个函数库是某个学生编写的并且是他论文工作的一部分，除非你或其他人愿意维护它否则它是不会更新的。如果需要某个特性而指令提供商并不认为该特性有广泛的需求，那么他们并不会仅仅为了你的应用程序而进行开发。

当有一个高效的单 CPU/ 单 GPU 实现方案时，将其移植到适合你的工作量的多核 / 多 GPU 解决方案。对于以 GPU 为主而 CPU 没有充分利用的工作流来说，简单的单 CPU 核控制和所有的 GPU 异步模型能够正常工作。如果 CPU 核也加载了，那么使用多线程且每个线程使用一个 GPU 会带来什么帮助呢？在 CPU 负载没有充分利用的情况下，多核 CPU 是否做了一些无用的工作？最好的设计方案大概是最有效地使用所拥有的资源来解决某个给定的问题。

移植到多线程 / 多 GPU 的方法可能是很容易的，也可能是很费劲的。GPU 全局内存数据现在划分到多 GPU 内存空间上。现在需要怎样进行 GPU 间通信呢？ P2P 模型（如果支持）通常是这种通信的最佳方法。或者，GPU 之间的协调和传输需要由主机完成。让一个单独的 CPU 协调 N 个 GPU 可能比让多 CPU 线程协调同样的这些 GPU 更容易。

你的指令和函数库是否支持多 GPU 方法呢？它们是线程安全的吗？或者假设只有一个实例或 CPU 线程，它是否会维护一个内部状态？对数据交换和并发操作提供了什么支持？是否强制串行与每个 GPU 交换数据，并在不同 GPU 间轮流进行？或者能否同时执行 N 个传输呢？

在选择工具或函数库时，考虑一下它们是否成熟以及编写它们的目的是什么。在代码错误时会如何调试（因为这是不可避免的）。你是自己解决问题还是存在技术支持，能够负责你的漏洞修复、特征请求等问题？它们是什么时候编写的并且针对哪一种 GPU 工作得最好？对于不同代的 GPU，它们是否优化过了（或者意识到需要优化）？

通过提前思考设计并且搞清要在哪里结束，能够决定开始时需要哪一种软件和工具。可以用某种方法创建一个解决方案的原型，但是最终可能需要使用 CUDA 以获得理想的性能和效率。在软件开发中没有神秘的“银弹”。而是需要认真考虑设计方案，计划如何实现它以及理解某些特定的方法能带你走多远。

10.6 本章小结

在这一章，我们已经看到了大量 GPU 代码开发的方法。哪一种方法适合你，很大程度取决于你的背景以及目前使用 CUDA 的感受和经验。我特别鼓励你看一看 NVIDIA 提供的函数库，因为它们涵盖了许多常见的问题。

我们考察了 SDK 中大量非特定领域的例子，通过学习这些程序，每个人都可以采用并且从中受益。SDK 中也有很多特定领域的例子。我鼓励去探索这些例子。在对 CUDA 已经有了很好的了解基础上，将会获得很多额外的知识。

我希望从这一章能够认识到使用 CUDA 直接编写所有的代码，并不是唯一的途径。使用函数库能够获得显著的生产收益。指令集也允许更高层次的编程，而相对于较低层次的 CUDA 方法，人们可能更加青睐高层次的方法。由于各种原因，人们会做出不同的选择。理解对你来说很关键的标准是什么并且据此做出选择。

第 11 章

规划 GPU 硬件系统

11.1 简介

服务器一般放置在比较宽敞、特设的空调房里。通常要密封房间，以隔离它们产生的过大噪音。它们消耗的能量以十万瓦乃至百万瓦计量。计算机通常会组织成 1U、2U 或者 4U 的节点，插入一个大机架单元。这些机架往往进一步使用如 InfiniBand 的高速连接器互连，如图 11-1 所示。

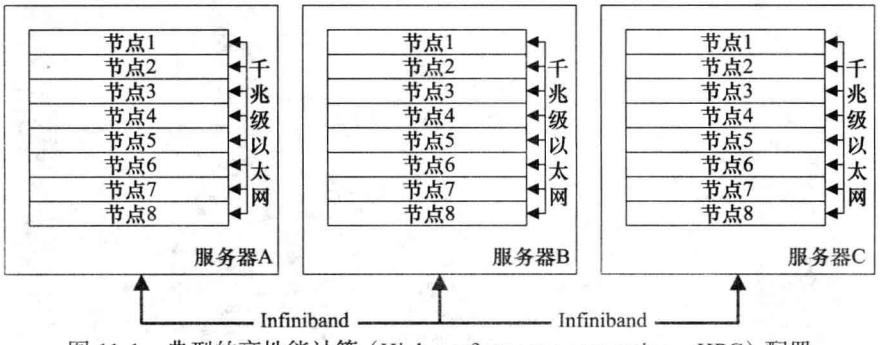

图 11-1 典型的高性能计算（High-performance computing，HPC）配置

在一个给定的服务器里，每个节点都借由一个高速交换机连接到所有其他的节点。这种连接开关与千兆以太网一样简单。大多数节点的主板附带两个千兆以太网端口：一个负责内部连接，另一个负责外部连接。所有的外部连接通向一个共同交换机。交换机本身处在诸如 InfiniBand 的高速的主干网上。

这样的组织方式有一个很有趣的属性。服务器机架内的节点间通信可能会明显快于跨机架节点间的通信。这种组织方式导致了非统一内存访问（Nonuniform Memory Access，NUMA）架构。作为一个程序员，必须处理这个传输过程。可以武断地忽略这个问题，但是这会导致低性能。需要考虑数据驻留在哪里以及节点间需要共享哪些数据。

如果查看一个多 GPU 硬件系统，会看到它与图 11-1 所示的单服务器节点非常相似。每个节点是一个 GPU 板卡，它们连接到中央 PCI-E 总线而不是千兆级以太网。每组的 GPU 卡

构成了一个更为强大的节点，它通过高速链路与其他类似节点连接，如图 11-2 所示。

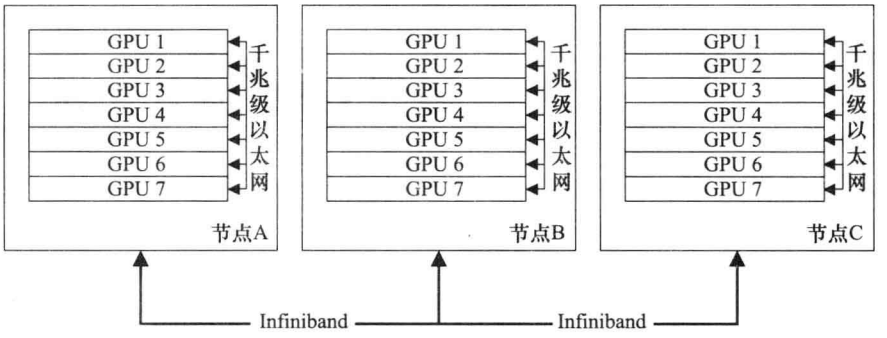

图 11-2 GPU 的高性能计算配置

请注意，图 11-2 中每个节点包含 7GPU。在实践中，这是唯一可能使用专门的机架或液体冷却的 GPU 系统。我们在 CudaDeveloper 搭建了一个同样的系统，如图 11-3 所示。

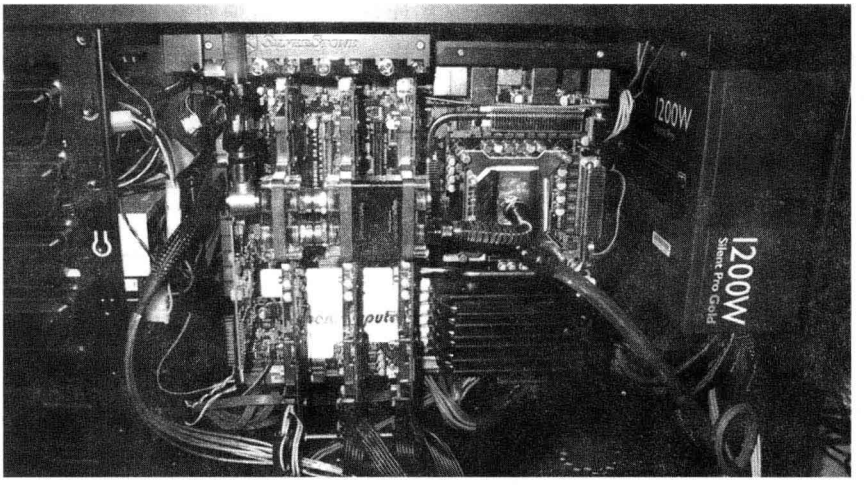

图 11-3 CudaDeveloper 搭建的液冷计算机，其中搭载了 3 个 GTX290 卡（共 6 个 GPU）

除了一些古老 G80 系列系统，大多数的 GPU 卡占用双卡槽位。多数主板支持最多 4 个 PCI-E 插槽。这意味，如果使用安装了风冷的台式机箱，每个节点仅限于 4 个 GPU。由于每个开普勒系列卡拥有每秒 3 万亿次浮点操作的处理能力，一个不用放置在远程服务器房间的桌面节点可以拥有每秒 12 万亿次浮点操作能力。

限制使用高速运算能力的主要问题之一是能量和发热问题。随着时钟频率的增加，产生的热量也相应增加。随着不断增加的热量，相同时钟频率下消耗的功率也会上升。费米型设备的温度极限略超过 212 华氏度（100 摄氏度）。系统中并排放置两个以上的 GPU，在通风不佳的环境中，很迅速就能攀升到这个阈值。

把手伸到现代 GPU 的排气孔背部，类似于把手靠近一个吹风机。利用万亿次浮点操作

的 GPU 工作站，可以把热量加大到 4 倍，一个小型办公室很快就可以免费享用热度不错的暖气系统。

580 系列的费米型 GPU 卡（GF110）引入了一种更好的均热板散热系统。由于它输出的热量更小，而沿用到 GTX680 上。这一散热系统在空心铜管里装有液体，能够迅速将热量带给散热片和风扇。它跟液冷系统非常相似，只是它的热量仍需要通过对着散热片吹风进行散发，而风扇要装在 GPU 卡的狭小空间里。保持 GPU 越冷，意味着更少的功耗和更少的发热量。然而，用风冷是有限制的，最终也会限制 GPU 性能的增长。一个典型的 480/580 系列 GPU 卡可以消耗高达 250 瓦 / 卡的能量。因此，单节点上的 4 卡系统很容易超过 1000 瓦。而后期发布的开普勒型 GTX680，功耗不到 200 瓦 / 卡；而双芯的 GTX690 控制在 300 瓦以下。

然而，一个典型的高速工作站或服务器并不是只有 GPU 部件。我们将依次讨论这些部件，看看它们如何影响整个系统的设计。在设计任何系统时，要记住一个重要原则，不管 GPU 速度有多快，总体吞吐量受限于最慢的部件。

11.2 CPU 处理器

处理器的选择主要限于英特尔和 AMD 之间。不考虑过时的处理器，可以选择最新的英特尔 I7 系列或 AMD 的 Phenom 系列。请注意，由于提供的 PCI-E 通道有限，Sandybride 架构的 1155/1156 插槽设计没有考虑在内。充分考虑上述因素，符合条件的有：

英特尔 I7 Nehalem 处理器（1366 插槽，参见图 11-4）

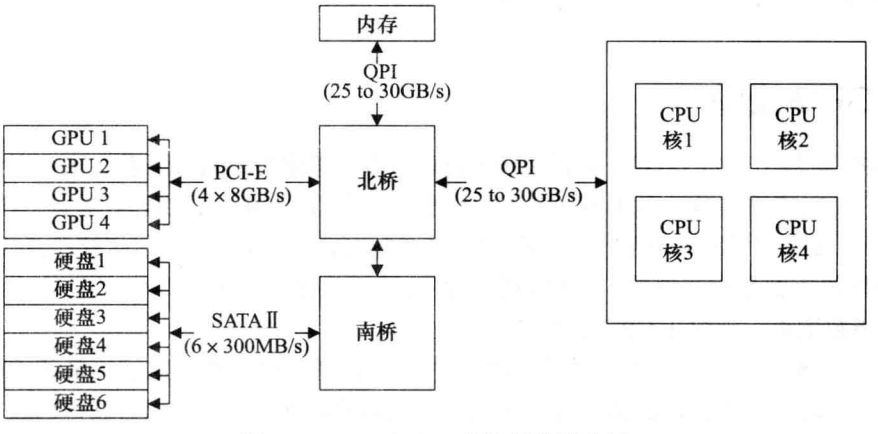

图 11-4 I7 Nehalem 架构的典型布局

- 4 ~ 6 核
- 基于快速通道互联（QPI）的 DDR3 三通道内存接口
- 125 瓦的热设计功耗
- 36 条 PCI-E 2.0 通道

英特尔 I7 沙桥 -E 处理器（2011 插槽）

- 4 ~ 6 核（最多 8 个 Xeon 变种）
- 基于快速通道互联（QPI）的 DDR3 4 通道内存接口
- 130 瓦的热设计功耗
- 40 条 PCI-E 2.0 通道

AMD 的 Phenom Ⅱ /FX

- 基于 Hypertransport 技术的 DDR-2/DDR-3 内存接口
- 125 瓦的热设计功耗
- 42 条 PCI-E 2.0 通道

从性能方面说，同等的核数和时钟频率下，英特尔处理器一般快于 AMD 处理器。从价格方面说，AMD 处理器是相当便宜的。也存在低功耗版本的处理器，这对需要保持机器持续运转的场合是很有吸引力的。然而，能够支持 4 个或更多 PCI-E 插槽的主板较少，带来的问题就是可能限制每个节点上 GPU 的数量。沙桥 -E 平台显著快于其他两种解决方案，但它在处理器和主板上的预算也会显著增加。

在需要大量 CPU 协作的应用程序中，每个 GPU 分配一个线程核。这提供了把一个线程或者进程固定分配到某个物理核上的可能。除非有 4 个以上的 GPU 或者有大量的工作量供 CPU 核处理，否则 6 核处理器中的两个核可能会浪费掉。此种情形下，I7 在性能方面是明显完胜的。当然，如果在 6 核处理器上插接 6 个 GPU 将更有优势。

另外一个选择是最近发布的 IvyBridge 架构的英特尔处理器产品线。它支持 PCI-E 3.0 标准。随着预计于 2012 年年底发布的基于 LGA 2011 插槽的 Ivybridge-E，将最终带来 PCI-E 3.0 解决方案，为 GPU 计算提供足够多 PCI-E 通道。

11.3 GPU 设备

在 GPU 机器中，每个 GPU 无疑是任何设计中需要考虑的最重要因素。GPU 的更新换代大约每 12 ~ 24 个月一次，比 CPU 端的更新略快。到目前为止，我们已经看到 GPU 的性能大约每 18 ~ 24 个月增加一倍，完全符合摩尔定律。CPU 持续符合摩尔定律已经很多年了，但是单个核的速度是有极限的。只要问题具备足够的并行性，那么 GPU 的性能应该可以持续以这一指数增长，延续在 CPU 上的多核增长。

那么，应该主要考虑 GPU 的哪些因素呢？首先，我们没必要一定使用最新的硬件。使用大致同样的能量预算，每次大的硬件换代都带来加倍的性能。如果已有硬件已经能够满足性能要求，则不必更新硬件；否则需要换用新型硬件。从 2 分钟降低到 1 分钟没什么大不了，但是从 10 小时降低到 5 小时或 10 天降低到 5 天，不论从可用性上来说，还是从功耗和空间预算上来说，都有较大不同了。

GPU 市场是由游戏玩家推动的——并行硬件能够以商用价格带给大众应该归功于他们。GPU 硬件分为两种主要类型：游戏 GPU 和计算服务器 GPU。英伟达推出的 Tesla 系列 GPU

就是针对服务器和工作站市场的。这一系列的 GPU 相较它们的桌面兄弟 GPU 具有如下优势：

- 大容量内存的支持
- ECC 内存的支持（自费米架构开始）
- Tesla 计算集群驱动程序
- 更高的双精度数学运算
- 大内存总线带宽
- 系统管理中断（System Management Interrupt，SMI）
- 状态指示灯

下面，我们看看它们是什么以及为什么它们对于服务器市场如此重要。

11.3.1 大容量内存的支持

从 GPU 上传送和传回数据是缓慢的。最好拥有 5GB/s 的双向 PCI-E 总线（总计 10GB/s）带宽通向主 CPU 内存。GPU 的内存越大，可以在 GPU 存放越多的数据。这可以避免频繁地向 GPU 传送和传回数据。Tesla 卡通常配备 4GB ~ 6GB 的内存。随着费米架构的引进，我们终于摒弃了 32 位内存空间的限制，允许 GPU 拥有多达 6GB 的内存。如果每个 CPU 连接 4 个 GPU，不用违反多数服务器主板对于内存大小的限制，很容易就可以拥有总计 24GB 的内存。

11.3.2 ECC 内存的支持

ECC 内存是在服务器环境中使用的一种特殊类型的内存。在这种环境下，内存很容易出错。普通内存面对大量的电磁干扰，其内存单元很可能改变为随机值。设备周围的电子密度越高，则产生越多的电磁辐射和越高的错误率。每个机架放入多个 GPU，而机架又同其他机架并排放置，就会产生可观的电子噪声。CPU 端服务器已经使用了 ECC 很多年。ECC 可以同时检测和纠正内存错误，使得它非常适合服务器类型的环境。

GPU 内存数据的错误对于游戏玩家来说无关紧要，通常会完全忽视。它可能会导致一个错误的像素，或一个不应出现的物体。然而，由于帧缓冲通常每秒把整个画面重新绘制 50 ~ 60 次，因此很难看出某个出错的孤立像素。

然而，当转向计算领域，内存数据的错误意味着输出数据集中有一个甚至多个元素的答案是错误的，这显然是不能接受的。这一问题或者可以使用 ECC 或每次运行两次计算来检验结果。后一种方式，需要双倍的硬件。这实际上意味着需要最初投资和运行代价的两倍，显然这一方式不是最佳的方案。

11.3.3 Tesla 计算集群驱动程序

这是仅支持 Tesla 卡的驱动程序。Tesla 卡没有图形输出，专为计算而设计。由于需要支持图形接口，内核调用将会带来相当大的开销和延迟。通过移除图形功能，TCC 驱动程序会带来明显超过标准 GeForce 驱动程序的性能提升。也有若干硬件部件是专供 Tesla 设备使用的，如 ECC 和双 PCI-E 复制引擎。

TCC 驱动程序包含在标准的英伟达驱动程序下载包中，但只在基于 Tesla 的硬件上有效。

11.3.4 更高双精度数学运算

由于大多数游戏很少涉及双精度数学运算，因此费米系列卡在每个 SM 中的两个双精度单元禁掉了一个。因此，标准 GeForce 费米卡的双精度性能相当于 Tesla 同等卡的一半左右。单就单精度浮点运算性能而言，GeForce 跟 Tesla 相当，而且在许多情况下，得益于它更高的时钟频率，可以比 Tesla 更快。但是，如果双精度浮点运算对于你的应用程序非常重要，如在许多金融应用程序中，那么只安装 Telsa 型 GPU 是有道理的。

11.3.5 大内存总线带宽

Tesla 卡，作为最高端的 GPU 卡，通常全部的 SM 都是有效的。英伟达对服务器级别的 GPU 卡收取更多的费用，所以可以按照起作用的 SM 数目筛选 GPU。那些包含无效 SM 的 GPU 卡可以处理成更便宜的 GeForce 卡，即使禁掉一个或两个 SM 单元，对整体游戏性能的影响并不大。

让所有 SM 都有效，意味着全部总线带宽都可以用来从 GPU 卡上的全局内存传输数据。由于内存带宽往往是很多算法的唯一限制因素，因此采用 512 位宽对比采用 448 位宽，将带来明显差异。GeForce 285 和 275 卡同属较老的 G200 系列，前者拥有更大的总线带宽，带来额外的成本，但相对后者也有一定的性能增长。GeForce 480 和 580 卡也有同样的问题，从 320 位宽到 384 位宽，单就内存总线带宽而言提高了 20%，更不要说额外的 SM 单元了。专为通用计算设计的开普勒架构的 Tesla K20 模型配置了 384 位宽的总线，而 GTX 680 是 256 位宽的总线。

11.3.6 系统管理中断

系统管理中断（SMI）是一个非常有用的功能，它可以通过网络远程查询设备。在一个大的数据中心，可能安装有成千上万的 GPU。CPU 节点已经存在集中管理的解决方案，加入 SMI 的支持，可以把集中管理扩展到 GPU。因此，GPU 具有响应请求并报告了一些对于中央管理系统有用的信息。

11.3.7 状态指示灯

Tesla 卡的背面有一些指示灯，可以显示此卡的状态。除了 GeForce 295 卡之外，所有的标准 GeForce 卡都不存在状态指示灯。这些指示灯允许技术人员信步走在这些 GPU 之间，识别出失败的 GPU。在有上千个 GPU 的数据中心，可以快速断定任何节点是否有问题，这将会大大方便看管这些系统的 IT 员工。

11.4 PCI-E 总线

英特尔系统采用的北桥 / 南桥芯片组设计。北桥芯片本质上是一个高速开关，连接所

有的高速外设。南桥的速度更慢，处理所有普通的外围请求，如 USB、鼠标、键盘等。在 AMD 处理器系统上，以及英特尔后期的设计，PCI-E 总线控制器的某些功能集成到了 CPU，而不是作为一个完全独立的设备。

在英特尔 I7 Nehalem 系统上，总计有 36 线 PCI-E 总线带宽（在沙桥 -E 上是 40 线）。它们每 16 线一组，形成单个的 PCI-E 2.0 X16 链路。这条链路就是 GPU 使用的，在任一方向提供 4GB/s 的带宽。单个 I7 或 AMD 处理器最多支持两个 GPU 运行在完整的 X16 模式。当添加更多的 GPU 时，每个 GPU 可用的通道数量就会减少，相应地分配给每个 GPU 的带宽也会减少。如果使用 4 个 GPU，运行的链路将是 X8，即任一方向 2GB/s 的带宽。

大多数主板不支持 4 个以上的 PCI-E 插槽。然而，有些主板可以使用英伟达特制的多路分线器（NF200）拆分出更多的通道，达到支持 4 个 PCI-E 插槽的目的，如华硕 supercomputer 主板就是一个这样的例子，该主板支持 7 个 PCI-E 插槽。

在设计系统时，请记住，其他设备也可能需要占用 PCI-E 总线。如图 11-3 所示包含 6 个 GPU 工作站，在最后一个 PCI-E 插槽里安装了一个 24 通道的 PCI-E RAID 卡。其他系统可以在空闲 PCI-E 插槽上插上 InfiniBand 或千兆以太网卡，因此并不是只有 GPU 可以使用 PCI-E 总线。

现在许多主板也支持 PCI-E 3.0。这将大大提高目前每个 GPU 可以使用的总线带宽，因为每条 PCI-E 3.0 通道的带宽是 PCI-E 2.0 的两倍。然而，仅开普勒产品线的显卡支持 PCI-E 3.0。

11.5 GeForce 板卡

GeForce 卡可以作为 Tesla 卡的替代卡。Tesla 卡瞄准的是服务器和企业级市场。如果一个学生或自学 CUDA 的工程师，无法通过学校或者公司使用到 Tesla 卡，那么使用 GeForce 卡完全适合开发 CUDA 程序。如果正在开发消费级市场，显然正需要在这些卡上进行开发。

消费级 GPU 卡之间的差别主要在计算水平上的不同。目前，购买的几乎所有 400 或 500 系列的卡，将包含一个费米型 GPU。600 系列卡大多是基于开普勒架构的设计。如果偏偏需要较老的卡，则前一个代次（计算能力 1.3）是 200 系列。计算能力 1.1/1.2 的卡通常是编号为 9000 系列的卡。最后，计算能力 1.0 的卡通常是 8000 系列，实际上，在这种老卡上编程相较现代的卡是相当困难的。

在同一代次的 GPU 卡里，SM 的数量和全局内存的大小会有变动。在购买 GPU 卡时，应保证它至少有 1GB 的内存。当前，GeForce 卡内存的最大容量为 4GB。事先有个心理准备，多数的 GPU 卡要比一般的安静 PC 嘈杂。如果无法忍受噪音问题，可以选择一个不那么强大的卡，或者选择如微星 Frozr 系列的定制散热器。请注意，500 系列卡一般比 400 系列卡安静，因为它们对硅晶体作了调整，降低了功耗和发热量。开普勒架构的 GPU 卡由于发热量更少，往往比 500 系列卡略微安静。然而，正如任何事情，有失才有得。因此，在同一系列卡（如 GTX560、570 和 580 等）里，价高的一般比低端的安静些。

在 GPU 卡的设计方面，几乎所有的卡都是基于英伟达标准布局。因此，它们的主体是相同的，差别在于生产商品牌、配件和附带的软件。而在较高端的 GPU 卡上，有所例外，生产商

要做些实际的创新，技嘉的 SOC（超级超频）品牌也许是最好的例子。单风扇的原装散热器替换为三风扇散热器。经过速度分级，筛选出那些可以在更高速度（通常超频 10%）上稳定运行的 GPU。电源电路已重新设计，可以提供额外的能量，以驱动 GPU 在此规格下稳定地运转。

在低端卡方面，GTX520/GTX610 是最便宜的一类，售价不到 50 美元（约 30 英镑或 35 欧元）。它不需要任何特殊设计的电源连接器，适合安装在任何 PC 上。它们是合适 CUDA 开发的、理想预算的 GPU 卡。

考虑液体冷却（液冷）系统，Zoltac Infinity 版 GPU 卡可能是最有用的。它带有一个密封好的、配备齐全的液体冷却系统，类似 CPU 的某些冷却系统。因此，所需做的只是取下已有排气风扇，替换成提供的散热器和风扇。这是理想的单卡解决方案，但对多 GPU 系统不太理想。POV TGT 发布的 Beast GTX580 液冷版配备了 3GB 的内存并配备水冷头，可以很容易地连接其他模块。配备了液冷的 GPU 卡也可从 EVGA、微星、PNY 购买。

11.6 CPU 内存

CPU 内存似乎不必特别在意。然而，任何数据的传输必须来自内存，并最终返回给发送者。鉴于 PCI-E 2.0 带宽在双向中任一方向最大可达 4GB/s，每个 GPU 卡最多可以使用 8GB/s 的内存带宽。

需要的带宽在很大程度上取决于使用的数据结构和需要在 GPU 卡上保存多少数据。你面对的可能是输入规模很大但输出很小的问题，或者相反。

假设输入和输出规模比较均衡，采用 3 个 GPU 卡（共 24GB/s 的峰值带宽），即使 CPU 端空置，CPU 的内存带宽也能达到饱和。使用 4 个或更多的 GPU 卡，当应用程序需要大规模的输入和输出带宽时，意味着要搭载 I7 Nehalem 服务器版或者沙桥 -E 这类具有 6GT/s QPI 总线连接器的处理器，否则无法保证每个卡都有充足的数据供应。

如果有大量的数据需要传输，那么标准 1066/1333 MHz 的内存时钟将构成多 GPU 系统的瓶颈。如果应用程序涉及少量计算，那么问题很难有所改观。I7 平台上，DDR-3 内存可以安全超频到 2GHz，而在 AMD 平台很难达到这么高。正式说来，两家的设备均不支持超过 1333MHz 的内存时钟频率。内存还带有一定的时序信息（timing information），有时简称为 CL7、CL8 或者 CL9，大致度量了对数据请求的响应时间。因此，同一条在 1066MHz 频率下时序为 CL7 的内存，也可能当作 1333MHz 频率下时序为 CL9 的内存来出售。正如大多数计算机硬件，时钟速率越高，响应时间越低，内存价格越昂贵。

特定 DIMM 内存包含嵌入信息（英特尔 XMP 认证信息）。在合适的主板支持下，这些信息可以自动用作安全地设定内存的最佳频率。当然，由于许可费用的原因，这种经过认证的内存比没有认证的同等内存更贵。

然而，请注意，内存设备也遵从如下规律：时钟速率越高，发热量和功耗也越大。通常情况下，应该为主板上的 DDR-3 内存按照每 GB 大约 1 瓦的功耗来预算。

在考虑内存速度的同时，也要考虑需要多大的内存总量。使用锁页内存是最快的传输方

式，这需要为系统中的每个 GPU 卡分配专用的内存块。对于 Tesla 卡，你可能希望传送多达该卡的最大内存容量，即 6GB 的数据。由于 Tesla 卡没有输出口（无法接入显示器），一个典型的桌面配置将采用 3 个 Tesla 卡和 1 个专用图形卡。因此，单就锁页内存而言，可能需要高达 18GB 的内存。

另外，操作系统也需要约 1 ~ 2GB 的内存自用。这样，额外的 2GB 应分配给磁盘高速缓存。综上，对于一个 3 Tesla 卡的系统，大约需要 20GB 的内存。

DDR3 内存系统在英特尔系统上通常是 3 通道或者 4 通道的；而在 AMD 系统上是双通道的。大多数英特尔系统有 4 ~ 8 个 DIMM（双列直插式内存模块）插槽；而大多数 AMD 系统具有 4 个 DIMM 插槽。通常，每个插槽必须使用同等大小的内存：4GB 的 DIMM 是相当标准的，而 8GB 的 DIMM 内存也可买到，只是后者每 GB 的售价大约是前者的两倍。因此，4 插槽的 AMD 系统中，通常会发现多达 16GB 或 32GB 的内存；而在英特尔系统上可以得到 16GB、24GB、32GB、64GB 的内存。需要注意，4GB 内存的 32 位操作系统仍然是目前最常见的消费级平台。

对于非 Tesla 卡，卡上的内存一般只有 2GB 的容量，意味着锁页内存占用的总内存量要少很多。采用 4 张卡，只需 8GB 的内存。即使使用最大支持的 7 张卡，所需的 14GB 内存也较好地保持在一般高端主板的支持容量之内。

11.7 风冷

发热和耗电问题是每个系统设计者都头痛的事。当增加时钟速度时，所需能耗增加，转而又产生更多的热量。设备越热，驱动栅极所需的能耗越大。时钟速度越快，它也越容易出问题。

CPU 设计者在很早之前就已经不再努力冲刺 4GHz 的极限频率，而是转向并行核的路线。超频发烧友会告诉你他们可以在 4GHz 甚至更高时可靠地运行系统。然而，产生的热量和能耗相对标准时钟和设备的能源使用量（power footprint）是巨大的。

GPU 一直是需要消耗大量的能源并产生大量的热量。这并不是因为它们低效，而是因为它们在一台设备上搭载了如此多的核。一个 CPU 通常会有 4 个核，对于一些高端服务器设备可以多达 16 个。当意识到高端 GPU 有 512 个 CUDA 核要冷却时，就要开始理解这个问题了。对于应该在 SM 层还是 CUDA 核层与 CPU 核进行公平比较的问题，仍存在争议。但无论使用哪种度量方式，GPU 设备的核数最终仍会是 CPU 核的许多倍。

零售 CPU 通常附带一个很基本的散热片和风扇单元。它们属于低成本的、批量生产的单元。如果把标准散热器和风扇替换为高级的单元，CPU 温度可以很容易下降 20 度或更多。

GPU 通常采用双倍高度的板罩（2 个 PCI-E 插槽大小），上层为冷却用的风扇。拆开后，通常可以看到相当庞大的散热器（参见图 11-5）。

GeForce 580 甚至带有均热板散热器的特色设计，紧靠 GPU 的铜表面充满了液体，以利于热量从 GPU 传递到散热片组。仅就冷却 GPU 而言，这是非常先进的技术。然而，其中的

一个问题是 GPU 的散热片只有被冷空气包围时才有好的效果，如果把 GPU 一个挨着另一个放置将会耗尽空气供应。

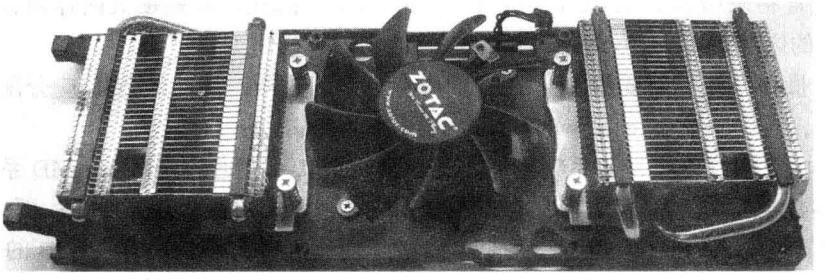

图 11-5 GTX295（双核 GPU）显卡的散热片

把 4 个 GPU 卡放置在一台标准 PC 机箱里，看起来像只气垫船。发出的热量可以代替储水式加热器。不幸的是，一旦开始装载 GPU，短短的 10 分钟就很可能进入过热状态。持续过热会导致计算错误，操作员也必须穿上 T 恤和短裤来上班。

风冷条件下，要想运行 4 块 GPU，要么需要送入事先冷却后的空气（代价高），要么购买包含定制型散热器的专用卡（参见图 11-6）。大多数服务器环境采用前种方案，服务器放置在特制空调房间里。定制型散热器解决方案更适合用于办公室工作站。然而，不能全部使用 Tesla 卡，或者至多使用 2 个 Tesla 卡，卡间留出空隙，否则期望把它们安静地放置在办公桌旁边。在更大机箱里配备如 ASRock X58 Extreme6 这样的主板，由于 PCI-E 插槽之间相距三个插槽的间距，可以保证上述系统能够正常运行，甚至可能支持 3 卡的风冷系统。

图 11-6 含 4 个 GPU 的风冷系统（多块消费级 GPU 卡）

互联网上，很多评论网站对 GeForce 卡进行评论。他们几乎全都要测量该卡的噪声输出。微星、技嘉和耕昇为风冷版 GPU 生产了一些非常有趣的冷却方案。多数硬件（GPU 或 CPU）会附带常规原装冷却器，通常应不惜一切代价避免。因为它们往往噪声过大，很难放

置在办公桌旁。花 20 美元购置一个定制的散热器系统常常让生活很清静并保持 GPU 凉爽，节省运行成本。

11.8 液冷

考虑冷却效果，液体相较空气有两个有趣的优势。它具有更高的热导性，同时具有更高的比热容。这意味着它更容易吸收热量，并带走更多热量。

液体冷却听起来像是散热问题的一个另类解决方案，但它实际上是相当实用的。早期的超级计算机中，使用绝缘液体进行降温是冷却领域的重大突破之一。例如，Cray- Ⅱ使用了一种由 3M 公司开发的 Fluorinert 特殊绝缘液体，整个电路板浸泡到其中进行降温。该液体流经整个系统，然后送到外部的冷却单元进行散热。

对于 GPU 计算，我们前进了一小步。尽管把整个主板和 GPU 浸泡在绝缘液体里，如常用的油，是可以解决散热问题的，但它不是一个很好的解决方案。液体最终可以穿透敏感元件，从而导致系统故障。

液冷发烧友萌生出液冷组块的想法。它们是中空的铜组件，液体可以在其中流动，并不会与任何电子元器件物理接触（参见图 11-7）。可以买绝缘液体，将其放到我们的液冷系统中，即使出现泄漏事故，也能把元器件的损伤风险降到最小。

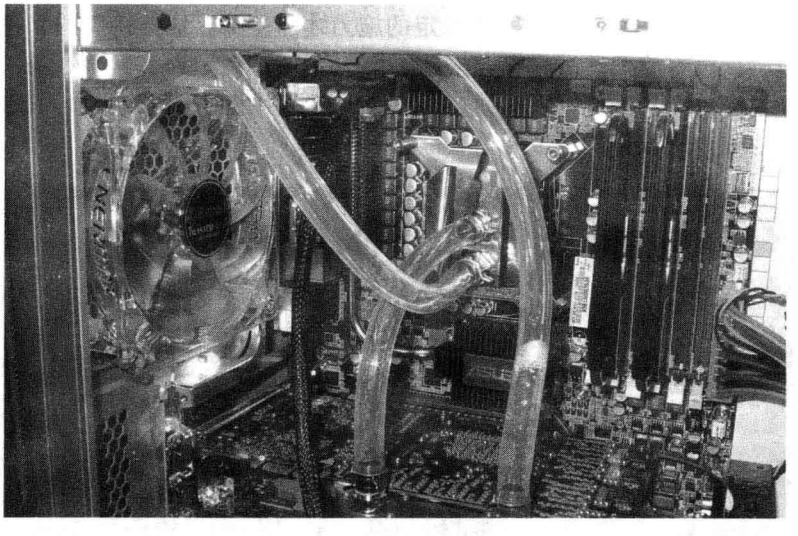

图 11-7 单个 CPU 和 GPU 水冷却回路

最新的液冷系统由多个集热器、一个 CPU 组块，一个或多个 GPU 组块，以及根据需要任选的内存和芯片组块。中空的铜组块从水箱注入液体，液体在其管道内流动。然后，被加热的液体输出到冷却系统，冷却系统通常是一个或多个散热器或热交换器，典型的系统布局如图 11-8 中所示。

上述布局存在许多变化。如图 11-8 中所示，顺序流经的单元数目越多，液体在流动过程中的阻力越大。并行流动方案试图解决这一问题，但由于液体通常会选择阻力小的路径，实际上很难保证平行路径上得到相同的流量。

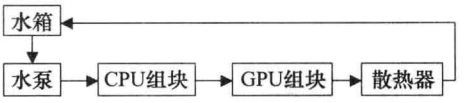

图 11-8 典型的液冷回路

液冷的主要问题是，它并没有真正解决发热问题。它只允许将热能移动到其他更易于散热的地方。因此，散热器可能是外置的一个大块头，当冷却量不大时甚至可能安装在工作站的机箱内部。

散热器是任何水冷系统的关键部件，最为关键的因素包括它的尺寸、气流的流量以及气流的温度。外置 Watercool MO-RA3 是最好的散热器之一，有 9 × 120mm 和 4 × 180mm 两种外形规格。内置的散热器最好使用能放进机箱的最大尺寸（包括高度、宽度、深度），并应该充分排出气体。尽可能地考虑温度升高方面的物理定律。把散热器安装于顶部，往往是最好的解决方案，但在初始灌装系统时需要一些方法来清除残留的空气。水泵应尽可能放低，水箱应尽可能放高，以确保水泵能一直泵出液体而不是空气。考虑下如何填充和清空这样的系统，以及空气会聚集在何处。通常，这样的系统要标记出液体排空点和空气排空点。

液冷接头有多种尺寸。大多数液冷系统使用 G1/4 螺纹接头。它们的管间内径（Intertube Diameter，ID）为 10mm。因此，常常采用 13mm/10mm（内径为 3/8 英寸）的水管。所标示的规格中，第一个尺寸是外径（Outertube Diameter，OD），然后是内径。常见的接头有三种，分别是宝塔型（barb）、快插型（push fit）和快拧型（compression-type fitting）。对于快拧和宝塔型接头，即使没有使用管箍，也需要一定的施力才能移除。快拧接头的管箍要滑过倒钩，旋紧到位，以确保没有旋开顶端时不会松动。相反，宝塔接头使用一个软管夹，没有管箍那么紧，但往往更容易在较小机箱内移动。快拧接头是其中最不可能发生泄漏或者松动的连接器，值得大力推荐，请参阅图 11-9。

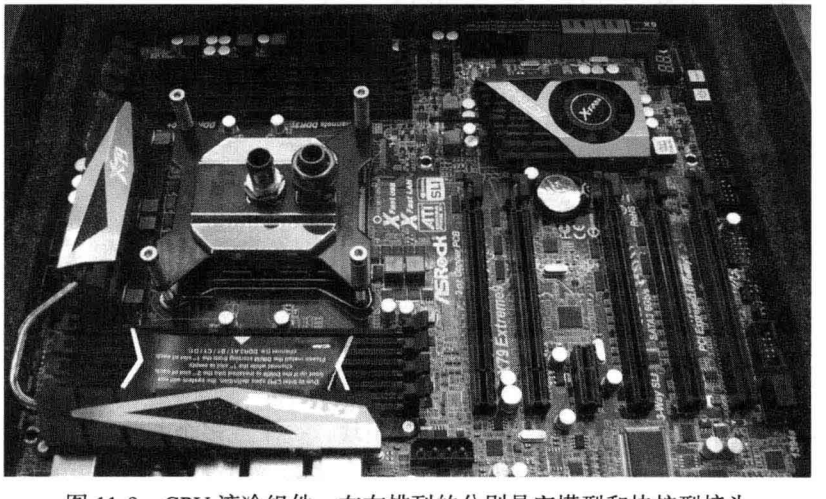

图 11-9 CPU 液冷组件，左右排列的分别是宝塔型和快拧型接头

至于冷液，有很多人使用各种预混合的液体。它们通常包含必要的抗菌剂，以防止藻类生

长。有些是绝缘的，虽然大多数在某种程度上至少有一些导电性。虽然可以使用蒸馏或去离水，但一定不能使用自来水，因为它的成分比较复杂，里面包含液冷回路中不希望包含的东西。

系统中的多 GPU 必须连接在一起。这是通过专用冷头组块实现的，比如 AquaComputer 型孪生连接器或其他相似系统。组块中包括一个硬质塑料连接器，所有的显卡以 90 度角相接。鉴于这些装置不仅提供了良好的固定效果也保证了显卡间的正确间距，它们比金手指类型的“速力”(Scalable Link Interface，SLI）连接器更受欢迎，请参阅图 11-10。

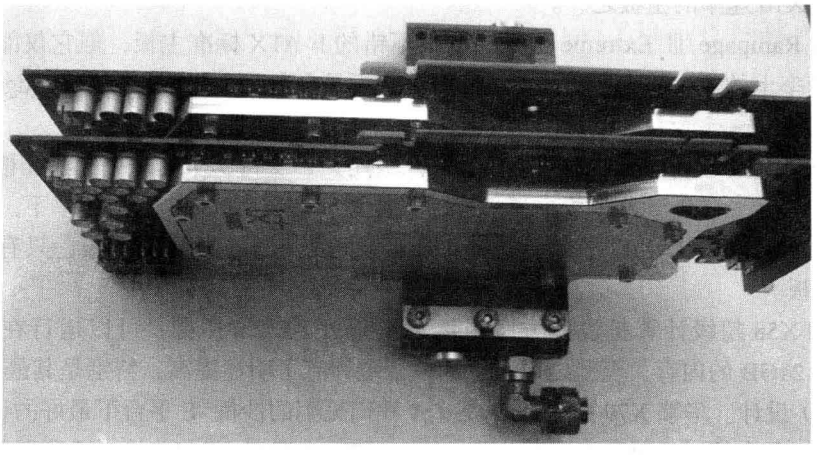

图 11-10 安装在硬质连接器组件上的孪生液冷 GPU 卡

液冷系统的主要优点是，它允许创建一个几乎静音的工作站。而同时，它的冷却效果远胜气冷系统，这反过来意味着更低的功耗。它还允许时钟速度增加到超出原始设定的规格，即所谓的超频。这类超频的 GeForce 显卡，在运行单精度任务时，轻松地超过工作站和服务器环境的 Tesla 卡约 20% 以上的性能。甚至可以购买很多现成可用的液冷版本的显卡，或者带有液冷组件或者一整套密闭系统。

液冷系统也有两个不足之处。首先，尝试全部的组件需要额外成本和精力。其次，存在冷液泄漏的风险，这种风险一般只在系统最初安装时存在。由于大多数液体每年必须更换，导致维护费用也较高。

11.9 机箱与主板

有意搭建属于自己 GPU 系统的人需要把设备放置在一定规格的机箱内。机箱必须要有合适的尺寸，选取的准则主要取决于期望安插几个 GPU 以及所采用主板的尺寸。大多数的主板是 ATX 或者 E-ATX 标准的，这意味着它们可以放于大多数台式机箱内，然而也有一些小的机箱，不支持 E-ATX。

许多支持 4 个 PCI-E 插槽的主板超出 E-ATX 规格范围，EVGA 公司的主板是典型的例子。EVGA 销售的唯一一种双 X58 主板，称为 EVGA Classified SR-2，它支持两个基于 Xeon

的 Nehalem I7 处理器和高达 48GB 的内存。但是，一旦选择了这样的主板，可供选择的配套机箱只有少数几个型号可用（查看最新的支持列表，参见 EVGA 网站 http://www.evga.com）。

华硕公司是最早为 CUDA 计算平台定制主板的厂商之一，这种类型的最早主板是 P6T7 WS 超级计算机主板。它支持 X58 平台（Nehalem I7），可用的双槽间距的 PCI-E2.0 插槽有 4 个，可以全速运行在 X16 PCI-E 2.0 下。值得注意的是，此板是紧凑式电子结构（Compact Electronics Bay，CEB）规格，通常适合大多数 E-ATX 机箱，它是少数几个支持所有 4 个插槽都运行在 X16 速率的主板之一。

华硕的 Rampage Ⅲ Extreme 也是一款很不错的 E-ATX 标准主板，但它仅能支持运行在 X8 PCI-E 速率上的 4 个显卡。华硕 Extreme V 主板是少数几个能够兼容 Ivybridge 架构 PCI-E 3.0 插槽的主板之一。

微星公司生产的 BigBang 系列主板针对高级用户，支持 7 个物理 PCI-E 插槽。然而，当插满 4 张显卡后，它会与大多数主板一样，只能运行在 X8 PCI-E 总线速率下。微星推出的微星 890FXA-GD70 是 AMD 平台上支持 4 个双槽间距 PCI-E 插槽的主板，只有少数厂商推出过类似主板。

华擎的 X58 超级计算机设计方案提供了 4 个 PCI-E 2.0 插槽，可以运行在 X8 速率下，并支持高达 24GB 的内存。至此之后，它的设计能力有了明显提高，特别是其最新的 2011 插槽（沙桥 -E）设计。华擎 X79 Extreme9 是迄今我们见到的沙桥 -E 平台下最好的设计之一（参见图 11-9）。它支持 5 个 PCI-E X8 插槽、8 个 SATA-3 端口、PCI-E 3.0 标准以及高达 64GB 的内存。难能可贵的是，它的外形尺寸仍符合 ATX 设计规格。华擎日前发布 2011 插槽接口，Extreme 11 主板，引以为豪地支持了 7 个 PCI-E 3.0 X16 插槽。

技嘉也是一个受人尊敬的主板制造商。它出品的 UD9-X58 平台，与华硕的超级计算机主板一样，具有双 NF200 芯片，这意味着它支持 4 个全速 X16 PCI-E 2.0 插槽。它的 GA-990FXA-UD7 主板面向 AMD 平台，支持最新的 990 芯片组，提供了对 SATA-3 和 4 个 PCI-E 2.0 高达 X8 速率的插槽。

决定了主板之后，需要选择能够支持主板规格和计划采用的 PCE-E 插槽数量的机箱。标准 PC 机箱只能容纳 7 个 PCI-E 插槽，这在使用 4 个双槽 PCI-E 显卡时，会导致问题。

在选择机箱时需要关注散热和气流问题，尤其是在多 GPU 系统下。Silverstone 出品了一系列机箱，主板旋转了 90 度，因此便于 CPU 和 GPU 上的热空气直接从机箱排出。如图 11-3 所示的设计来自 Raven 的 RV02 机箱。我们发现这样的设计具有最有效的冷却效果。向上流动的空气会使机箱内部温度下降好几度。Raven 的 Fortress FT02 和 Temjin TJ11 机箱均遵循类似的设计。Raven 机箱的美学特性比较极端，要么你会爱不释手，要么你唯恐避之不及。虽然这三种机箱都相当大，但 Fortress 和 Temjin 的设计更加传统。请注意，新版本的 Raven（RV02-evolution）和 Fortress 机箱仅支持 7 个 PCI-E 插槽，而 Temjin 能支持 9 个插槽。

作为替代方案，Coolermaster HAF 和 Antec 1200 系列的机箱也有很好的气流。然而，它们只支持 7 个 PCI-E 插槽。Raven RV03 是比 RV02 更为精简的版本，它支持 8 个 PCI-E 插槽，是市场上最便宜的机箱。

液冷机箱多数是面向单个 CPU 的冷却系统。因此缺少足够的空间放置多个 GPU 的液冷配置。当运行 4 个 GPU 和一个 I7 CPU 时，系统的功率将超过 1kW，会产生大量的热量。这样的系统最好使用外置的冷却系统。预计你将需要一个 120mm 的散热器来冷却每台设备（CPU 或 GPU）。Silverstone Temjin TJ11 允许移除机箱下部的内置硬盘驱动器，换上一个 4 × 140mm 的散热和水箱组件。这也许是当前市场上最好的机箱，但同时也是最昂贵的。

11.10 大容量存储

11.10.1 主板上的输入 / 输出接口

大容量存储子系统是相当重要的部分，需要能够从系统中轻松地导入、导出数据。如果考虑到每个 GPU 同时有多达 5GB/s 的输入和输出带宽，让大容量存储供应这么大量的数据，将是一个难题。

典型的硬盘有大约 160MB/s 的最大传输速率。根据硬盘的制作特点，越接近盘中心，数据的密度越低。因此，当它的外部磁道写满，开始使用内部磁道时，数据的传输速率将降为最大速率的一半左右。

多数英特尔 I7 主板配备一个内置的控制器，最多支持 6 个 SATA 硬盘。这是南桥芯片组的一部分。南桥芯片组控制低速外设，例如键盘和鼠标等，它也处理 SATA 硬盘和网络接口。

在 SATA-2 标准定义下，每个 SATA 通道的速度高达 300MB/s。新的 SATA-3 标准，支持两倍于此的速度。内置的控制器支持多达 6 个硬盘，这意味着从 SATA 端口到主存的传输能力理论上可以达到 1.8GB/s。如果使用 SATA-2 固态硬盘，由于它的读取速度超过 300MB/s，通过连接多达 6 个磁盘可以得到一个较理想的数据输入速率。但即使采用这一方案，只能达到单个 PCI-E X16 显卡带宽的一半。

然而，还有雪上加霜的事。实际上，基于南桥芯片组的内置控制器将在约 600MB/s ~ 700MB/s 处达到峰值。这跟期望的 1.8GB/s 的总体硬盘最高速率相去甚远。对于速率仅为 160MB/s 的普通硬盘，这可能比较匹配。但对于可能不低于 SATA-2 接口速度的固态硬盘来说，标准主板的 SATA 控制器将没有多大用处。只使用 4 个固态硬盘，控制器已经成为系统的瓶颈。

更先进的主板有了一些进步，AMD 平台上目前已经完全迁移到 SATA-3 接口，而在英特尔平台上仍处于 SATA-2 和 SATA-3 接口共存阶段。SATA-3 接口相较 SATA-2 的速度增加了一倍，这意味着固态硬盘可以达到它的峰值速率 550MB/s（SATA-3 的速率为 600MB/s）。同时使用 6 个这样的固态盘，峰值速度迅速趋近与单个 GPU 匹配的速率。然而，与 SATA-2 控制器类似，多数主板的内置 SATA3 控制器的传输峰值大约在 1GB/s，因此无法支持数量较多的固态硬盘。

11.10.2 专用 RAID 控制器

为了更快地输入数据，需要借助置于 PCI-E 总线上的专用硬盘控制器。但这会跟同一总线上的图形计算卡的 PCI-E 插槽需求产生冲突。在基于风冷的冷却方案上，所有的 GPU 都是

双槽间距的显卡，为了插入一个专用硬盘控制器或者高速网卡，可能需要移除一个 GPU 卡。

对于液冷系统，则更容易些，因为每张 GPU 卡都是单槽间距的。然而，仍然会受制于 PC 机的总体功耗，通常最多 1.5kW。这意味着，至少在使用高端显卡的环境下，将有空闲的 PCI-E 插槽。

假设拥有 550MB/s 的 SATA-3 固态硬盘驱动器，为了满足单个 GPU 卡所需的 5GB/s 的输入需求，则需要 10 个固态硬盘驱动器。如果使用的 RAID 卡支持与 PCI-E 总线间的双向同步传输，那么则需要有 20 个 SATA-3 固态硬盘驱动器，以匹配一个 PCI-E X16 的 RAID 控制器的全部带宽。

因此，为了能够实时提供单个 GPU 卡所需的数据供应和数据存储的峰值要求，即使使用固态硬盘，也将需要 20 个。即使每个驱动器托架包含 6 个固态硬盘，也需要 4 个驱动器托架予以支持。

如果你考察一个高端 GPU 配置，该系统可能采用的是支持 7 个 PCI-E 总线插槽的主板，上面放置了包含 4 个 GPU 的液冷解决方案。不采用任何其他类型卡的情况下，可以让 4 个 GPU 显卡同时运行在 X8 速度（2.5GB/s 的输入速率和 2.5GB/s 的输出速率）或者让两个 GPU 卡运行于 X16 速度。

使用基于液冷的系统时，鉴于大部分液冷方案是单槽间距的，因此在显卡之间会存在空闲插槽。只要添加了一个 RAID 控制器卡，相应的显卡和 RAID 控制器卡的插槽都会下降到 X8 或者 X4 速度。除非你特意留出一个 X16 插槽给 RAID 控制器专用，我们推荐这么做。

实际包含在一个工作站规格下的驱动器托架的数目，是有物理限制的。即使是一个有 7 个 PCI-E 插槽的主板（经常称为超级计算机主板），一旦使用了 4 个基于液冷的 GPU 卡，就只剩下 3 个插槽可用了。这样还可以允许两个 RAID 控制器和一个高速网卡塞进系统中。

虽然 RAID-0 模式是用来提高速度的，但 RAID 并不只为了速度。RAID-1 支持数据镜像，所有的数据都会完全复制到另一个磁盘上。一个磁盘出现故障，系统即时切换到另一个冗余盘，不会对系统的运行产生严重影响。显然，出现故障的磁盘是需要尽快更换的。它可以避免由于硬盘驱动器出错，丢失已经计算了几周的中间数据。

在小型集群下，硬盘失效情况很少，不会构成太大的问题。而在大型集群下，存在上千块硬盘在运行，你将频繁地更换硬盘。

RAID-5 系统平衡了存储性能和数据冗余，允许数据以安全的方式分割到多个驱动器存储。硬盘组的其中的一个硬盘专门存储奇偶校验信息，如果某个硬盘出现了故障，可以用其来恢复 RAID 阵列。如果你的工作性质不允许在另一台机器上重新启动而且对于失去当前已有的中间计算结果无法接受的话，RAID 是你一定要考虑的方案。

检查点（check pointing）技术经常用来避免故障造成的影响。每隔一段时间，当前的全部数据结果要进行检查或转储为永久存储。因此，只需把检查点数据和相应的程序代码移动过去，工作就可以转移到另一个节点继续执行。在设计运行时间较久的应用程序时，应该常常考虑设置检查点系统在该应用程序中。

11.10.3 HDSL

HDSL是一家名为OCZ的公司开发的标准，该公司已在固态硬盘市场上推出多款创新产品。其中最值得注意的是RevoDrive系列，该产品本质上是数个固态硬盘驱动器，它们都放置在一张PCI-E卡上，并包含一个内置的硬盘控制器。这种原版卡可达到500MB/s级别的速率，还是比较适度的。高端卡（R4 C系列）宣称可达2800MB/s。为了实现同等的带宽，则需要一个SATA-3控制器和至少5个顶级配置的固态硬盘。

OCZ出品的HDSL驱动器也是很有趣的产品，并能通过它可以洞察未来存储的可能发展方向。它在一个标准的3.5英寸硬盘里嵌入了4个老式固态硬盘驱动器，并附有嵌入式RAID-0控制器。它使用了一个特殊的控制器卡，通过电缆直接连接驱动器接口，扩展出了4条PCI-E总线的通道。这4条PCI-E2.0通道，相当于两个方向均为约1GB/s的速率，大大优于单向SATA-3接口。

作为一项新技术，在硬盘速率与带宽匹配之前，还有一段路要走。目前，硬盘的峰值速率大约是750MB/s，与1000MB/s的链路容量相比有些差距。在硬盘出货时，附带一个单端口的X4 HDSL控制器，而双端口和四端口的X8以及X16控制器已在计划中。假设硬盘速度提高了一些，达到接口的全部带宽（随着技术的进步，这几乎是一定的），这将是一个非常有趣的、令人期待的技术。

由于硬盘本身是3.5英寸的规格，这意味着更多的硬盘可以放进同样大小的物理空间。分配两个X8插槽将支持4个HDSL驱动器，提供大约3GB/s的读/写能力。

11.10.4 大容量存储需求

除了对大容量存储设备输入速度的需求之外，我们还对总存储容量有要求。以拥有世界上最大规模数据的谷歌公司为例。据2008年的统计资料，他们每天处理20PB（petabyte）的数据。1PB是1000TB（terabyte），而1TB又是1000GB。假设当前最大的单块大容量硬盘是4TB，那么仅存储谷歌每天生产的数据就要（20 × 1000）÷ 4 = 5000块硬盘！

因此，很显然，在设计任何节点时都需要考虑大容量存储的需求。在实践中，多数大型的安装环境都使用专用的存储节点，这些节点不执行任何计算功能。因此，在另外的计算节点上只需要配置存储容量足够存放单次计算所需的数据大小即可。它们可以通过与数据中心的集群之间的高速互联链路下载数据，这意味着可以使用高速而小容量的固态硬盘。这也正是我们在搭建CudaDeveloper社区测试机所做的。

11.10.5 联网

当所考虑的系统包含不止单个节点时，联网是关键议题之一。随着便宜的商用硬件变得稀松平常，搭建节点的集群已经在大学和商业组织中变得司空见惯。配置一个小型计算机网络，是相对直接的、用于协同解决同一个问题的方案。

通常会看到两种类型的网络：基于千兆以太网的网络和使用稍快、却相当昂贵的InfiniBand网络。千兆以太网的价格非常便宜，通常附带在主板上无须额外费用，并且可以

非常容易地与 16 口、24 口或 32 口的交换机连接。有些主板提供了双千兆以太网接口，通常支持链路聚合（Link Aggregation）功能。如果交换机也支持这一功能，即允许两个物理链路合并为一个逻辑通道，则该节点提供的带宽量将增加一倍。

对于特定任务，网络的作用有多关键，在很大程度上取决于需要共享的数据量。大多数情况下，如果可以将任务限制到单个节点并采用多 GPU 的路线，将会比采用多节点路线高效得多。

诸如谷歌的 MapReduce 这类系统，面对的情况有些不同。它要处理庞大的数据，要把数据分割，分发到多个节点。MapReduce 工作在一个共享的、分布式文件系统下，使得整个文件系统看起来像一块大硬盘。数据在每个节点的本地存储上按照组块形式进行保存。MapReduce 把程序分发到数据所在的物理位置，而不是将数据传给程序所在的节点。Hadoop 是 MapReduce 的一个开源实现，它允许你搭建一个非常类似的框架，分发和调度工作任务。通常，数据集是非常大的，而程序是非常小的，因此这种方式在极大地降低网络流量方面效果显著。

使用类似 MPI 的专用通信建立系统也比较常见。只要网络通信成为程序在时间因素上的主导瓶颈，就需要升级到如 InfiniBand 这样更快的网络架构。显然，这会导致额外的成本。也可以通过巧妙的编程技巧，例如异步通信、压缩数据包等，避免额外成本。

CUDA 4.0 SDK 现在支持同节点下 GPU 间的对等通信模式。此外，GPU 可以直接与特定 InfiniBand 卡以同样的方式对话，而不需要主机 CPU 的干预。因此，对于较大规模的 GPU 装置，在网络流量起重要作用的场合，InfiniBand 以及其他更高速的互连方式可能成为一种必然。

11.11 电源选择

设计持续运行的机器时，电力消耗是个大问题。运行一台超级计算机，短短几年的运营成本往往可以等同于它首次安装的成本。当然，这种机器在其整个生命周期内的成本很容易超过原来的安装成本。

电力消耗不仅来自部件本身，还来自为保障计算机正常运行配置的冷却系统。即使是一台只有 4 个 GPU 的高端工作站，怎样让它保持凉爽也需要做些规划。除非住在气候寒冷的地域并可以把机器放置在低温的地方，这样它将很好地为你的办公室做好加热工作。将多个这样的机器放进一个房间里，那么这个房间的温度会迅速攀升到相当不可接受的水平。

为了确保计算机气温不过高以及避免因温度造成的计算操作出错，大量的电能将进一步花费到安装空调系统上，在夏季气温可达到 85 华氏度 /30 摄氏度或更高的地方，尤其如此。空调系统的运行成本是昂贵的。非常值得考虑如何以最佳方式冷却计算机系统并考虑热能是否可以在某种程度上重用。在这方面，液冷方案比较高效，因为液体可以在热交换器内循环流动，最后流入传统的暖气系统，在此过程中，不需要把两种液体混合在一起。一直没人考虑如何重用计算机装置产生的多余热量，这令人很费解。随着天然资源的成本不断增加以及公司在能源绿色化进程中与日俱增的压力，简单地把热量排出窗外不再是经济上或社会上可接受的。

如果调查下高端的 GPU 卡，会发现它们的功耗等级通常标记在 250W 附近。一个典型 CPU 的功率则在 125W 左右。因此，对一个含 4GPU 的典型系统，其功率预算可能会

如表 11-1 所示。

表 11-1 典型电能使用量

部件	数量	单位功率（W）	合计功率（W）
GPU	4	250	1000
CPU	1	125	125
内存	16	1	16
主板	1	50	50
启动盘	2	5	10
数据盘	8	5	40
外设	1	10	10
总计			1251

从表 11-1 中可以看到，此配置下每个节点的大概功率是 1250W（1.3kW）。商用的电源功率最高大约在 1.5kW。超过此界限，就只能寻求非常昂贵的定制解决方案。

GPU 的选择会导致整体功耗的巨大差异。如果考察每核消耗的瓦特数和每核的 gigaflop，我们看到如表 11-2 所示的一些有趣的事情。注意，在 500 系列的费米卡上的架构改进带来更好瓦特数和浮点数的表现。费米设备也能自动降低为相较 G80 或 G200 系列老卡更低的时钟频率，当系统空闲时，这会大大节省能耗。事实上，就每瓦 gigaflop 这一指标而言，基于 GF114 的 560Ti 系列是性能最好的卡之一。560Ti 明确面向游戏市场，具有很高的内部时钟速度，其性能达到 1.2 gigaflop，与 580 卡的接近 1.6 gigaflop 可以相比。但是，前者是在仅消耗 170W 的情况下达到的，而 580 卡则是消耗了 240W。所以 560Ti 的每瓦性能是迄今为止最好的。实际上，560Ti 是在包含 448 个核的 570 基础上设计，并于 2011 年年底重新发布，后来的 GTX680 是基于 560 设计的。双 GPU 的 690 包含了两个 GTX680 芯片，每个都是经过精心选片和频率细调等步骤的，最后该设备的功率设定为 300W，此卡在每瓦 gigaflop 指标上具有最好表现。

表 11-2 每核的十亿次浮点操作数

显卡	CUDA 核数	时钟频率（MHZ）	功率使用量（W）	gigaflop	每核的 gigaflop	每瓦的 gigaflop
430	96	700	49	269	2.8	5.5
450	192	790	106	455	2.37	4.3
460	336	675	160	907	2.7	5.7
470	448	607	215	1089	2.43	5.1
480	480	700	250	1345	2.8	5.4
560Ti（GF114）	384	822	170	1260	3.28	7.4
560（GF110）	448	732	210	1312	2.93	6.2
570	480	732	219	1405	2.93	6.4
580	512	772	244	1581	3.09	6.5
590	1024	607	365	2488	2.43	6.8
680	1536	1006	195	3090	2.01	15.8
690	3072	915	300	5620	1.83	18.7

在选择电源时，要认识到并非所有的电源都是一样的。很多便宜的电源声称具有某个额定功率，但却无法在 12V 导轨上达到这一数值，而这个电压正是这类系统主要功耗部件所需要的（来自图形卡）。此外，别的一些电源没有提供足够的 PCI-E 连接头，无法支持更多的 GPU 卡。

最值得关注的问题之一是电源的供电效率。它可以低至 80% 也可以高达 96%。其中 16% 的差异意味着每付出 1 元（欧元 / 英镑 / 瑞郎）的电费，将节省 0.16 美分。

电源按照其供电效率进行评级。那些符合 80+ 标准的电源，保证在各个负载程度下都至少有 80% 的效率。更有效的模型分别评定为铜（82%）、银（85%）、金（87%）、铂（89%）和钛（91%）等级别，这些效率均在 100% 满载时评定。在 50% 负载下的效率一般高出几个百分点，另外，欧洲 240V 的电源效率略高于美国的 115V 标准。认证过的电源列表，参见网站 http://www.80plus.org。

如果以典型的欧洲电力成本为例，也就是，每千瓦时 0.20 欧元，那么功率为 1.3kW 的机器每小时的运行成本是 $0.20 \times 1.3 = 0.26$。对应的，每天是 6.24 欧元，一个星期是 43.48 欧元。所以不间断地运行一年，仅电力成本就是 2271 欧元。这里假设有 100% 的电源效率，实践中并不存在，参见表 11-3。

表 11-3 每年电能消耗的典型成本

功率（W）	使用时间（小时 / 天）	单位成本（欧元 /kW）	每天功率（kW）	每周功率（kW）	每年功率（kW）	每天成本（欧元）	每周成本（欧元）	每年成本（欧元）
CPU								
65	24	0.2	1.56	10.92	568	0.31	2.18	114
95	24	0.2	2.28	15.96	830	0.46	3.19	166
125	24	0.2	3	21	1092	0.6	4.2	218
GPU								
50	24	0.2	1.2	8.4	437	0.24	1.68	87
100	24	0.2	2.4	16.8	874	0.48	3.36	175
150	24	0.2	3.6	25.2	1310	0.72	5.04	262
200	24	0.2	4.8	33.6	1747	0.96	6.72	349
250	24	0.2	6	42	2184	1.2	8.4	437
300	24	0.2	7.2	50.4	2621	1.44	10.08	524
600	24	0.2	14.4	100.8	5242	2.88	20.16	1048
900	24	0.2	21.6	151.2	7862	4.32	30.24	1572
1200	24	0.2	28.8	201.6	10 483	5.76	40.32	2097
1500	24	0.2	36	252	13 104	7.2	50.4	2621

如果电源效率是 80%，对于 1.3kW 的输出功率，则需要输入功率为 1.625kW 的电源，浪费掉了 325W。原本每年 2271 欧元的电费账单就增加到了 2847 欧元。如果能把电源效率提高到 92%，则只需要使用 1.413kW 的电源（比 80% 效率时降低了 212W），这时每年花费 2475 欧元。这 12% 的效率提高，每年节省约 400 欧元。这些节省的费用很容易就抵消掉了

购买高效率电源的额外费用。

如果以美国市场为例，电费会便宜些，大概是每千瓦 0.12 美分。因此，额定功率为 1.3kW 的机器在 80% 的电源效率下（1.625kW 的输入功率），每小时消耗约 0.19 美分的运营费用。换成 92% 效率的电源下（1.413kW 的输入功率），它的成本为每小时 0.17 美分。后者在每小时里节省 0.02 美分，在系统持续运行一年后将省出 175 美元。推广到 N 个节点，你将很快明白为什么许多公司在购买计算机系统时把效率作为重要标准。

当然，在自己的机器上，我们总是使用当时最有效的电源。很多公司诸如谷歌等也遵循类似的策略，使用 90% 以上效率的高效电源。能源价格会随着时间不断增加，因此上述考虑是很明智的。

就回收利用废弃的热能而言，液冷系统提供了一个有趣的方法。液冷装置的冷液在泵出过程中可以把热量带去别处，而气冷系统只能用于加热它所在的局部区域。通过使用热交换器，冷液可以使用普通水进行冷却。这些水可以泵入加热系统，甚至用来加热一个室外游泳池或其他大量的水。在大规模安装这种系统的地方，例如公司或大学的计算中心，利用其中的废弃热能来节省其他地方的暖气账单是很明智的。

许多超级计算机装置选址在较大河流的附近，恰恰是为了方便地获得冷水。其他的装置采用大型冷却塔来释放多余热量，这两种解决方案都不够绿色环保。我们已经为这些热量付了钱，如果在可以很容易把它们用于加热的条件下却无情地扔掉它们，无疑是很不明智的。

当考虑能源使用量时，我们还必须记住，程序设计实际上在决定能耗大小上起着非常大的作用。从功率角度看，最昂贵的操作是芯片的数据迁移。因此，高效地使用设备内的寄存器和共享内存将大大降低能源使用量。如果还考虑到写得很好的程序的总执行时间远少于写得不好的程序，那么重写旧程序以利用诸如更大的共享内存等新特性，可以进一步降低大型数据中心的运营成本。

11.12 操作系统

11.12.1 Windows

CUDA 开发环境为 Windows XP、Windows Vista 和 Windows 7 等平台的 32 位和 64 位两个版本正式支持。它也为 Windows HPC（高性能计算）服务器版本所支持。

由于 XP 不支持 DirectX 10 和 11，因此基于 DirectX 9.0 版本之后的某些渲染特性 XP 并不支持。支持超过 4 个 GPU 卡，可能会出现问题，不光有操作系统的原因，也与主板的基本输入输出系统（Basic Input Output System，BIOS）有关。不同版本的 CUDA 驱动程序对各种特性的支持不尽相同。但对于大多数特性，都是支持的。

使用 Windows 远程桌面是不支持 GPU 计算的，因为输出桌面所在的系统中并不包含任何 CUDA 设备。有一些其他软件包提供 SSH（Secure Shell）类型连接，能够支持上述远程

GPU 应用，UltraVNC 就是比较常用的一款软件。

Windows 平台上安装驱动程序比较简单，著名的调试工具 Parallel Nsight 也可用，它是很好用的。对于多 GPU 解决方案，使用 64 位操作系统是必须的，只有这样 CPU 可访问的内存空间才能突破 4GB 的限制。

11.12.2 Linux

CUDA 为大多数主要 Linux 发行版所支持。Linux 发行版和 Windows 发行版之间的主要区别，在于对安装者知识的预期水平。对于大多数 Linux 发行版，CUDA 驱动程序需要显式安装。针对每个主要发行版的具体安装过程请参见第 4 章。

Linux 对多 GPU 的支持比 Windows 好得多。当引导超过 4 个以上 GPU 的系统时，一些 BIOS 问题可以通过自定义 BIOS 来修正。常见的问题是，多数老版本的 BIOS 是 32 位的，因此无法把大内存的地址空间映射到多个 GPU 的内存空间。如果想尝试这种方法，可以看看 Fastra Ⅱ项目（http://fastra2.ua.ac.be/），其中使用的 BIOS 支持在单个桌面系统上配置 13 个 GPU 卡的 BIOS。

Linux 支持的基本调试器是 GNU GDB 工具包。它没有 Parallel NSight 工具包（目前也有 Linux 版本）的功能全面，但它也在稳步改进。其他常见的并行调试器已经支持多数 CUDA 调试功能，或在不断完善中以支持 CUDA 功能。

与 Windows 版本一样，多 GPU 解决方案必须使用 64 位版本，否则可用的 CPU 内存空间将限制在共 4GB 以内。而由于 Linux 操作系统所占内存量较 Windows 小得多，因此有更多的内存可以提供给应用程序。

11.13 本章小结

在本章中，我们从数据中心使用 GPU 和个人应用搭建 GPU 机器的角度，考察了搭建 GPU 计算节点的一些方面。如果你是研究人员，并且需要一个超高速计算机，那么自己从头开始搭建 GPU 系统是一个非常有用的经验。对于那些希望使用现成解决方案的用户，NVIDIA 提供了预先建立的桌面和服务器系统，测试和认证以确保可以可靠地工作。无论决定建立自己的系统还是直接购买，通过阅读本章，都会更加了解在做出购买任何硬件的重大决策之前需要考虑的问题。

第 12 章

常见问题、原因及解决方案

12.1 简介

在本章，我们将考虑一些困扰 CUDA 开发人员的问题，以及如何使用一些相对简单的做法避免或者缓解这些问题。与 CUDA 程序相关的问题通常归入以下类别之一：

- 错误使用各种 CUDA 指令而导致错误。
- 常见的并行编程错误。
- 算法错误。

最后，我们通过探讨如何开展后续的 CUDA 学习来结束本章的内容。总体来说，以 CUDA 和 GPU 编程为主题的有很多，并且还有许多在线资料可供参考。在此，我们为该阅读哪些书籍以及如何找到它们提供一些指引。同时，我们还会简要地讨论英伟达对 CUDA 开发人员启动的专业认证计划。

12.2 CUDA 指令错误

错误地使用 CUDA API 是目前为止我们看到人们在学习 CUDA 过程中最常遇到的问题。由于 CUDA API 对许多开发者来说是全新的，因此使用时出现错误是预料之中的。

12.2.1 CUDA 错误处理

在第 4 章，我们引入了 CUDA_CALL 宏。所有的 CUDA API 函数返回一个错误代码。除了 cudaSuccess 之外的所有信息都表明在调用 API 时犯了某些错误。然而，也有为数不多的例外，比如 cudaEventQuery，它返回当前事件的状态而不是错误状态。

CUDA API 本质上是异步的，这意味着在查询处返回的错误代码，可能与之前发生在某个较远地方的事件有关。实际上，它经常产生于紧临检测出的错误之前的调用。当然可以在每个 API 调用后强制同步（例如调用 cudaDeviceSynchronize 函数）。尽管这种策略对于调试可能是一个很好的选择，但是它不应该存在于任何发布版本的代码中。

每一个错误代码都能变成有潜在意义的错误字符串，而不是一个必须在 API 文档中查

找的数字。为了识别问题产生的可能原因，核查错误字符串是最先能够获取信息的尝试。然而，它依赖于程序员在主机程序中显式地检查返回代码。当然，如果在运行调试版本时，CUDA 运行时能够捕获这些异常并且进行错误提示（正如我们显式地执行 CUDA_CALL 宏一样），那就更加理想了。这对把错误引入用户程序有极大的帮助。我们在 4.1 版本的 CUDA SDK 中会看到这方面的改进。

CUDA 错误处理从某种程度上来说是不成熟的。通常，你会获得一个有用的错误信息。但是，也经常会得到一些不那么有用的消息，例如，未知的错误。它经常在内核程序调用后发生。这基本上意味着你的内核执行了一些不应该执行的活动。例如，对全局内存或共享内存中的数组末尾越界写入。本章稍后还将提及许多调试工具和调试方法，它们有助于识别这类问题。

12.2.2 内核启动和边界检查

CUDA 中最常见的错误之一是数组溢出。应该确保在每次内核调用前，对其使用的数据（无论是为了读取或写入）进行检查，以确保这些数据处于某个判定条件的保护中。例如：

```
if (tid < num_elements)
{
... array[tid] = ....
}
```

这种条件占用最低限度的时间，但是会在调试阶段节省大量的精力。你通常会看到这样的问题：你有许多数据元素不是线程块大小的整数倍。

假设每个线程块有 265 个线程和 1024 个数据元素。这样将会调用 4 个线程块且每个线程块有 256 个线程，每个线程都会对结果产生影响。现在，假设有 1025 个数据元素，那么你通常会在这里犯两种错误。其一是由于使用了整数除法而无法调用足够数量的线程，这样常常会减少所需线程块的数量。通常人们这样写：

```
const int num_blocks = num_elements / num_threads;
```

这种情况下，只有在数据元素的数目恰好是线程数的整数倍时才能正常工作。在有 1025 个数据元素的情况下，我们会启动 1024 个（4×256）线程，而最后一个数据元素没有处理。我也看到了试图“回避”这个问题的手段以及其他一些大同小异的方法。例如：

```
const int num_blocks = ((float) num_elements / num_threads);
```

但是这样并没有真正解决问题。不能使用 4.1 个线程块，因为对整数赋值后依然会缩减为 4 个块。这个问题的解决方法其实很简单，可以改用下面的代码实现：

```
const int num_blocks = (num_elements + (num_threads-1)) / num_threads;
```

这样就能确保你分配了足够多的块数。

下面我们将介绍第二个经常遇到的问题。现在调用 5 个线程块中的总共 1280 个线程。如果没有对内核中数组的访问进行这种保护，那么第 5 个线程块中除了第 1 个线程外的其他线程都将越界访问内存。CUDA 运行时很少执行运行时检查，如数组边界检查。你永远不会

看到 CUDA 暂停内核程序并且显示这样一条消息：array overrun in line 252 file kernel.cu。然而，它也不是在运行失败时不提供任何提示，这其实是最糟糕的情形。它至少以某种方式捕获错误，然后返回一条像这样的消息：unknown error。

12.2.3 无效的设备操作

另一种普遍的错误是各种操作的错误结合，最常见的是指针。当你在设备或主机上分配内存空间时，会接收到一个指向该内存空间的指针。但是，这个指针的使用有一个隐含的要求：只有主机可以访问主机指针，同样，只有设备才可以访问设备指针。然而，也有少数的例外，例如零拷贝内存，此时主机指针可以转换成指向主机内存的设备指针。但是，即使在这种情况下也应该区分使用。

由于指针是不可互相转换的，因此人们希望使用不同的类型来声明设备指针。这样可以在编译时对 API 调用进行基于类型的检查，以标记上述问题。遗憾的是，设备指针和主机指针属于相同的基本类型，这意味着编译器不会执行静态类型检查。

当然，自己定义这种类型也未尝不可。然后可以根据 API 函数开发自己的包装器函数，执行类型检查。对于那些正在着手编写 CUDA 程序的人来说，这是莫大的帮助，或许随着 CUDA 的发展我们会看到这方面的改善。第 10 章中介绍的 Thrust 库有主机向量和设备向量两个概念。它使用 C++ 函数重载来确保系统总是为给定的数据类型调用正确的函数。

标准的 CUDA 运行时会检查这种设备指针和主机指针的不正确混用，以将主机指针传递给设备函数的方式验证其是否正确。CUDA API 检查指针的来源，并且如果在没有把一个主机指针转换成指向主机内存的设备指针的情况下，将其传递给一个内核函数，CUDA API 会生成一个运行时错误。但是，对于标准 C 或 C++ 系统库来说，却不是这样的。如果对一个设备指针调用了标准的 free 函数而不是 cudaFree 函数，系统库将尝试释放主机上的这块内存，然后很可能出现崩溃的现象。主机库函数认为没有它们不能访问的内存空间。

另一种无效的操作来源于某个数据类型在被初始化前使用。这就类似于在变量被赋值前使用该变量。例如：

```
cudaStream_t my_stream;
my_kernel<<<num_blocks, num_threads, dynamic_shared, my_stream>>>(a, b, c);
```

在这个例子中，我们缺少了对 cudaStreamCreate 函数和随后的 cudaStreamDestroy 函数的调用。create 调用会执行一些初始化操作以便于在 CUDA API 注册该事件。而 destroy 调用会释放所有的资源。正确的代码如下：

```
cudaStream_t my_stream;
cudaStreamCreate(&my_stream);
my_kernel<<<num_blocks, num_threads, dynamic_shared, my_stream>>>(a, b, c);
cudaStreamSynchronize(my_stream);
cudaStreamDestroy(my_stream);
```

不幸的是，CUDA 多设备模型是基于选择设备上下文优于执行操作的。一个转为清晰的接口可能在每一个调用中指定一个可选的 device_num 参数，如果没指定可能会默认为 0。这就会出现下列代码：

```
{
cudaStream_t my_stream(device_num); // constructor for stream
my_kernel<<<num_blocks,num_threads,dynamic_shared,my_stream,device_num>>>(a,b,c);
cudaStreamSynchronize(my_stream);
} // destructor for stream
```

虽然这是从 C 移植到 C++ 的，但它提供了较为清晰的接口，因为资源会随着构造方法自动创建并随着析构方法自动销毁。当然，也可以轻松地编写一个这样的 C++ 类。

但是，无效的设备操作并不是简单地由于忘记创建它们而引起的，也有可能是在设备未使用完就销毁它们引起的。试着在源代码中删除 cudaStreamSynchronize 调用，这将会导致被异步内核使用的流销毁，然而此时内核可能仍然在设备上运行着。

由于流的异步特性，cudaStreamDestroy 调用并不会失败。它将返回 cudaSuccess，所以它甚至不会被 CUDA_CALL 宏检测到。事实上，你不久后才会从一个完全无关的对 CUDA API 的调用中获取错误。对此，一种解决方法是将 cudaSynchronizeDevice 调用嵌入到 CUDA_CALL 宏中。这样有助于识别问题的确切原因。但是，千万小心，别把它放到产品代码中。

12.2.4 volatile 限定符

C 中的“volatile”关键字对编译器进行指定，所有对某一变量的引用，无论是读或者写，必须引发一个内存引用，并且这些引用必须按照程序制定的顺序。看看下面的代码段：

```
static unsigned int a = 0;
void some_func(void)
{
 unsigned int i;
 for (i=0; i<= 1000; i++)
 {
   a += i;
 }
}
```

这里，我们声明了一个初值为 0 的全局变量 a。每次调用这个函数时，它会把 i 从 0 遍历到 1000 并且把 i 的值加到变量 a 上。在该代码未优化的版本中，很有可能每次对 a 的写入都会导致对物理内存的写入。然而，在代码优化的版本中就很少发生。

在这里优化器可以采用两种方法。首先，最常见的是在循环开始时将 a 的值加载到寄存器中，然后运行到循环结束，接着把结果寄存器以一个单独存储操作的方式写回到内存。然而，这仅仅是程序员不了解或不关心内存访问成本的一个简单例子。C 语言代码可以写成如下方式：

```
static unsigned int a = 0;
void some_func(void)
{
 unsigned int register reg_a = a;
 unsigned int i;
 for (i=0; i<1000; i++)
 {
   reg_a += i;
 }
 a = reg_a;
}
```

编译器会有效地替换成这些代码。一个较为先进的优化器可能会用一条单独的表达式展开循环，就好像它有一个恒定的边界一样。由于该表达式包含 a 和一系列常量的相加，因此所有这些常量在编译时简化成一个常量，这样就完全消除了循环。例如：

```
static unsigned int a = 0;
void some_func(void)
{
 a += (1 + 2 + 3 + 4 + 5 + 6 + 7 .......);
}
```

或

```
static unsigned int a = 0;
void some_func(void)
{
 a += 500500;
}
```

尽管有许多编译器会展开循环，但是我认为许多编译器不能产生后来简化过的代码。但是从理论上讲，如果其他一些进程在循环迭代过程中需要共享参数 a 的值，则这两种方法都有可能导致问题。在 GPU 中，这种共享的参数可能在共享内存中也可能在全局内存中。大多数这类问题对程序员都是透明的，因为对 __syncthreads() 的调用会引起任何当前线程块的共享内存或全局内存对内存的写入被隐式地清空。大多数共享内存代码通常执行一些相同的操作：写入结果，然后同步。同步操作也有助于在线程间自动分发数据。

如果程序员考虑到同一个线程束内的线程以同步的方式运行，于是省略了同步原语，那么就会发生问题。当使用归约操作且最后 32 个值不需要同步原语时，通常会看到这样的优化。这只有在共享内存附加 volatile 声明的情况下才会发生，否则编译器不会向共享内存写入任何值。

共享内存有两个目的：其一，作为一个本地的、高速的每线程内存；其二是为了促进同一块内线程间的通信。只有在后一种情况才需要用 volatile 声明共享内存。因此，__shared__ 指令并不隐式地将参数声明成 volatile。因为当编译器能够使用一个寄存器来优化其中一些的时候，程序员可能并不总是希望执行读取和写入。当一个线程束内的线程合作时，不使用 syncthread 调用是非常有效的做法，但是必须意识到，共享内存对于线程束中的线程来说已经不再是一致的了。

当通过全局内存进行块间通信时，如果没有通过显式同步，各个块所看到的全局内存也是不一致的。此时我们遇到了和共享内存一样的问题，因为编译器可能优化掉了中间的全局写入，只向内存写入了最后一个。仅涉及块内访问时，可以通过使用 volatile 关键字来解决。但是，CUDA 并不指定块的执行顺序，因此这无法解决块间的依赖问题。这类问题存在两种解决方案。首要的也是最常见的，结束当前内核程序，启动另一个内核程序，这里面隐含着所有待处理的全局内存事务的结束和所有缓存的清空。当想在同一个内核调用中执行一些操作时，可以使用第二种方案。在这种情况下，需要调用 __threadfence 原语，如果调用 __threadfence 原语的线程包含写操作，则该原语会让写操作可见于所有受影响的线程，直至操

作完成。对于共享内存来说，受影响的线程就是同一块中的各个线程，因为只有这些线程能看到共享内存被分配给了某一个给定块。对于全局内存来说，这等同于设备中所有的线程。

12.2.5 计算能力依赖函数

计算能力为 2.x 的硬件支持许多额外的、在早期硬件系统中没有出现过的函数。对于计算能力为 1.3 的设备也是如此。如果在 CUDA 编程指南上查找，可以列出许多只有在特定计算能力才能使用的函数，例如，__syncthreads_count 是计算能力 2.0 下的函数。

不幸的是，默认的 CUDA 工程（例如，Visual Studio 上的新建工程向导）使用 CUDA1.0。因此，如果安装了基于费米架构的显卡（一个计算能力为 2.x 的设备）并且使用一条计算能力 2.0 的指令来编译工程，那么编译器会相当遗憾地显示如下信息：

```
Error 1 error: identifier "__syncthreads_count" is undefined j:\CUDA\Chapter-009-
OddEvenSort\OddEven\kernel.cu 145 OddEven
```

它并没有说该函数仅在计算能力 2.0 架构下才支持，但是，至少有助于我们识别问题。它只是说该指令未被定义，这会促使大多数编程人员以为他们遗漏了某条包含语句或是做错了某些事。因此，他们被指向一个错误的方向来寻找问题的解决方案。

如图 12-1 所示，通过改变 GPU 中的 CUDA 运行时选项的属性来设置 GPU 架构级别，可以轻松地解决该问题。这会导致下列的命令行选项添加到编译器调用命令中：

```
-gencode=arch=compute_20,code=\"sm_20,compute_20\"
```

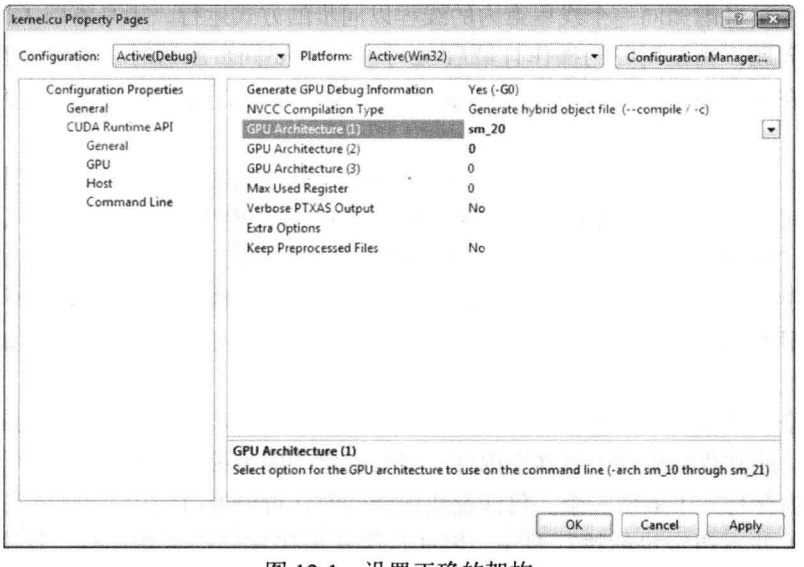

图 12-1 设置正确的架构

请注意，可以在 Visual Studio CUDA 工程创建的标准工程中最多设置 3 种架构。通过编译器预处理器，我们可以为各种不同的计算能力编写相应的代码。事实上，这也是目前为了可以使用更高的计算能力函数所使用的方法。

CUDA 定义了一个预处理器标志 __CUDA_ARCH__，目前它包含 100、110、120、130、200、210 和 300 几种取值。很显然，随着未来更多架构的定义，这些数值也会增加。因此，可以这样写：

```
#if (__CUDA_ARCH__ >= 200)
 my_compute_2x_function();
#else
 my_compute_1x_function();
#endif
```

此外，可以编写一个函数，该函数在需要时使用条件编译确定是使用计算水平高的函数还是为计算水平低的设备提供替代方案。

许多计算能力 2.x 的函数简化了编程需求，因此使开发变得更加容易。然而，大多数后来计算能力高的函数也可用计算能力低的设备通过较慢的方式或者更多的编程来实现。CUDA 通过不提供具体的实现来保证向后兼容性，CUDA 强制程序员做出这样一个选择：是不使用这些新特性，还是使用它们但是排除那些使用旧硬件的客户，或使用它们并且向使用旧硬件的客户编写自己的实现方法。

大多数客户都希望你的软件可以在他们的硬件上运行。对于一条告诉他们把 9800GT 或是 GTX260 显卡换成一个 400/500/600 系列的费米 / 开普勒显卡的消息，他们并不会太理解。大多数客户并不知道计算能力到底是什么，他们购买显卡只是为了能够运行最新版本的某个游戏。

如果工作在科研或者商业领域，那么你的硬件基本上是由机构或是公司规定的。如果你有自己选择的余地，那一定要选择计算能力至少在 2.x 或以上的硬件，因为这会使编程更加容易。然后可以完全忘掉迄今为止 GPU 的发展，并且使用一个对大多数 CPU 程序员来说更为熟悉的、基于缓存的系统进行工作。如果你有一种混合硬件，就像许多客户一样，那么就需要考虑对于每一代的硬件如何达到它们最佳的性能，并且依此编写相应的程序。

12.2.6 设备函数、全局函数和主机函数

在 CUDA 中，如果一个函数或数据项存在于 PCI-E 数据总线的主机（CPU 端）或设备（GPU 端）上时，就必须进行标识。因此有 3 种标识符可以使用，如表 12-1 所示。如果省略了这些标识符，那么 CUDA 编译器会假设函数存在于主机上并且只允许从主机上调用该函数。这是在编译时就能检测到的错误，从而容易纠正。我们也可以同时用 __device__ 和 __host__ 标识符来标识某个既存在于主机上（CPU），又存在于设备上（GPU）的函数。然而，我们并不把 __global__ 和 __host__ 标识符结合使用。

表 12-1 GPU 和主机函数

标识符	代码位置	可能的调用者
__device__	GPU	全局函数或者设备函数
__global__	GPU	主机函数，通过内核函数调用
__host__	主机（Host）	常规 C 函数调用

这种双重标识是非常有用的，因为这样可以编写在 GPU 和 CPU 上都通用的代码。你可以在一个全局函数中把什么数据交由哪个线程处理抽象出来。然后，全局函数可以调用设备函数，并且传递给它一个指向其执行任务所要使用的数据的指针。主机函数能简单地循环调用设备函数来实现相同的功能。

就设备函数和全局函数如何编译而言，设备函数和 C 语言中的静态函数类似。也就是说，CUDA 编译器希望在编译时而不是在链接时，就能看到设备函数的整个作用域。这是因为设备函数默认地内联到全局函数中。

内联是这样一个过程：该过程不需要正式参数且没有调用开销，每一个对函数的调用都被扩展，就好像被调用的函数体被包含在了调用处一样。这可能会让你认为编译器在浪费代码空间，因为程序的内存空间可能有两个同一设备函数的副本。然而，通常函数调用的上下文会允许使用额外的优化策略，所以虽然设备函数很大程度上被复制了，但它们的用法可能稍有不同。

这给程序员带来的问题是：编译器期望得到源文件。如果你想有两个内核源文件（.cu 文件）共享一个通用的设备函数，那么需要使用 #include 将 .cu 源文件包含到每个调用处而不是用声明通常的头文件的方法让链接器决定调用。请注意，在 CUDA 5.0 发布版本的 SDK 中，GPU 目标代码链接库新特性可以允许对设备内核代码进行标准对象代码生成，甚至可以把这些代码放入静态链接库。这一特性带来了更好的代码重用和更快的编译。

12.2.7 内核中的流

让一个异步的操作按你预想的工作是很棘手的，因为流模型并没有很好地反映实际的硬件，至少直到计算能力 2.1 的设备都是这样的。因此，要创建两个流，先将大量的内存副本填入 A 流，再填入 B 流。你可能会认为 A 流和 B 流是不同的，因为硬件会从各个流中隔行扫描副本。实际上，硬件只有一个单独的队列，并且按照命令提交的顺序执行它们。因此，这两个实现复制到设备、执行内核、从设备复制回操作的流将会以顺序的方式执行而不是以互相交错的方式执行。

计算能力在 3.0 及以下的消费级硬件只有两个队列：一个是为了内存副本，另一个是为了内核。在内存队列中，每一个处于队列前面的操作必须在新的操作发起前完成。这就再清楚不过了，因为一个单独的 DMA（直接内存访问）引擎在同一时间内只能进行一次传输。然而，这意味着以深度优先的方式将流填充到队列，从而使流操作串行执行。这就有悖于使用流以获取更高水平的并发内核、内存传输的目标。

解决的方法是仍然首先填充队列的深度，但是不执行把内存操作从队列复制回的操作。因此，复制和内核操作会彼此重叠执行。在输入数据大于输出数据的情况下很有效。一旦一批内核中的最后一个压入队列，所有的复制回操作则被压入传输队列。

基于 GF100/GF110 设备的费米设备（例如，GTX 470、GTX480、GTX570、GTX580、Tesla C2050、C2070、C2075、Tesla M2050/2070）有两个 DMA 引擎。然而，只有 Tesla 设备能够启用驱动器中的第二个传输引擎，这称为“异步引擎计数”。因此，在费米 Tesla 设备

上，之前提到的“深度优先”方法有改进的空间。由于不再有一个单独的传输队列，事实上我们应该以广度优先的方式向流发出命令。这极大简化了流处理，因为我们可以忽视硬件内部的处理，然后期待它像逻辑流模型预测那样工作。

然而，要注意一种硬件的优化会引发问题。硬件会根据完成的时间把连续的传输捆绑在一起。因此，启动两个内存副本后，紧接着执行两个内核调用会导致两个内存副本都必须在任意一个内核启动前完成。可以通过向内存副本之间的流中插入一个事件来打破这种模式。那么，每一个副本都会单独地处理。

12.3 并行编程问题

解决了 API 使用的问题后，下一个大多数 CUDA 开发人员会陷入的陷阱是那些更普遍的、困扰着并行软件开发的问题。在本节，我们会看到这些问题以及它们如何影响 GPU 开发。

12.3.1 竞争冒险

在单线程应用程序中，生产者 / 消费者问题是很容易解决的。仅仅是需要查看数据流，确认是否存在某个变量在写入前被读取了。许多更好的编译器高亮显示这类问题。然而，即使有了这种帮助，那些复杂的代码也可能会遇到这类问题。

只要把多线程引入到程序中并且没有预先仔细地考虑，那么生产者 / 消费者问题会很让人头痛。大多数操作系统（CUDA 也不例外）上的线程机制通过多线程运行以达到最佳的整体吞吐量。这通常意味着，多线程能以任意的顺序运行并且程序不能对这种顺序敏感。

考虑这样一个循环，该循环的第 i 次迭代依赖于循环的第 i–1 次迭代。如果我们简单地为数组的每一个元素分配一个线程，并且该线程不进行其他任何操作，那么只有在处理器按照线程 ID 从低到高的顺序每次执行一个线程时程序才能正常运行。如果改变了这个顺序或者多个线程并行执行，那么程序会中断。但是，这只是一个相对简单的例子，并不是所有的程序都会中断。许多程序有时会运行并且产生正确的结果。如果曾经发现在某些运行上得到了正确的结果，而在其他运行上得到了错误的结果，那么很有可能遇到了一个生产者 / 消费者或者竞争冒险问题。

竞争冒险，正如它的名字表明的一样，发生在多个程序段竞争某个临界点的时候，比如一个内存的读 / 写。有时线程束 0 会赢得竞争，此时结果是正确的。其他时候，线程束 1 会发生延迟并且线程束 3 会首先命中临界区，从而产生错误的结果。

竞争冒险的主要问题是它并不总是发生，这使得调试错误或是在错误处设置断点变得很困难。竞争冒险第二个特性是它对时序干扰极端敏感。因此，增加断点并且单步执行代码总是会使正在被观察的线程产生延迟。该延迟经常会改变其他线程束的调度模式，这意味着产生错误的特定条件永远不会发生。

在这样的情况下，出现第一个问题的地方不是代码所在地，而是需要后退一步了解全局

的情况。考虑在什么情况下结果会发生改变。如果你的假设与设计中线程以及块的执行顺序有关，那么其实我们已经找到问题的原因了。由于 CUDA 并不对块的顺序和线程束的执行顺序提供任何保证，因此任何这样的假设都意味着设计存在缺陷。例如，用简单的基于求和的归约方法把一个大型数组的每个数字相加。如果每一次运行都产生一个不同的结果，那么这很可能是因为这些块以不同的顺序运行。这并不是我们所期望的，顺序不应该也不可能影响最终的结果。

在这个例子中，通过给数组排序并且按照定义好的从低到高的顺序将数值加和的方法我们可以修复这个顺序问题。我们可以而且应该针对这些问题定义一个顺序。然而，某个已知同步点在硬件上实际的执行顺序是未定义的。

12.3.2 同步

CUDA 中的“同步”是用来使同一个线程块内的各个线程或者同一个线程网格里的各个线程块共享信息的术语。一个线程既可以访问寄存器空间也可以访问本地内存空间，这两个空间对于该线程都是私有的。为了使多个线程共同运行以解决同一问题，人们经常使用片上共享内存。在先前介绍的归约问题中，我们看到了一些这方面的例子。

每 32 个线程组成了一个线程束，每一个线程束对于硬件来说是一个独立的可调度单元。SM 自身包含 8、16、32、48 或更多的 CUDA 核。因此，它能在单一的时间点调度许多线程束并且可以切换线程束以维持设备的吞吐量。这也会导致一些同步方面的问题。假设一个单独的块中有 256 个线程，这相当于 8 个线程束。在一个计算能力为 2.0 且有 32 个 CUDA 核的设备上，两个线程束可以在某一时刻同时运行。有两个线程束而不是一个，这是因为硬件在每个着色器时钟周期（shader clock）实际运行两个独立的半个线程束（halfwarp）(每个 GPU 时钟周期运行两个完整的线程束)。因此，两个线程束会使程序得到提升，然而其他数目的线程束会使程序处于闲置状态。

我们假设线程束 0 和 1 是硬件最初选择运行的。SM 不使用传统的时间片方法，而是一直运行直到线程束阻塞或者到达最大运行时间。原则上来说，这都是调度器需要的。只要线程束 0 发起一个操作（算数运算或内存操作），它都会发生延迟并且线程束会被切换。如果所有的线程束都按照相同的方式运行，会产生这样一个后果：这些操作会在线程块内依次传递，每次传递一个线程束。这反过来允许指令流在 N 个线程束之间以极高的效率执行。

然而，这样的情况很少会持续较长的时间，因为一个或多个外部依赖会使某个线程束发生延迟。例如，假设线程块中的每个线程束从全局内存读取数据，除了最后一个线程束其他的都会命中一级缓存，最后一个线程束是不幸的，它的数据正在从全局内存中取出。如果假设有 20 个时钟周期的指令延迟和 600 个时钟周期的内存延迟，那么当内存请求满足时其他线程束已经进行了 30 个指令。如果内核存在循环，那么编号在 0 ~ 6 之间的线程束可能在线程束 7 之前迭代了数次。

让我们看看第 9 章中出现的这样的例子，在此我们新添加一个数据集。为了实现这一点我们在循环开始处加上如下的代码段：

```
#define MAX_WARPS_PER_SM 8
__shared__ u64 smem_start_clock_times[MAX_WARPS_PER_SM];
__shared__ u64 smem_sync_clock_times[MAX_WARPS_PER_SM];
__global__ void reduce_gmem_loop_block_256t_smem(const uint4 * const data,
                                                 u64 * const result,
                                                 const u32 num_elements)
{
 // Calculate the current warp id
 const u32 log_warp_id = threadIdx.x >> 5;

 // For the first SM only, store the start clock times
 if (blockIdx.x == 0)
     smem_start_clock_times[log_warp_id] = clock64();

 // Shared memory per block
 // Divide the number of elements by the number of blocks launched
 // ( 4096 elements / 256 threads) / 16 blocks = 1 iteration
 // ( 8192 elements / 256 threads) / 16 blocks = 2 iterations
 // (16384 elements / 256 threads) / 16 blocks = 4 iterations
 // (32768 elements / 256 threads) / 16 blocks = 8 iterations
 const u32 num_elements_per_block = (( (num_elements/4) / 256) / gridDim.x);
 const u32 increment = (gridDim.x * 256);
 const u32 num_elem_per_iter = (num_elements>>2);

 // Work out the initial index
 u32 idx = (blockIdx.x * 256) + threadIdx.x;

 // Accumulate into this register parameter
 u64 local_result = 0;

 // Loop N times depending on the number of blocks launched
 for (u32 i=0; i<num_elements_per_block; i++)
 {
  // If still within bounds, add into result
  if (idx < num_elem_per_iter)
  {
   const uint4 * const elem = &data[idx];

   local_result += ((u64)(elem->x)) + ((u64)(elem->y)) + ((u64)(elem->z)) +
((u64)(elem->w));
   // Move to the next element in the list
   idx += increment;
  }
 }

 // Create a pointer to the smem data area
 u64 * const smem_ptr = &smem_data[(threadIdx.x)];

 // Store results - 128..255 (warps 4..7)
 if (threadIdx.x >= 128)
 {
```

```
  *(smem_ptr) = local_result;
 }

 // For the first SM only, store the clock times before the sync
 if (blockIdx.x == 0)
     smem_sync_clock_times[log_warp_id] = clock64();

 __syncthreads();
...
}
```

我们这里所做的是在累加开始时，优先于同步操作前将内部 GPU 时钟存入共享内存，原始数据结果如表 12-2 所示。从这些数据中注意几点：首先，第一次运行数据花费的时间更多。这是因为数据是从内存存取的而不是缓存。第二，注意线程束间实际的启动时间各不相同。正如你可能预期的那样，奇数线程束和偶数线程束在几个时钟内一个接一个地调度。

表 12-2 来自归约示例的时钟数据

Processing 48 MB of data, 12M elements								
CPU Serial Time: 10.33 ms Parallel Time: 4.79 ms								
ID:0 GeForce GTX 470:GMEM loop E 384 passed Time 0.77 ms								
Warp:	0	1	2	3	4	5	6	7
Start:	3989317792	3989317764	3989317798	3989317768	3989317802	3989317772	3989317806	3989317776
Sync :	3989402014	3989401760	3989403480	3989401716	3989402022	3989402188	3989401620	3989401960
Delta:	84222	83996	85682	83948	84220	84416	83814	84184
Start:	3990392804	3990392798	3990392806	3990392802	3990392812	3990392806	3990392816	3990392810
Sync :	3990468116	3990471366	3990471466	3990470474	3990472008	3990468490	3990472060	3990470096
Delta:	75312	78568	78660	77672	79196	75684	79244	77286
Start:	3991404250	3991404244	3991404252	3991404248	3991404258	3991404252	3991404262	3991404256
Sync :	3991479140	3991478900	3991481020	3991479272	3991480994	3991481338	3991479262	3991481158
Delta:	74890	74656	76768	75024	76736	77086	75000	76902
ID:3 GeForce GTX 460:GMEM loop E 192 passed Time 0.89 ms								
Warp:	0	1	2	3	4	5	6	7
Start:	3854029986	3854029982	3854029988	3854029992	3854029994	3854029996	3854029998	3854030000
Sync :	3854182306	3854181636	3854181640	3854181850	3854182082	3854181916	3854181956	3854182702
Delta:	152320	151654	151652	151858	152088	151920	151958	152702
Start:	3855394420	3855394406	3855394424	3855394408	3855394428	3855394502	3855394432	3855394418
Sync :	3855543696	3855543430	3855541802	3855543036	3855542248	3855543322	3855541356	3855541446
Delta:	149276	149024	147378	148628	147820	148820	146924	147028
Start:	3856767160	3856767224	3856767164	3856767310	3856767168	3856767314	3856767172	3856767318
Sync :	3856914982	3856915592	3856915748	3856913948	3856918534	3856914904	3856915896	3856916670
Delta:	147822	148368	148584	146638	151366	147590	148724	149352

不过即便如此，在早期，启动时间仍有一些差异。图 12-2 是一个标准化版本启动时间的散点图，x 轴方向为线程束，y 轴方向为周期。注意线程束交替的调度器是如何向 SM 提交

线程束的。

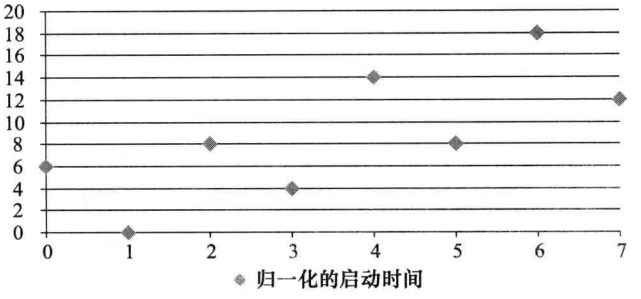

图 12-2 线程束启动时间的归一化分布

正如我们所预料的，考虑到线程束不按照固定顺序执行，当命中同步操作时，时序变化大约为 4000 个时钟周期。即使线程束 1 在线程束 0 之后启动，它也仅在大约 3000 个时钟周期后命中同步操作（参见图 12-3）。

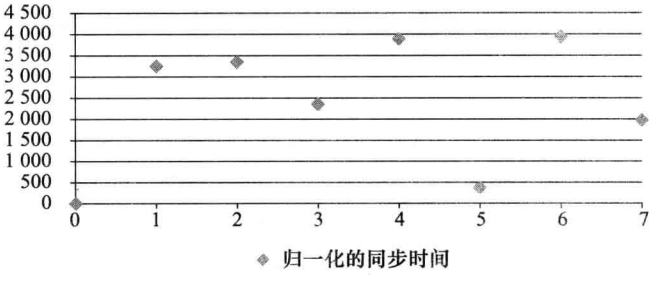

图 12-3 线程束同步时间的归一化分布

很显然，可以看到，我们不可能通过依赖任何的执行顺序实现正确的操作。任何不同线程束中的线程需要交换数据的地方都需要同步点。

当我们尝试从不同的块进行数据交换时，会看到同样的问题：

```
Block  Id: 16 SM: 0 Start: 10420984500 End: 10421078132 Delta: 93632
Block  Id: 22 SM: 0 Start: 10420984504 End: 10421079614 Delta: 95110
Block  Id: 36 SM: 0 Start: 10420984508 End: 10421086198 Delta: 101690
Block  Id: 50 SM: 0 Start: 10420984512 End: 10421105046 Delta: 120534
Block  Id: 64 SM: 0 Start: 10420984592 End: 10421137178 Delta: 152586
Block Id: 171 SM: 0 Start: 10421223384 End: 10421308772 Delta: 85388
Block Id: 172 SM: 0 Start: 10421223406 End: 10421311256 Delta: 87850
Block Id: 176 SM: 0 Start: 10421223424 End: 10421322372 Delta: 98948
Block Id: 177 SM: 0 Start: 10421223518 End: 10421350178 Delta: 126660
Block Id: 178 SM: 0 Start: 10421233178 End: 10421381276 Delta: 148098
Block Id: 303 SM: 0 Start: 10421449580 End: 10421535186 Delta: 85606
Block Id: 304 SM: 0 Start: 10421449618 End: 10421538246 Delta: 88628
Block Id: 305 SM: 0 Start: 10421449800 End: 10421546884 Delta: 97084
Block Id: 306 SM: 0 Start: 10421449822 End: 10421577204 Delta: 127382
Block Id: 313 SM: 0 Start: 10421469888 End: 10421606770 Delta: 136882
```

在此，我们从线程 0 记下了运行在 SM 0 上的线程块的启动时间和完成时间。

起初，你可以看到，SM 0 上的线程块 ID 的分布很广泛，同时各个线程块也分布在不同的 SM 上。我们希望看到这种模式继续下去，因为单独的线程块都从 SM 上撤回，并且新的线程块被引入。

实际上，我们看到调度器将几组 ID 近线性分布的线程块添加到每个 SM 中。这表明线程块调度器只有在某个给定 SM 中的空闲线程块槽达到特定阈值时才会分配新的线程块。从本地缓存访问的角度看，这是很有好处的，并且这会提升一级缓存的命中率。然而，它导致的代价是可能会减少可调度的线程束的个数。我们可以看到线程束和线程块都是在时间上按某种规律分布的。因此，任何基于线程或线程块的合作都可以使计算中的每个元素被计算出来，这是很重要的。

对于线程同步，需要使用 __synthread 原语并且可以利用片上共享内存。对于基于线程块的同步，你要把数据写入全局内存并且启动下一个内核。

最后一个困扰着人们的、关于同步的问题是：需要记住一个线程块内的所有线程都要到达某个屏障同步（barrier）原语，例如 __syncthreads，否则内核会挂起。因此，在 if 语句或是循环结构中使用这些原语时千万要小心，因为这样使用可能会导致 GPU 挂起。

12.3.3 原子操作

在前面的章节我们可以看到，不能依赖或者假定通过顺序的方法可以确保正确的输出。然而，也不能假设一个读取 / 修改 / 写入操作会在同一个设备的其他 SM 中同步完成。考虑这样的一个场景：SM 0 和 SM 1 都执行了一个读取 / 修改 / 写入操作。它们必须串行执行该操作以确保获得正确的结果。如果 SM 0 和 SM 1 都从某一内存地址读取数字 10，将它加 1，然后都把 11 写回，那么其中一个对该数字的增加会丢失。由于一级缓存不一致，很有可能在一个内核调用中发生不止一个线程块向同一输出地址写入。

在多个线程需要向某一公共输出地址写入数据的情况下，我们可以使用原子操作。这些原子操作可以确保读取 / 修改 / 写入操作作为一个整体的串行操作执行。然而，它并不确保任何读取 / 修改 / 写入操作的顺序。因此，如果 SM 0 和 SM 1 都请求在相同地址上执行一个原子操作，哪一个 SM 先执行是无法预测的。

让我们考虑一下传统的并行归约算法。它可以看成一个如图 12-4 所示的简单树。我们有很多方式看待这个操作，可以把 A、B、C、D 各分配到一个单独的线程，这些线程执行原子加法操作并且把结果（A、B）和（C、D）分别输出保存，然后就归结到两个线程，每个线程都将部分结果添加到最终结果上。

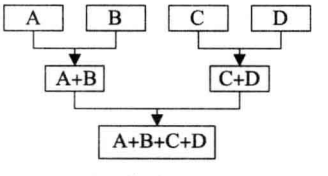

图 12-4 传统的归约操作

或者，我们可以直接从第二行开始，仅使用两个线程。线程 0 将读取 A 和 B 的内容，并把结果输出到指定地址。线程 1 将处理从 C 和 D 获取的输入，然后线程 1 退出，留下线程 0 将两部分结果相加。同样，我们也可以简单地用线程 0 计算 A+B+C+D，将问题简化成由一个单独的线程处理。

第一种方法的工作原理是将目标数据写入到共享的输出地址，这是一个分散操作。另一种方法是聚集源数据，并在下一阶段使用。由于不止一个贡献者向输出地址写入，因此分散的操作需要使用原子操作。聚集的方法完全避免了使用原子操作，因此通常是更可取的解决方案。

如果有（事实上存在）超过一个线程，那么在完全相同的时刻试图执行写操作，原子操作会引入串行。如果这些写操作在时间分布上不发生写冲突，那么原子操作就没有显著的代价。但是，无法断定，在一个复杂系统中的任何时间点绝对不存在两个同时发生的写操作。所以，即使写操作预期是在时间上稀疏分布的，我们也要使用原子操作确保始终是这样的情况。

考虑到可以使用聚集操作取代原子写操作，且聚集操作不需要任何形式的数据加锁，那么使用原子操作还有意义吗？在大多情况下，聚集方法确实会更快，然而，这是有代价的。

在归约操作的例子中，两个数字的加法是微不足道的。由于只有 4 个数字，我们可以很容易地消减线程，并让一个单独的线程依次相加 4 个数字。很显然在数据很少时，这是有效的，但是，如果有 32 000 000 个数据要使用某种形式的归约方法处理呢？

在第 9 章介绍归约方法的例子中可以看到，仅使用一个 CPU 上单个线程慢于使用两个线程，而使用两个线程又比使用三个线程更慢。对于给定线程完成的工作量与运行线程的总数之间，存在一个折衷。对于 CPU 来说，由于主机上的内存带宽问题，AMD Phenom Ⅱ 905e 系统的最大吞吐量仅取决于 3 个线程。

一个更现代的处理器（例如沙桥 -E），具有较高的内存带宽，但同时会有两个附加的处理器核（一共 6 个而不是 4 个）。在沙桥 -E I7 3930K 系统上运行相同的 OpenMP 归约操作产生的结果如表 12-3 和图 12-5 所示。因此，即使大量增加内存带宽和核数量，我们仍会遇到之前的问题。由于加入了越来越多的核，在基于 CPU 的架构上使用更多的线程会使产生的回报逐步降低。

表 12-3 OpenMP 在沙桥 -E 上的可扩展性

使用线程数	执行时间（s）	实际时间百分比（%）	预期时间百分比（%）	开销所占百分比（%）
1	5.78	100	100	0
2	3.05	53	50	3
3	2.12	37	33	4
4	1.71	30	25	5
5	1.52	26	20	6
6	1.46	25	17	8

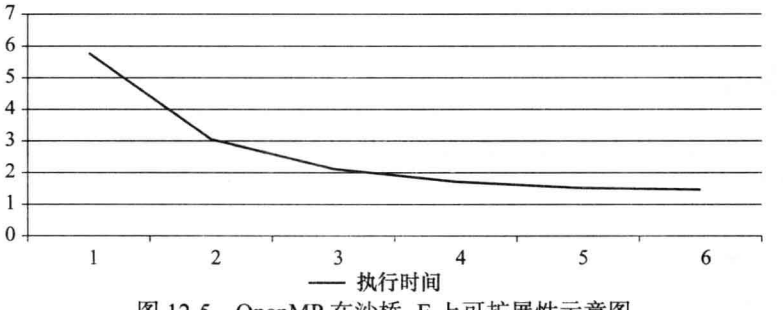

图 12-5 OpenMP 在沙桥 -E 上可扩展性示意图

只运行两个线程不能有效地利用硬件。在 CPU 中运行 16 000 000 个线程并在每一轮归约完后结束一半的线程，也不是一个好方法。在 GPU 中我们可以采用这种方法。因为 GPU 创建一个线程池，该线程池会在程序员提交的 32 000 000 个线程中逐步移动。我们当然可以在 CPU 中手动创建一个类似的线程池，只是能够运行线程的核数要少得多。

我们在第 9 章归约操作示例中使用了聚集操作和分散操作相结合的方法。我们基于设备上实际拥有的 SM 数量的倍数调度一些线程块，然后把数据集划分成 N 块，并让线程从内存读取所需数据，从而执行本地的、片上的累加操作。

每个线程都执行了大量的工作。从前面计时的例子中可以看到，相较 GTX460，更宽的数据总线宽度和双倍的 SM 数量，是 GTX470 能够更快完成该操作的原因。不管是针对 GPU 还是 CPU，我们都期望确保把设备中的并行性发挥到最大。

已经在单线程基础上计算出了部分和，那么接下来的问题就是如何累加所有部分和。因为待累加的数据对于每个线程来说是私有的，所以必须使用原子操作。在源线程没有把数据写出之前，另一个线程是无法收集该数据的。

计算能力为 2.0 的典型 GPU 的 SM 数量高达 16 个，每一个 SM 可以运行的线程束数目多达 48 个，且每个线程束包括 32 个线程。因此，在任何时间点都会有多达 24 500 个活动线程。原子操作可以在共享内存（从计算能力为 1.1 或更高的设备开始）或全局内存上执行。毫无疑问，共享内存的原子操作比跨越 SM 到达全局内存的基于全局内存的原子操作要快很多。由于我们有多达 16 个的 SM，基于共享内存的原子操作是基于全局内存写操作带宽的 16 倍。因此，我们期望尽可能地使用基于共享内存的原子操作。

原子操作只有在计算能力 1.1 及以上的设备上是可用的。基本上任何除了早期 GTX8800 系列之外的显卡都支持。基于共享内存的 32 位整数原子操作，在计算能力为 1.2 的设备（9800 系列及以后）上也可以使用。64 位整数原子操作从计算能力 1.2 的设备开始，可以在全局内存中使用，而从计算能力 2.0 的设备（GTX400 系列）开始，可以在共享内存中使用。

单精度浮点运算的原子操作只在计算能力 2.0 及以后的设备可用。现有的硬件本身还不支持双精度原子操作。然而，可以通过软件实现它，CUDA 编程指南提供了一个例子，展示了如何使用原子对比和交换（Compare And Swap，CAS）操作实现它。

理解什么时候使用聚集操作以及什么时候使用分散操作，经常是实现正确性和高性能的关键。务必考虑最好的构造设计方案，从而最小化地使用（分散的）原子操作，最大化地使用聚集操作。

12.4 算法问题

最后一种问题是很棘手的。程序能够运行而且不产生任何错误提示，但由于算法的问题可能导致错误的结果。

12.4.1 对比测试

测试是判断一个程序员是写出了“好的代码”还是仓促地集成出一些偶尔可以使用的东

西的关键。作为专业的程序员，你应该在限定的时间内努力地交付最优质的软件。如何实现这一点呢？

对比测试认为，编写并行执行的代码比编写功能上等同的串行代码集要难很多。考虑到这一点，每次都要在编写 CUDA 应用程序之前或同时，为某一问题开发出串行的实现方案。然后在两套代码上运行相同的数据集，并且比较两次输出结果。任何输出结果的差异都表明，你的 CUDA 程序可能遇到了问题。

那么，为什么我仅仅说“可能”遇到问题呢？答案很大程度上归结为是否使用了浮点（单精度或双精度）数。浮点数的问题是舍入和精度。在串行 CPU 上，如果有一个大规模数组由随机浮点数构成，当从数组最小下标累加到最大下标得到的结果与从最大下标开始反方向累加的结果是不同的，试试看就知道了。

那么，为什么会这样呢？单精度和双精度数使用 24 位存放尾数（mantissa value），用 8 位存放指数。如果把 1.1e+38 和 0.1e–38 相加，你认为结果会是什么呢？结果是 1.1e+38。因为 0.1e–38 代表的值太小了，无法用尾数表示出来。在一大组数字中，会有很多这样的问题。因此，处理数字的顺序就变得尤为重要了。为了保持精度，解决这种问题最佳的方法往往是将数字集排序，然后从最小的数加到最大的数。然而，为了更高的精度，可能会引入大量的工作（从排序的角度来说）。

还有一些其他的、有关在计算能力 1.x 的设备上处理浮点数的问题（特别是很小的、接近 0 的数）。因此，在检验浮点数是否相等时，最好的方式往往是妥协，允许一定范围的误差。

如果有现成的 CPU 解决方案，那么比较结果就相对简单了。对于基于整数的问题，C 语言标准库函数 `memcmp`（内存比较）已经足够用来查看两个输出集合是否存在差异。通常当 GPU 端有编程错误时，结果并不会只有一点点差异而是会有很大的差别。因此，我们很容易判断这段代码是否工作，并且在输出结果的哪个地方出现了差异。

更加困难的是，结果一直是匹配的，直到到达某一点。通常，这可能是最开始的 256 个值。由于 256 经常作为线程个数，因此这表明线程块索引计算出现错误。只有前面 32 个值正确时，表明线程索引计算错误。

如果没有一个现成的 CPU 实现方案，则需要编写一个或者使用一个别人编写的而且确保正确的实现。然而，事实上，编写自己的串行实现可以让你在尝试并行实现之前确切地阐述问题，更好地理解问题。当然，在开始并行工作前必须确保串行实现产生了预期的结果。

对比测试还提供了一个有用的指标来判断使用 GPU 是否提供了良好的加速比。在这个评价过程中，我们考虑 PCI-E 总线的任何传输时间。对于归约操作的例子，我们可以在 GPU 上编写一个归约算法，这比在 CPU 上实现相同功能的 OpenMP 算法执行得要快得多。然而，仅仅把数据送往 GPU 就浪费掉了任何节省下来的执行时间。要知道 GPU 并不总是最好的解决方案。有一个 CPU 上相对应的方案，可以更容易地进行评估。无论是 CPU 还是 GPU，问题的解决方案应该是最大限度地利用任何可用资源。

一旦创建了对比测试（在本书的很多例子中我们都这样做了），可以立即看到是否引入了错误。因为是在引入它的时候发现的，所以查找和识别错误更加容易。将这个与版本控制系

统相结合（或者是简单地在每次大的进展后创建新的备份）可以在以后的开发周期中免除很多艰苦的调试工作。

12.4.2 内存泄漏

内存泄漏是一个常见的问题而且并不仅仅局限于 CPU 领域。正如它的名字所表明的一样，内存泄漏是程序在运行过程中不断失去可用内存空间。该问题最常见的原因是程序分配了内存空间但是使用后并没有释放这块空间。

如果你曾经让电脑连续开机几个星期，它迟早会慢下来，然后可能会开始显示内存不足的警告。这是由于程序中糟糕的代码没有清理所申请的内存引起的。

显式的内存管理是需要你在 CUDA 编程中负责的。如果分配了内存，那么在程序完成任务时要负责释放这些内存。同样，你也不能使用已经释放回 CUDA 运行时的设备句柄或指针。

有几个 CUDA 操作（在特定的流和事件中）需要你为当前的流创建一个实例。在最初的创建期间，CUDA 运行时可能会在内部分配内存。未能成功调用 `cudaStreamDestroy` 或 `cudaEventDestroy` 意味着内存（可能在主机上也可能在 GPU 上）仍然占用着。你的程序可能会退出，但是如果程序员没有显式地释放这些数据，那么运行时可能不知道这些数据应该被释放。

对于这类问题，一个好的、通用的解决方法是调用 `cudaResetDevice`，它会完全清空在设备上分配的内存。这应该是退出主机程序之前执行的最后一个调用。即使已经释放了所有你认为分配过的内存（对于一个合理大小的程序），你或者是团队中的某个同事仍然可能忘记了曾经的一个或多个内存分配。这种方法可以简单、容易地确保一切内存分配都清除了。

最后，`cuda-memcheck` 是一个对编程人员非常有用的工具（支持 Linux、Windows 和 Mac）。它可以集成到 cuda-gdb 中为 Linux 和 Mac 用户使用。对于 Windows 用户，只需使用如下命令行运行它：

```
cuda-memcheck my_cuda_program
```

假如内核函数包含任何以下的问题，那么程序会执行内核函数并打印相应的错误信息：

- 未指定的启动失败。
- 全局内存访问越界。
- 未对齐的全局内存访问。
- 在计算能力 2.x GPU 上发生的某些硬件检测异常。
- 由 `cudaGetLastError API` 调用检测到的错误。

不论是在内核程序的调试模式还是发布模式下，上述工具均可以运行。在调试模式下，凭借可执行文件中的附加信息，可以从源代码中识别出导致问题的那行代码。

12.4.3 耗时的内核程序

执行时间很长的内核程序会导致很多问题。其中最明显的，当内核程序在设备后台执行的同时该设备也用来显示，就会导致屏幕刷新变得缓慢。为了能在运行 CUDA 内核程序的同

时支持屏幕的显示，GPU 必须在显示刷新和内核之间切换上下文。当内核程序占用很短时间时，用户很少会察觉这一点。然而，当它的时间变得越来越长时，这会变得很讨厌以至于用户都不想再使用这个程序了。

费米架构曾经试图解决这个问题，使用计算能力 2.x 或以上的用户比使用早期硬件的用户受到的困扰更少。然而，这个问题还是很明显的。因此，如果你的应用程序像 BOINC 那样使用“空闲的”GPU 周期，那么它可能会被用户关闭，这显然是不好的。

该问题的解决方案是首先确保有耗时较少的内核。如果你认为屏幕需要每 60ms 更新一次，这意味着每个屏幕更新发生在约 16ms 的间隔内，你可以把内核划分成合适的块然后在这段时间内执行。然而，这很可能意味着整体执行时间将大大增加，因为 GPU 需要在图形上下文和 CUDA 上下文之间不断切换。

对这个特定的问题尚没有简单的解决方案。如果 CUDA 的工作量变得很大，那么低配置的机器和老式的显卡（例如计算能力只有 1.x 的显卡）在试图执行 CUDA 和图形任务时会叫苦连天。要意识到这一点，并且在老式的硬件上测试你的程序，以保证它表现良好。用户往往更喜欢可以让机器同时执行其他任务的程序，即使它稍微慢一点也没关系。

12.5 查找并避免错误

12.5.1 你的 GPU 程序有多少错误

我们在 CudaDeveloper 社区中做的最有利于程序开发的改变，是把所有 CUDA API 调用封装到 `CUDA_CALL` 宏中，在第 4 章搭建 CUDA 环境时已经见过它了。这是一种非常有用的方式，它能让你免受检查返回值的辛苦，这一点可以在 CUDA 程序中引入错误的地方看到。

如果没有使用这种检测机制，那么内核生成的错误个数会在某些工具中显示，例如，Parallel Nsight。不幸的是，它们并不会定位出错误所在。它们只是简单地告诉你执行后返回的错误个数。很显然，任何非 0 的值都代表错误的产生，这时试图定位这些错误是很麻烦的。常见的情况是你没有检查返回值，这当然是一个不好的编程习惯。要么在函数内部处理这些错误，要么调用者应该处理它们。

运行时检测出的错误是很容易修复的。简单地在每个 CUDA API 中使用 `CUDA_CALL` 宏，同时在内核执行完成时使用 `cudaGetLastError()`，这样能检查出大多数的错误。借助 CPU 代码进行的对比测试能够检测出任何内核中出现的大部分功能性和算法性错误。

诸如 Memcheck 和 Parallel Nsight 中的 Memory Checker 都是非常有用的工具（参见图 12-6）。内存越界访问是最常见的导致在内核调用后返回“未知错误”的操作。我们已经介绍了 Memcheck 工具。同样，Parallel Nsight 调试器也能检查内存访问的越界错误。

选择 Parallel Nsight 的 Options（选项）菜单可以在 Nsight 作为调试器运行时启动内存检查器。如果内核在共享内存或全局内存越界写入，那么调试器会中断该越界访问。

请注意，当内存越界访问发生在线程本地的变量上时，这并不起作用，而且启用这个特性会减缓内核总体的执行速度。由于我们仅仅是在用 Parallel Nsight 进行调试时才启用它，

所以这往往不是一个问题。

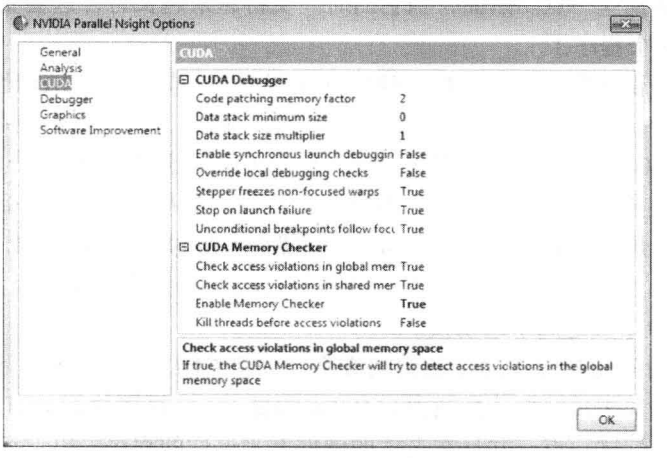

图 12-6 默认启用 CUDA 内存检查器

启用这个选项也会提供一些关于内存未对齐访问的有用信息。未对齐访问从严格意义上来说并不算是错误，然而它只是指出了在哪里（如果能使它对齐访问）能够显著地提高内核的速度。这些消息写到了 Parallel Nsight 的输出窗口，该窗口是可通过微软 Visual Studio 的下拉框选择的众多输出窗口的一种。同时，该输出窗口也显示编译错误消息，它通常是 Visual Studio 工程中打开的 3 个标准窗口中的那个底部窗口。

12.5.2 分而治之

分治法是一种常见的调试方法，它并不针对于特定的 GPU。然而，这是非常有效的，这也是为什么我们要在这里提到它，它在处理内核引起的一些运行时无法处理的异常时非常有效。这通常意味着获取了一个错误消息，然后程序停止运行，或者在最坏的情况下，机器会完全挂起。

解决这类问题的第一个方法是使用调试器运行，在一个较高的层次上单步执行每一行代码。这样迟早会找到引起程序崩溃的那个调用。从主机调试器开始（确保使用的是 CUDA_CALL 宏），然后查看在什么地方发生了错误。最有可能发生错误的是内核调用或内核调用后的第一个 CUDA API 调用。

如果发现问题出现在内核中，那么切换到 GPU 调试器，例如，Parallel Nsight 或 CUDA-GDB。随后重复如下过程：在内核执行过程中跟踪单个线程的活动。这可以看到引起错误的最上级的调用。如果没有的话，那么可能是跟踪线程之外的其他线程引起的。通常，任何特定的线程块中“受关注的”线程是线程 0 和线程 32。大多数没有被检测到的 CUDA 内核错误可能是由于线程束间或线程块间的行为不像程序员所设想的那样运作。

单步执行代码，并检查每一个计算的答案是否都是预期的那样。一旦得到了一个错误的答案，则需要明白它为什么是错的，然后解决的方案就很清晰了。现在所要做的是一个高层次的折半搜索。通过跳过子函数执行代码直到命中错误，就可以避免在单一层次逐步浏览代

码，然后就可以非常迅速地找出问题函数 / 代码行。

如果在你的环境中无论如何也无法使用调试器或是调试器以某种方式干扰了可见的问题，那么也可以采取这种策略但不使用任何调试器。只需将 `#if 0` 和 `#endif` 预处理器指令放在本次运行需要移除的代码处即可。编译并运行内核然后检查结果。如果代码没有任何错误地执行，那么错误很有可能发生在移除的代码段中的某处。逐渐减少移除代码段的大小，直到程序再次中断。程序中断的地方明确地指示了问题可能的来源。

你可能也希望试图用这种方法观察程序是否以如下方式运行：

- 1 个线程块，每个线程块有 1 个线程。
- 1 个线程块，每个线程块有 32 个线程。
- 1 个线程块，每个线程块有 64 个线程。
- 2 个线程块，每个线程块有 1 个线程。
- 2 个线程块，每个线程块有 32 个线程。
- 2 个线程块，每个线程块有 64 个线程。
- 16 个线程块，每个线程块有 1 个线程。
- 16 个线程块，每个线程块有 32 个线程。
- 16 个线程块，每个线程块有 64 个线程。

如果一个或多个这样的测试失败了，这就表明同一线程束中的线程，同一线程块中的线程或者一个内核启动中的线程块之间的交互引起了问题。它为在代码中定位对象提供了指示。

12.5.3 断言和防御型编程

防御型编程假定调用者会做一些错误的事情。例如，下列代码有什么错误呢？

```
char * ptr = malloc(1024);
free(ptr);
```

该代码假定 malloc 会返回一个指向 1024 字节内存的有效指针。由于我们需要的内存数量小，因此这实际上是不可能失败的。如果失败了，malloc 返回一个空指针。为了让代码能够正常工作，需要使用 free() 函数处理空指针。因此，释放内存函数的开头可能是：

```
if (ptr != NULL)
{
... search list of allocated memory areas for ptr and de-allocate memory.
}
```

free() 既需要考虑接受空指针也需要考虑接受明显有效的指针。然而，NULL 指针并不指向一块有效地被分配内存的区域。通常，如果调用 free() 处理空指针或无效指针，一个防御型函数不会损坏堆存储，相反它会什么都不做。防御型编程在函数接受不合法输入时不会进行任何错误的操作。

但是，这也有一个令人讨厌的副作用。在这种情况下，用户再不会看到程序的失败，然而测试部门或质量保证部门抑或程序编写者也无法看到程序崩溃。事实上，不管调用处是否有编程错误，程序现在都会悄无声息地失败。如果一个函数对边界或输入的范围有隐含的要

求，都应该被检查。比如，如果一个参数是数组的索引，那么绝对应该检查这个值以确保不会对该数组产生越界访问。然而，这个问题经常被错误地处理了。

C 语言提供了一个非常有用的设计，然而除了那些熟悉良好的软件工程习惯的程序员却很少有人使用——断言指令（assert directive）。当一个程序失败时，让它保持默不作声是糟糕的实践之道。这会使错误仍然残留在代码中无法发现。而 assert 背后的思想却是相反的。如果调用者传递的参数出现错误，那么程序就出现了错误。被调用的函数应该给出警告直到它被修复。因此，如果不允许一个空指针作为输入参数传递给函数，那么用下列代码替换 if ptr! = NULL 检查

```
// Null pointers not supported
assert(ptr_param != NULL);
```

这意味着我们不再需要额外的缩进，并且我们在代码中记录了进入该函数的前提条件。请务必确保在每个断言之前有一行注释，解释为什么该断言是必须的。程序在将来某些时候可能会失败，因此应希望函数的调用者尽快知道为什么它们的调用是无效的。这个调用者经常可能就是你自己，所以确保注释断言对你自己有最大的好处。

从现在算起，经过 6 个月后，你就会忘记为什么这个前提条件是必要的。然后你会四处搜寻试图记起它为什么是必需的。同样，它也有助于防止未来程序员移除“看起来错误的”断言，从而保证某些错误在发布前没有被忽视。在没有完全理解为什么断言放置于首要位置前请不要这样做。几乎在所有情况下，移除 assert 检查都会在后面掩盖程序的错误。

当使用断言时，千万不要把处理程序错误和有效的失败条件混淆。例如，下列代码是错误的：

```
char * ptr = malloc(1024);
assert(ptr != NULL);
```

malloc 返回空指针是有效的条件。当堆空间被耗尽时就会出现这种情况。程序员对此应该有一个有效的错误处理判断，因为最终它总是会发生的。断言应该保留以便处理无效的条件，例如，索引越界，枚举操作中的默认 switch 条件处理等。

使用防御性编程和断言的附带问题之一是处理器需要花费时间检查条件，然而在大多数情况下这些条件都是有效的。在每个函数调用、循环迭代等过程中都会这样做，这取决于断言使用得多广泛。该问题的解决方法很简单——生成两套软件，一个调试版本和一个发布版本。如果你已经使用了一个软件包，例如 Visual Studio，那么它本身就包含在默认工程设置中。老一些的系统（特别是基于非 IDE 的系统）需要人为创建。

一旦完成，就可以简单地生成 assert 宏的一个版本，ASSERT。

```
#ifdef DEBUG
#define ASSERT(x) (assert(x))
#else
#define ASSERT(x)
#endif
```

这个简单的宏只会在调试代码（你和质量保证人员在发行版本的同时进行测试所用的版本）中包含断言检查。

截至 CUDA 4.1 版本发布，对于计算能力 2.x 的设备，现在也可以把断言添加到设备代码中。由于 GPU 的能力限制，之前的 GPU 设备上无法抛出这样的异常消息。

12.5.4 调试级别和打印

除了有单独的发布版本和调试版本，拥有一个可以灵活改变（例如通过设置全局变量的值，#define 或者其他常量）的调试级别也是很有用的。你可能还希望能通过命令行设置这一参数，例如通过 -debug = 5 将调试级别设置成 5 等等。

在开发过程中，可以在代码中添加有用的信息，例如：

```
#ifdef DEBUG
#ifndef DEBUG_MSG
// Set to 0..4 to print errors
// 0 = Critical (program abort)
// 1 = Serious
// 2 = Problem
// 3 = Warning
// 4 = Information
#define DEBUG_ERR_LVL_CRITICAL (0u)
#define DEBUG_ERR_LVL_SERIOUS (1u)
#define DEBUG_ERR_LVL_PROBLEM (2u)
#define DEBUG_ERR_LVL_WARNING (3u)
#define DEBUG_ERR_LVL_INFO (4u)

// Define the global used to set the error indication level
extern unsigned int GLOBAL_ERROR_LEVEL;
void debug_msg(char * str, const unsigned int error_level)
{
 if (error_level <= GLOBAL_ERROR_LEVEL)
 {
  if (error_level == 0)
   printf("\n**********%s%s", str, "**************\n");
  else
   printf("\n%s", str);

  fflush(stdout);

  if (error_level == 0)
   exit(0);
 }

}
#define DEBUG_MSG(x, level) debug_msg(x, level)
#else
#define DEBUG_MSG(x, level)
#endif
#endif
```

在这个例子中，我们已经创建了 5 个级别的调试消息。在不使用调试版本软件的地方，这些消息以不引起编译错误的方式从可执行程序中剥离出来。

```
#define DEBUG
#include "debug_msg.h"
unsigned int GLOBAL_ERROR_LEVEL = DEBUG_ERR_LVL_WARNING;

int main(int argc, char *argv[])
{
 DEBUG_MSG("Error from level four", DEBUG_ERR_LVL_INFO);
 DEBUG_MSG("Error from level three", DEBUG_ERR_LVL_WARNING);
 DEBUG_MSG("Error from level two", DEBUG_ERR_LVL_PROBLEM);
 DEBUG_MSG("Error from level one", DEBUG_ERR_LVL_SERIOUS);
 DEBUG_MSG("Error from level zero", DEBUG_ERR_LVL_CRITICAL);

 return 0;
}
```

为了调用该函数，只需简单地把宏添加到代码中去，如以前的例子中所示。这在主机代码中工作良好，但是不经过修改无法在设备代码上工作。

首先，在内核中打印消息时需要注意一些问题。只有计算能力 2.x 的设备才支持内核级的 printf。如果试图在将要被计算能力 1.x 设备编译的内核中使用 printf，会看到一个错误，告诉你不能从全局或设备函数调用 printf。这个说法其实并不正确——仅仅是因为该函数并不被计算能力 1.x 的设备支持，而且目标架构必须是计算能力 2.x 的设备。

假设现在有一个费米型的设备，那么 printf 调用是支持的。线程束中 32 个线程（线程束的大小）将分别打印消息，除非你特意不这么做。很显然，由于要启动成千上万个线程，因此简单地打印一条消息可能会导致从终端窗口顶端开始滚动上万条附加的消息行。由于 printf 的缓冲有固定的大小（线程束也是），这样会丢失早些时间得到的输出。

另一个问题是，这些消息行能以任意的顺序打印出来。在没有确定消息准确的创建时间的情况下，需要某些时间参考，否则我们不能用打印消息的顺序来体现程序的执行顺序。因此，我们需要确定每个消息的来源并且用时间戳记录它。

第一个问题通过让线程块或线程束中的一个线程打印消息就能简单地处理。按照惯例，这通常都是线程 0。我们可能还希望从每个线程束中打印一条消息，所以再一次仅选择每一个线程束的第一个线程来打印消息。也可以使用其他的准则，例如选择计算边缘重叠区域（halo region）的线程等。下面是一个样例代码集。

```
if ( (blockIdx.x == some_block_id) && ((threadIdx.x %32) == 0) )
{
 // Fetch raw clock value
 unsigned int clock32 = 0;
 asm("mov.u32 %0, %%clock ;" : "=r"(clock32));

 // Fetch the SM id
 unsigned int sm = 0;
 asm("mov.u32 %0, %%smid ;" : "=r"(sm));

 printf("\nB:%05d, W:%02d, SM:%02u, CLK:%u", blockIdx.x, (threadIdx.x>>5), sm,
clock32);
}
```

它仅仅寻找指定的线程块编号，然后打印线程块编号、线程束编号、所在的 SM 编号和原始时钟值。

```
B:00007, W:05, SM:13, CLK:1844396538
B:00001, W:04, SM:05, CLK:1844387468
B:00002, W:09, SM:09, CLK:1844387438
B:00007, W:10, SM:13, CLK:1844396668
B:00002, W:06, SM:09, CLK:1844387312
B:00007, W:00, SM:13, CLK:1844396520
B:00005, W:12, SM:06, CLK:1844396640

B:00005, W:13, SM:02, CLK:24073638
B:00006, W:03, SM:04, CLK:24073536
B:00005, W:15, SM:02, CLK:24073642
B:00002, W:03, SM:05, CLK:24076530
B:00006, W:00, SM:04, CLK:24073572
B:00002, W:00, SM:05, CLK:24076570
```

在这里，我们打印了线程块编号、线程束编号、线程束执行时所在的 SM 编号和原始时钟值，可以简单地把这些输出重定向到一个文件中，然后绘制出散点图。由于我们把设备端的 printf 放在内核的开始处，因此该图显示了每个内核是何时被调用的。

在图 12-7 中，纵轴代表 SM 编号，横轴代表绝对时钟时间。我们可以看到，所有的 SM 大约在相同的时间启动（除了个别 SM 晚一些启动，但也是一起启动的）。之后，随着每一个线程块在其开始运行时打印的执行细节，时间戳基本上是随机分布的。这个分布完全取决于所执行的程序，与外部资源变得可用的时间有关，全局内存就是最好的例子。

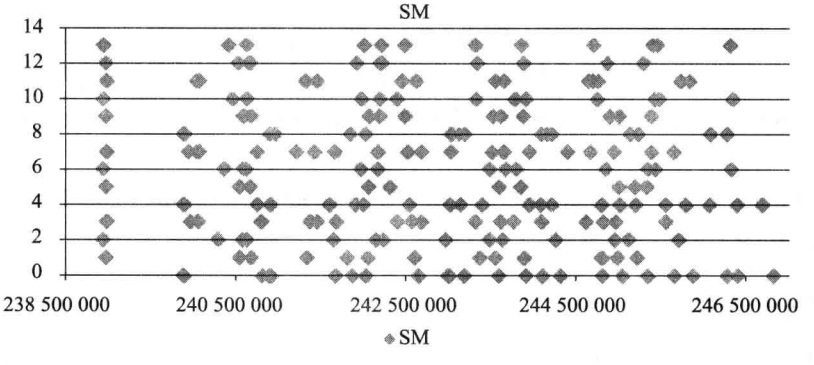

图 12-7 线程束在 14 个 SM 上执行的图示（显卡为 GTX470）

考虑到多 GPU 和多数据流的情形，我们要考虑能够识别出消息来源的方案。这同样可以通过在消息前添加唯一的前缀标识符来解决。在有些例子中，当我们使用多 GPU 时，用一个从设备 ID 字符串创建的字符串 device_prefix 也完全能够做到这一点。然而，提取这个信息的 API 是主机端调用而不是设备端调用。这是有道理的，因为我们不希望 30 000 个线程每个都获取设备 ID 字符串，因为该字符串对它们来说都是相同的。因此，我们能

做的是通过全局内存或常量内存提供这个主机端的信息。如果只有一个 GPU 和一个数据流，那么这不是必需的，但是任何重要的程序都会在可能的情况下同时使用多数据流和多 GPU。

在多 GPU 情况下，可能会看到时钟值有明显的变化。因此，很容易能看出输出流是来自于不同设备的，但是哪个来自于设备 0、1、2 或 3 呢？对于同样的设备，我们无法确定。万一这些消息来自于同一设备的不同数据流呢？

使用绝对 TID（线程 ID）值足以为每个 GPU 内核识别消息。然而，使用了多数据流或多设备就需要使用设备号、TID 和数据流编号。

仅从查看输出的角度来说，时序是一个问题。应该用以下方式创建一个前缀

```
GPU ID : Stream ID : TID : Message
```

有了这个前缀，就可能把输出重定向到文件并通过保持相对顺序来进行排序。我们最后就按顺序得到了每个 GPU 和数据流排序的消息。

请注意，虽然 printf 是一种在主机端显示信息的简单方法，要知道它在 GPU 内存开辟了 1MB 的缓冲区并且在特定事件下将这个缓冲区传回到主机。

因此，printf 输出只有在下列条件下才会看到：

1. 在下一个内核启动的开始。

2. 在一个内核执行结束且环境变量 CUDA_LAUNCH_BLOCKING 被设置时（如果使用多 GPU 或多数据流则不推荐这样做）。

3. 主机端发起的同步点，例如，同步设备、流或事件的结果。

4. 阻塞版本的 cudaMemcpy。

5. 程序员发起的设备重启（cudaDeviceReset 或驱动程序的 cuCtxDestroy API 调用）。

因此，大多数情况下会看到输出被打印出来。如果没有看到，在退出主机程序前调用 cudaDeviceReset 或者在工作集结束时在流里调用 cudaStreamSynchronize，这样丢失的输出就会出现。

如果需要一个更大的缓冲区，可以调用 cudaDeviceSetLimit（cudaLimitPrintFifoSize，new_size_in_bytes）API 进行设置。

12.5.5 版本控制

版本控制是任何专业软件开发的一个重要方面。它没有必要使用非常昂贵的工具或过于细致的流程，诸如某人可以更新某处这样的细节信息就多余了。在大型项目中，版本控制是绝对必不可少的。然而，即使是对于单人开发的软件（可能适用于很多读者）这也是很重要的。

考虑一下，调试 30 000 个线程的程序是很容易的。如果你认为这种说法是可笑的，那么就能理解如果没有对程序定期地或者每达到一个重要环节就进行版本控制时，所面临的任务有多困难。程序员通常是一群过于自信的人，他们可以确保一开始“较小的”变化不会带来

问题。然而，当事情没有像计划那样进展时，准确地记住所做的变化是很困难的。如果没有对可工作的程序进行备份，之后再想变回修改前的程序是很困难的。

专业领域的大多数程序是由团队开发的。同事是很有帮助的，因为他可以让你从另一个角度看问题。如果你有一个经过版本控制或创建过基线的可工作代码，那么很容易看出不同版本的差异以及是什么破坏了以前可用的解决方案。没有这些周期性的基线，很难识别错误可能发生的位置，你需要查看几千行代码而不是几百行代码。

12.6 为未来的 GPU 进行开发

12.6.1 开普勒架构

英伟达从费米及更高版本架构展现出的路线图是开普勒 GK104（K10），开普勒 GK110（K20），和 Maxwell。截至 2012 年 3 月第一个开普勒产品发布，即 GK104。该产品的目标是直接面向消费级市场，因此缺少一些 HPC（高性能计算）市场喜欢看到的一些特性，特别显著的是对双精度数学的支持。开普勒 GK110 毫无疑问会是更以 HPC 为目标的产品且最终可能会以某种形式转化为面向消费级别的显卡。GK110 计划在 2012 年年底发布，但是它的设计版已经在英伟达内部开发 CUDA 5 发布版本的时候被使用了。

让我们通过表格简要看看开普勒所带来的变化。首先，它的能量效率更高。而相比 GTX580 的 244 瓦功耗，开普勒 GTX680 的 TDP 功耗为 195 瓦。就消费级市场的高端单卡而言，它的绝对消耗减少了超过 20%。通过更加深入地了解 GTX680，在 GTX580（GF110）和 GTX560（GF114）之间，它更接近于后者，有点像内部折叠起来的 GTX560。

然而，如果我们考虑每瓦能耗带来的十亿次浮点数操作数，那么你看到开普勒 GK104 在多达 2 个因素上超过了费米 GF110。英伟达对常见的消费级游戏的研究（英伟达，2012 年 5 月 18 日）表明每瓦有平均 1.5 倍性能的提升。现在的许多游戏都是非常复杂的，因此希望在基于计算的应用程序上建立一个可比较的能耗分析数据是很有道理的。

通过使用特别精选的部件，GTX690（双 GTX680 卡版本）在每瓦能耗得到的浮点操作数目上，大大地超越了 GTX680。在每瓦能耗带来的性能上增加一倍或更多是英伟达团队的巨大成就。GTX690 是 Tesla K10 系列的基础。这是首次在 Tesla 产品线上使用双 GPU 板卡方案。

虽然从 GTX580 到 GTX680 全局内存带宽的峰值保持不变，但是现在我们已经从 PCI-E 2.0 转到 PCI-E 3.0 标准。因此，GTX680 配置到支持 PCI-E 3.0 标准的主板和 CPU 上，发送和接收数据的速率将是 PCI-E 2.0 标准 400/500 系列显卡的两倍。对那些受制于 PCI-E 带宽的内核程序而言，带宽的加倍意味着显著的加速。

开普勒 GTX680/GTX690 把我们从计算能力 2.1 转移到计算能力 3.0。开普勒 GK110 的目标是达到计算能力 3.5。新的计算能力一览表如表 12-4 所示。

表 12-4 开普勒架构的新计算能力

描述 / 设备	G80（GTX8800）/ G92（GTX9800）	G200（GTX280）	费米 GF110（GTX580）	费米 GF114（GTX560）	开普勒 GK104/K10（GTX680）	开普勒 GK110/ K20(TBD)
计算能力	1.0/1.1	1.3	2.0	2.1	3.0	3.5
每 SM 的最大线程束数量	24	32	48	48	64	64
每 SM 的最大线程数	768	1024	1536	1536	2048	2048
每 SM 的全部寄存器大小（单位：B）	32K	64K	128K	128K	256K	256K
每线程的最大寄存器数	63	127	63	63	63	255
每线程块的最大线程数	512	512	1024	1024	1024	1024
每 SM 的 CUDA 核数	8	16	32	48	192	192*
每设备的最大 SM 数	8	32	16	8	8	15
每 SM 的最大共享内存 / 一级缓存（单位：B）	16K	16K	48K	48K	48K	48K
每设备的最大二级缓存（单位：B）	N/A	N/A	768K	512K	512K	1536K

* 另加上每 SM 上的 64 个双精度单元

开普勒的一个重大变化是去除了着色器时钟。在开普勒之前，GPU 在一个给定的 GPU 时钟频率下运行，而着色器时钟运行在两倍于 GPU 时钟频率上。在前几代，是着色器时钟而不是 GPU 时钟驱动 CUDA 核在设备上执行。

在任何处理器设计中，时钟速率都是功率消耗的重要驱动因素。去除了着色器时钟，英伟达不得不在每个 SM 放置两倍的 CUDA 核以达到相同的吞吐量。这种折衷方案显著地减少了总体的功率消耗并且允许英伟达把核时钟从 772MHZ 提升到略超过 1GHZ。

实际上，开普勒 GK104 的设计使 CUDA 核数量增加了 4 倍。加载存储单元（LSUs）、特殊功能单元（SFUs）、指令调度器的数目以及寄存器文件大小都加倍了。共享内存 / 一级缓存仍保持 64KB 不变，但是除了通常的 16KB/48KB 的划分方式，现在也可以分成 32KB/32KB。

这个选择是很有趣的，因为大量额外的计算能力被添加进来。如果查看前几代，我们可以看到从 GT200（计算能力 1.3）到 GF110（计算能力 2.0）设备，每个 SM 包含的线程束数目从 24 个升级到 48 个。开普勒 GK104 的设计把每个 SM 的线程束总数增加到 64 个，把每个 SM 的线程总数增加到 2048 个。

GTX680 宣称有 3 万亿次浮点运算的峰值性能，而与之相比，GTX580 则为 1.5 万亿次浮点运算。这种峰值性能是基于浮点乘法和加法（FMAD）操作执行的。当然，在任何实际使用中，指令构造和内存访问模式上都有很大的不同，这都最终决定了实际的性能水平。

此外，开普勒 GK104 现在有动态时钟调整的特性，它会根据当前的 GPU 负载上下调整时钟。我们几年前就已经在 CPU 端看到这个特性了，这十分有助于节省功率，尤其是设备处于空闲状态时。

从指令进化的角度来说，我们看到的主要好处是 shuffle 指令允许同一个线程束内的线程

进行通信。线程束内的线程在不需要通过共享内存共享数据的情况下，依然能够合作，这是很有好处的。使用这样的操作可以很容易地对归约操作和前缀求和的最后阶段进行加速。由于附加了48KB的只读缓存，额外的编译器内部函数已经支持硬件级的平移、旋转操作，纹理内存的访问也变得可用，而不再需要编写纹理内存操纵代码的开销。另外，4字节的包裹向量指令（add、subtract、average、abs、min、max）也被引入了。

从计算的角度来说，开普勒GK110有一些非常吸引人的特性，英伟达称这些技术为动态并行、Hyper-Q和RDMA。它几乎使每个设备的SM数量翻了一倍并且添加了许多HPC应用程序必需的双精度浮点单元。初期的（英伟达）数据表明，双精度性能超过了1万亿次浮点运算。内存总线宽度从256位增加到了384位，如果时钟类似于GK104，那么会确保内存带宽超过250GB/s。

先来说第一个技术——动态并行。它第一次允许我们从GPU内核轻松地启动其他的工作。以前这是通过超额预定线程块并且让一些线程块处于闲置状态或是运行多个内核实现的。前者会浪费资源且工作效率很差，尤其是对于大型问题。后者意味着有一段时间GPU未被充分利用，同时无法让内核在高速共享内存或高速缓存上维护数据，因为这个内存在多个内核启动之间并不是持久的。

再来说第二个技术，Hyper-Q。一直到（也包括）开普勒GK104，所有的数据流在硬件中都是作为单一管道被实现的。Hyper-Q解决了程序员表现出的数据流模型与实际在硬件上实现的差异问题。因此，即使程序员通过把这些内核分别放入不同的流中显式地指定，来自于数据流0的内核流也不会和来自于数据流1的内核流混合，因为它们是单独的工作单元。

Hyper-Q把一个单独的硬件流分成32个分离的硬件队列。因此，几百个程序员定义的数据流集合中最多有32个数据流能够在硬件上独立运行。这带来的主要好处在于设备的负载。随着每个SM拥有超过192个核，SM的粒度明显地增加了。如果小内核被执行且部分地载入SM，那么SM中的资源就会被浪费。

最后，RDMA（远程直接数据存取，Remote Direct Memory Access）也是一个非常有意思的技术。英伟达已经和某些特定的供应商合作（主要是在Infiniband端）以改进GPU到GPU节点之间通信的延迟。现在，对等方式支持一个节点中的多个GPU直接通过PCI-E总线进行通信。对于支持这一特性的显卡和操作系统，避免了使用CPU内存空间转发的间接访问方式。

然而，为了从一个非GPU设备（例如，诸如网卡一样的I/O设备）接收或发送数据，最好使用主机端锁页内存[⊖]的共享区域。RMDA这一特性对此做出了改变，它允许GPU通过PCI-E总线直接与其他PCI-E显卡通信，而不仅仅是英伟达的GPU。目前，这仅仅被一些Infiniband显卡支持，但是它开创了使用其他显卡的可能，例如直接数据采集、FPGA、RAID控制器等类似能够直接与GPU对话的操作。这会是一个让人期待的有趣技术。

⊖ 锁页内存能够保证存在于物理内存中，始终不会被分配到低速的虚拟内存，并能够通过DMA加速与设备端的通信。CUDA中，使用cudaHostAlloche和cudaFreeHost来分配和释放锁页内存。

12.6.2 思考

开发将来可以运行很多年的代码或者至少能够在将来运行的代码始终是一个棘手的问题。针对特定硬件所做的调优越多，在将来开发中的可移植性越差。因此，一个策略是确保任何所开发的代码都是参数化的，那么它就可以很容易地适应未来的 GPU。

通常，一个应用程序是针对特定的架构开发的。可能会有如下的代码段：

```
if ( (major == 2) && (minor == 0) ) // Compute 2.0
 num_blocks = (96*4);
else if ( (major == 2) && (minor == 1) ) // Compute 2.1
 num_blocks = (96*2);
else
 num_blocks = 64; // Assume compute 1.x
```

现在如果一个计算能力为 2.2 或 3.0 的架构发布，那么会发生什么呢？在这个样例程序中，我们会下降到计算能力 1.x（G80/G92/G200 系列）的那条路径。程序的用户不想把费米型 GPU 替换成一个新的开普勒型板卡，然后他们发现程序运行得更慢了或者干脆无法在他们崭新的显卡上运行。当编写这样代码的时候，要假定将来也可能会遇到一个未知的计算能力架构并且要相应地满足它。

从 G200 转移到费米有一个过渡期。此时程序编写者不得不重新发行程序，因为每 SM 执行的线程块数在各代都保持不变，只有每线程块的线程数增加了。如果一个内核已经使用了每 SM 里的最多的线程块数，这种情况下允许最佳指令混杂从而带来更好的性能，则没有额外的线程块被调度到 SM 上。因此，新的硬件没有被充分利用，现存的软件没有在新的硬件上执行得更快。

G200 到费米最重要的变化是需要增加每线程块所包含的线程数。每线程块最大线程数（该属性可被查询）为 512 ~ 1024。同时常驻的线程束数量从 24（计算能力 1.0/1.1）增加到 32（计算能力 1.2/1.3）再增加到 48（计算能力 2.0/2.1）。因此，随着线程块包含的线程数越来越多，未来很有可能会继续看到这样的趋势。开普勒也是第一个增加每 SM 内线程块个数的 GPU 架构，从 8 翻倍到 16 个线程块。因此，在调度的线程块达到允许上限时，每线程块的最优线程数为 2048 个线程 ÷ 16 线程块 = 128 个线程每线程块。

我们可以通过查询线程个数和线程束大小计算出可用线程束个数。cudaDeviceProp 结构体返回 warpSize 和 maxThreadPerBlock。因此我们可以调用 cudaGetDeviceProperties（&device_props）API，然后用每线程块的线程数除以线程束个数，以计算出给定 GPU 的最大线程束个数。

这种方法对开普勒 GK104 以及即将上市的开普勒 GK110 都适用。然而，它没有考虑到 GK110 将会带来的编程模型的变化。GK110 动态并行的方面（现在是开放的）可以清晰地进行规划。英伟达在 GPU 技术大会（GTC）展示了一些工作成果，并且声称仅这项技术（主要是消除 GPU 控制开销）就可以使很多代码的编写速度显著提高，这也带来了更加简化的递归形式，其中递归的部分能随着节点数的扩张提高并行性或者根据遇到的数据减少并行性。

现在，运行在开普勒硬件上的程序可以利用48KB的专用、只读纹理内存。一个很重要的变化在于不再需要进行纹理内存编程。只需要使用C99标准的__restrict__关键字声明只读指针，例如：

```
void my_func(float * __restrict__ out_ptr, const float * __restrict__ in_ptr)
```

在这个例子中，通过添加这个关键字，任何对参数out_ptr的写入不会影响in_ptr指向的内存区域。事实上，这两个指针不是彼此的别名。这将导致通过in_ptr的读取被缓存到纹理内存，并提供额外的48KB的只读缓存内存。这样很可能明显减少片外访问全局内存的次数从而显著提高内存吞吐量。

应该从现有内核的哪些元素可以被并行执行的角度思考Hyper-Q的逻辑。任务级的并行性在GPU上第一次真正成为可能。作为前期准备，如果现在运行了一系列的内核，则把这些分成独立的数据流，每个流负责一个独立的任务。当在当前的平台运行时，这不会反过来影响代码的性能，但是一旦这些特性可用时，将会使这些内核在开普勒上更好地执行。

最后，新的Tesla K10产品，是一款基于当前可用的GTX690消费级显卡的双GPU。由于有了多GPU，如果你打算只使用一个单独的核，那么就会浪费50%的额外计算能力。因此，任何打算安装K10产品的人都需要移植他们的代码以支持多GPU，我们在第8章已经提及这些了。你需要考虑数据驻留在哪里以及有些GPU之间的通信是否必要。如今移植到一个多GPU的解决方案会使过渡更加容易，并对许多应用程序提供几乎线性的扩展性。

12.7 后续学习资源

12.7.1 介绍

现有CUDA资源可以很容易通过网络和各个大学获得。我们也为渴望学习CUDA的专业人员和团队举办了多次学习交流会。我每年也尽力地通过面对面或者网络在线的方式参与CUDA的课程。因为十分热爱CUDA的缘故，我阅读学习了很多关于这方面的书籍，而且我十分乐意将这些CUDA资料分享给CUDA爱好者。这些信息也可以通过我们的网站www.learncuda.com获得，该网站是一个在全世界范围内获取各种CUDA资源的入口。

12.7.2 在线课程

CUDA成功之道很大程度上归于英伟达的承诺，英伟达愿意把CUDA推广给更多的用户。回顾历史，我们可以发现很多次他们欲将并行程序设计推广为主流的尝试，也设计了能够支持并行运算结构的编程语言。但是可能除了OpenMP和MPI之外，都无一例外的以失败告终。这大概是因为他们始终没有将并行运算推广出他们的交际圈，或者因为没有足够雄厚的资金支持，再或者是局限在了大学、政府部门和公司的少数计算机上。

在此，我们首先推荐英伟达官方培训网站，它是最好的 CUDA 资源之一。具体网址见 http://developer.nvidia.com/cuda-training。当然，你也可以通过各个大学获得讲座视频，获取源分别有：

- ECE-498AL，http://courses.engr.illinois.edu/ece498al/。这是第一本 CUDA 专业书籍的作者 Wen-mei W. Hwu 教授的课程。讲座的音频记录和幻灯片可以在 2010 年的课程里获得。
- Stanford CS193G，http://code.google.com/p/stanford-cs193g-sp2010/。这是一门斯坦福大学在 ECE-498 课程基础上开设的课程，网站里面有 iTunes 格式的讲座视频，讲座的主讲人是 Jared Hoberock 和 David Tarjan。
- Winsconsin ME964，http://sbel.wisc.edu/Courses/ME964/。这是一门关于高性能计算应用程序在工程领域的课程，由 Dan Negrut 主讲。网站里有许多课程视频和贵宾讲座的链接。
- EE171 并行计算机架构，http://www.nvidia.com/object/cudau_ucdavis。这门优秀的课程由加州大学戴维斯分校的 John Owens 教授主讲。课程里从构架的角度阐述了数据级并行、指令级并行和线程级并行。

紧随其后，值得推荐的在线资源是 GPU 会议的会议记录。英伟达一般每年会在加州的圣何塞举办 GPU 技术会议（GTC），当然会议也会在世界各地举行。在会议举办的一个月后，各类会议的资料会陆续上传到英伟达的 GPU 技术网站 http://www.gputechconf.com/gtcnew/on-demand-gtc.php。自 GTC 开办以来，有太多会议可以参加。许多会议的内容交叉重叠。可以在网络上查阅到近几年来的所有会议资料，当然也包括 NVIDIA 相关的会议记录。

那些主题演讲，特别是 Jen-Hsun Huang 主讲的，都十分值得一听，这些演讲能够拓宽个人关于 GPU 科技发展的视野。而 Sebastian Thrun 主讲的关于 DARPA 挑战的主题演讲展现了 CUDA 应用程序巨大的发展前景，例如汽车自动控制的应用。Paulius Micikevicius 的多个演讲关注于 CUDA 的性能优化，而 Vasily Volkov 的演讲更加关注于占有率，这些演讲也十分值得去听。

另一个主要的 CUDA 网络学习资源就是英伟达提供的线上研讨会存档。可以通过以下网站获得：http://developer.nvidia.com/gpu-computing-webinars。线上研讨会主要面向注册的 CUDA 开发者，可以通过免费的注册来获得线上研讨会的直播视频。在直播的互动参与中，可以提出自己的问题，并通过感兴趣的特别话题讨论的方式获得回馈。通常线上研讨会结束后不久就会有存档可供下载。线上研讨会主要针对于 CUDA 和 API 的新特性，也有关于供应商相关工具的介绍。

我们还有更多关于 CUDA 和并行计算的学习资源。如有需要，可以通过下面的网站获取完整资源列表：www.learncuda.com。

12.7.3 教学课程

许多大学会在并行计算课程中加入 CUDA 的内容。它通常与 OpenMP 和 MPI 一起讲授。

OpenMP 和 MPI 分别是当今同一处理器核之间以及不同节点之间进行并行程序设计的主要模型。英伟达提供了一个十分好用的工具可以让我们快速找出周围的 CUDA 课程讲授地点，工具地址：http://research.nvidia.com/content/cuda-courses-map。根据英伟达的统计，到 2012 年上半年为止，世界上有 500 多家高校开设了 CUDA 课程。

12.7.4 书籍

我们可以找到很多讲授 CUDA 内容的书籍，但是没有任何一本可以涵盖 CUDA 或者并行编程的所有方面。如果需要的话，建议另外阅读下列文章：

- Jason Sanders 的《 CUDA by Example 》[㊀]
- Rob Farber 的《 CUDA Application Design and Development 》[㊁]
- D. Kirk 与 Wen-mei W. Hwu 合著的《 Programming Massively Parallel Processors 》[㊂]
- 多名作者编著的《 GPU Computing Gems 》，分成绿宝石版（ Emerald Edition）和绿玉版（Jade Edition）。

我根据 CUDA 和 GPU 新手学习者对以上书籍的接受度进行了排列。所有这些书籍在 Amazon 等购物网站都得到好评，也十分值得买下来细读。

12.7.5 英伟达 CUDA 资格认证

CUDA 资格认证程序是英伟达鉴定开发者 CUDA 能力水平的测试流程。测试包括部分多选题和一部分需要在规定时限内完成的编程任务。CUDA 考试大纲发布在英伟达的网站上：http://developer.nvidia.com/nvidia-cuda-professional-developer-program-study-guide 以及 http://developer.nvidia.com/cuda-certification。

需要涉猎的资料大多数都包含在了《 Programming Massively Parallel Processors 》里面。由于测试问题十分重视编程设计能力，因此你需要很好地掌握 CUDA 知识，不仅要能够从头写出大量 CUDA 内核，还要理解如何编写出高效、高性能的代码。本书覆盖了资格认证考试的大部分重要考点，但也会有涉及不到的内容。在本书涉及的一些领域中，已经远远超出了资格认证所需要的内容。要完成书中的思考题，就需要去研究各章节中的例子，通过学习这些例子和解答课后习题可以得到非常好的学习效果。

你还要关注 CUDA 的最新发展趋势，这些可能没在考试大纲中列出，但是会在英伟达的培训课和线上研讨会中出现。

㊀ 本书的中文翻译版《 GPU 高性能编程 CUDA 实战》，已由机械工业出版社出版，书号为 9787111326793。——编辑注

㊁ 本书的中文翻译版《高性能 CUDA 应用设计与开发》，已由机械工业出版社出版，书号为 9787111404460。——编辑注

㊂ 本书的第 2 版已由机械工业出版社出版，书号为 9787111416296。——编辑注

12.8 本章小结

行文至此，本书即将结束。本书主要从 CUDA 实践者的角度介绍了 CUDA。我诚挚地希望读者能从这本书里学到更多关于 CUDA、GPU 和 CPU 的有用知识，以及高效的编码方法。

我希望你能保持对 GPU 和 CUDA 应用的热情。陈旧的串行编程模型已经成为了过去时，应用在 GPU 和 CPU 上的并行架构才是未来计算的发展方向。我们现在正处于计算产业的历史转折点，而并行计算作为加大计算吞吐量的唯一方法，已经吸引了足够多的专业人士的关注。

编程人员的并行思考习惯越来越普遍，我们日常使用的智能手机已经或者即将是双核处理器。大部分平板电脑也是双核的。市场上占绝大部分的家庭娱乐计算机中有 92% 也都是多核的。而这些智能机中有大约 50% 用的都是英伟达的 GPU（Steam，2012 年 4 月）。

不论是消费级市场还是商业领域，CUDA 将可能给并行处理带来巨大的革命。你可以花费大概 500 美元购买一个诸如开普勒这样的高端显卡（GeForce GTX680）。GPU 行业正在沿着每隔几年性能翻倍的曲线上升，而且这种情况可能持续好多年。所以，现在正是学习 GPU 编程的大好时机。

参考文献

NVIDIA, "NVIDIA's Next Generation Compute Architecture: Kepler GK110." Available at *http://www.nvidia.com/content/PDF/kepler/NVIDIA-Kepler-GK110-Architecture-Whitepaper.pdf*, accessed May 18, 2012.

Steam, "Consumer Hardware Survey." Available at *http://store.steampowered.com/hwsurvey*, accessed April 14, 2012.

[General Information]
书名=CUDA并行程序设计　GPU编程指南
作者=ShaneCook著；苏统华译；李东译；李松泽译；魏通译
页数=522
SS号=13453675
出版日期=2014.01
出版社=北京：机械工业出版社
ISBN号=978-7-111-44861-7
中图法分类号=TP391.41
原书定价=71.55
参考文献格式=ShaneCook著；苏统华译；李东译；李松泽译；魏通译.CUDA并行程序设计　GPU编程指南.北京：机械工业出版社,2014.01.